THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms. ……R.SHARMILA…… Reg.No:20524UCSC041……, a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course Incharge: Dr. S Manimekalai

Head of the Department: Ms. KManohari

Course Incharge	Dr. S Manimekalai

Course In-Charge Head of the Department

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join		

	• Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX.NO:01: Creation of DDL operations using Books Table**

DATE:19.6.2026

AIM

To create a Books table, insert book records, display all records, and retrieve books published from the year 2020 onwards

Software Used

Oracle/MySQL/sqlite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects in a database.

DDL COMMANDS:

● Create

● Alter

● Drop

● Truncate

CREATE:

Create command is used to create a new table and also use constraints.

Syntax

sql

```
CREATE TABLE table_name(
    column1 datatype constraints,
    column2 datatype constraints,
    column3 datatype constraints
);
```

Table Structure

sql

```
SQL> CREATE TABLE books(
    book_id INTEGER PRIMARY KEY,
   title TEXT,
```

```
    author TEXT,
    published_year INTEGER
);
```

Table Created.

ALTER:

Alter command is used to alter a table or make some changes in a table. It has methods such as ADD, MODIFY, DROP and RENAME.

ADD:

Add command is used to insert a new attribute/column with data type.

Syntax:

sql

```
ALTER TABLE table_name ADD column_name datatype;
```

Query:

sql

```
SQL> ALTER TABLE books ADD price INTEGER;
```

Table Altered.

MODIFY:

Modify command is used to modify or change the data type and to increase the data size in the table.

Syntax:

sql

ALTER TABLE table_name MODIFY column_name datatype;

Query:

sql

SQL> ALTER TABLE books MODIFY title VARCHAR(150);

Table Altered.

DROP:

Drop command is used to delete the attribute/column in a table.

Syntax:

sql

ALTER TABLE table_name DROP COLUMN column_name;

Query:

sql

SQL> ALTER TABLE books DROP COLUMN price;

Table Altered.

RENAME:

Rename command is used to re-change the table name.

sql

SQL> RENAME TABLE books TO library_books;

```

Table renamed.
```

sql

```
ALTER TABLE books RENAME TO library_books;
```

Column Rename

sql

```
ALTER TABLE library_books RENAME COLUMN title TO book_title;
```

TRUNCATE:

Truncate command is used to delete all values in the table.

Syntax:

sql

```
TRUNCATE TABLE table_name;
```

Query:

sql

```
TRUNCATE TABLE library_books;
```

DROP:

Drop command is used to delete the table and all its data permanently.

Syntax:

sql

```
DROP TABLE table_name;
```

Query:

sql

DROP TABLE library_books;

Explanation

● Create a table named `books`.

● Insert book details into the table.

● Display all book records.

● Retrieve books published in or after the year 2020 and sort them by published year.

Create Table

sql

```
CREATE TABLE books(
    book_id INTEGER PRIMARY KEY,
    title TEXT,
    author TEXT,
    published_year INTEGER
);
```

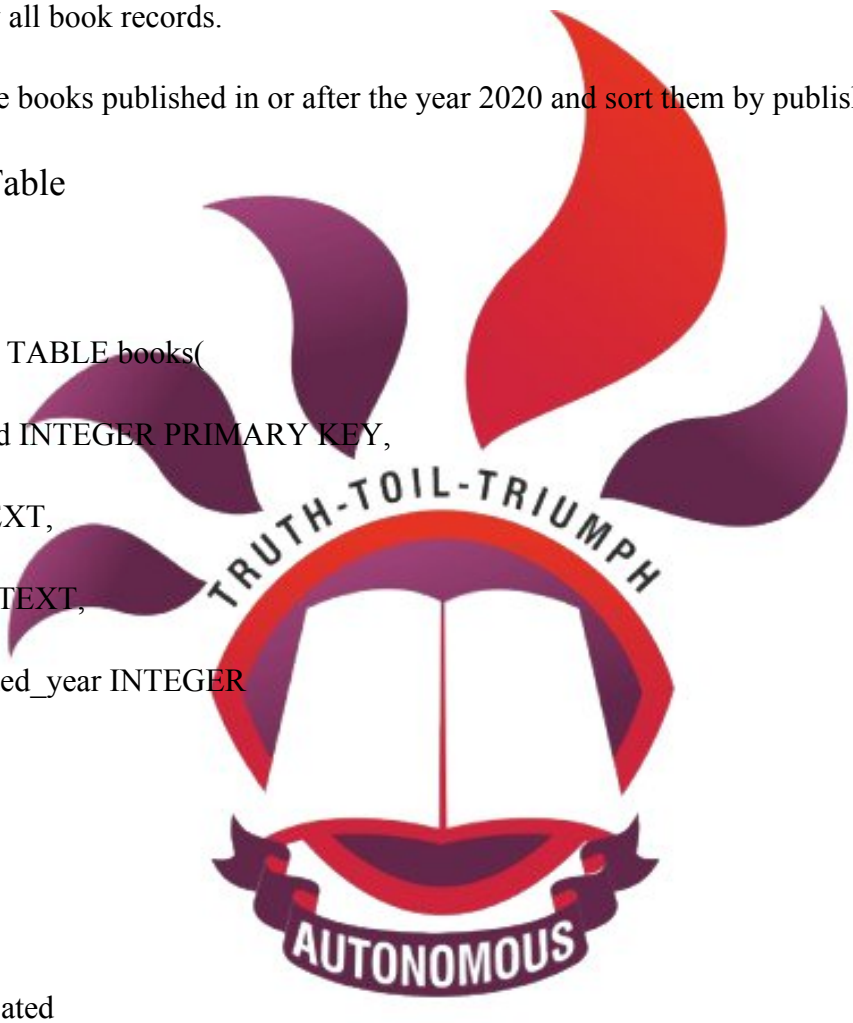


Table Created

Sample Data

sql

INSERT INTO books VALUES(1, 'Atomic Habits', 'James Clear', 2018);

INSERT INTO books VALUES(2, 'The Midnight Library', 'Matt Haig', 2020);

INSERT INTO books VALUES(3, 'Project Hail Mary', 'Andy Weir', 2021);

INSERT INTO books VALUES(4, 'The Psychology of Money', 'Morgan Housel', 2020);

INSERT INTO books VALUES(5, 'Klara and the Sun', 'Kazuo Ishiguro', 2021);

INSERT INTO books VALUES(6, 'Tomorrow and Tomorrow and Tomorrow', 'Gabrielle Zevin', 2022);

Records Inserted

SQL Query

Query1:

Display all book records

sql

SQL> SELECT * FROM books;

Resultant Query

book_id	title	author	published_year
1	Atomic Habits	James Clear	2018
2	The Midnight Library	Matt Haig	2020

3	Project Hail Mary	Andy Weir	2021
4	The Psychology of Money	Morgan Housel	2020
5	Klara and the Sun	Kazuo Ishiguro	2021
6	Tomorrow	Tomorrow Gabrielle Zevin	2022

Query2:

Display books published from the year 2020 onwards

sql

```
SQL> SELECT *
FROM books
WHERE published_year >= 2020
ORDER BY published_year;
```

Resultant Query

book_id	title	author	published_year
2	The Midnight Library	Matt Haig	2020
4	The Psychology of Money	Morgan Housel	2020
3	Project Hail Mary	Andy Weir	2021
5	Klara and the Sun	Kazuo Ishiguro	2021
6	Tomorrow and Tomorrow	Tomorrow Gabrielle Zevin	2022

RESULT:

Thus, the Books table was created successfully, records were inserted, all book details were displayed, and the books published from the year 2020 onwards were retrieved successfully.

EX.NO:02: Creation of DML Operations using Products Table

DATE:25.6.2026

Aim

To create a Products table, insert product records, display all records, update product details, delete a product record, and retrieve products based on their prices.

Software Used

Oracle/MySQL/sqlite3

Description

DML (Data Manipulation Language) is used to insert, modify, delete, and retrieve data stored in database tables.

DML COMMANDS:

- INSERT
- UPDATE
- DELETE
- SELECT

INSERT

INSERT command is used to add new records into a table.

Syntax

sql id="jv7v6q"

INSERT INTO table_name(column1,column2,column3)

VALUES(value1,value2,value3);

Query

sql id="qz2m6a"

SQL> INSERT INTO products VALUES(1, 'Laptop', 55000);

1 row inserted.

UPDATE

UPDATE command is used to modify existing records in a table.

Syntax

sql id="k8p0xv"

UPDATE table_name

SET column_name=value

WHERE condition;

Query

sql id="nq5m2e"

SQL> UPDATE products

SET price=58000

WHERE id=1;

1 row updated.

DELETE

DELETE command is used to remove existing records from a table.

Syntax

sql id="c7q1hs"

DELETE FROM table_name

WHERE condition;

Query

sql id="6v7j3n"

SQL> DELETE FROM products

WHERE id=5;

1 row deleted.

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

sql id="0yq7fr"

SELECT * FROM table_name;

Query

sql id="9q5z8k"

SQL> SELECT * FROM products;

Explanation

● Create a table named `products`.

● Insert product details into the table.

● Display all product records.

● Update the price of a product.

● Delete a product record.

● Retrieve products whose price is 30000 or above and sort them by price.

Create Table

sql id="a4n8ws"

```
CREATE TABLE products(
    id INTEGER PRIMARY KEY,
    name TEXT,
    price INTEGER
```

);

Table Created.

Sample Data

sql id="m3x7pk"

INSERT INTO products VALUES(1, 'Laptop', 55000);

INSERT INTO products VALUES(2, 'Keyboard', 1500);

INSERT INTO products VALUES(3, 'Smartphone', 35000);

INSERT INTO products VALUES(4, 'Headphones', 2500);

INSERT INTO products VALUES(5, 'Tablet', 28000);

INSERT INTO products VALUES(6, 'Monitor', 32000);

Records Inserted.

SQL Query

Query1:

Display all product records

sql id="7d5k2m"

SQL> SELECT * FROM products;

Resultant Query

id	name	price
1	Laptop	55000

2	Keyboard	1500
3	Smartphone	35000
4	Headphones	2500
5	Tablet	28000
6	Monitor	32000

Query2:

Update the price of Laptop

sql id="c4v8tz"

SQL> UPDATE products

SET price=58000

WHERE id=1;

1 row updated.

Updated Record

id	name	price
1	Laptop	58000

Query3:

Delete Tablet's record

sql id="w7k1fc"

SQL> DELETE FROM products

WHERE id=5;

1 row deleted.

Remaining Records

id	name	price
1	Laptop	58000
2	Keyboard	1500
3	Smartphone	35000
4	Headphones	2500
6	Monitor	32000

Query4:

Display products whose price is 30000 or above

sql id="h5q8vn"

```
SQL> SELECT *

FROM products

WHERE price >= 30000

ORDER BY price;
```

Resultant Query

id	name	price
6	Monitor	32000

3	Smartphone	35000
1	Laptop	58000

RESULT

Thus, the Products table was created successfully, records were inserted, product details were displayed, the product price was updated, a product record was deleted, and the products whose price was 30000 or above were retrieved successfully.

EX.NO:03: Creation of DQL Operations using Sales Table

DATE:

Aim

To create a Sales table, insert sales records, display all records, calculate total sales and average sales for each category, and retrieve categories based on total sales using DQL operations.

Software Used

Oracle/MySQL/sqlite3

Description

DQL (Data Query Language) is used to retrieve and display data stored in database tables. The main command used in DQL is SELECT.

DQL COMMANDS:

- SELECT
- GROUP BY
- SUM

● AVG

● HAVING

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

sql id="m4t6nx"

SELECT * FROM table_name;

Query

sql id="r8v2kp"

SQL> SELECT * FROM sales;

GROUP BY

GROUP BY command is used to group rows having the same values in specified columns.

Syntax

sql id="f7q3wc"

SELECT column_name, SUM(column_name)

FROM table_name

GROUP BY column_name;

SUM

SUM function is used to calculate the total value of a numeric column.

Syntax

sql id="k9z5ad"

SELECT SUM(column_name)

FROM table_name;

AVG

AVG function is used to calculate the average value of a numeric column.

Syntax

sql id="p2n6yv"

SELECT AVG(column_name)

FROM table_name;

HAVING

HAVING command is used to filter grouped records based on a condition.

Syntax

`sql id="x3c8lm"

SELECT column_name, SUM(column_name)

FROM table_name

GROUP BY column_name

HAVING condition;

Explanation

● Create a table named `sales`.

● Insert sales details into the table.

● Display all sales records.

● Calculate the total sales for each category.

● Calculate the average sales for each category.

● Retrieve categories whose total sales are greater than 50000.

Create Table

sql id="a7k2pd"

```
CREATE TABLE sales(
    sale_id INTEGER PRIMARY KEY,
    product_name TEXT,
    category TEXT,
    amount INTEGER
);
```

Table Created.

Sample Data

sql id="v5m9qr"

```
INSERT INTO sales VALUES(1, 'Laptop', 'Electronics', 55000);
INSERT INTO sales VALUES(2, 'Mobile', 'Electronics', 30000);
INSERT INTO sales VALUES(3, 'Chair', 'Furniture', 12000);
INSERT INTO sales VALUES(4, 'Table', 'Furniture', 18000);
INSERT INTO sales VALUES(5, 'Headphones', 'Electronics', 8000);
INSERT INTO sales VALUES(6, 'Sofa', 'Furniture', 25000);
```

Records Inserted.

SQL Query

Query1:

Display all sales records

`sql id="n6w4sz"

SQL> SELECT * FROM sales;

Resultant Query

Saleid	product name	category	amount
1	Laptop	Electronics	55000
2	Mobile	Electronics	30000
3	Chair	Furniture	12000
4	Table	Furniture	18000
5	Headphones	Electronics	8000
6	Sofa	Furniture	25000

Query2:

Display total sales for each category

sql id="c8p3vy"

SQL> SELECT category, SUM(amount) AS total_sales

FROM sales

GROUP BY category;

Resultant Query

category	total_sales

Electronics	93000
Furniture	55000

Query3:

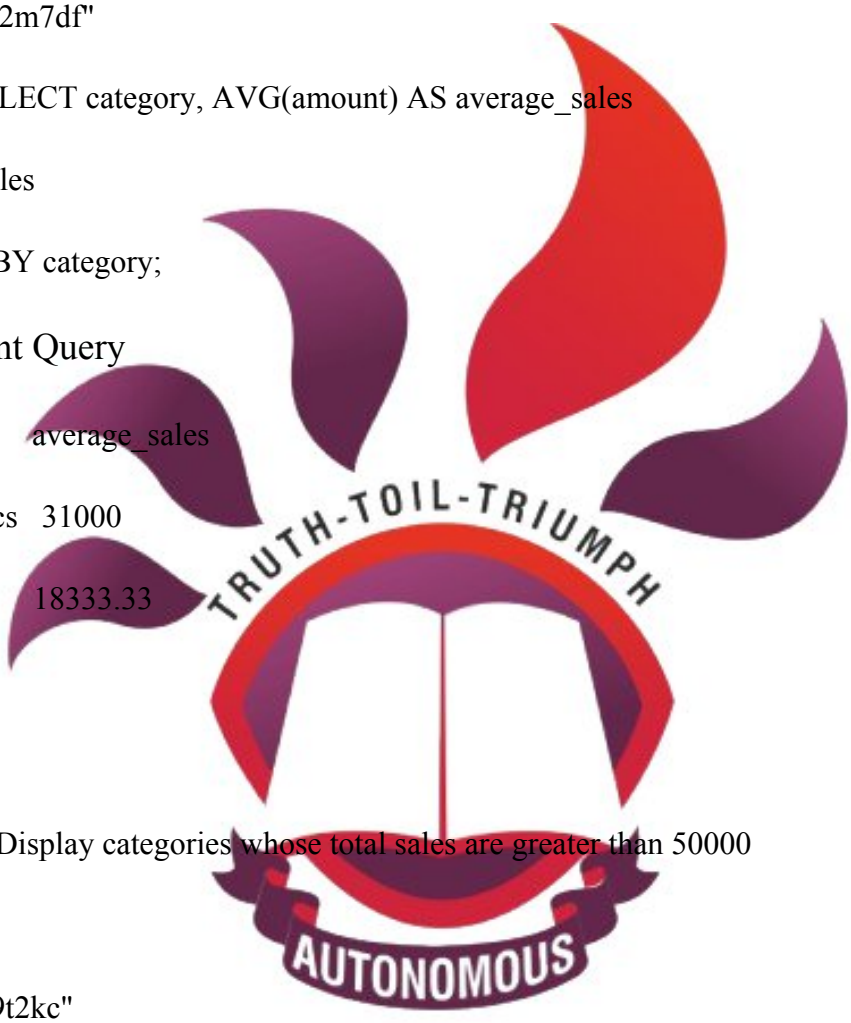
Display average sales for each category

sql id="w2m7df"

```
SQL> SELECT category, AVG(amount) AS average_sales
FROM sales
GROUP BY category;
```

Resultant Query

```
category      average_sales
Electronics   31000
Furniture     18333.33
```



Query4: Display categories whose total sales are greater than 50000

sql id="e9t2kc"

```
SQL> SELECT category, SUM(amount) AS total_sales
FROM sales
GROUP BY category
HAVING SUM(amount) > 50000;
```

Resultant Query

category total_sales

Electronics 93000

Furniture 55000

RESULT:

Thus, the Sales table was created successfully, records were inserted, all sales details were displayed, total and average sales were calculated for each category, and the categories having total sales greater than 50000 were retrieved successfully.

EX.NO: 04: Creation of Transaction Operations using Bank Transactions Table

DATE: _____

Aim

To create a `bank_transactions` table, insert transaction records, perform transaction operations using **COMMIT, ROLLBACK, SAVEPOINT**, and verify the changes made to the database.

Software Used

Oracle / MySQL / SQLite3

Description

Transaction Control Language (TCL) is used to manage transactions in a database. It helps to save or undo the changes made to the database.

TRANSACTION COMMANDS:

- COMMIT
- ROLLBACK

● SAVEPOINT

● ROLLBACK TO SAVEPOINT

COMMIT

COMMIT command is used to permanently save all the changes made during a transaction.

Syntax

sql

```
COMMIT;
```

Query

sql

```
SQL> COMMIT;
```

Transaction committed.

ROLLBACK

command is used to undo the changes made to the database before the changes are committed.

Syntax

sql

```
ROLLBACK;
```

Querysql

```
SQL> ROLLBACK;
```

Transaction rolled back.

SAVEPOINT

SAVEPOINT command is used to create a temporary point within a transaction. The transaction can be rolled back to that specific point.

Syntax

sql

SAVEPOINT savepoint_name;

Query

sql

SQL> SAVEPOINT sp1;

Savepoint created.

ROLLBACK TO SAVEPOINT

ROLLBACK TO SAVEPOINT is used to undo the changes made after a particular savepoint without undoing the entire transaction.

Syntaxsql

ROLLBACK TO SAVEPOINT savepoint_name;

Query

sql

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

Explanation

● Create a table named `bank_transactions`.

● Insert transaction details into the table.

● Use COMMIT to permanently save the inserted records.

● Update a customer's balance and create a SAVEPOINT.

● Make another change after the savepoint.

● Use ROLLBACK TO SAVEPOINT to undo the changes made after the savepoint.

● Use ROLLBACK to undo uncommitted changes.

Create Table

sql

```
CREATE TABLE bank_transactions(
    transaction_id INTEGER PRIMARY KEY,
    customer_name TEXT,
    transaction_type TEXT,
    amount INTEGER,
    balance INTEGER
);
```

Table Created

Sample Data

sql

```
INSERT INTO bank_transactions VALUES(201, 'Rahul', 'Deposit', 10000, 50000);

INSERT INTO bank_transactions VALUES(202, 'Anitha', 'Deposit', 15000, 60000);

INSERT INTO bank_transactions VALUES(203, 'Vijay', 'Withdraw', 5000, 25000);

INSERT INTO bank_transactions VALUES(204, 'Sneha', 'Deposit', 20000, 70000);

INSERT INTO bank_transactions VALUES(205, 'Karthik', 'Withdraw', 8000, 32000);
```

Records Inserted

SQL Query

Query 1:

Display all transaction records

`sql

```
SQL> SELECT * FROM bank_transactions;
```

Resultant Query

transaction_id	customer_name	transaction_type	amount	balance
201	Rahul	Deposit	10000	50000
202	Anitha	Deposit	15000	60000
203	Vijay	Withdraw	5000	25000
204	Sneha	Deposit	2500	70000
205	Karthik	Withdraw	8000	32000

Query 2:

Commit the inserted records

sql

SQL> COMMIT;

Transaction committed.

The inserted transaction records are now permanently saved.

Query 3:

Update Rahul's account balance

sql

SQL> UPDATE bank_transactions

SET balance = 55000

HERE transaction_id = 201;

1 row updated.

Updated Record

transaction_id	customer_name	balance
201	Rahul	55000

Query 4:

Create a SAVEPOINT

sql

SQL> SAVEPOINT sp1;

Savepoint created.

Query 5:

Update Anitha's account balance

sql

SQL> UPDATE bank_transactions

SET balance = 65000

WHERE transaction_id = 202;

1 row updated.

Updated Record

Transaction id	Customer name	balance
| 202	Anitha	65000

Query 6:

Rollback to SAVEPOINT

sql

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed

The changes made after SAVEPOINT `sp1` are undone.

Resultant Query

transaction id	Customer name	balance
201	Rahul	55000
202	Anitha	60000

Query 7:

Rollback an uncommitted change

sql

SQL> UPDATE bank_transactions

SET balance = 20000

WHERE transaction_id = 203;

1 row updated.

sql

SQL> ROLLBACK;

Transaction rolled back.

The balance of Vijay is restored to its previous value

Resultant Query

transaction_id	customer_name	balance
| 203	Vijay	25000

RESULT

Thus, the `bank_transactions` table was created successfully, transaction records were inserted, and the transaction operations **COMMIT, ROLLBACK, SAVEPOINT, and ROLLBACK TO SAVEPOINT** were performed successfully.

EX.NO:05 Creation of Join Operations Using Games and Players Table

DATE: 22.6.2026

AIM

To create Games and Players tables, insert records, and retrieve related data using different types of joins such as Inner Join, Left Join, Right Join, Full Join, and Self Join.

Software Used

Oracle / MySQL / SQLite3

DESCRIPTION

Joins are used to combine records from two or more tables based on a related column between them.

JOIN COMMANDS:

• Inner Join

• Left Join

• Right Join

• Full Join

• Self Join

CREATE TABLE

sql

```
CREATE TABLE Games (
    Game_ID INT PRIMARY KEY,
    Game_Name VARCHAR(30),
    Game_Type VARCHAR(20)
);
```

sql

```
CREATE TABLE Players (
    Player_ID INT PRIMARY KEY,
    Player_Name VARCHAR(30),
    Game_ID INT
);
```

INSERT RECORDS

sql

```
INSERT INTO Games VALUES (101, 'Free Fire', 'Action');
INSERT INTO Games VALUES (102, 'Minecraft', 'Adventure');
INSERT INTO Games VALUES (103, 'Chess', 'Strategy');
INSERT INTO Games VALUES (104, 'Subway Surfers', 'Arcade');
```

INSERT INTO Games VALUES (105, 'Valorant', 'Shooting');

sql

INSERT INTO Players VALUES (1, 'Arun', 101);

INSERT INTO Players VALUES (2, 'Bala', 102);

INSERT INTO Players VALUES (3, 'Cathy', 103);

INSERT INTO Players VALUES (4, 'Divya', 101);

INSERT INTO Players VALUES (5, 'Esha', 106);

INNER JOIN

Inner Join returns only the records that have matching values in both tables.

Syntax:

sql

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

Query:

sql

```
SELECT Games.Game_Name, Players.Player_Name
FROM Games
INNER JOIN Players
ON Games.Game_ID = Players.Game_ID;
```

Result:

GAME	PLAYER NAME
Free Fire	Arun
Minecraft	Bala
Chess	Cathy
Free Fire	Divya

LEFT JOIN

Left Join returns all records from the left table and the matching records from the right table.

Syntax:

sql

SELECT columns

FROM table1

LEFT JOIN table2

ON table1.column = table2.column;

Query:

sql

SELECT Games.Game_Name, Players.Player_Name

FROM Games

LEFT JOIN Players

ON Games.Game_ID = Players.Game_ID;

Result:

GAME_NAME	PLAYER_NAME
Free Fire	Arun
Free Fire	Divya
Minecraft	Bala
Chess	Cathy
Subway Surfers	NULL
Valorant	NULL

RIGHT JOIN

Right Join returns all records from the right table and the matching records from the left table.

Syntax:

sql

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

Query:

sql

```
SELECT Games.Game_Name, Players.Player_Name
FROM Games
RIGHT JOIN Players
ON Games.Game_ID = Players.Game_ID;
```

Result:

GAME_NAME	|PLAYER_NAME
Free Fire |	Arun
Minecraft	Bala
Chess	Cathy
Free Fire	Divya
NULL	Esha |

FULL JOIN

Full Join returns all records from both tables. It includes matching and non-matching records.

Syntax:

sql

```
SELECT columns
FROM table1
FULL JOIN table2
ON table1.column = table2.column;
```

Query:

sql

```
SELECT Games.Game_Name, Players.Player_Name
FROM Games
FULL JOIN Players
ON Games.Game_ID = Players.Game_ID;
```

Result query :

GAME_NAME	|PLAYER_NAME
Free Fire	Arun
Free Fire	Divya
Minecraft	Bala
Chess	Cathy
Subway Surfers	NULL
Valorant	| NULL
NULL	| Esha

SELF JOIN

Self Join is a join in which a table is joined with itself.

Query:

`sql

```
SELECT P1.Player_Name AS Player1,
    P2.Player_Name AS Player2
FROM Players P1
JOIN Players P2
ON P1.Game_ID = P2.Game_ID
AND P1.Player_ID < P2.Player_ID;
```

Result query:

PLAYER1	PLAYER2
Arun	Divya

RESULT

Thus, the Games and Players tables were created successfully, records were inserted, and different types of join operations such as Inner Join, Left Join, Right Join, Full Join, and Self Join were executed successfully.

EX. NO: 05: Creation of SQL Functions Using Travel_Bookings Table

DATE:

AIM

To create a Travel_Bookings table, insert booking records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / sqlite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL FUNCTION TYPES:

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS

Aggregate functions are used to perform calculations on a group of rows and return a single result.

Common aggregate functions are:

● COUNT()

● SUM()

● AVG()

● MAX()

● MIN()

Syntax

sql

```
SELECT aggregate_function(column_name)
FROM table_name;
```

STRING FUNCTIONS

String functions are used to perform operations on text or character values.

Common string functions are:

● UPPER()

● LOWER()

● LENGTH()

Syntax

sql

```
SELECT string_function(column_name)
FROM table_name;
```

DATE FUNCTIONS

Date functions are used to perform operations on date values.

Common date functions are:

● CURRENT_DATE

● YEAR()

● MONTH()

● DAY()

Syntax

sql

```
SELECT date_function(date_column)
FROM table_name;
```

Explanation

● Create a table named travel_bookings.

● Insert travel booking details into the table.

● Display all travel booking records.

● Calculate the total number of bookings using COUNT().

● Calculate the total and average booking amount using SUM() and AVG().

● Find the highest and lowest booking amount using MAX() and MIN().

● Display destination names in uppercase and lowercase using UPPER() and LOWER().

● Find the length of each destination name using LENGTH().

● Retrieve booking details based on the travel date.

Create Table

sql

CREATE TABLE travel_bookings (

booking_id INTEGER PRIMARY KEY,

traveller_name TEXT,

destination TEXT,

booking_amount INTEGER,

travel_date DATE

);

Table Created

Sample Data

sql

INSERT INTO travel_bookings VALUES (1, 'Arun', 'Chennai', 4500, '2026-01-10');

INSERT INTO travel_bookings VALUES (2, 'Meena', 'Ooty', 6500, '2026-02-15');

INSERT INTO travel_bookings VALUES (3, 'Kavin', 'Goa', 8500, '2026-03-12');

INSERT INTO travel_bookings VALUES (4, 'Divya', 'Mysore', 5500, '2026-04-18');

INSERT INTO travel_bookings VALUES (5, 'Priya', 'Kerala', 7500, '2026-05-20');

INSERT INTO travel_bookings VALUES (6, 'Rahul', 'Bangalore', 4000, '2026-06-25');

Records Inserted

SQL Query

Query 1

: Display all travel booking records

sql

SELECT * FROM travel_bookings;

Resultant Query

booking_id	Btraveller_name	destination	booking_amount	travel_date
1	Arun	Chennai	4500	2026-01-10
2	Meena	Ooty	6500	2026-02-15
3	Kavin	Goa	8500	2026-03-12
4	Divya	Mysore	5500	2026-04-18
5	Priya	Kerala	7500	2026-05-20
6	Rahul	Bangalore	4000	2026-06-25

AGGREGATE FUNCTIONS

Query 2:

Count the total number of bookings

sql

SELECT COUNT(*) AS Total_Bookings

FROM travel_bookings;

Query 3:

Calculate the total booking amount

sql

SELECT SUM(booking_amount) AS Total_Amount

FROM travel_bookings;

Resultant Query

Total_Amount

42000

Query 4:

Calculate the average booking amount

sql

```
SELECT AVG(booking_amount) AS Average_Amount
FROM travel_bookings;
```

Resultant Query

Average_Amount

7000

Query

Find the highest and lowest booking amount

sql

```
SELECT MAX(booking_amount) AS Highest_Amount,
MIN(booking_amount) AS Lowest Amount
FROM travel_bookings;
```

Resultant Query

Highest_Amount	Lowest_Amount
8500	4000

STRING FUNCTIONS

Query 6:

Display destination names in uppercase

sql

```
SELECT destination, UPPER(destination) AS Upper_Destination
```

FROM travel_bookings;

Resultant Query

destination	Upper Destination
Chennai	CHENNAI
Ooty	OOTY
Goa	GOA
Mysore	MYSORE
Kerala	KERALA
Bangalore	BANGALORE

|Query 7:

Display destination names in lowercase

sql

SELECT destination, LOWER(destination) AS Lower_Destination

FROM travel_bookings;

Resultant Query

| destination | Lower_Destination |

| Chennai | chennai |

| Ooty | ooty |

| Goa | goa |

| Mysore | mysore |

| Kerala | kerala |

| Bangalore | bangalore |

Query 8: Display the length of each destination name

```sql
SELECT destination, LENGTH(destination) AS Destination_Length
FROM travel_bookings;
```

Resultant Query

destination	Destination_Length
Chennai	7
Ooty	4
Goa	3
Mysore	6
Kerala	6
Bangalore	9

DATE FUNCTIONS

Query 9: Display the current date

```sql
SELECT CURRENT_DATE AS Current_Date;
```

Resultant Query

Current_Date

Current system date

Query 10:

Display bookings made for travel after March 2026

sql

```
SELECT traveller_name, destination, travel_date
FROM travel_bookings
WHERE travel_date > '2026-03-31';
```

Resultant Query

traveller_name | destination | travel_date |

| Divya | Mysore | 2026-04-18 |

| Priya | Kerala | 2026-05-20 |

| Rahul | Bangalore | 2026-06-25 |

RESULT:

Thus, the Travel_Bookings table was created successfully, travel booking records were inserted, and the required data was retrieved successfully using Aggregate Functions, String Functions, and Date Functions.

EX. NO: 06: Creation of SQL Functions Using Travel_Bookings Table

DATE:10.7.2026

AIM

To create a Travel_Bookings table, insert booking records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / sqlite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL FUNCTION TYPES:

● Aggregate Functions

● String Functions

● Date Functions

AGGREGATE FUNCTIONS

Aggregate functions are used to perform calculations on a group of rows and return a single result.

Common aggregate functions are:

● COUNT()

● SUM()

● AVG()

● MAX()

● MIN()

Syntax

```sql

SELECT aggregate_function(column_name)
```

FROM table_name;

```

### STRING FUNCTIONS

String functions are used to perform operations on text or character values.

Common string functions are:

- UPPER()
- LOWER()
- LENGTH()

### Syntax

```sql
SELECT string_function(column_name)
FROM table_name;
```

DATE FUNCTIONS
```

Date functions are used to perform operations on date values.

Common date functions are:

- CURRENT_DATE
- YEAR()
- MONTH()
- DAY()

Syntax

```sql
SELECT date_function(date_column)
FROM table_name;
```

Explanation

- Create a table named travel_bookings.
- Insert travel booking details into the table.

● Display all travel booking records.

● Calculate the total number of bookings using COUNT().

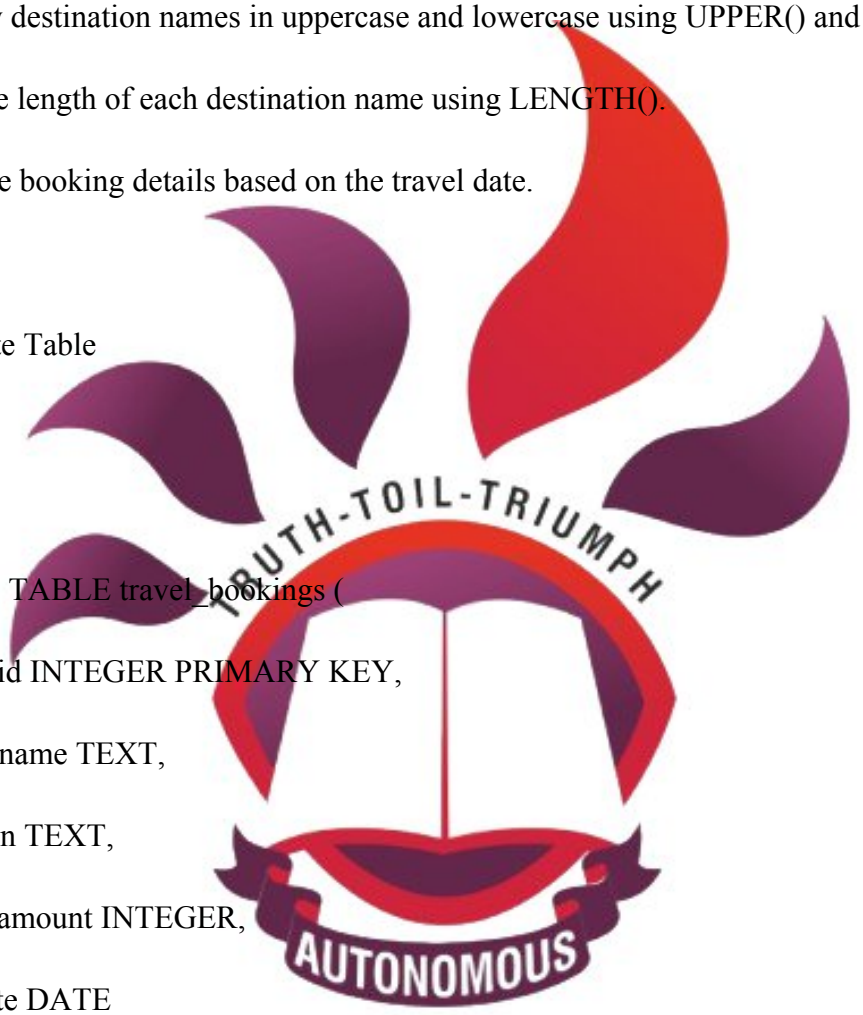
● Calculate the total and average booking amount using SUM() and AVG().

● Find the highest and lowest booking amount using MAX() and MIN().

● Display destination names in uppercase and lowercase using UPPER() and LOWER().

● Find the length of each destination name using LENGTH().

● Retrieve booking details based on the travel date.



Create Table

```sql
CREATE TABLE travel_bookings (
booking_id INTEGER PRIMARY KEY,
traveller_name TEXT,
destination TEXT,
booking_amount INTEGER,
travel_date DATE
);
```

Table Created

Sample Data

```sql
INSERT INTO travel_bookings VALUES (1, 'Arun', 'Chennai', 4500, '2026-01-10');

INSERT INTO travel_bookings VALUES (2, 'Meena', 'Ooty', 6500, '2026-02-15');

INSERT INTO travel_bookings VALUES (3, 'Kavin', 'Goa', 8500, '2026-03-12');

INSERT INTO travel_bookings VALUES (4, 'Divya', 'Mysore', 5500, '2026-04-18');

INSERT INTO travel_bookings VALUES (5, 'Priya', 'Kerala', 7500, '2026-05-20');

INSERT INTO travel_bookings VALUES (6, 'Rahul', 'Bangalore', 4000, '2026-06-25');
```

Records Inserted

SQL Query

Query 1: Display all travel booking records

```sql
SELECT * FROM travel_bookings;
```

Resultant Query

booking_id	traveller_name	destination booking_amount	travel_date
1	Arun	Chennai	4500	2026-01-10
2	Meena	Ooty	6500	2026-02-15
3	Kavin	Goa	8500	2026-03-12
4	Divya	Mysore	5500	2026-04-18
5	Priya	Kerala	7500	2026-05-20
6	Rahul	Bangalore	4000	2026-06-25

AGGREGATE FUNCTIONS

Query 2: Count the total number of bookings

```sql

SELECT COUNT(*) AS Total_Bookings

FROM travel_bookings;

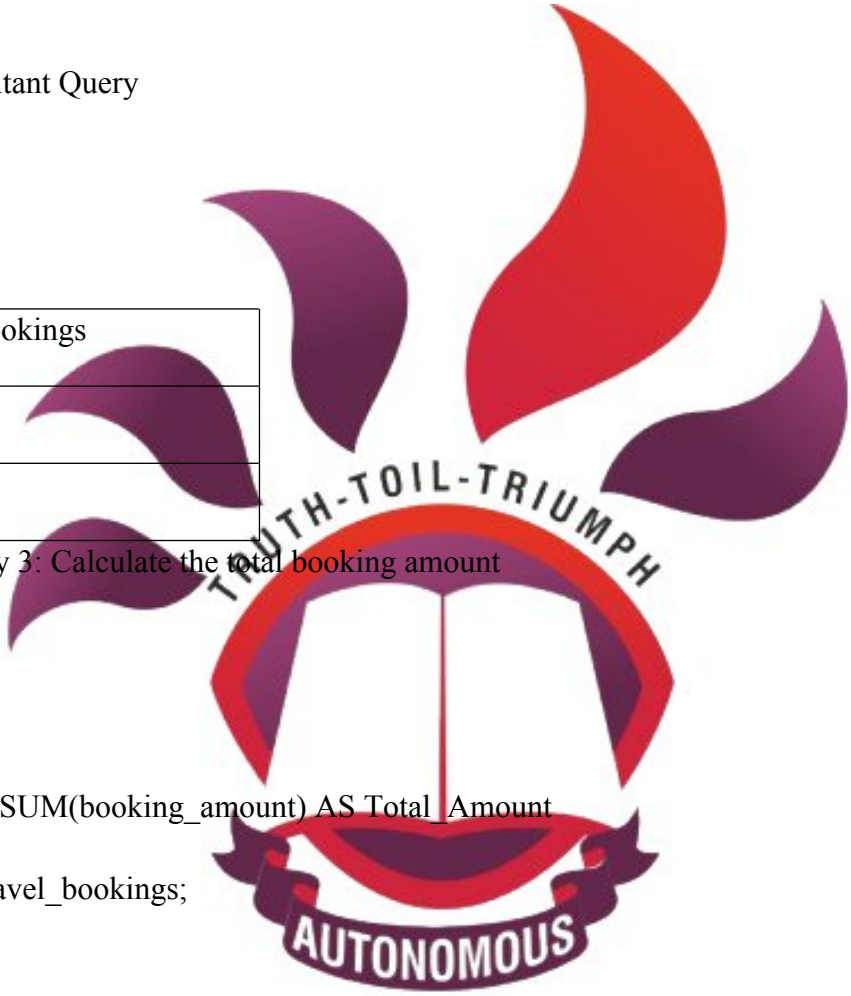
```

Resultant Query

|

Total Bookings
6

Query 3: Calculate the total booking amount




```sql

SELECT SUM(booking_amount) AS Total_Amount

FROM travel_bookings;

```

Resultant Query

|

Total Amount
42000

|

Query 4: Calculate the average booking amount

```sql
SELECT AVG(booking_amount) AS Average_Amount
FROM travel_bookings;
```

Resultant Query

Average_Amount
7000

Query 5: Find the highest and lowest booking amount

```sql
```

SELECT MAX(booking_amount) AS Highest_Amount,

MIN(booking_amount) AS Lowest_Amount

FROM travel_bookings;

```

### Resultant Query

|

| Highest Amount | LowestAmount |
|---|---|
| 8500 | 4000 |

|

# STRING FUNCTIONS

### Query 6: Display destination names in uppercase

```sql

SELECT destination, UPPER(destination) AS Upper_Destination

FROM travel_bookings;

```
```

Resultant Query

destination	Upper Destination
Chennai	CHENNAI
Ooty	OOTY
Goa	GOA
Mysore	MYSORE
Kerala	KERALA
Bangalore	BANGALORE

Query 7: Display destination names in lowercase

sql

```
SELECT destination, LOWER(destination) AS Lower_Destination
FROM travel_bookings;
```

Resultant Query

|

destination	Lower Destination
Chennai	chennai

Ooty	ooty
Goa	goa
Mysore	mysore
Kerala	kerala
Bangalore	bangalore

|

Query 8: Display the length of each destination name

sql

```
SELECT destination, LENGTH(destination) AS Destination_Length
FROM travel_bookings;
```

Resultant Query

|

destination	Destination_Length
| Chennai	7
| Ooty	4
| Goa	3
| Mysore	6
| Kerala	6
| Bangalore	9

DATE FUNCTIONS

Query 9: Display the current date

sql

```
SELECT CURRENT_DATE AS Current_Date;
```

Resultant Query

Current_Date
Current system date

Query 10: Display bookings made for travel after March 2026

sql

```
SELECT traveller_name, destination, travel_date
FROM travel_bookings
WHERE travel_date > '2026-03-31';
```

Resultant Query

|

traveller_name	destination	travel_date |

| Divya	Mysore	2026-04-18
| Priya	Kerala	2026-05-20
| Rahul	Bangalore	2026-06-25 |

RESULT:

Thus, the Travel_Bookings table was created successfully, travel booking records were inserted, and the required data was retrieved successfully using Aggregate Functions, String Functions, and Date Functions.

EX. NO: 07: Creation of Subqueries and Nested Queries using Products

DATE:14.7.2026

Aim

To create a Products table, insert product records, and retrieve data using Subqueries and Nested Queries.

Software Used

Oracle / MySQL / sqlite3

Description

A Subquery is a query written inside another SQL query. It is used to retrieve intermediate results and use them in the main query.

A Nested Query is a query placed inside another query to perform operations based on the result of the inner query.

SUBQUERY TYPES:

• Single Row Subquery

• Multiple Row Subquery

• Nested Subquery

SINGLE ROW SUBQUERY

A Single Row Subquery returns only one value or one row and is used with comparison operators such as =, >, <, >= and <=.

Syntax:

```sql
SELECT column_name
FROM table_name
WHERE column_name >
(SELECT column_name
FROM table_name
WHERE condition);
```

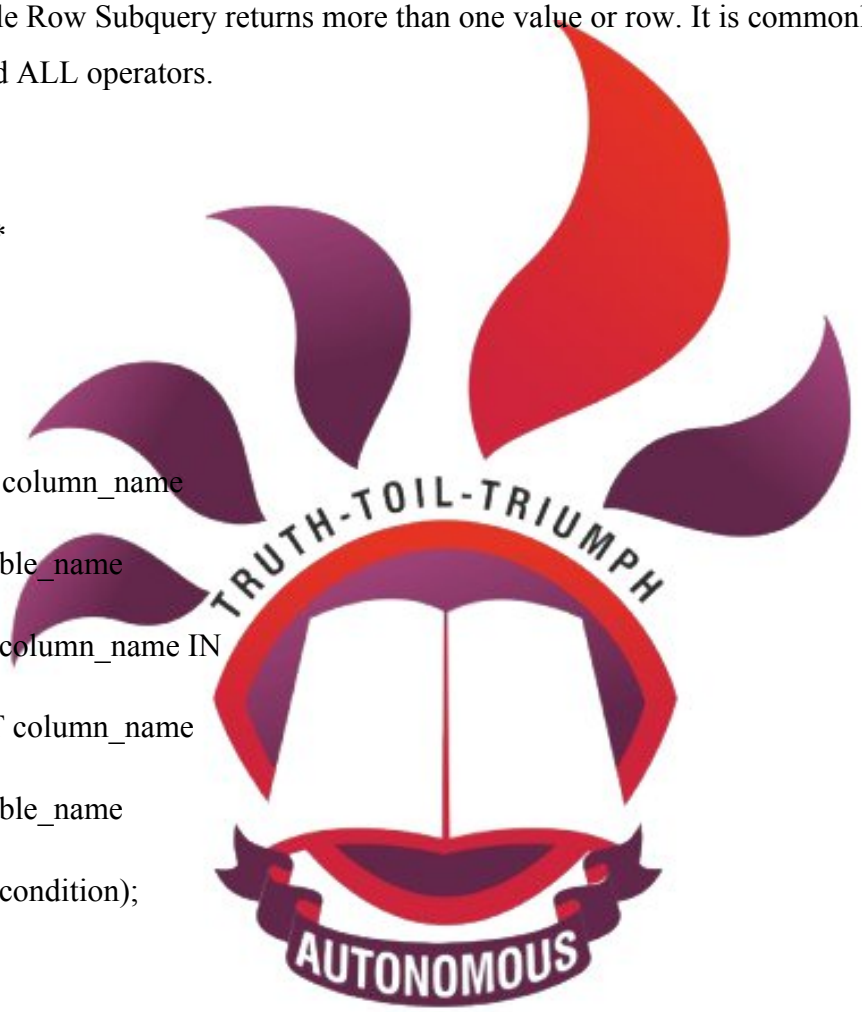
```

### MULTIPLE ROW SUBQUERY

A Multiple Row Subquery returns more than one value or row. It is commonly used with IN, ANY, and ALL operators.

Syntax:**

```sql
SELECT column_name
FROM table_name
WHERE column_name IN
(SELECT column_name
FROM table_name
WHERE condition);
```



NESTED SUBQUERY

A Nested Subquery contains one query inside another query. The result of the inner query is used by the outer query.
```

Syntax:

sql

SELECT column_name

FROM table_name

WHERE column_name =

(SELECT column_name

FROM table_name

WHERE condition);

```

### Explanation




• Create a table named products.

• Insert product details into the table.

• Display all product records.

• Find products whose price is greater than the average price.

• Find products that belong to the Electronics or Clothing categories.

• Find the product that has the highest price.

• Retrieve products based on the price of another product.
```

Create Table

```sql
CREATE TABLE products (
product_id INTEGER PRIMARY KEY,
product_name TEXT,
category TEXT,
price INTEGER
);
```

Table Created

Sample Data

```sql
INSERT INTO products VALUES (1, 'Laptop', 'Electronics', 55000);

INSERT INTO products VALUES (2, 'T-Shirt', 'Clothing', 1200);
```

INSERT INTO products VALUES (3, 'Headphones', 'Electronics', 3500);

INSERT INTO products VALUES (4, 'Smartphone', 'Electronics', 30000);

INSERT INTO products VALUES (5, 'Jeans', 'Clothing', 2500);

INSERT INTO products VALUES (6, 'Watch', 'Accessories', 5000);

```

**Records Inserted**

SQL Query

### Query 1: Display all product records

sql

SELECT * FROM products;

Resultant Query

| productid | productname | category | price |
|---|---|---|---|
```

1	Laptop	Electronics	55000
2	T-Shirt	Clothing	1200
3	Headphones	Electronics	3500
4	Smartphone	Electronics	30000
5	Jeans	Clothing	2500
6	Watch	Accessories	5000

Query 2: Display products whose price is greater than the average price

```sql
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price)
OM products);
```

Resultant Query

productname	price
Laptop	55000
Smartphone	30000

Query 3: Display products that belong to Electronics or Clothing category

```sql

SELECT product_name, category

FROM products

WHERE category IN (SELECT category

FROM products

WHERE category IN ('Electronics', 'Clothing'));

```

Resultant Query

|

product_name	category
Laptop	Electronics
T-Shirt	Clothing
Headphones	Electronics
Smartphone	Electronics
Jeans	Clothing

|

Query 4: Display the product with the highest price

```sql
```

```
SELECT product_name, price

FROM products

WHERE price = (SELECT MAX(price)

FROM products);
```

Resultant Query

|

productname	price
Laptop	55000

Query 5: Display products whose price is greater than T-Shirt's price

```sql
SELECT product_name, price

FROM products

WHERE price > (SELECT price

FROM products

WHERE product_name = 'T-Shirt');
```

Resultant Query

|

product_name	price
| Laptop	55000
| Headphones	3500
| Smartphone	30000
| Jeans	250|
| Watch	5000

RESULT:

Thus, the Products table was created successfully, product records were inserted, and the required product details were retrieved successfully using Subqueries and Nested Queries.

EX. NO:08: Creation of Views, Indexes and Sequences Using Patient Details**

DATE:

Aim

To create and perform operations on Views, Indexes, and Sequences using SQL commands.

Software Used

Oracle SQL / SQL Command Line

Description

View

A View is a virtual table created from one or more tables using a SELECT query. It does not store the actual data separately.

Index:**

An Index is used to improve the speed of data retrieval from a table.

Sequence:**

A Sequence is a database object used to generate unique numeric values automatically.

1. VIEW

Definition

A View is a virtual table based on the result of a SQL query.

Syntax

```sql
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Create Table

sql

```
CREATE TABLE patient_details (
patient_id NUMBER PRIMARY KEY,
patient_name VARCHAR2(50),
department VARCHAR2(30),
consultation_fee NUMBER
);
```

Table Created

Insert Records

sql

INSERT INTO patient_details VALUES (301, 'Kavin', 'Cardiology', 800);

INSERT INTO patient_details VALUES (302, 'Meena', 'General', 400);

INSERT INTO patient_details VALUES (303, 'Arun', 'Neurology', 1000);

INSERT INTO patient_details VALUES (304, 'Divya', 'Cardiology', 900);

INSERT INTO patient_details VALUES (305, 'Priya', 'General', 500);

COMMIT;

Records Inserted

Create View

sql

CREATE VIEW cardiology_patients AS

SELECT patient_id, patient_name, consultation_fee

FROM patient_details

WHERE department = 'Cardiology';

```

View Created**

Display View

```sql

SELECT * FROM cardiology_patients;

```

Resultant Query

|

Patient id	patient name	consultation fee |
301	Kavin	800 |
304	Divya	900 |

Explanation
```

The `cardiology_patients` view displays only the patients who belong to the Cardiology department.

2. INDEX

Definition

An Index is a database object that improves the speed of searching and retrieving records from a table.

Syntax

```sql
CREATE INDEX index_name
ON table_name (column_name);
```

Create Index

sql

```
CREATE INDEX idx_patient_name
ON patient_details(patient_name);
```

```

Index Created**

Use Index in Search

sql

SELECT *

FROM patient_details

WHERE patient_name = 'Kavin';

Resultant Query

|

| patientid | patientname | department | consultation_fee |
|---|---|---|---|
| 301 | Kavin | Cardiology | 800 |

### Explanation

The index `idx_patient_name` is created on the `patient_name` column. It helps improve the performance of searches based on patient names.
```

3. SEQUENCE

Definition

A Sequence is a database object that automatically generates a series of unique numeric values.

Syntax

sql

```
CREATE SEQUENCE sequence_name
START WITH initial_value
INCREMENT BY increment_value;
```

Create Sequence

sql

```
CREATE SEQUENCE patient_seq
START WITH 306
INCREMENT BY 1;
```

Sequence Created

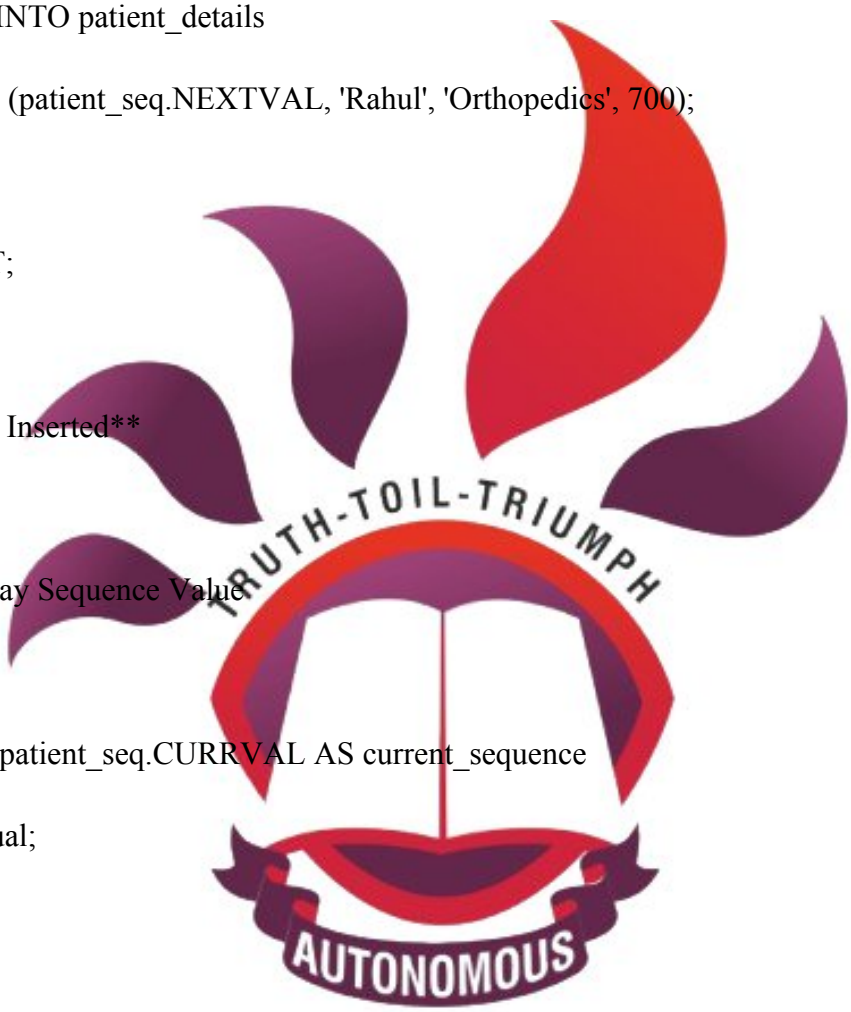
Insert Record Using Sequence

sql

```
INSERT INTO patient_details
VALUES (patient_seq.NEXTVAL, 'Rahul', 'Orthopedics', 700);

COMMIT;
```

Record Inserted



Display Sequence Value

sql

```
SELECT patient_seq.CURRVAL AS current_sequence
FROM dual;
```

Resultant Query

|

Current sequence

306

Insert Another Record Using Sequence

sql

```
INSERT INTO patient_details
VALUES (patient_seq.NEXTVAL, 'Anitha', 'Dermatology', 600);

COMMIT;
```

Record Inserted**

Display All Patient Records

sql

```
SELECT * FROM patient_details
ORDER BY patient_id;
```

Resultant Query

|

Patientid	patientname	department	consultationfee
301	Kavin	Cardiology	800
302	Meena	General	400
303	Arun	Neurology	1000
304	Divya	Cardiology	900
305	Priya	General	500
306	Rahul	Orthopedics	700
307	Anitha	Dermatology	600

Important Commands

Drop View

sql

DROP VIEW cardiology_patients;

```

**View Dropped**

Drop Index

sql

DROP INDEX idx_patient_name;`
```

Index Dropped**

Drop Sequence

```sql
DROP SEQUENCE patient_seq;
```

Sequence Dropped**

Result

Thus, the View, Index, and Sequence were created and successfully operated using SQL commands. The View was used to display selected records, the Index was used to improve data retrieval, and the Sequence was used to generate unique numeric values automatically.

EX. NO:09: Creation of Stored Procedures and Functions

DATE:25.7.2026

Aim

To create and execute Stored Procedures and Functions using PL/SQL.

Oracle SQL / SQL Command Line

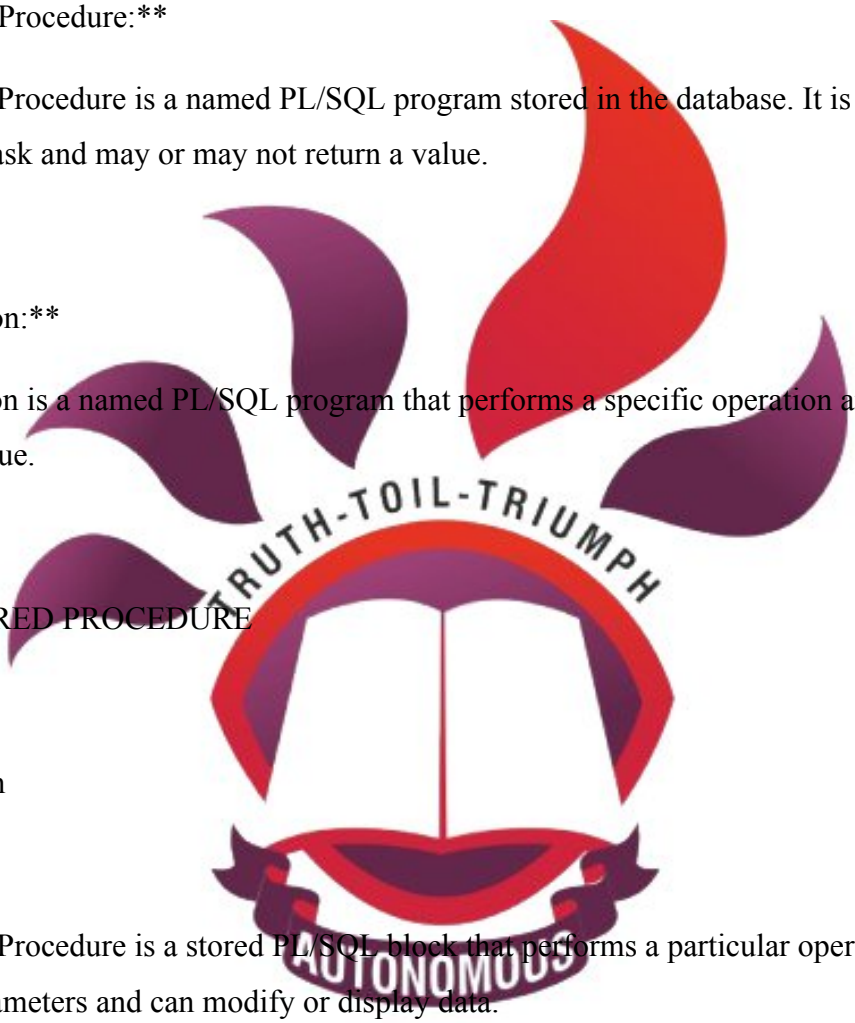
Description

Stored Procedure:

A Stored Procedure is a named PL/SQL program stored in the database. It is used to perform a specific task and may or may not return a value.

Function:

A Function is a named PL/SQL program that performs a specific operation and always returns a single value.



1. STORED PROCEDURE

Definition

A Stored Procedure is a stored PL/SQL block that performs a particular operation. It can accept input parameters and can modify or display data.

Syntax

`sql

CREATE OR REPLACE PROCEDURE procedure_name

(parameter_name IN datatype)

IS

BEGIN

    statements;

END;

/

Create Table

```sql
CREATE TABLE food_orders (
    order_id NUMBER PRIMARY KEY,
    customer_name VARCHAR2(50),
    food_item VARCHAR2(50),
    order_amount NUMBER
);
```

Table Created**

Insert Records

```sql

INSERT INTO food_orders VALUES (101, 'Arun', 'Burger', 250);

INSERT INTO food_orders VALUES (102, 'Meena', 'Pizza', 550);

INSERT INTO food_orders VALUES (103, 'Kavin', 'Biryani', 450);

INSERT INTO food_orders VALUES (104, 'Divya', 'Pasta', 350);

INSERT INTO food_orders VALUES (105, 'Priya', 'Fried Rice', 300);

COMMIT;

```

Records Inserted**

Display Food Order Records

sql

SELECT * FROM food_orders;

Resultant Query

|

order_id customername food_item order_amount	

101	Arun	Burger	250 |
102	Meena	Pizza	550 |
103	Kavin	Biryani	450 |
104	Divya	Pasta	350 |
105	Priya	Fried Rice	300 |

Create Stored Procedure

The following procedure displays the details of a food order based on the order ID.

```sql
CREATE OR REPLACE PROCEDURE display_order (
    p_order_id IN NUMBER
)
IS
    v_customer food_orders.customer_name%TYPE;
    v_food food_orders.food_item%TYPE;
    v_amount food_orders.order_amount%TYPE;
BEGIN
    SELECT customer_name, food_item, order_amount
    INTO v_customer, v_food, v_amount
    FROM food_orders
    WHERE order_id = p_order_id;
```

```
    DBMS_OUTPUT.PUT_LINE('Customer Name: ' || v_customer);
    DBMS_OUTPUT.PUT_LINE('Food Item: ' || v_food);
    DBMS_OUTPUT.PUT_LINE('Order Amount: ' || v_amount);
END;
/
```



Procedure Created**

Execute Stored Procedure

```
sql
SET SERVEROUTPUT ON;

EXEC display_order(103);
```

Resultant Query

Customer Name	Food Item	Order Amount

Kavin	Biryani	450

Output

```text
Customer Name: Kavin
Food Item: Biryani
Order Amount: 450
```

Explanation

The procedure `display_order` accepts the order ID as an input parameter and retrieves the corresponding customer name, food item, and order amount from the `food_orders` table.

2. FUNCTION

Definition

A Function is a stored PL/SQL program that performs a specific operation and returns a single value.

Syntax

```sql
CREATE OR REPLACE FUNCTION function_name
(parameter_name IN datatype)
RETURN datatype
IS
BEGIN
    statements;
    RETURN value;
END;
/
```

Create Function

The following function returns the order category based on the order amount.

```sql
CREATE OR REPLACE FUNCTION get_order_category (
    p_amount IN NUMBER
)
```

```
RETURN VARCHAR2
IS
    v_category VARCHAR2(15);
BEGIN
    IF p_amount >= 500 THEN
        v_category := 'Premium';
    ELSIF p_amount >= 400 THEN
        v_category := 'Large';
    ELSIF p_amount >= 300 THEN
        v_category := 'Medium';
    ELSE
        v_category := 'Small';
    END IF;

    RETURN v_category;
END;
/
```

Function Created

Execute Function

```sql
SELECT order_id, customer_name, order_amount,
    get_order_category(order_amount) AS category
FROM food_orders;
```

Resultant Query

orderid| customername	orderamount	category
101	Arun	250	Small
102	Meena	550	Premium
103	Kavi	450	Large
104	D	350	Medium
105	Priy	300	Medium

Explanation

The function `get_order_category` accepts the order amount as an input and returns the corresponding order category. The function is called inside the SELECT statement for each order.

Important Commands

Display Stored Procedure

sql

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'DISPLAY_ORDER';

```

### Resultant Query

| object_name | object_type |
| --- | --- |
| DISPLAY_ORDER | PROCEDURE |

|

Display Stored Function

```sql

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'GET_ORDER_CATEGORY';

```
```

Resultant Query

|

object_name	object_type |
GET_ORDER_CATEGORY	FUNCTION

Drop Procedure

```sql
DROP PROCEDURE display_order;
```

Procedure Dropped

Drop Function

```sql
DROP FUNCTION get_order_category;
```

Function Dropped**

Result

Thus, the Stored Procedure and Function were created and executed successfully using PL/SQL. The Stored Procedure was used to retrieve food order details, and the Function was used to determine the order category based on the order amount.

EX. NO: 10Creation of Triggers using Bank Accounts Table**

DATE:12.8.2026

Aim

To create and execute Triggers using PL/SQL to automatically perform an action when an INSERT, UPDATE, or DELETE operation occurs on a table.

Software Used

Oracle SQL / SQL Command Line

Description

A Trigger is a stored PL/SQL program that is automatically executed when a specified event occurs on a table or database.

Triggers can be used to automatically perform operations such as validating data, preventing invalid values, and maintaining audit records when a particular event takes place.

1. CREATE TABLE

```sql
CREATE TABLE bank_accounts (
    account_id NUMBER PRIMARY KEY,
    customer_name VARCHAR2(50),
    account_type VARCHAR2(20),
    balance NUMBER
);
```

Table Created

Insert Records

sql

```
INSERT INTO bank_accounts VALUES (301, 'Arun', 'Savings', 25000);
INSERT INTO bank_accounts VALUES (302, 'Meena', 'Current', 40000);
INSERT INTO bank_accounts VALUES (303, 'Kavin', 'Savings', 18000);
INSERT INTO bank_accounts VALUES (304, 'Divya', 'Salary', 30000);
INSERT INTO bank_accounts VALUES (305, 'Priya', 'Savings', 22000);
```

sql

COMMIT;

Records Inserted**

Display Bank Account Records

sql

SELECT * FROM bank_accounts;

Resultant Query

account_id	customer_name	account_type	balance
301	Arun	Savings	25000
302	Meena	Current	40000
303	Kavin	Savings	18000
304	Divya	Salary	30000
305	Priya	Savings	22000

---

2. BEFORE INSERT TRIGGER

Definition

A BEFORE INSERT Trigger is executed automatically before a new record is inserted into a table.

Syntax

```sql
CREATE OR REPLACE TRIGGER trigger_name
BEFORE INSERT ON table_name
FOR EACH ROW
BEGIN
   statements;
END;
/
```



Create Trigger

The following trigger checks whether the balance is negative before inserting a bank account record.

```sql
CREATE OR REPLACE TRIGGER check_account_balance
BEFORE INSERT ON bank_accounts
FOR EACH ROW
BEGIN
    IF :NEW.balance < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Balance cannot be negative');
    END IF;
END;
/
```

Trigger Created

Execute Trigger

```sql
INSERT INTO bank_accounts
VALUES (306, 'Rahul', 'Savings', -5000);
```

Output

```text
ORA-20001: Balance cannot be negative
```

The trigger prevents the insertion of an invalid negative balance.

---

3. BEFORE UPDATE TRIGGER

Definition

A BEFORE UPDATE Trigger is executed automatically before an existing record is updated.

Create Trigger

The following trigger prevents the account balance from being updated to a negative value.

```sql
```

```
CREATE OR REPLACE TRIGGER check_updated_balance

BEFORE UPDATE OF balance ON bank_accounts

FOR EACH ROW

BEGIN

   IF :NEW.balance < 0 THEN

      RAISE_APPLICATION_ERROR(-20002, 'Updated balance cannot be negative');

   END IF;

END;

/
```



Trigger Created

Execute Trigger

```sql
UPDATE bank_accounts

SET balance = -1000

WHERE account_id = 303;
```

Output

```text
ORA-20002: Updated balance cannot be negative
```

The trigger prevents the invalid balance from being updated.

---

4. AFTER INSERT TRIGGER

Definition

An AFTER INSERT Trigger is executed automatically after a new record is successfully inserted into a table.

Create Audit Table

```sql
CREATE TABLE account_audit (
```

```
    audit_id NUMBER,
    account_id NUMBER,
    action_type VARCHAR2(20)
);
```

Table Created

Create Sequence

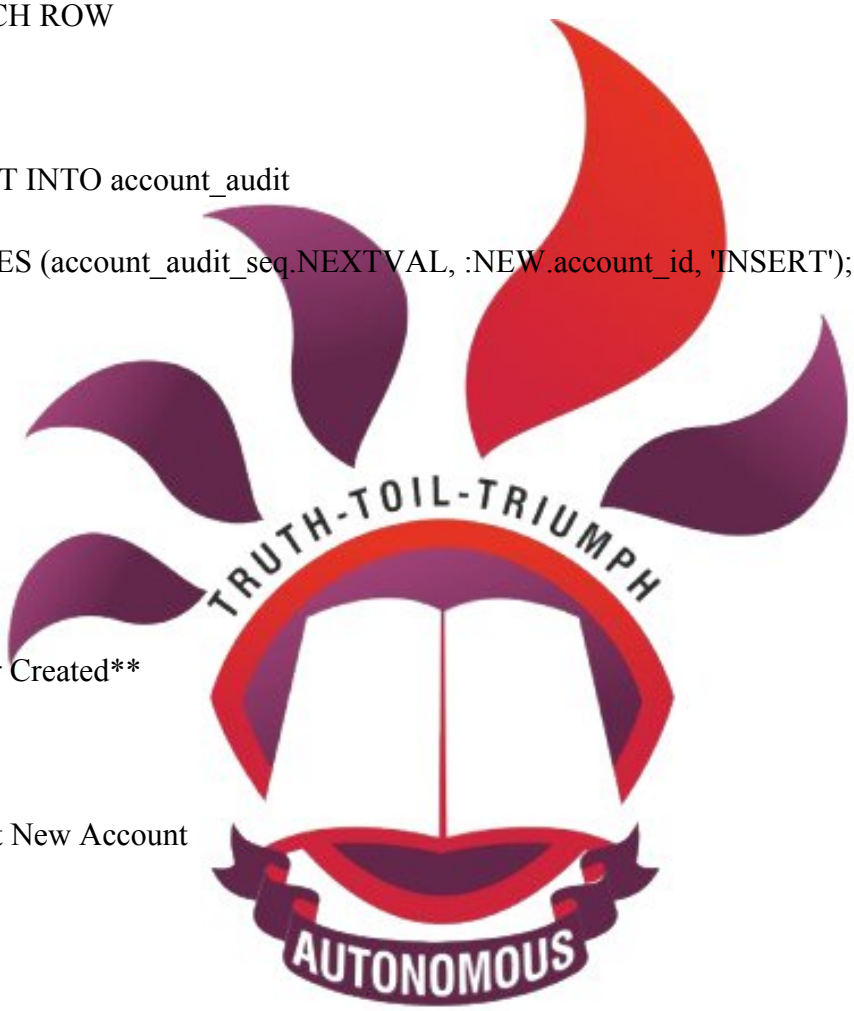
Before executing the above trigger, create the required sequence:

```sql
CREATE SEQUENCE account_audit_seq
START WITH 1
INCREMENT BY 1;
```

Sequence Created

Create Trigger

```sql
CREATE OR REPLACE TRIGGER account_insert_audit
AFTER INSERT ON bank_accounts
FOR EACH ROW
BEGIN
    INSERT INTO account_audit
    VALUES (account_audit_seq.NEXTVAL, :NEW.account_id, 'INSERT');
END;
/
```



Trigger Created

Insert New Account

```sql
INSERT INTO bank_accounts
VALUES (306, 'Rahul', 'Savings', 15000);

COMMIT;
```

```

**Record Inserted**

### Display Audit Records

```sql
SELECT * FROM account_audit;
```

Resultant Query

audit_id	account_id	action_type
1	306	INSERT

---

5. DISPLAY FINAL BANK ACCOUNT RECORDS

```sql
SELECT * FROM bank_accounts
```
```

ORDER BY account_id;

```

### Resultant Query

|

| account_id customer_name account_type \| balance \| |
|---|
| 301 Arun \| Savings \| 25000 \| |
| 302 \| Meena \| Current \| 40000 \| |
| 303 \| Kavin \| Savings \| 18000 \| |
| 304 \| Divya \| Salary \| 30000 \| |
| 305 \| Priya \| Savings \| 22000 \| |
| 306 \| Rahul \| Savings \| 15000 \| |

Important Commands

Display Triggers

```sql

SELECT trigger_name, status

FROM user_triggers;

```

Resultant Query
```

| trigger_name | status |

| ---------- | ------- |

| CHECK_ACCOUNT_BALANCE | ENABLED |

| CHECK_UPDATED_BALANCE | ENABLED |

| ACCOUNT_INSERT_AUDIT | ENABLED |

Drop Trigger

```sql
DROP TRIGGER check_account_balance;
```

Trigger Dropped

```sql
DROP TRIGGER check_updated_balance;
```

Trigger Dropped

```sql

DROP TRIGGER account_insert_audit;

```

Trigger Dropped

Result

Thus, the Triggers were created and executed successfully using PL/SQL. The triggers automatically validated account balance before insertion and updation and recorded new account insertion details in the audit table.

EX. NO: 11 Creation of Cursor Implementation using Bus Reservation Table

DATE:19.8.2026

Aim

To implement and execute a Cursor using PL/SQL to retrieve and process records from a database table.

Software Used

Oracle SQL / SQL Command Line

Description

A Cursor is a pointer used to process the rows returned by a SQL query one row at a time.

Cursors are useful when each record needs to be processed individually.

There are two types of cursors:

1. Implicit Cursor
2. Explicit Cursor

In this practical, an Explicit Cursor is implemented.

1. CREATE TABLE

```sql
CREATE TABLE bus_reservation (
    reservation_id NUMBER PRIMARY KEY,
    passenger_name VARCHAR2(50),
```

```
    bus_route VARCHAR2(50),
    ticket_fare NUMBER
);
```

Table Created

Insert Records

```sql
INSERT INTO bus_reservation VALUES (401, 'Arun', 'Chennai to Madurai', 650);
INSERT INTO bus_reservation VALUES (402, 'Meena', 'Madurai to Coimbatore', 550);
INSERT INTO bus_reservation VALUES (403, 'Kavin', 'Chennai to Pondicherry', 300);
INSERT INTO bus_reservation VALUES (404, 'Divya', 'Trichy to Chennai', 700);
INSERT INTO bus_reservation VALUES (405, 'Priya', 'Salem to Madurai', 450);
```

```sql
COMMIT;
```

Records Inserted

Display Reservation Records

sql

SELECT * FROM bus_reservation;

```

Resultant Query

|

| reservation_id | passenger_name | bus_route | ticket_fare |
|---|---|---|---|
| 401 | Arun | Chennai to Madurai | 650 |
| 402 | Meena | Madurai to Coimbatore | 550 |
| 403 | Kavin | Chennai to Pondicherry | 300 |
| 404 | Divya | Trichy to Chennai | 700 |
| 405 | Priya | Salem to Madurai | 450 |

|

---

## 2. EXPLICIT CURSOR
```

Definition

An Explicit Cursor is a cursor that is declared and controlled by the programmer. It is used to retrieve multiple rows from a table and process them one by one.

Syntax

```sql
DECLARE
   CURSOR cursor_name IS
      SELECT statement;
BEGIN
   OPEN cursor_name;

   LOOP
      FETCH cursor_name INTO variables;

      EXIT WHEN cursor_name%NOTFOUND;

      statements;
   END LOOP;
```

```
    CLOSE cursor_name;
END;
/
```

Create Explicit Cursor

The following cursor retrieves each passenger name and ticket fare from the `bus_reservation` table.

```sql
SET SERVEROUTPUT ON;

DECLARE
    CURSOR bus_cursor IS
        SELECT passenger_name, ticket_fare
        FROM bus_reservation;

    v_passenger_name bus_reservation.passenger_name%TYPE;
    v_ticket_fare bus_reservation.ticket_fare%TYPE;
```

```
BEGIN
    OPEN bus_cursor;

    LOOP
        FETCH bus_cursor INTO v_passenger_name, v_ticket_fare;

        EXIT WHEN bus_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Passenger: ' || v_passenger_name ||
            ' | Fare: ₹' || v_ticket_fare
        );
    END LOOP;

    CLOSE bus_cursor;
END;
/
```

Output

```text
Passenger: Arun | Fare: ₹650
Passenger: Meena | Fare: ₹550
Passenger: Kavin | Fare: ₹300
Passenger: Divya | Fare: ₹700
Passenger: Priya | Fare: ₹450
```

---

3. CURSOR WITH CONDITION

The following cursor displays only the passengers whose ticket fare is greater than ₹500.

```sql
SET SERVEROUTPUT ON;

DECLARE
    CURSOR bus_cursor IS
        SELECT reservation_id, passenger_name, ticket_fare
        FROM bus_reservation
```

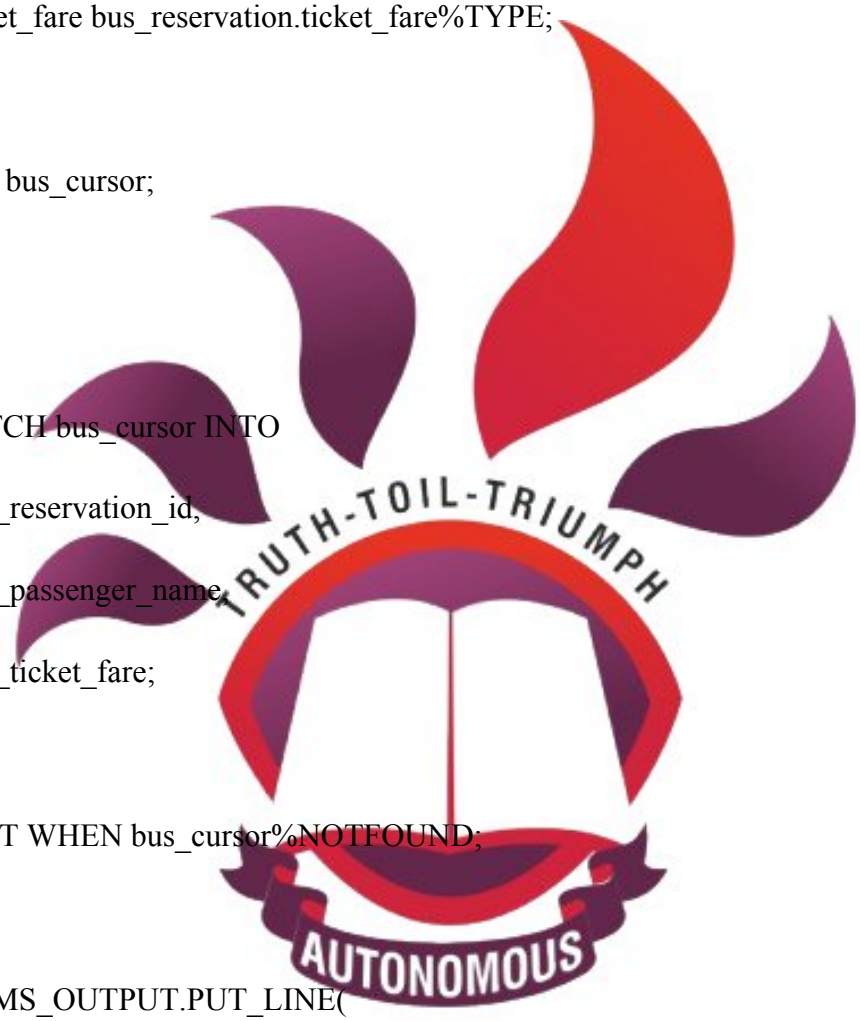
```
    WHERE ticket_fare > 500;

    v_reservation_id bus_reservation.reservation_id%TYPE;
    v_passenger_name bus_reservation.passenger_name%TYPE;
    v_ticket_fare bus_reservation.ticket_fare%TYPE;
BEGIN
    OPEN bus_cursor;

    LOOP
        FETCH bus_cursor INTO
            v_reservation_id,
            v_passenger_name,
            v_ticket_fare;

        EXIT WHEN bus_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Reservation ID: ' || v_reservation_id ||
            ' | Passenger: ' || v_passenger_name ||
            ' | Fare: ₹' || v_ticket_fare
        );
```



```
    END LOOP;

    CLOSE bus_cursor;
END;
/
```

Resultant Query

reservation_id	passenger_name	ticket_fare
401	Arun	650
402	Meena	550
404	Divya	700

---

4. CURSOR ATTRIBUTES

Cursor attributes provide information about the current status of a cursor.

Important Cursor Attributes

| Cursor Attribute | Description

| `%FOUND` | Returns TRUE if the last fetch returned a row |

| `%NOTFOUND` | Returns TRUE if the last fetch did not return a row |

| `%ROWCOUNT` | Returns the number of rows processed |

| `%ISOPEN` | Returns TRUE if the cursor is currently open |

---

5. CURSOR USING %ROWCOUNT

The following program displays the total number of reservations processed by the cursor.

```sql
SET SERVEROUTPUT ON;

DECLARE
   CURSOR bus_cursor IS
      SELECT passenger_name, ticket_fare
      FROM bus_reservation;

   v_passenger_name bus_reservation.passenger_name%TYPE;
   v_ticket_fare bus_reservation.ticket_fare%TYPE;
```

```
BEGIN
    OPEN bus_cursor;

    LOOP
        FETCH bus_cursor INTO v_passenger_name, v_ticket_fare;

        EXIT WHEN bus_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Passenger: ' || v_passenger_name ||
            ' | Fare: ₹' || v_ticket_fare
        );
    END LOOP;

    DBMS_OUTPUT.PUT_LINE(
        'Total Reservations Processed: ' || bus_cursor%ROWCOUNT
    );

    CLOSE bus_cursor;
END;
/
```



```

Output

```text
Passenger: Arun | Fare: ₹650
Passenger: Meena | Fare: ₹550
Passenger: Kavin | Fare: ₹300
Passenger: Divya | Fare: ₹700
Passenger: Priya | Fare: ₹450
Total Reservations Processed: 5
```

Result

Thus, the Cursor was successfully implemented using PL/SQL. The explicit cursor was used to retrieve and process bus reservation records one by one, apply conditions, and count the number of processed records.

EX. NO: 12 Creation of PL/SQL Exception Handling using Restaurant Orders Table

DATE:28.8.2026
```

Aim

To implement Exception Handling in PL/SQL and handle runtime errors using predefined and user-defined exceptions.

Software Used

Oracle SQL / SQL Command Line

Description

An Exception is an error that occurs during the execution of a PL/SQL program.

Exception Handling is used to handle runtime errors without terminating the program abnormally.

The basic structure of exception handling is:

```sql
DECLARE
    declarations;
```

```
BEGIN
    statements;
EXCEPTION
    WHEN exception_name THEN
        statements;
END;
/
```

Types of Exceptions

Type	Description
Predefined Exception	Built-in exceptions provided by Oracle
User-Defined Exception	Exception created by the programmer

---

1. CREATE TABLE

```sql
CREATE TABLE restaurant_orders (
```

```
    order_id NUMBER PRIMARY KEY,
    customer_name VARCHAR2(50),
    food_item VARCHAR2(50),
    amount NUMBER
);
```

Table Created**

Insert Records

```sql
INSERT INTO restaurant_orders VALUES (501, 'Arun', 'Biryani', 450);
INSERT INTO restaurant_orders VALUES (502, 'Meena', 'Pizza', 850);
INSERT INTO restaurant_orders VALUES (503, 'Kavin', 'Burger', 350);
INSERT INTO restaurant_orders VALUES (504, 'Divya', 'Fried Rice', 550);
INSERT INTO restaurant_orders VALUES (505, 'Priya', 'Pasta', 650);
```

```sql
COMMIT;
```

Records Inserted

Display Order Records

sql

SELECT * FROM restaurant_orders;

```

Resultant Query

|

| order_id | customer_name | food_item | amount |
|---|---|---|---|
| 501 | Arun | Biryani | 450 |
| 502 | Meena | Pizza | 850 |
| 503 | Kavin | Burger | 350 |
| 504 | Divya | Fried Rice | 550 |
| 505 | Priya | Pasta | 650 |

2. NO_DATA_FOUND EXCEPTION

Definition
```

`NO_DATA_FOUND` occurs when a `SELECT INTO` statement does not return any record.

Program

```sql
SET SERVEROUTPUT ON;

DECLARE
    v_customer restaurant_orders.customer_name%TYPE;
    v_amount restaurant_orders.amount%TYPE;
BEGIN
    SELECT customer_name, amount
    INTO v_customer, v_amount
    FROM restaurant_orders
    WHERE order_id = 599;

    DBMS_OUTPUT.PUT_LINE('Customer: ' || v_customer);
    DBMS_OUTPUT.PUT_LINE('Amount: ₹' || v_amount);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE(
            'Exception: Order not found.'
        );
END;
/
```

Output

```
text
Exception: Order not found.
```

Exception Handled Successfully

---

3. TOO_MANY_ROWS EXCEPTION

Definition

`TOO_MANY_ROWS` occurs when a `SELECT INTO` statement returns more than one record.

Program

```sql
SET SERVEROUTPUT ON;

DECLARE
    v_food restaurant_orders.food_item%TYPE;
BEGIN
    SELECT food_item
    INTO v_food
    FROM restaurant_orders
    WHERE amount > 400;

    DBMS_OUTPUT.PUT_LINE('Food Item: ' || v_food);

EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: More than one order found.'
        );
```

END;

/

```

Output

```text

Exception: More than one order found.

```

Exception Handled Successfully**



4. ZERO_DIVIDE EXCEPTION

Definition

`ZERO_DIVIDE` occurs when a number is divided by zero.

Program
```

```sql
SET SERVEROUTPUT ON;

DECLARE
    v_amount NUMBER := 850;
    v_persons NUMBER := 0;
    v_share NUMBER;
BEGIN
    v_share := v_amount / v_persons;

    DBMS_OUTPUT.PUT_LINE(
        'Amount Per Person: ₹' || v_share
    );

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: Number of persons cannot be zero.'
        );
END;
/
```

Output

text

Exception: Number of persons cannot be zero.

Exception Handled Successfully

5. USER-DEFINED EXCEPTION

Definition

A User-Defined Exception is an exception created by the programmer to handle a specific business condition.

In this example, an order amount greater than ₹800 is treated as a high-value order.

Program

sql

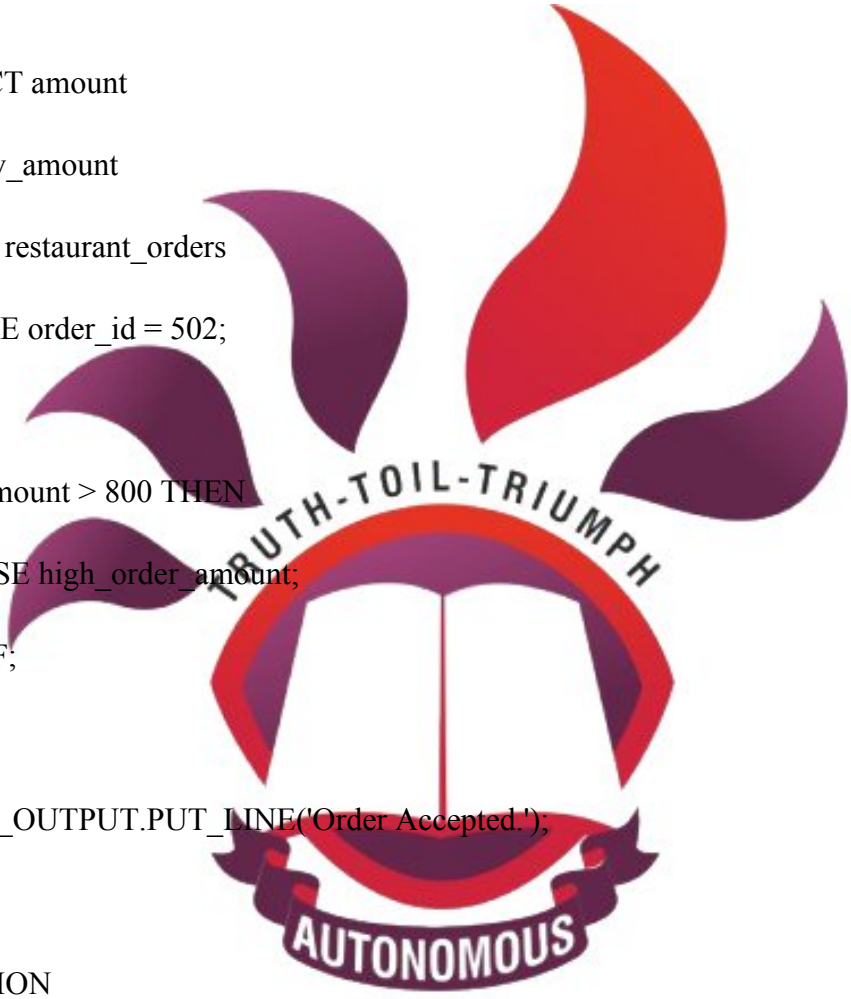
SET SERVEROUTPUT ON;

```
DECLARE
    v_amount restaurant_orders.amount%TYPE;
    high_order_amount EXCEPTION;
BEGIN
    SELECT amount
    INTO v_amount
    FROM restaurant_orders
    WHERE order_id = 502;

    IF v_amount > 800 THEN
        RAISE high_order_amount;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Order Accepted.');

EXCEPTION
    WHEN high_order_amount THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: High-value order detected.'
        );
```



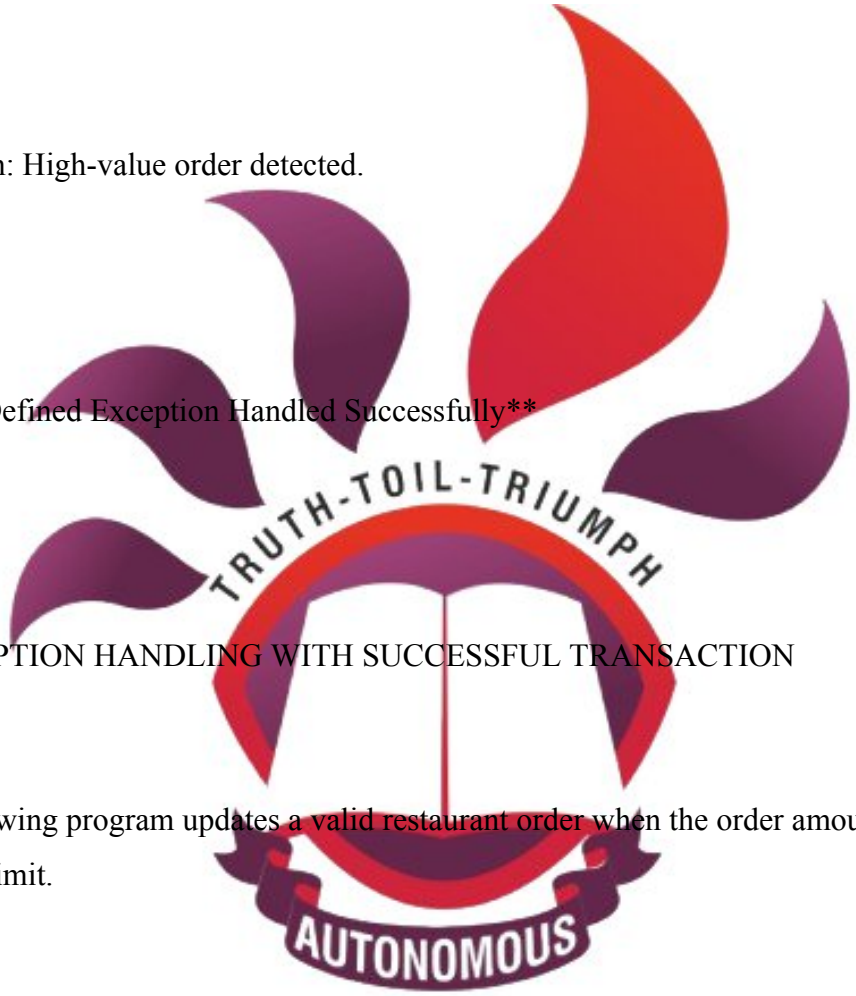
END;

/

Output

```text
Exception: High-value order detected.
```

User-Defined Exception Handled Successfully



6. EXCEPTION HANDLING WITH SUCCESSFUL TRANSACTION

The following program updates a valid restaurant order when the order amount is within the allowed limit.

sql

SET SERVEROUTPUT ON;

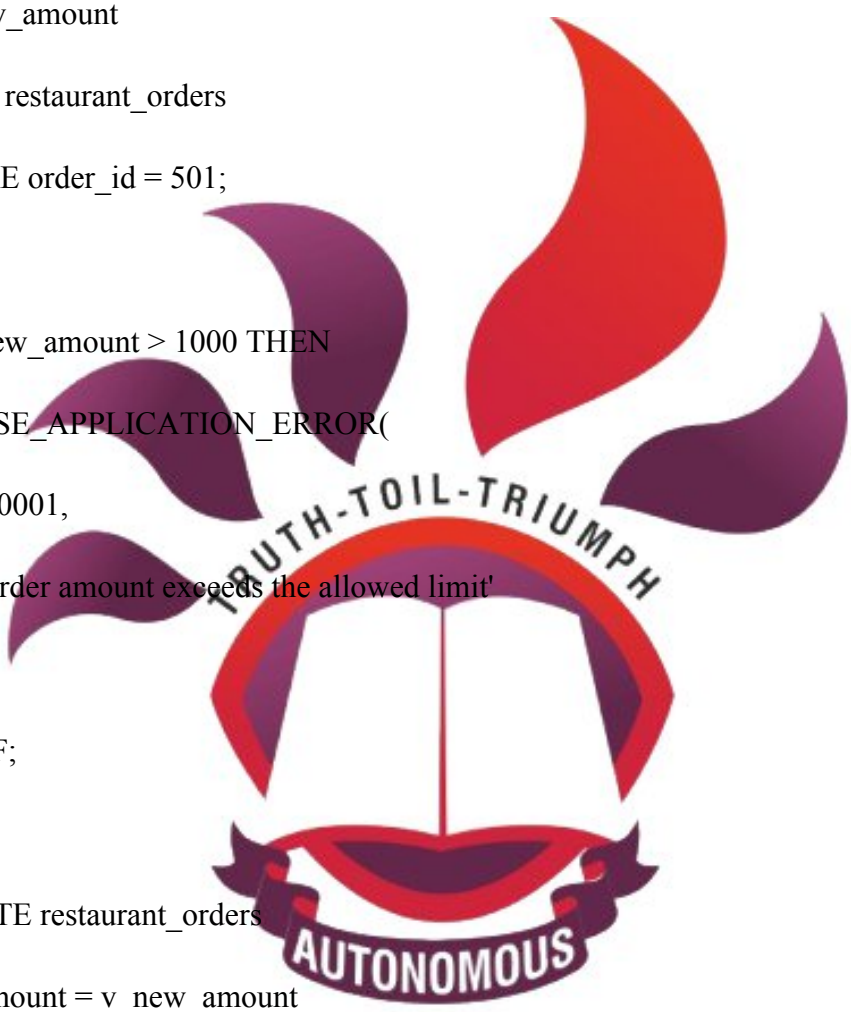
DECLARE

```
    v_amount restaurant_orders.amount%TYPE;
    v_new_amount NUMBER := 750;
BEGIN
    SELECT amount
    INTO v_amount
    FROM restaurant_orders
    WHERE order_id = 501;

    IF v_new_amount > 1000 THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Order amount exceeds the allowed limit'
        );
    END IF;

    UPDATE restaurant_orders
    SET amount = v_new_amount
    WHERE order_id = 501;

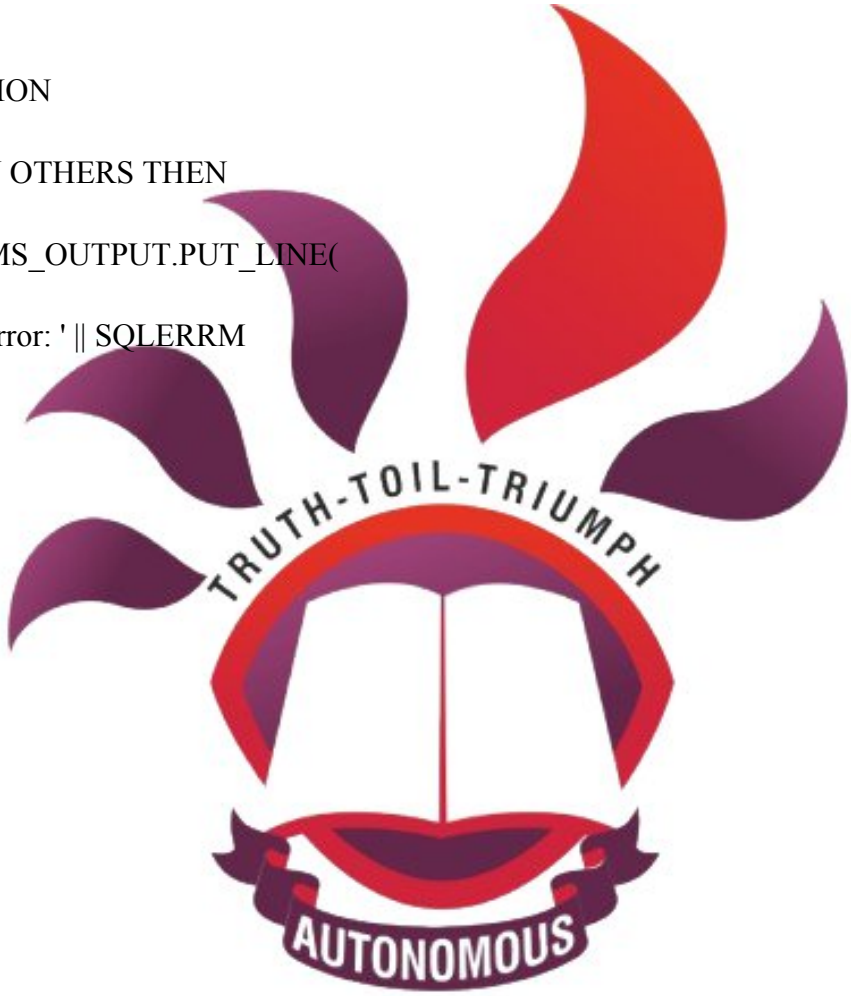
    COMMIT;
```



```
    DBMS_OUTPUT.PUT_LINE('Order Amount Updated Successfully.');
    DBMS_OUTPUT.PUT_LINE(
        'Updated Amount: ₹' || v_new_amount
    );

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/
```

Output




```
text
Order Amount Updated Successfully.
Updated Amount: ₹750
```

Display Updated Order

```sql
SELECT * FROM restaurant_orders
WHERE order_id = 501;
```

Resultant Query

order_id	customer_name	food_item	amount
501	Arun	Biryani	750

Important Predefined Exceptions

Exception	Description
NO_DATA_FOUND	No record is returned
TOO_MANY_ROWS	More than one record is returned
ZERO_DIVIDE	Division by zero occurs
DUP_VAL_ON_INDEX	Duplicate value is inserted into a unique/primary key column
VALUE_ERROR	Invalid value or conversion error occurs

Result

Thus, PL/SQL Exception Handling was successfully implemented. The `NO_DATA_FOUND`, `TOO_MANY_ROWS`, `ZERO_DIVIDE`, and user-defined exceptions were handled successfully using appropriate exception-handling mechanisms.

EX. NO: 13 Creation of PL/SQL Stored Procedures using Employee Attendance Table

DATE:2.9.2026

Aim

To create and execute PL/SQL Stored Procedures to retrieve, update, and process employee attendance records using input and output parameters.

Software Used

Oracle SQL / SQL Command Line

Description

A Stored Procedure is a named PL/SQL program stored in the database. It can be executed whenever required to perform a specific task.

Stored Procedures are useful for performing operations such as displaying records, updating values, and processing data using parameters.

There are three commonly used parameter modes:

| Parameter | Description

| IN | Used to pass a value to the procedure |

| OUT | Used to return a value from the procedure |

| IN OUT | Used to pass and return a value |

---

1. CREATE TABLE

```sql
CREATE TABLE employee_attendance (
    employee_id NUMBER PRIMARY KEY,
    employee_name VARCHAR2(50),
    department VARCHAR2(30),
```

attendance_days NUMBER

);

```

Table Created**

Insert Records

sql

INSERT INTO employee_attendance VALUES (601, 'Arun', 'IT', 24);

INSERT INTO employee_attendance VALUES (602, 'Meena', 'HR', 22);

INSERT INTO employee_attendance VALUES (603, 'Kavin', 'Finance', 26);

INSERT INTO employee_attendance VALUES (604, 'Divya', 'IT', 20);

INSERT INTO employee_attendance VALUES (605, 'Priya', 'Sales', 23);

sql

COMMIT;

**Records Inserted**
```

Display Attendance Records

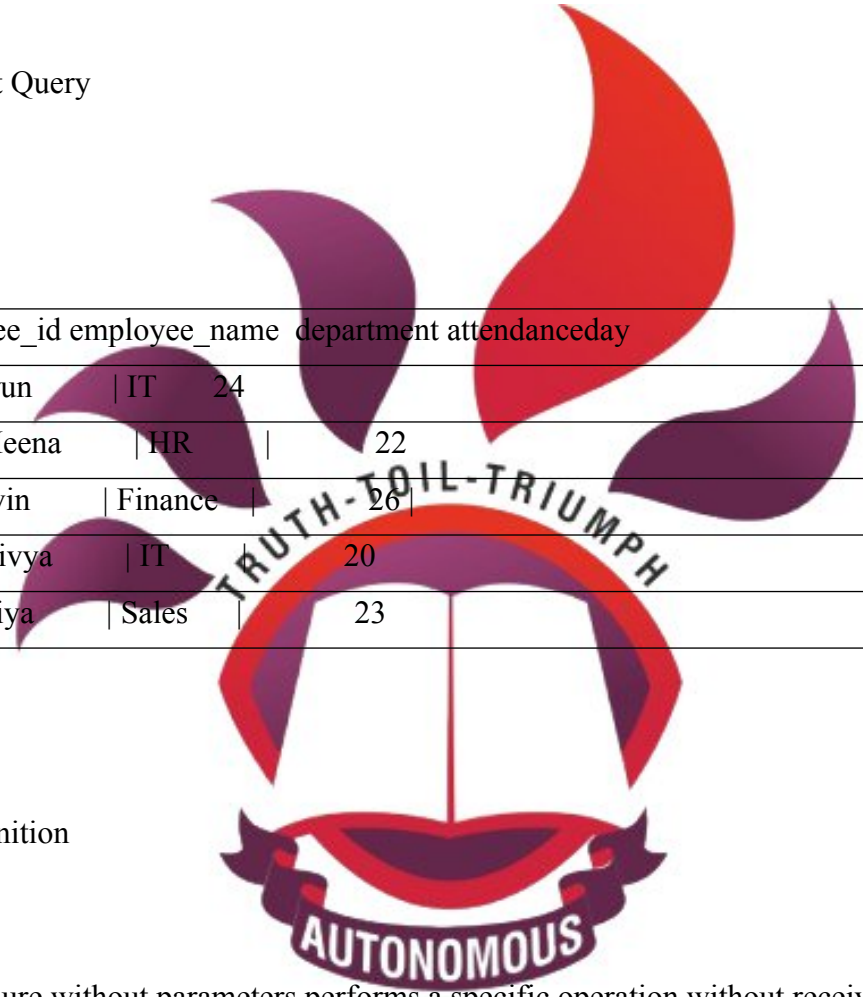
sql

SELECT * FROM employee_attendance;

Resultant Query

|

employee_id employee_name	department attendanceday
601 | Arun	| IT	24
602 | Meena	| HR	| 22
603 Kavin	| Finance	| 26 |
604 | Divya	| IT	| 20
605 | Priya	| Sales	| 23



Definition

A Procedure without parameters performs a specific operation without receiving any input value.

Syntax

```sql
```

```
CREATE OR REPLACE PROCEDURE procedure_name
IS
BEGIN
   statements;
END;
/
```

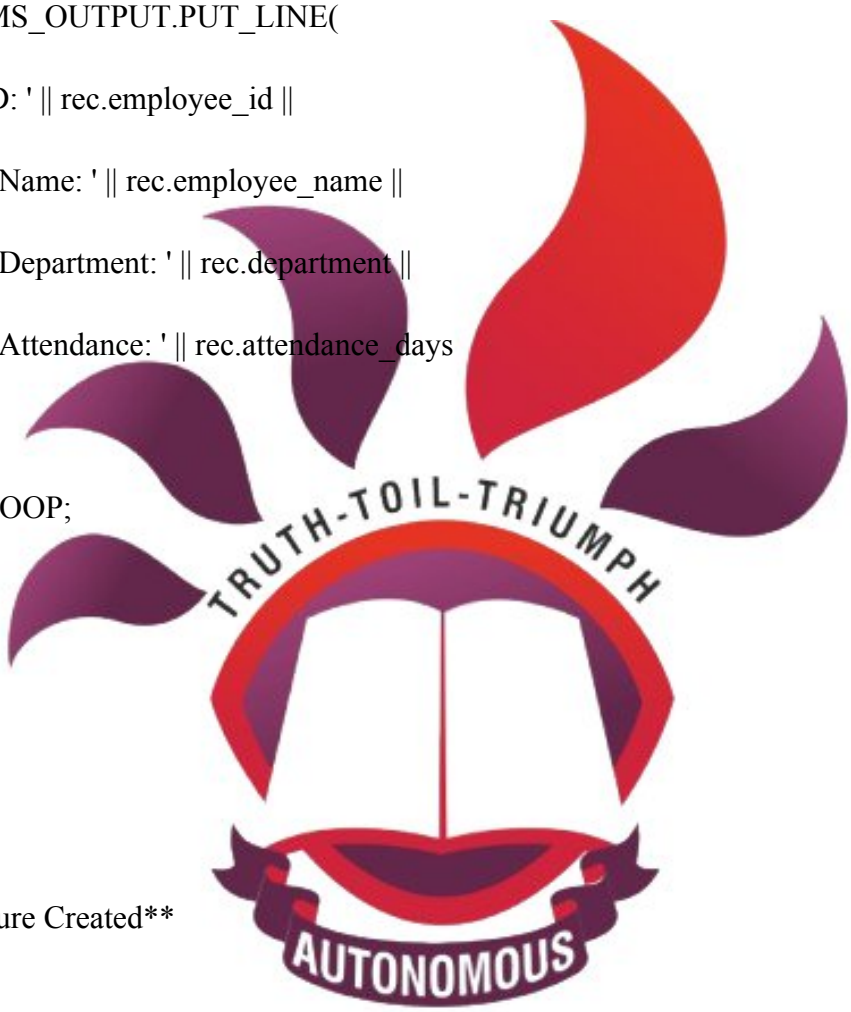
Create Procedure

The following procedure displays all employee attendance records.

```sql
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE display_attendance
IS
BEGIN
   FOR rec IN (
      SELECT employee_id, employee_name,
          department, attendance_days
```

```
        FROM employee_attendance
        ORDER BY employee_id
    )
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            'ID: ' || rec.employee_id ||
            ' | Name: ' || rec.employee_name ||
            ' | Department: ' || rec.department ||
            ' | Attendance: ' || rec.attendance_days
        );
    END LOOP;
END;
/
```



Procedure Created

Execute Procedure

```sql
EXEC display_attendance;
```

```

### Output

text

| | | | |
|---|---|---|---|
| 601 | Arun | Department: IT | Attendance: 24 |
| 602 | Meena | Department: HR | Attendance: 22 |
| 603 | Kavin | Department: Finance | Attendance: 26 |
| 604 | Divya | Department: IT | Attendance: 20 |
| 605 | Priya | Department: Sales | Attendance: 23 |

3. PROCEDURE WITH IN PARAMETER

Definition

An IN parameter is used to pass a value to a procedure.

## Create Procedure

The following procedure updates the attendance days of a particular employee.
```

```
sql

CREATE OR REPLACE PROCEDURE update_attendance (

    p_employee_id IN NUMBER,

    p_days IN NUMBER

)

IS

BEGIN

    UPDATE employee_attendance

    SET attendance_days = p_days

    WHERE employee_id = p_employee_id;


    COMMIT;


    DBMS_OUTPUT.PUT_LINE(

        'Attendance Updated Successfully.'

    );

END;

/
```



Procedure Created

Execute Procedure

sql

```
EXEC update_attendance(604, 25);
```

Output

text

Attendance Updated Successfully.

Display Updated Employee

sql

```
SELECT * FROM employee_attendance
WHERE employee_id = 604;
```

Resultant Query

Employee id	employee name	department	attendance days
604	Divya	IT	25

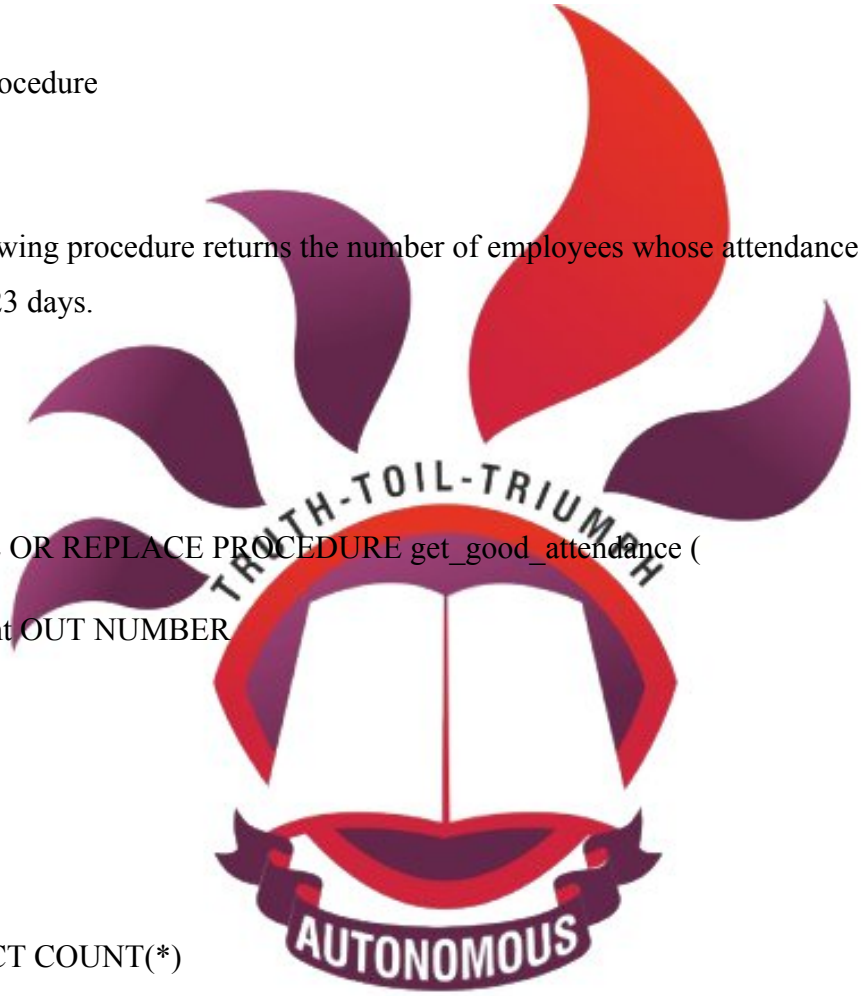
4. PROCEDURE WITH OUT PARAMETER

Definition

An OUT parameter is used to return a value from a procedure to the calling program.

Create Procedure

The following procedure returns the number of employees whose attendance is greater than or equal to 23 days.



sql

```
CREATE OR REPLACE PROCEDURE get_good_attendance (
    p_count OUT NUMBER
)
IS
BEGIN
    SELECT COUNT(*)
    INTO p_count
    FROM employee_attendance
    WHERE attendance_days >= 23;
END;
```

/

```

Procedure Created

Execute Procedure

sql
SET SERVEROUTPUT ON;

DECLARE
    v_count NUMBER;
BEGIN
    get_good_attendance(v_count);

    DBMS_OUTPUT.PUT_LINE(
        'Employees with Good Attendance: ' || v_count
    );
END;
/
```

Output

text

Employees with Good Attendance: 4

5. DISPLAY FINAL ATTENDANCE RECORDS

sql

```
SELECT * FROM employee_attendance
ORDER BY employee_id;
```

Resultant Query

Employee id	employee name	department	attendance_days
601	Arun	IT	24
602	Meena	HR	22
603	Kavin	Finance	26
604	Divya	IT	25
605	Priya	Sales	23

Important Commands

Display Stored Procedures

sql

SELECT object_name, object_type

FROM user_objects

WHERE object_type = 'PROCEDURE';

```

Resultant Query

| OBJECT | NAME | OBJECT_TYPE |
|---|---|---|
| DISPLAY | ATTENDANCE | PROCEDURE |
| UPDATE | ATTENDANCE | PROCEDURE |
| GETGOOD | ATTENDANCE | PROCEDURE |

|

Drop Procedures

sql

DROP PROCEDURE display_attendance;

Procedure Dropped

sql
```

DROP PROCEDURE update_attendance;

Procedure Dropped

sql

DROP PROCEDURE get_good_attendance;

Procedure Dropped

Result

Thus, the PL/SQL Stored Procedures were created and executed successfully. The procedures were used to

CODECHEF

1. **All Products**

Task

Write a query to output Products table.

Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

PROGRAM:

SELECT *

FROM Products;

OUTPUT:

AUTONOMOUS

Your Output

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture

✔ Hooray, you did it!

2. **High Price of Products**

Task

Write a query to find all product_name and category that have a price greater than 100.00 from the **Products** table.

Table name: **Products**

CODECHEF

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

PROGRAM:

```
SELECT product_name, category
FROM Products
WHERE price > 100;
```

OUTPUT:

Your Output

```
┌──────────────┬─────────────┐
│ product_name │  category   │
├──────────────┼─────────────┤
│ Laptop       │ Electronics │
│ Desk Chair   │ Furniture   │
│ Smartphone   │ Electronics │
│ Standing Desk │ Furniture  │
│ Tablet       │ Electronics │
└──────────────┴─────────────┘
```

✓ Excellent work!

3. **Average Salary**

Task

- Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

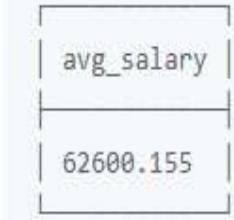
employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

PROGRAM:

```
SELECT AVG(salary) AS avg_salary
FROM Works;
```

OUTPUT:

Your Output



```
┌──────────────┐
│ avg_salary   │
├──────────────┤
│ 62600.155    │
└──────────────┘
```

✓ Well done, it's correct!

4. **Locate People**

Task

- Write a query to retrieve the department_name and location of departments located in a city that starts with 'S'.

Table name: **departments**

department_id	department_name	location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Boston

CODECHEF

PROGRAM:

```
SELECT department_name, location
FROM departments
WHERE location LIKE 'S%';
```

OUTPUT:

Your Output

department_name	location
Research and Development	San Francisco
IT Support	Seattle
Product Management	San Francisco

✓ Excellent work!

5. **Distinct Companies**

Task

Write a query to select all the distinct companies (company_name) in the Works table.
Table name: **Works**

CODECHEF

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

PROGRAM:

SELECT DISTINCT company_name

FROM Works;

OUTPUT:

Your Output

```
┌──────────────┐
│ company_name │
├──────────────┤
│ TechCorp     │
│ InnovateLtd  │
│ HealthPlus   │
│ EduWorks     │
│ GreenTech    │
└──────────────┘
```

✓ Great job, keep it up!

6. Fiction Collection Size

Task

Write a query to find the total count of books whose genre is Fiction.
Note: Output column name should be fiction_count.

Table name: Books

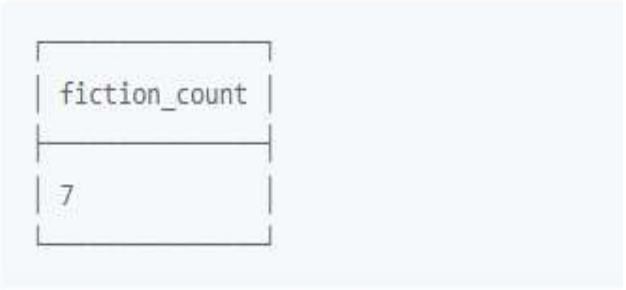
id	title	author	genre	price	published_yea
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869

PROGRAM:

```
SELECT COUNT(*) AS fiction_count
FROM Books
WHERE genre = 'Fiction';
```

OUTPUT:

Your Output



fiction_count
7

✓ Perfect answer!

7. List of Movies with Ratings

Task

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table: Cinema

List of Movies with Ratings

Task

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table: Cinema

PROGRAM:

```
SELECT Movie_name
FROM Cinema
WHERE Rating > 7
  AND Rating < 9;
```

OUTPUT:

Your Output

```
┌──────────────┐
| Movie_name   |
├──────────────┤
| War          |
| Science      |
| Ice Song     |
| The Escape   |
| Solar Flare  |
| Lost Dreams  |
| Galaxy War   |
```

✔ Hooray, you did it!

8. Handling NULL Values

Task

Write a query to retrieve book_id, title, author and published_year of the books which have NULL rating for their books.

Table name: Library

book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

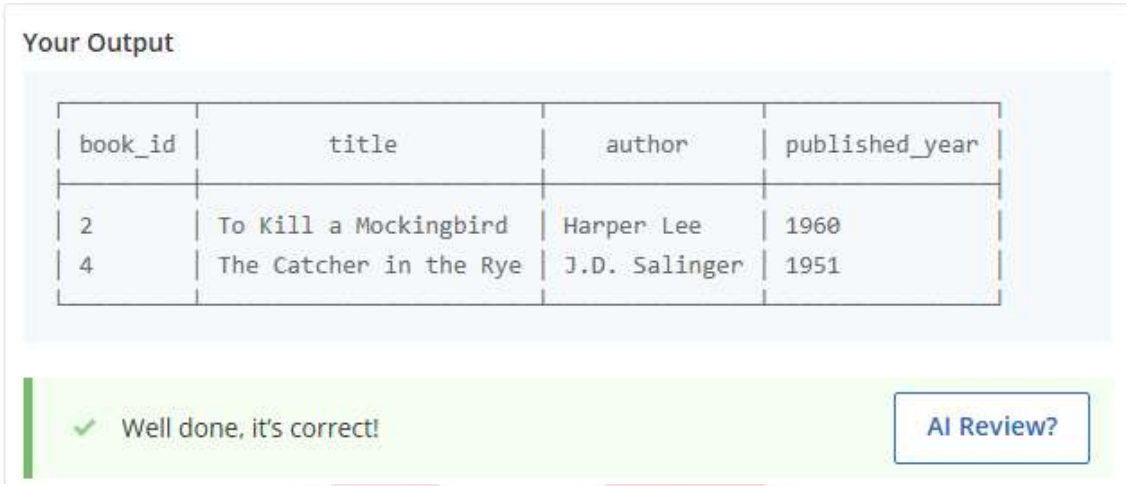
PROGRAM:

SELECT book_id, title, author, published_year

FROM Library

WHERE rating IS NULL;

OUTPUT:



9. **Salary of Employees**

Task

Create a query to retrieve the employee_name, company, and salary for employees in the full-time category, ordered by salary in **descending** order

Table name: Employees

Salary of Employees

Task

Create a query to retrieve the employee_name, company, and salary for employees in the full-time category, ordered by salary in **descending** order

Table name: Employees

PROGRAM:

SELECT employee_name, company, salary

FROM Employees

WHERE category = 'Full-Time'

ORDER BY salary DESC;

OUTPUT:

Your Output

employee_name	company	salary
Bob Brown	FinServ	90000
David Green	TechCorp	85000
John Smith	TechCorp	80000
Grace Grey	TechCorp	70000
Frank Black	HealthPlus	60000

✓ Perfect answer!

10. **Department of Each Employee**

Task

Write a query to group the employees by their department and display the total number of employees (as total_employees) in each department.

Table name: Employees

AUTONOMOUS

CODECHEF

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

PROGRAM:

```
SELECT department, COUNT(*) AS total_employees
FROM Employees
GROUP BY department;
```

OUTPUT:



11. **Article views**

Task

Write a query to retrieve the author_id, author_name, and publication_name for authors whose articles got zero views. The result should be sorted by author_id in ascending order.

Return the result table sorted by id in **ascending order**.

Table name: Views

CODECHEF

article_id	author_id	author_name	viewer_id	view_count	publication_nai
101	1	John Doe	1	3	The Daily Ti mes
102	2	Jane Smith	2	1	Global New s
103	3	Emily Clark	4	0	Tech Month ly
104	4	Mark Lewis	4	2	Health Dige st
105	5	Sara White	3	0	The Busines s Journal
106	1	John Doe	2	0	The Daily Ti mes
107	2	Jane Smith	2	2	Global New s

PROGRAM:

```
SELECT author_id, author_name, publication_name
FROM Views
WHERE view_count = 0
ORDER BY author_id ASC;
```

OUTPUT:

Your Output

author_id	author_name	publication_name
1	John Doe	The Daily Times
3	Emily Clark	Tech Monthly
5	Sara White	The Business Journal

✓ You got it right!

12. **Player Performance Insights**

Task

Write a query to find the names of the **top 3 distinct players** by highest score who have **won** matches, including their **scores**.

Table 1: **Players**

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: **Matches**

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

PROGRAM:

```
SELECT DISTINCT p.player_name, p.score
FROM Players p
JOIN Matches m
ON p.player_name = m.winner
ORDER BY p.score DESC
LIMIT 3;
```

OUTPUT:

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

✓ You got it right!

13. **Player Details**

Task

Write a query to retrieve the details of the **last five matches** played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named **Players** and **Matches**

CODECHEF

Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	jack	1400	10

Matches

match_id	player_1	player_2	winner	match_date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

PROGRAM:

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score
FROM Matches m
JOIN Players p ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

OUTPUT:

CODECHEF

Your Output

match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

✓ Perfect answer!

AI Review?

HACKERANK

1. Query all columns for all American cities in the **CITY** table with populations larger than 100000. The **CountryCode** for America is USA.

The **CITY** table is described as follows:

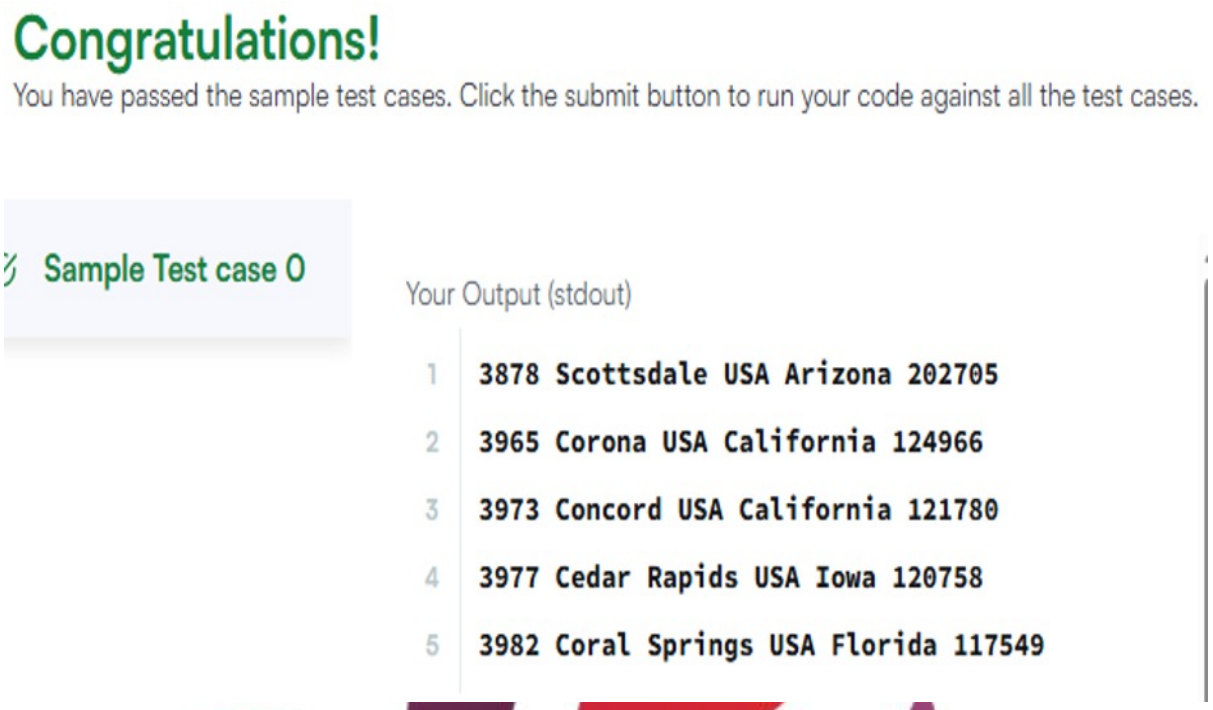
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
  SELECT *
FROM CITY
WHERE COUNTRYCODE = 'USA'
  AND POPULATION > 100000;
```

OUTPUT:



2. Query the **NAME** field for all American cities in the **CITY** table with populations larger than 120000. The *CountryCode* for America is USA.

The **CITY** table is described as follows:

CITY

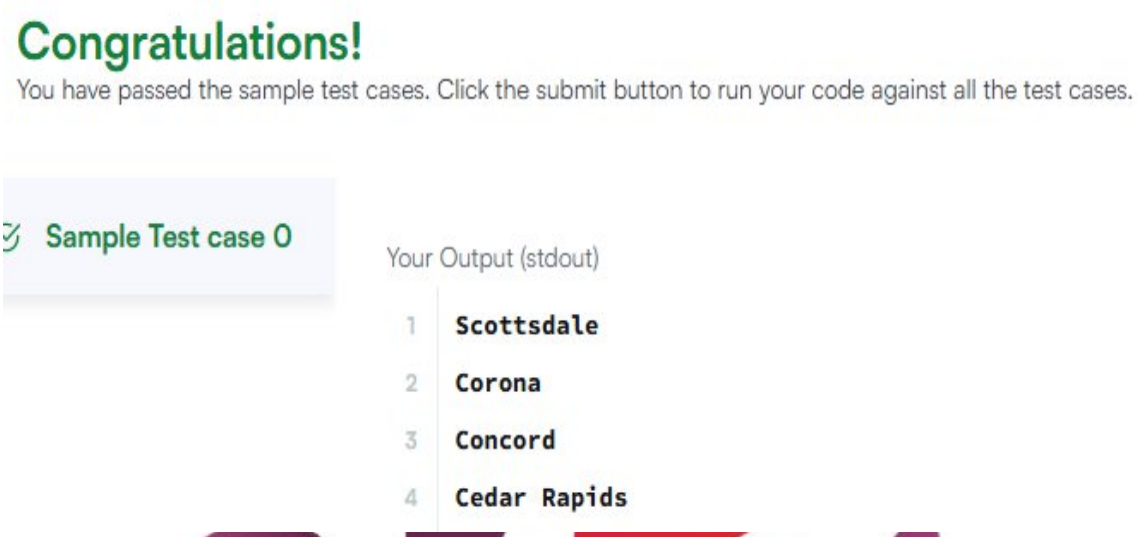
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
SELECT NAME
FROM CITY
```

```
WHERE COUNTRYCODE = 'USA'
  AND POPULATION > 120000;
```

OUTPUT:



3. Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

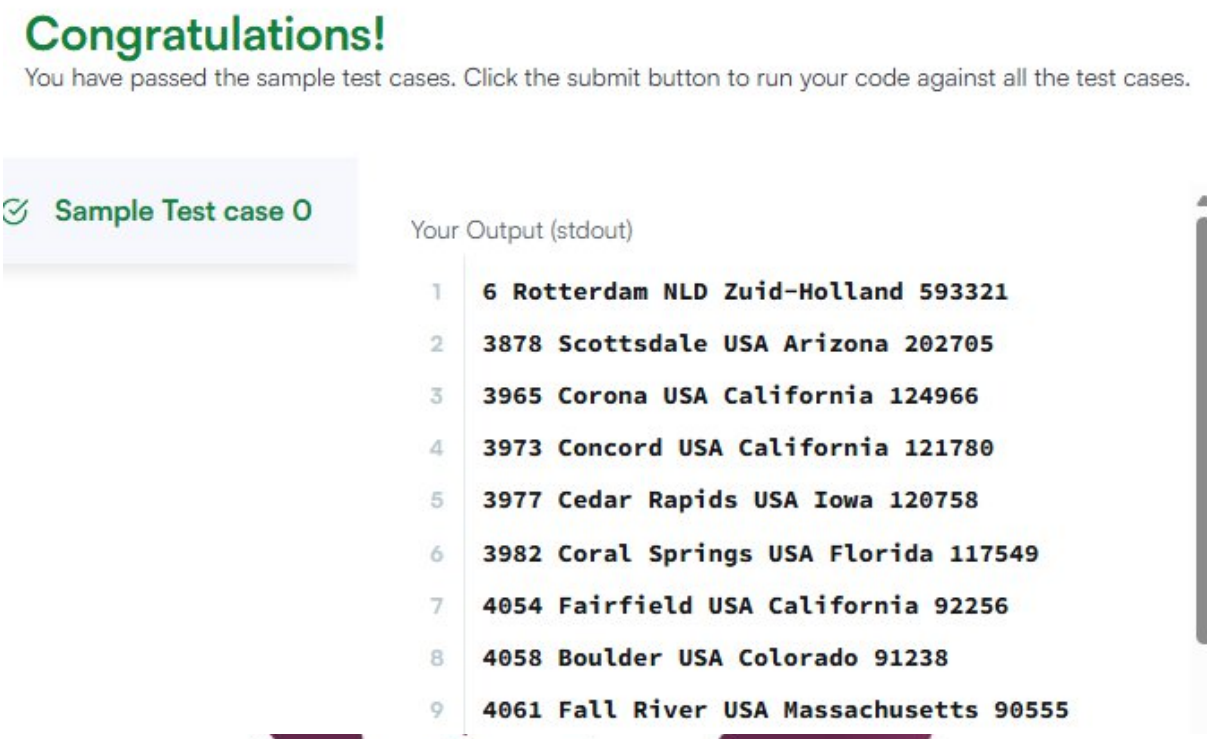
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
SELECT *
FROM CITY;
```

OUTPUT:



4Query all columns for a city in **CITY** with the *ID* 1661.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
SELECT *
FROM CITY
WHERE ID = 1661;
```

OURPUT:



5. Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is JPN.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
SELECT *
FROM CITY
```

WHERE COUNTRYCODE = 'JPN';

OUTPUT:



6. Query the names of all the Japanese cities in the **CITY** table. The **COUNTRYCODE** for Japan is JPN.
The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

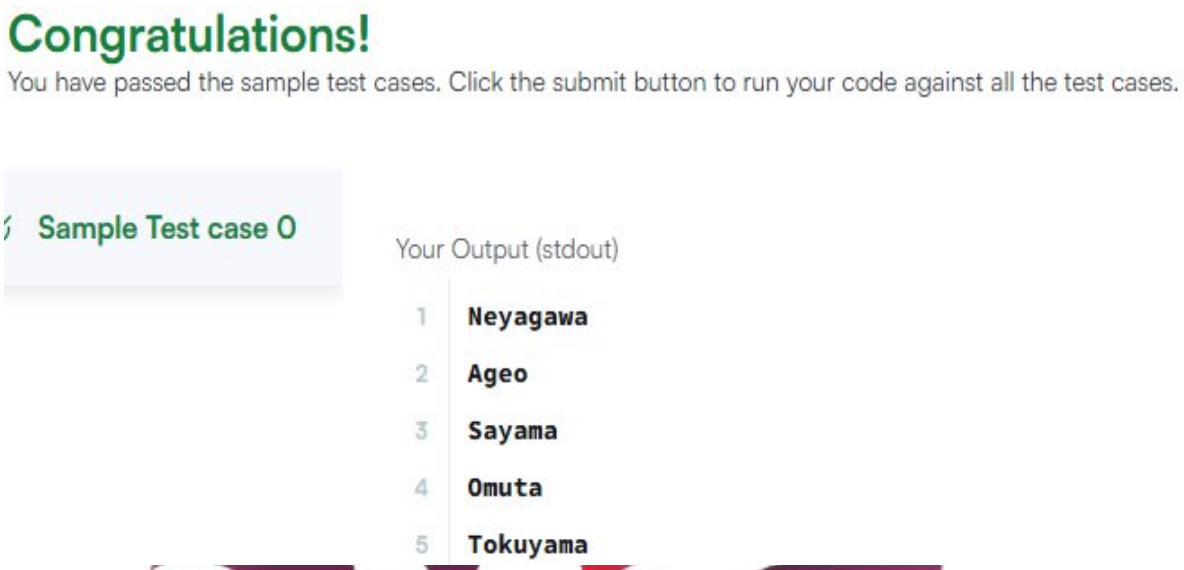
PROGRAM:

SELECT NAME

FROM CITY

WHERE COUNTRYCODE = 'JPN';

OUTPUT:



7. Query a list of **CITY** and **STATE** from the **STATION** table.
The **STATION** table is described as follows:

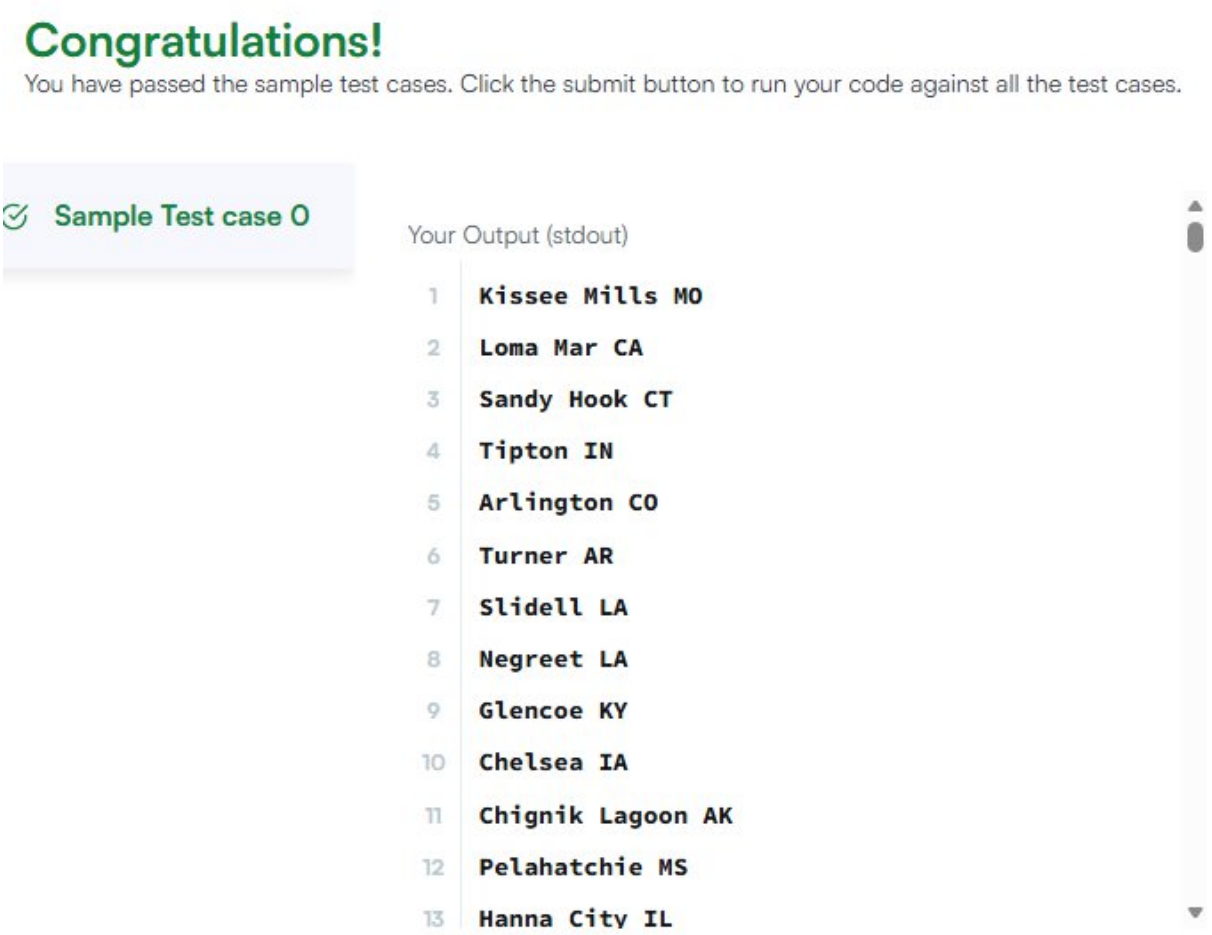
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT CITY, STATE
FROM STATION;
```

OUTPUT:



8. Query a list of **CITY** names from **STATION** for cities that have an even **ID** number. Print the results in any order, but exclude duplicates from the answer.
The **STATION** table is described as follows:

AUTONOMOUS

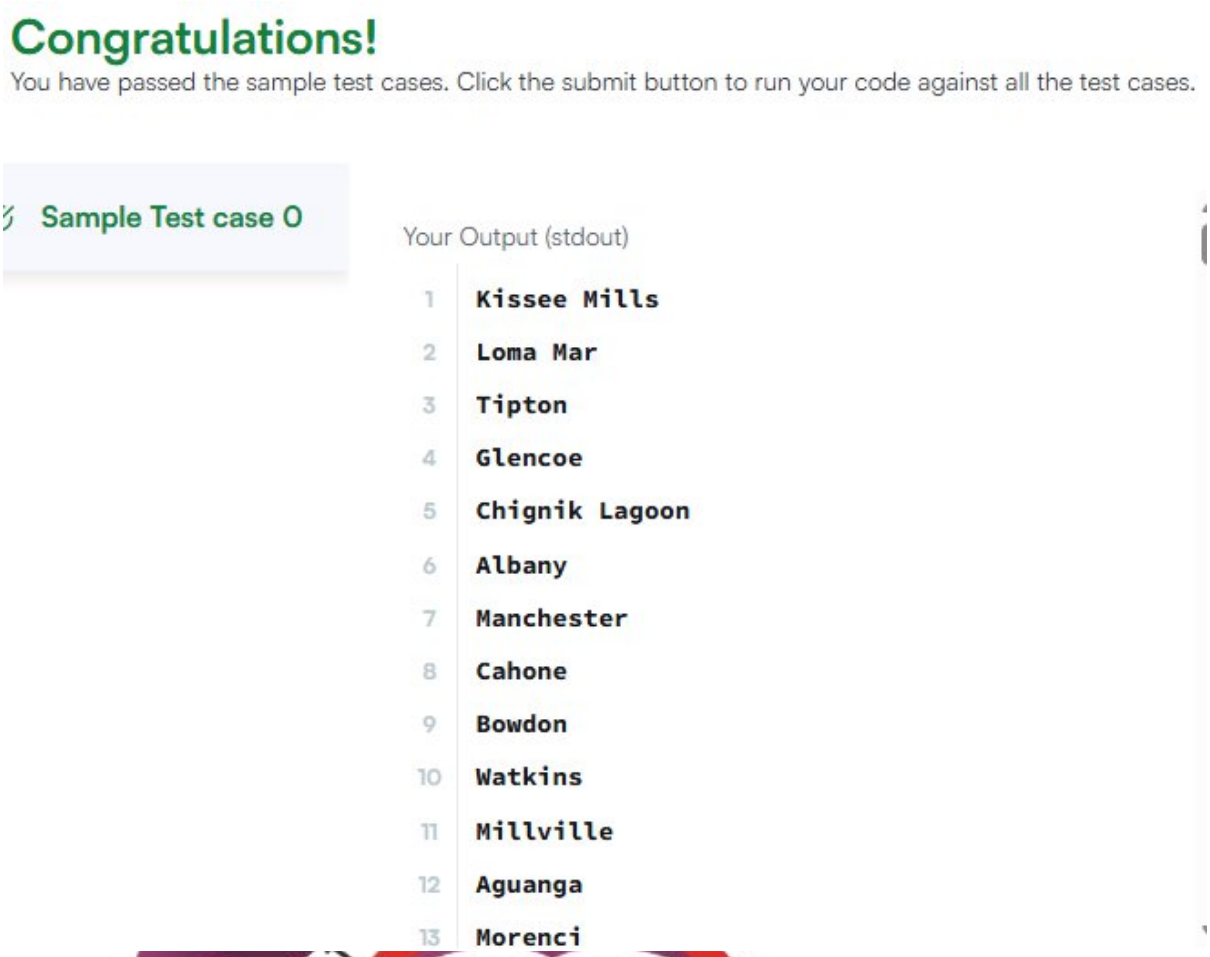
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT DISTINCT CITY
FROM STATION
WHERE ID % 2 = 0;
```

OUTPUT:



9. Find the difference between the total number of **CITY** entries in the table and the number of distinct **CITY** entries in the table.
The **STATION** table is described as follows:

AUTONOMOUS

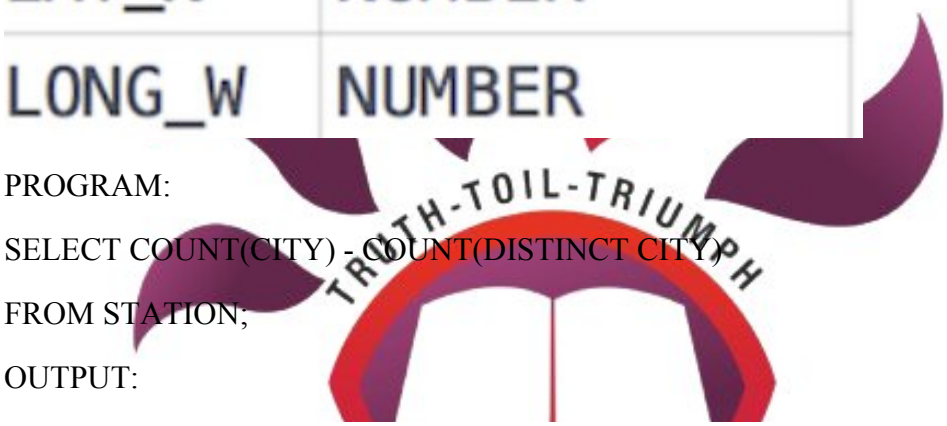
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT COUNT(CITY) - COUNT(DISTINCT CITY)
FROM STATION;
```

OUTPUT:



Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

1 13

10. Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.
The **STATION** table is described as follows:

STATION

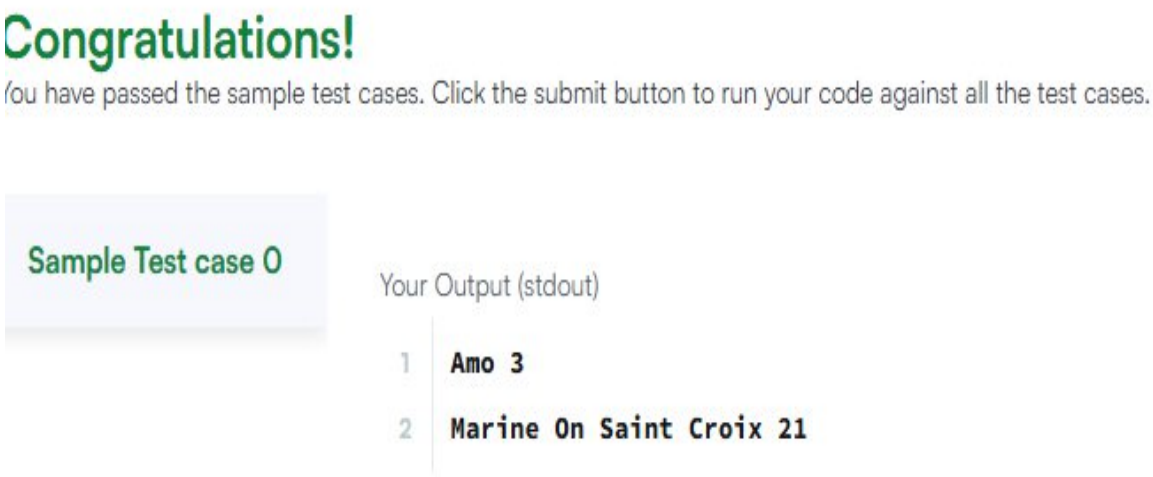
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY), CITY
LIMIT 1;

SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) DESC, CITY
LIMIT 1;
```

OUTPUT:



11. Query the list of *CITY* names starting with vowels (i.e., a, e, i, o, or u) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

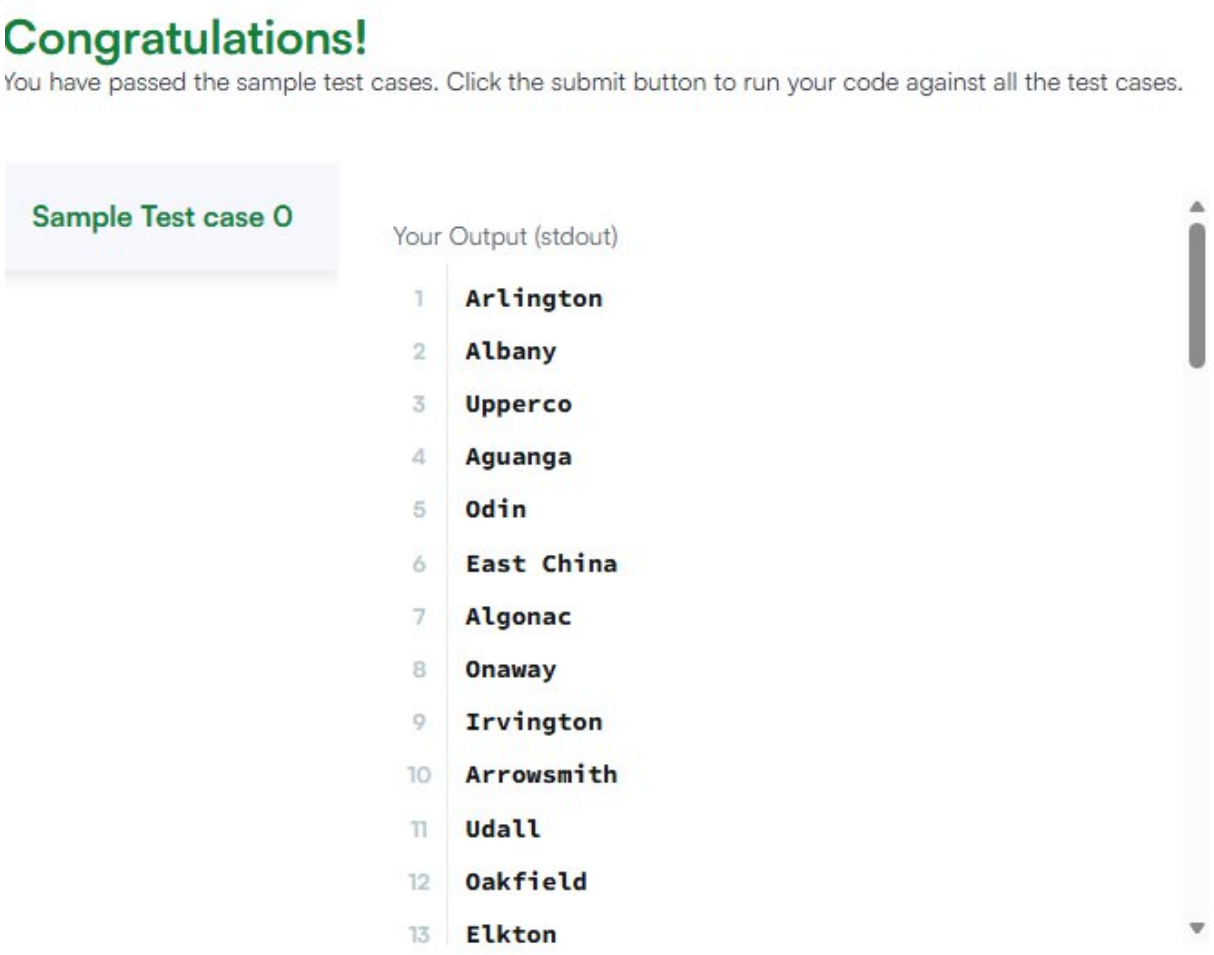
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT DISTINCT CITY
FROM STATION
```

```
WHERE CITY LIKE 'A%'
   OR CITY LIKE 'E%'
   OR CITY LIKE 'I%'
   OR CITY LIKE 'O%'
   OR CITY LIKE 'U%';
```

OUTPUT:



12. Query the list of *CITY* names ending with vowels (a, e, i, o, u) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

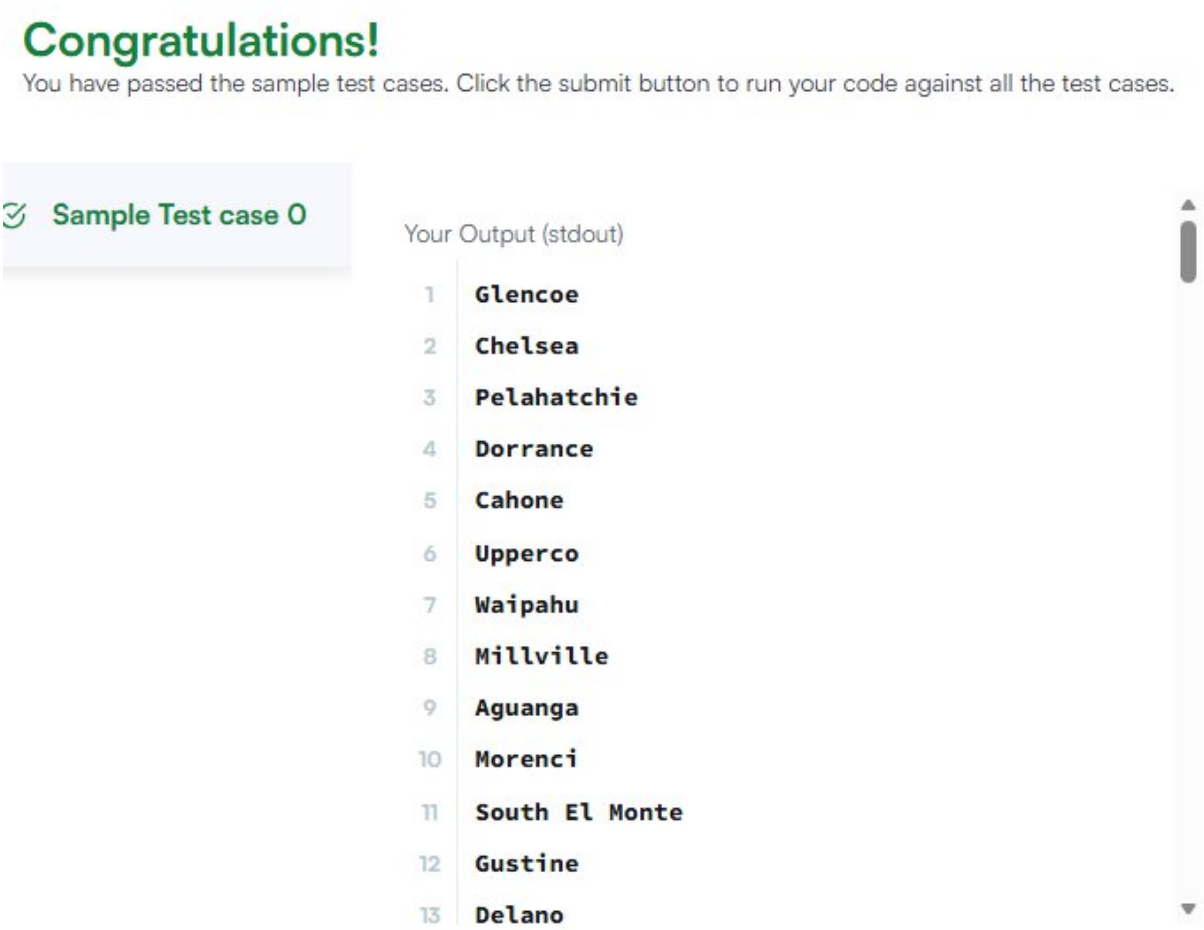
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY LIKE '%A'
   OR CITY LIKE '%E'
   OR CITY LIKE '%I'
   OR CITY LIKE '%O'
   OR CITY LIKE '%U';
```

OUTPUT:



13. Query the list of *CITY* names from **STATION** that *do not start* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

AUTONOMOUS

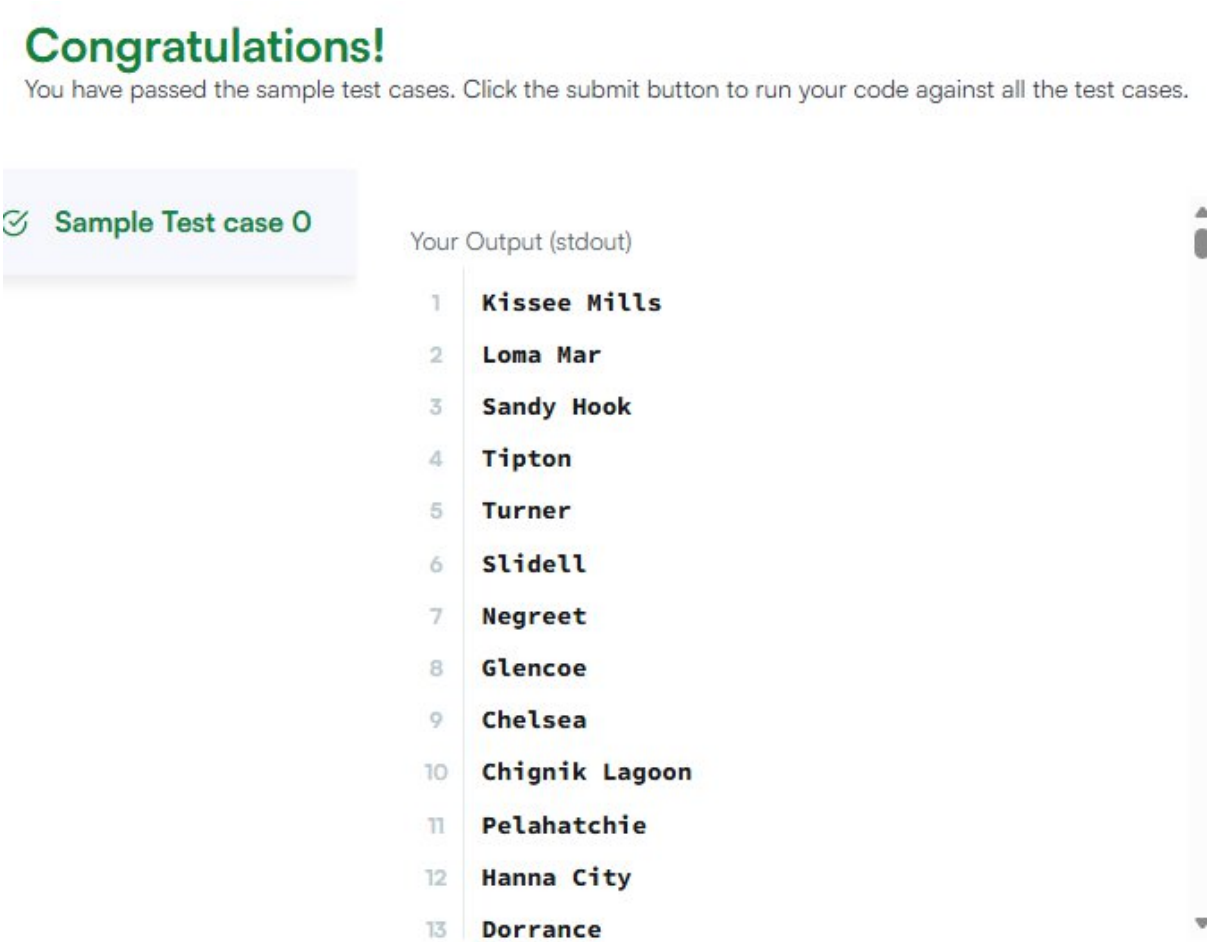
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT LIKE 'A%'
  AND CITY NOT LIKE 'E%'
  AND CITY NOT LIKE 'I%'
  AND CITY NOT LIKE 'O%'
  AND CITY NOT LIKE 'U%';
```

OUTPUT:



14. Query the list of *CITY* names from **STATION** that *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

AUTONOMOUS

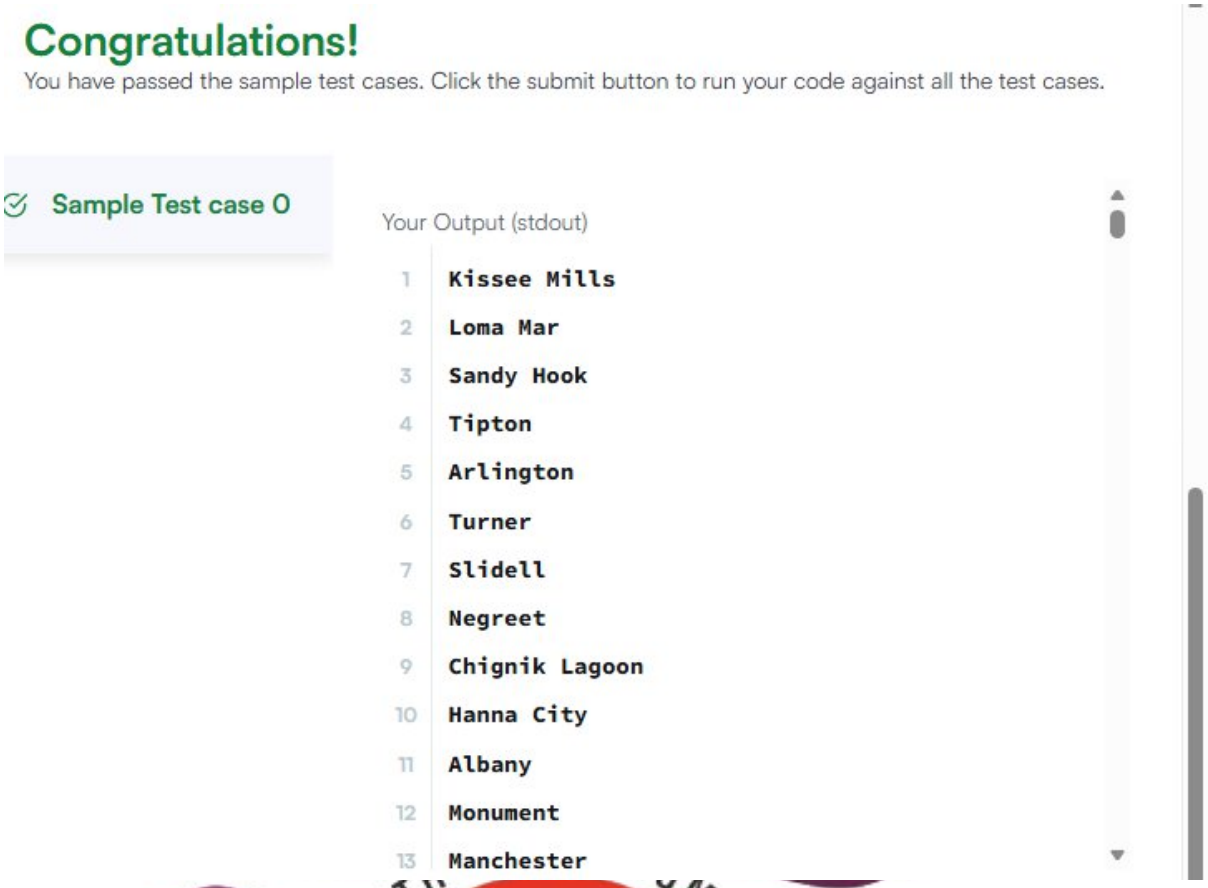
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT LIKE '%A'
  AND CITY NOT LIKE '%E'
  AND CITY NOT LIKE '%I'
  AND CITY NOT LIKE '%O'
  AND CITY NOT LIKE '%U';
```

OUITPUT:



154Query the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

AUTONOMOUS

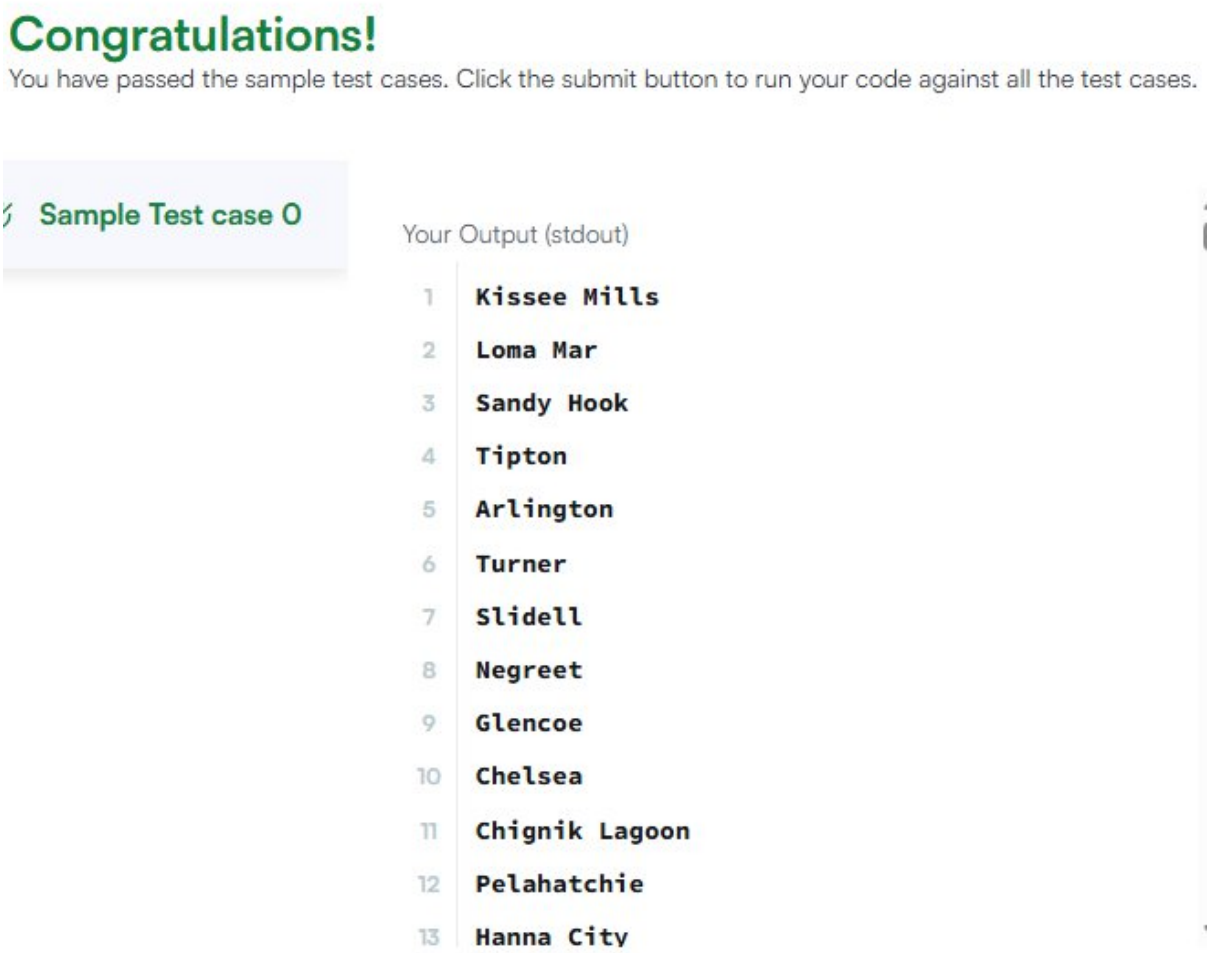
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT REGEXP '^[AEIOUaeiou]'
   OR CITY NOT REGEXP '[AEIOUaeiou]$';
```

OUTPUT:



16. Query the list of *CITY* names from **STATION** that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

AUTONOMOUS

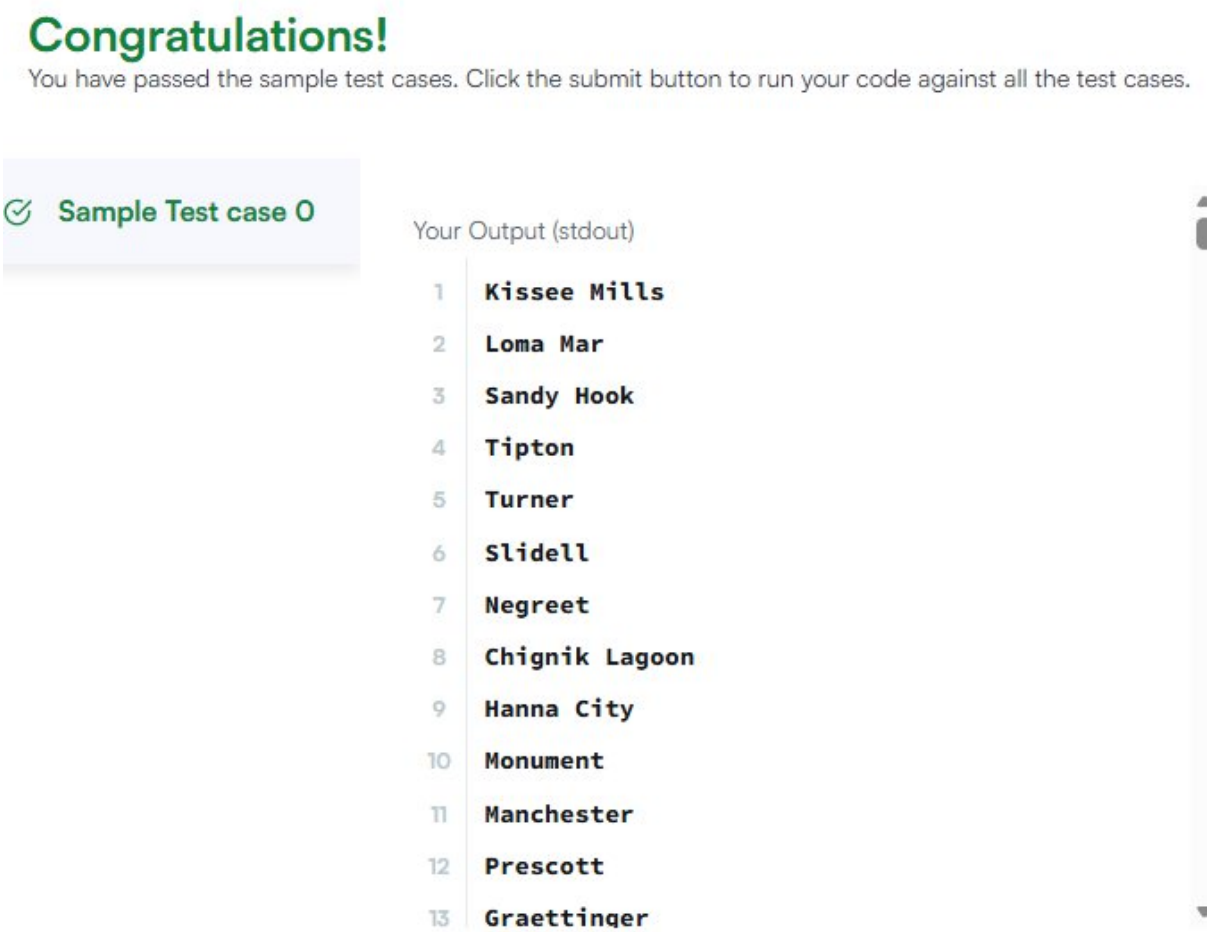
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT REGEXP '^[AEIOUaeiou]'
  AND CITY NOT REGEXP '[AEIOUaeiou]$';
```

OUTPUT:



17 Quer *Marks*. Order your output by the *last three characters* of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending *ID*.y the *Name* of any student in **STUDENTS** who scored higher than

Input Format

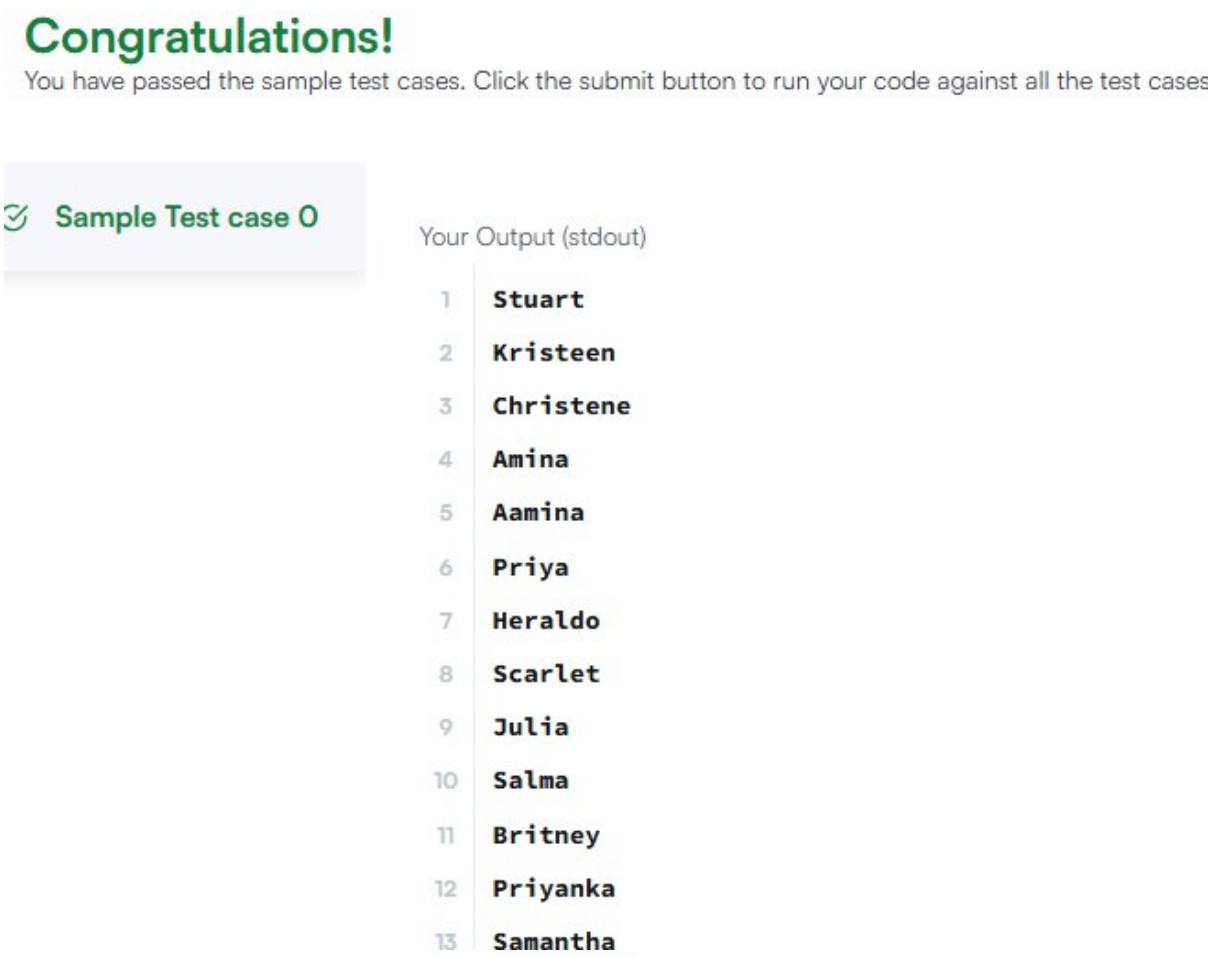
The **STUDENTS** table is described as follows:

Column	Type
ID	Integer
Name	String
Marks	Integer

PROGRAM:

```
SELECT Name
FROM STUDENTS
WHERE Marks > 75
ORDER BY RIGHT(Name, 3), ID;
```

OUTPUT:



18. Write a query that prints a list of employee names (i.e.: the *name* attribute) from the **Employee** table in alphabetical order.

Input Format

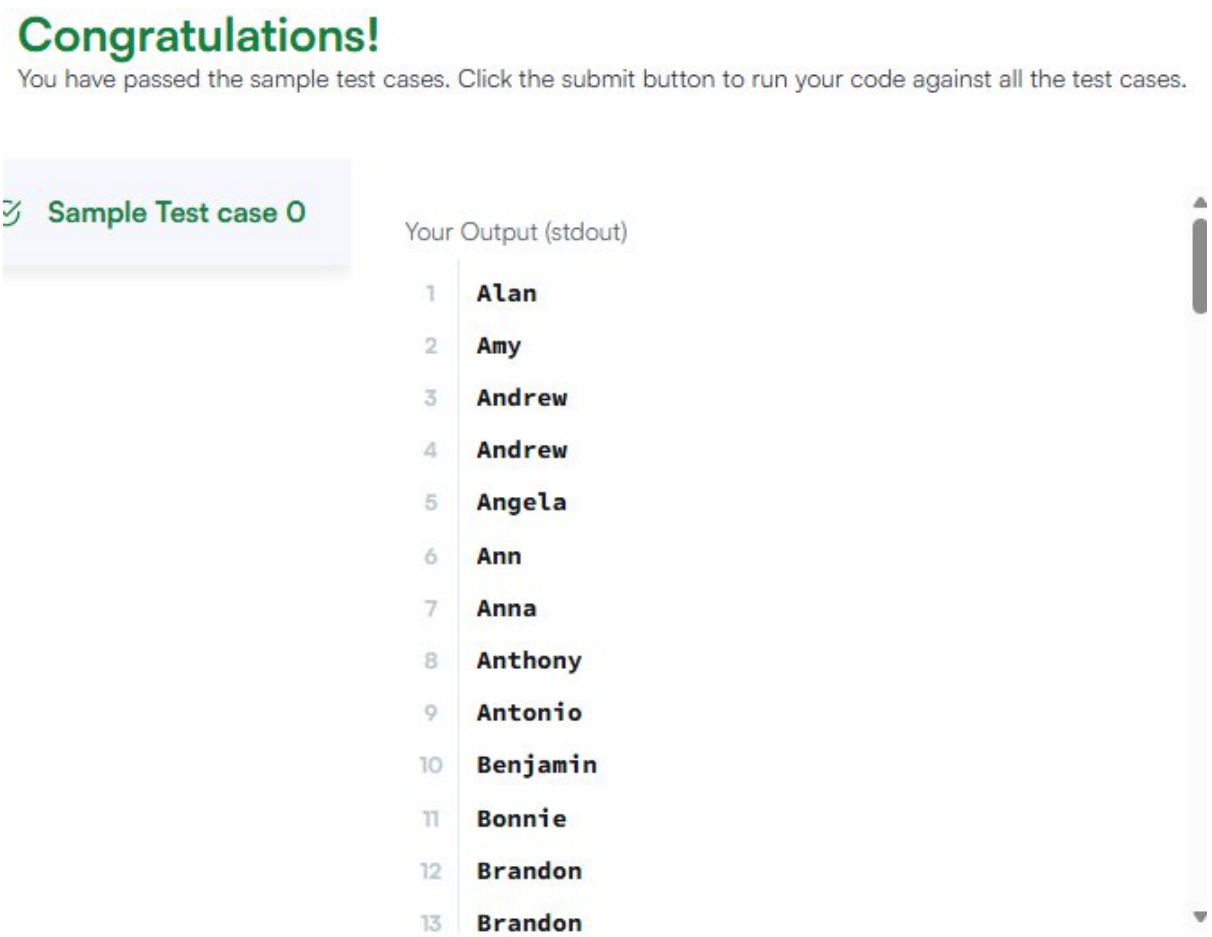
The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

PROGRAM:

```
SELECT name
FROM Employee
ORDER BY name ASC;
```

OUTPUT:



19. Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in **Employee** having a salary greater than

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

PROGRAM:

```
SELECT name
FROM Employee
WHERE salary > 2000
  AND months < 10
ORDER BY employee_id ASC;
```

OUTPUT:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Rose
Patrick
Lisa
Amy
Pamela
Jennifer
Julia
Kevin
Paul
Donna
Michelle
Christina
Brandon
```

20. Write a query identifying the *type* of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

- **Equilateral**: It's a triangle with
-
-
-

AUTONOMOUS

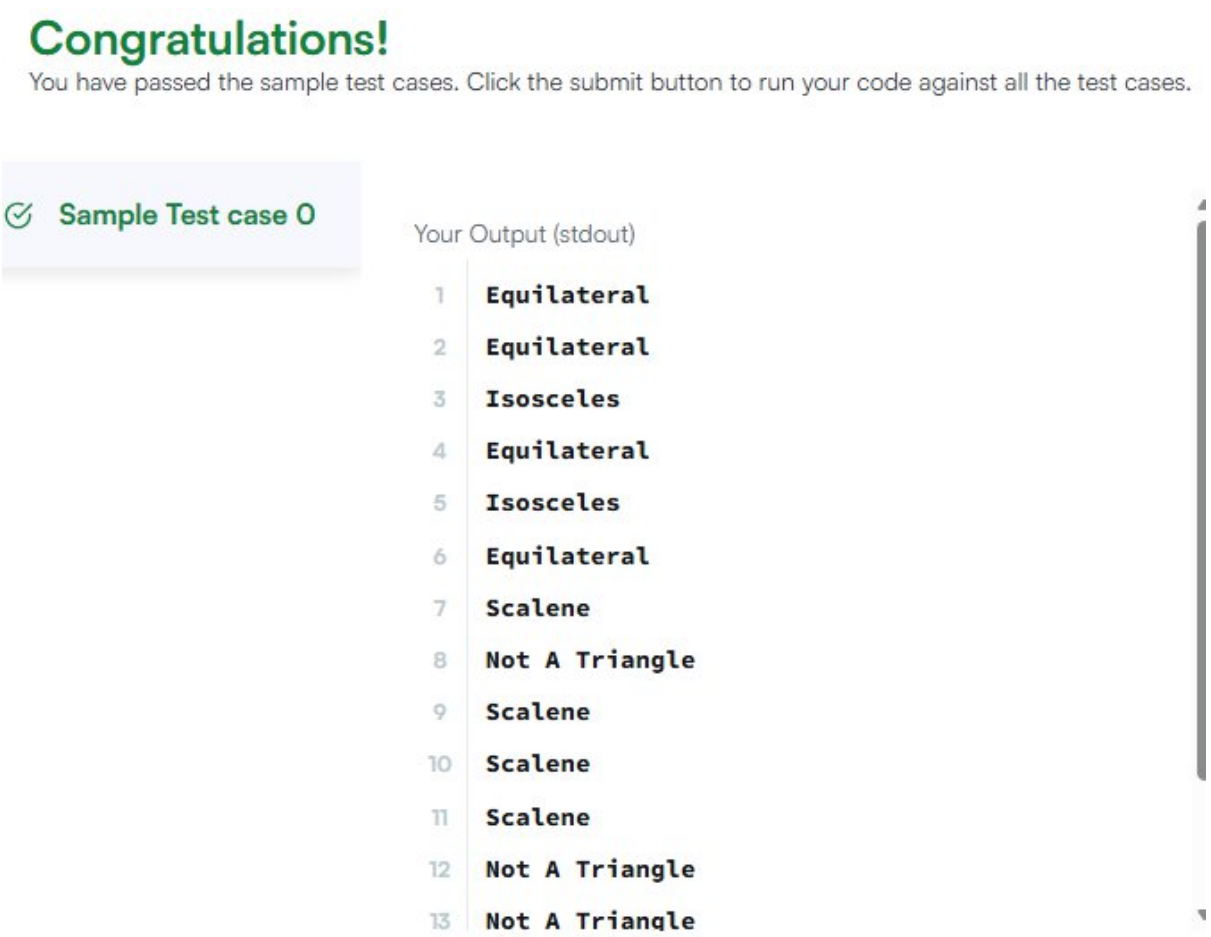
HACKERANK

Column	Type
A	Integer
B	Integer
C	Integer

PROGRAM:

```
SELECT
   CASE
      WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle'
      WHEN A = B AND B = C THEN 'Equilateral'
      WHEN A = B OR B = C OR A = C THEN 'Isosceles'
      ELSE 'Scalene'
   END
FROM TRIANGLES;
```

OUTPUT:



21. Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in **OCCUPATIONS**, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).
2. Query the number of ocurrences of each occupation in **OCCUPATIONS**. Sort the occurrences in *ascending order*, and output them in the following format:
3. There are a total of [occupation_count] [occupation]s.

where [occupation_count] is the number of occurrences of an occupation in **OCCUPATIONS** and [occupation] is the *lowercase* occupation name. If more than one *Occupation* has the same [occupation_count], they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

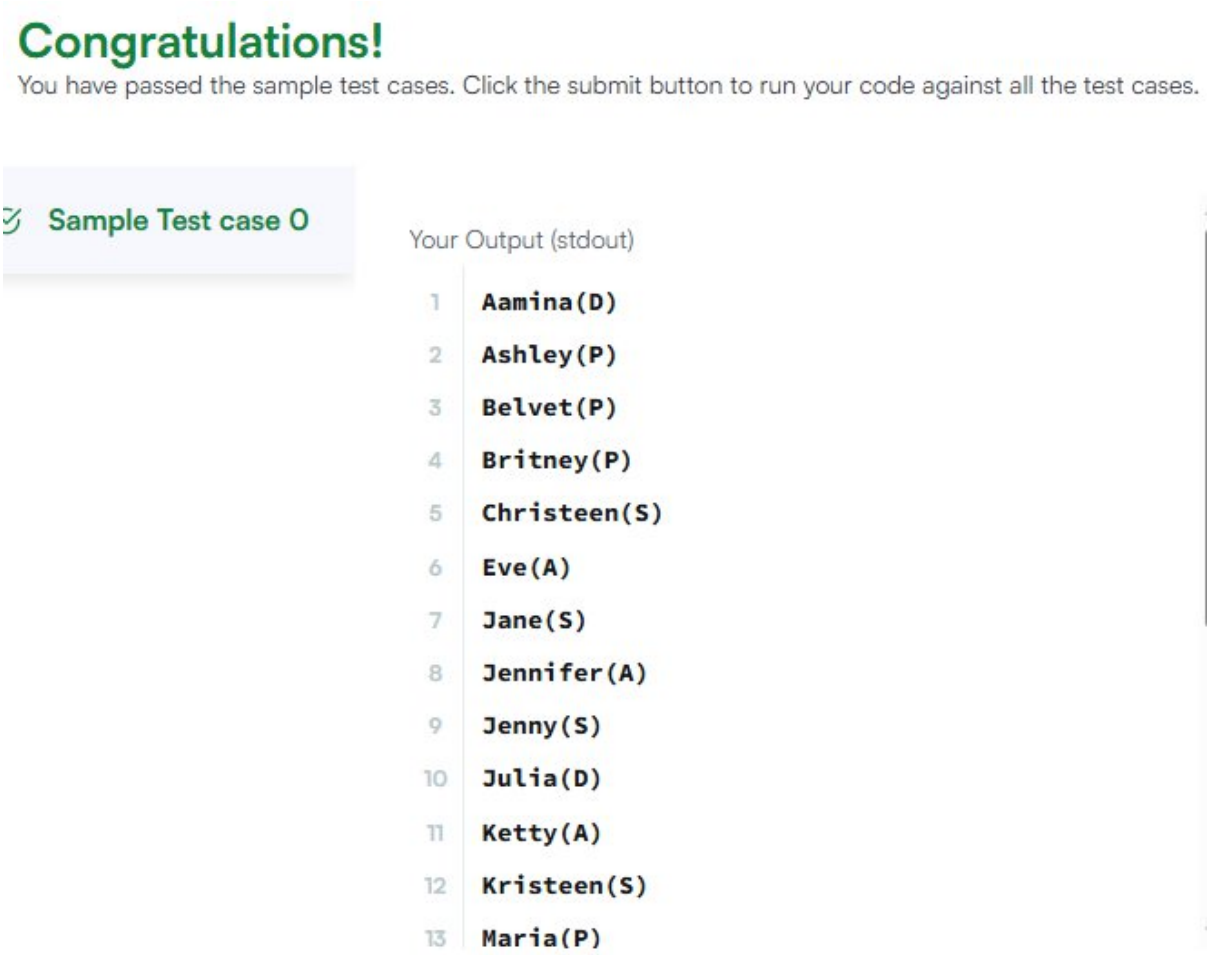
Column	Type
Name	String
Occupation	String

PROGRAM:

```
SELECT CONCAT(name, '(', LEFT(occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY name;

SELECT CONCAT(
    'There are a total of ',
    COUNT(*),
    ' ',
    LOWER(occupation),
    's.'
)
FROM OCCUPATIONS
GROUP BY occupation
ORDER BY COUNT(*), occupation;
```

OUTPUT:



22. Pivot the *Occupation* column in **OCCUPATIONS** so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print **NULL** when there are no more names corresponding to an occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

Column	Type
Name	String
Occupation	String

HACKERANK

PROGRAM:

```
SELECT
    MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
    MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
    MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
    MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM
(
    SELECT
        Name,
        Occupation,
        ROW_NUMBER() OVER(PARTITION BY Occupation ORDER BY Name) AS rn
    FROM OCCUPATIONS
) t
GROUP BY rn
ORDER BY rn;
```

OUTPUT:



Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1 Aamina Ashley Christeen Eve
2 Julia Belvet Jane Jennifer
3 Priya Britney Jenny Ketty
4 NULL Maria Kristeen Samantha
5 NULL Meera NULL NULL
6 NULL Naomi NULL NULL
7 NULL Priyanka NULL NULL
```

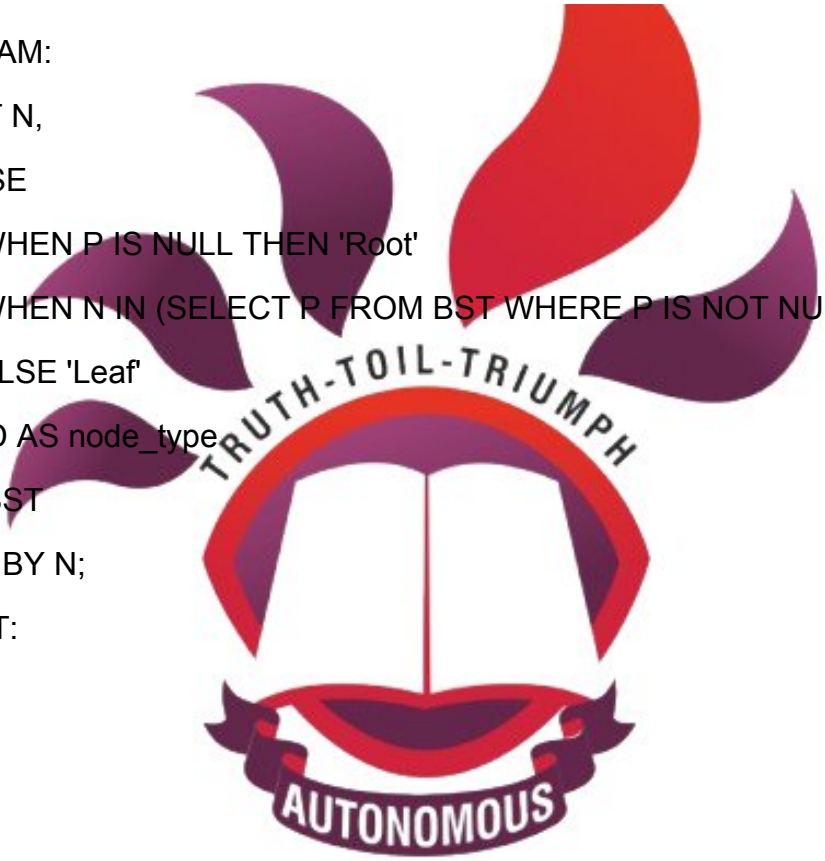
23. You are given a table, *BST*, containing two columns: *N* and *P,* where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*.

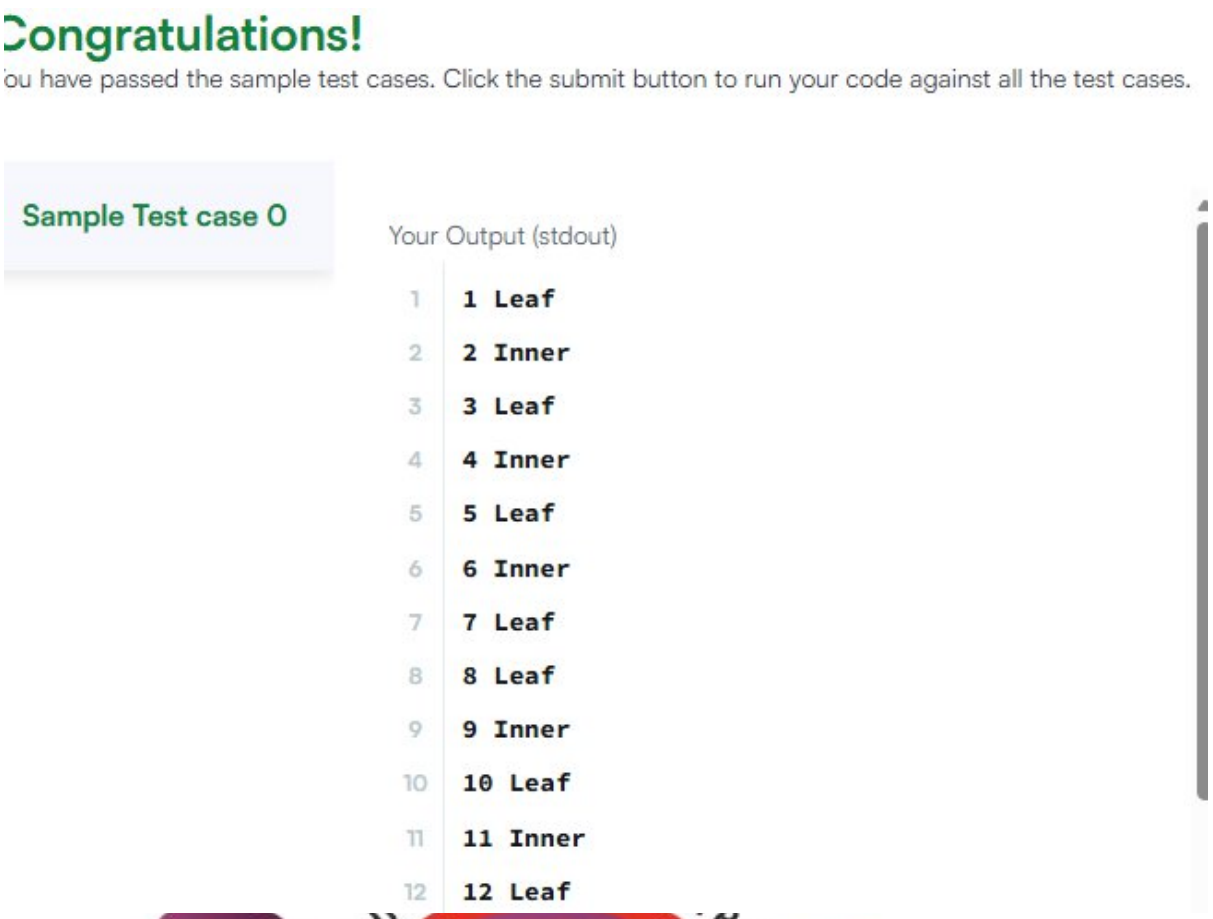
Column	Type
N	Integer
P	Integer

PROGRAM:

```
SELECT N,
    CASE
        WHEN P IS NULL THEN 'Root'
        WHEN N IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN 'Inner'
        ELSE 'Leaf'
    END AS node_type
FROM BST
ORDER BY N;
```

OUTPUT:





24. A Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

Note:

- The tables may contain duplicate records.
- The *company_code* is string, so the sorting should not be **numeric**. For example, if the *company_codes* are *C_1*, *C_2*, and *C_10*, then the ascending *company_codes* will be *C_1*, *C_10*, and *C_2*.

Input Format

The following tables contain company data:

- *Company:* The *company_code* is the code of the company and *founder* is the founder of the company.

mber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:

Column	Type
company_code	String
founder	String

PROGRAM:

```
SELECT c.company_code,
       c.founder,
       COUNT(DISTINCT l.lead_manager_code),
       COUNT(DISTINCT s.senior_manager_code),
       COUNT(DISTINCT m.manager_code),
       COUNT(DISTINCT e.employee_code)
FROM Company c
LEFT JOIN Lead_Manager l
    ON c.company_code = l.company_code
LEFT JOIN Senior_Manager s
    ON c.company_code = s.company_code
LEFT JOIN Manager m
    ON c.company_code = m.company_code
LEFT JOIN Employee e
    ON c.company_code = e.company_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code;
```

OUTPUT:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
C1 Angela 1 2 5 13
C10 Earl 1 1 2 3
C100 Aaron 1 2 4 10
C11 Robert 1 1 1 1
C12 Amy 1 2 6 14
C13 Pamela 1 2 5 14
C14 Maria 1 1 3 5
C15 Joe 1 1 2 3
C16 Linda 1 1 3 5
C17 Melissa 1 2 3 7
C18 Carol 1 2 5 6
C19 Paula 1 2 4 7
```

25. Query a *count* of the number of cities in **CITY** having a *Population* larger than

. **Input Format**

The **CITY** table is described as follows:put Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
SELECT COUNT(*)
FROM CITY
WHERE POPULATION > 100000
```

OUTPUT:



Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

1 6

26. Query the total population of all cities in **CITY** where *District* is **California**.

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
SELECT SUM(Population)
FROM CITY
WHERE District = 'California';
```

OUTPUT:



Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1  339002
```

27. Query the average population of all cities in **CITY** where *District* is **California**.

Input Format

The **CITY** table is described as follows:

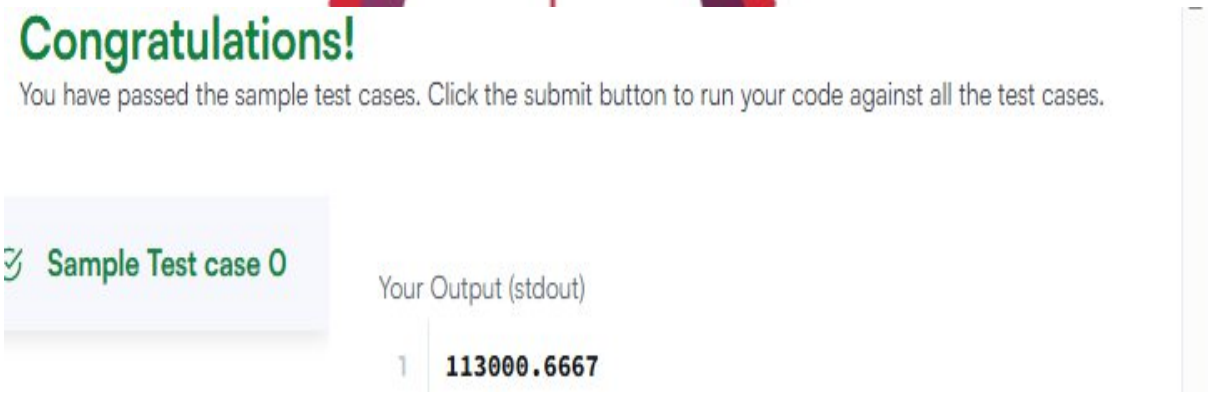
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
SELECT AVG(Population)
FROM CITY
WHERE District = 'California';
```

OUTPUT:



28. Query the average population for all cities in **CITY**, rounded *down* to the nearest integer.

Input Format

The **CITY** table is described as follows:

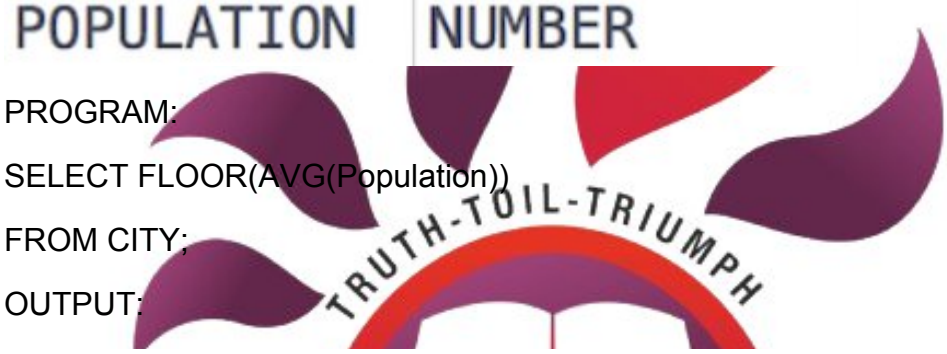
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
SELECT FLOOR(AVG(Population))
FROM CITY;
```

OUTPUT:



Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1  454250
```

29. Query the sum of the populations for all Japanese cities in **CITY**. The *COUNTRYCODE* for Japan is **JPN**.

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
SELECT SUM(Population)
FROM CITY
WHERE CountryCode = 'JPN';
```

OUTPUT:



Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

1 879196

30. Query the difference between the maximum and minimum populations in **CITY**.

Input Format

The **CITY** table is described as follows:

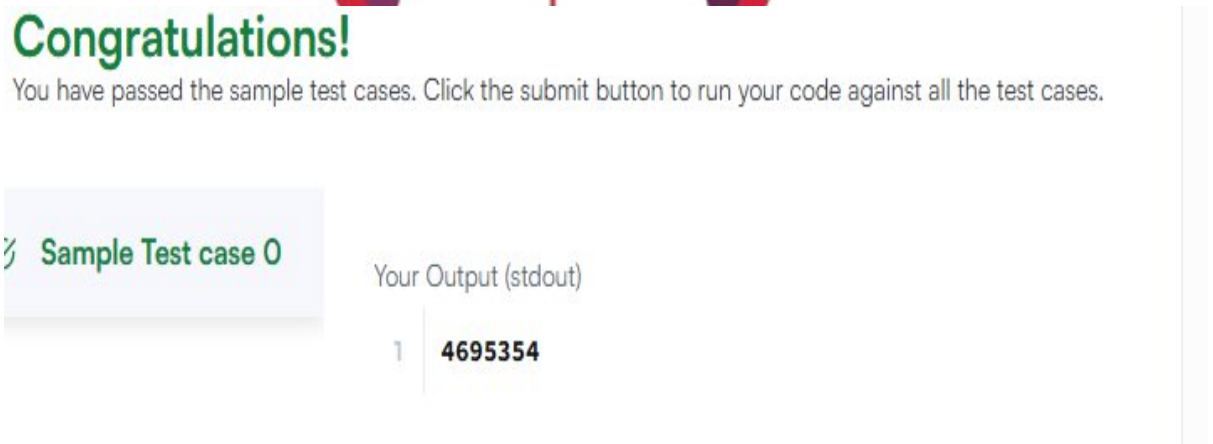
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
SELECT MAX(Population) - MIN(Population)
FROM CITY;
```

OUTPUT:



31. Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's

was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.:

average monthly salaries), and round it up to the next integer.

Input Format

The **EMPLOYEES** table is described as follPRO

Column	Type
ID	Integer
Name	String
Salary	Integer

PROGRAM:

```
SELECT CEIL(
    AVG(Salary) - AVG(REPLACE(Salary, '0', ''))
)
FROM EMPLOYEES;
```

OU TPUT:



Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

1 2253

32. We define an employee's *total earnings* to be their monthly

worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the **Employee** table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

Input Format

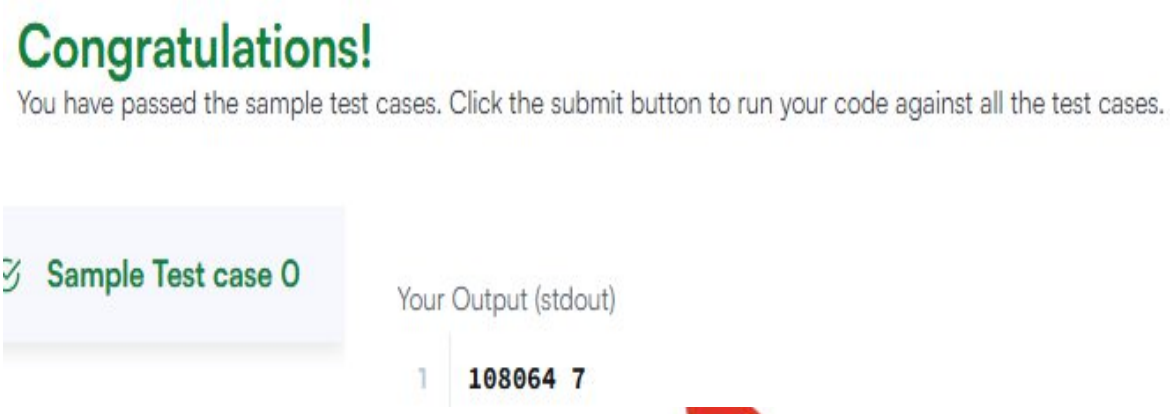
The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

PROGRAM:

```
SELECT
    MAX(months * salary),
    COUNT(*)
FROM Employee
WHERE (months * salary) = (
    SELECT MAX(months * salary)
    FROM Employee
);
```

OUTPUT:



33. Query the following two values from the **STATION** table:

1. The sum of all values in *LAT_N* rounded to a scale of

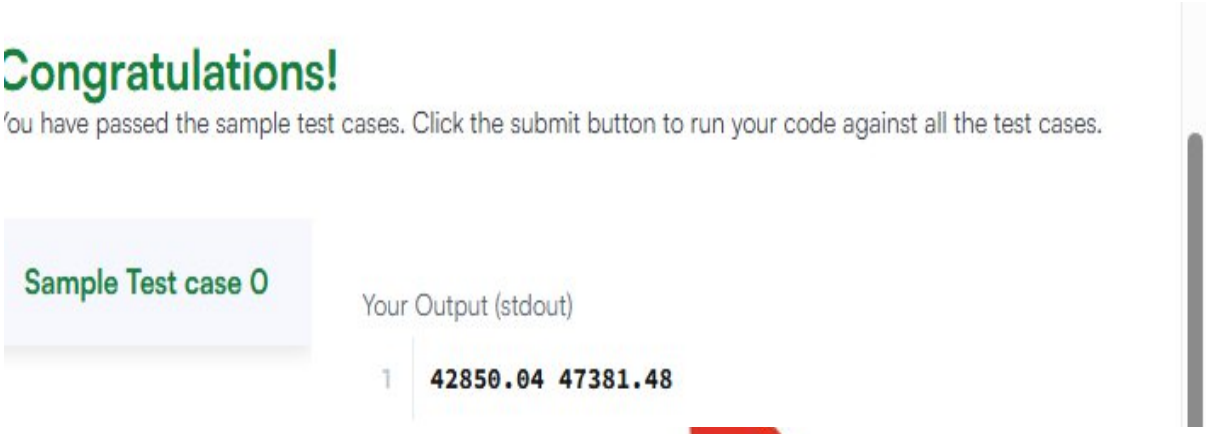
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT
   ROUND(SUM(LAT_N), 2),
   ROUND(SUM(LONG_W), 2)
FROM STATION;
```

OUTPUT:



34. Query the sum of *Northern Latitudes* (*LAT_N*) from **STATION** having values greater than

Truncate your answer to

Input Format

The **STATION** table is described as follows:

STATION

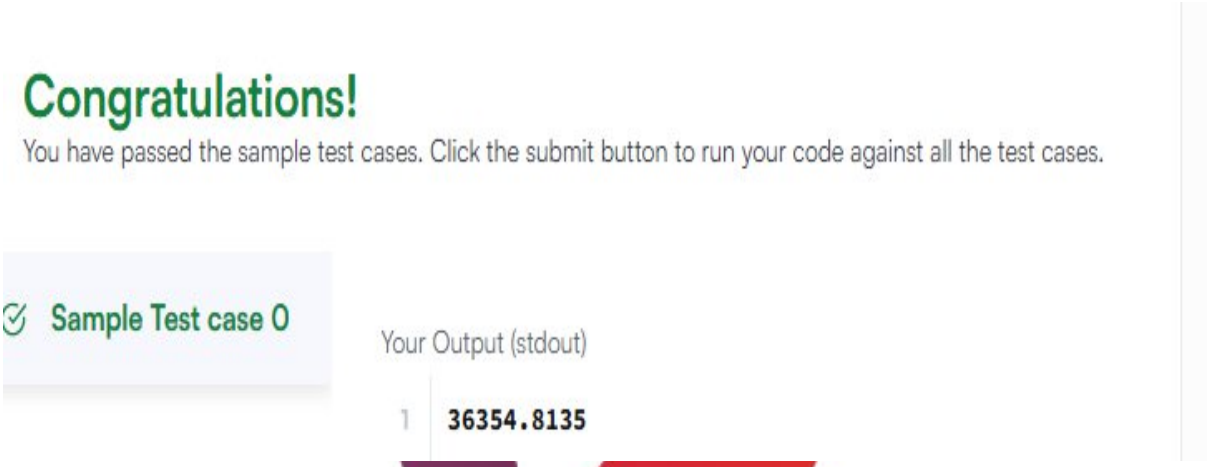
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT TRUNCATE(SUM(LAT_N), 4)
FROM STATION
```

```
WHERE LAT_N > 38.7880
  AND LAT_N < 137.2345;
```

OUTPUT:



35. Query the greatest value of the *Northern Latitudes* (*LAT_N*) from **STATION** that is less than

nput Format

The **STATION** table is described as follows:

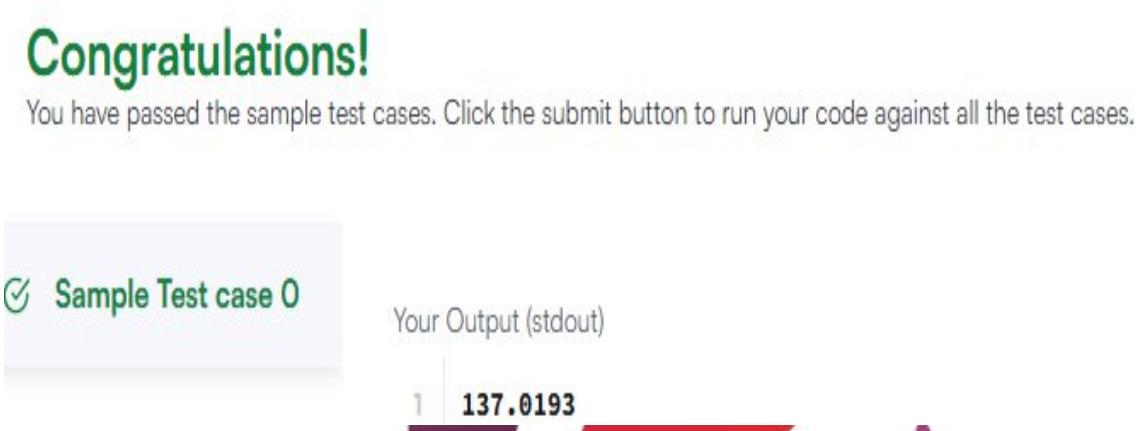
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
ELECT CAST(FLOOR(MAX(LAT_N) * 10000) AS DOUBLE) / 10000
FROM STATION
WHERE LAT_N < 137.2345;
```

OUTPUT:



36. Query the *Western Longitude* (*LONG_W*) for the largest *Northern Latitude* (*LAT_N*) in **STATION** that is less than137.2345 Round your answer to

Input Format

The **STATION** table is described as follows:

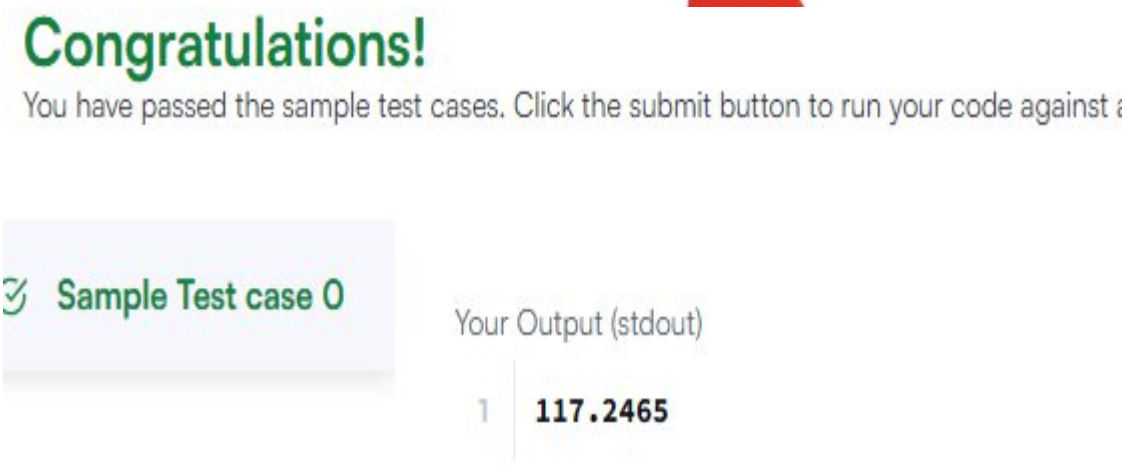
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MAX(LAT_N)
   FROM STATION
   WHERE LAT_N < 137.2345
```

OUTPUT:



37. Query the smallest *Northern Latitude* (*LAT_N*) from **STATION** that is greater than 38.7780 Round your answer to4 decimal places.

Input Format

The **STATION** table is described as follows:

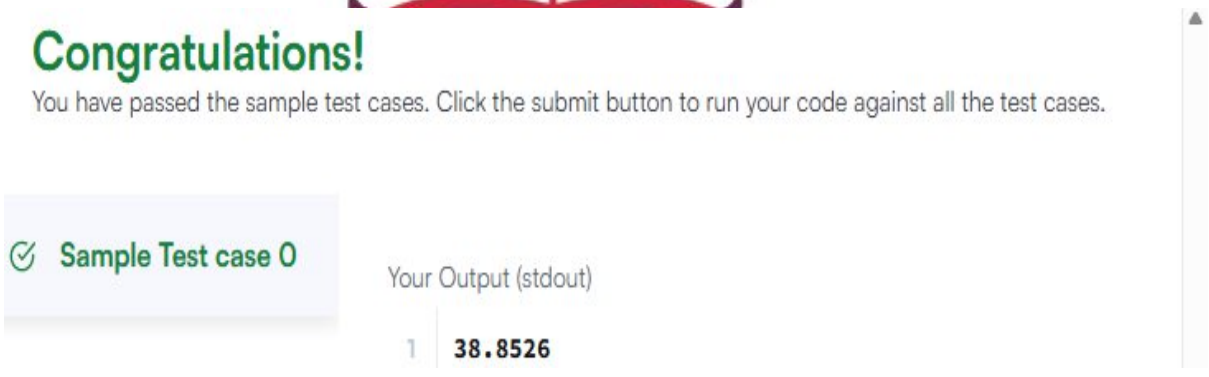
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT ROUND(MIN(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7780;
```

OUTPUT:



38. Query the *Western Longitude* (*LONG_W*)where the smallest *Northern Latitude* (*LAT_N*) in **STATION** is greater than 38.7780 Round your answer to 4 decimal places

Input Format

The **STATION** table is described as follows:

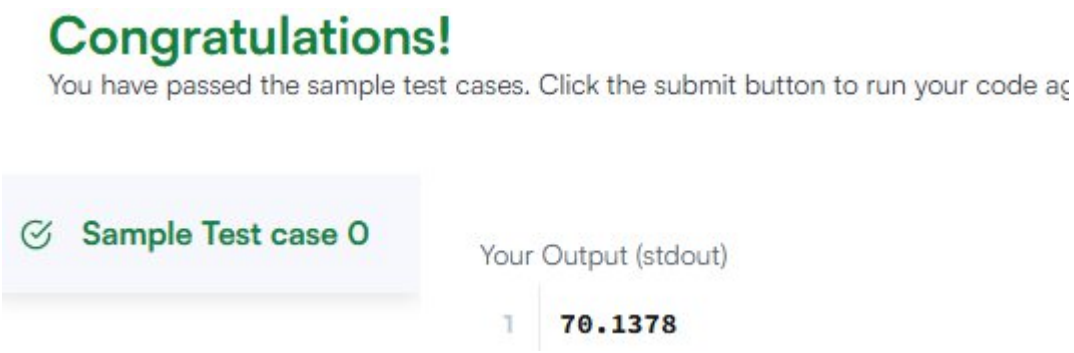
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

PROGRAM:

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
    SELECT MIN(LAT_N)
    FROM STATION
    WHERE LAT_N > 38.7780
);
```

OUTPUT:



39.consider p1(a,b)and p2(c,d) to be two points on a *2D* plane. happens to equal the minimum value in *Northern Latitude* (*LAT_N* in **STATION**).

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

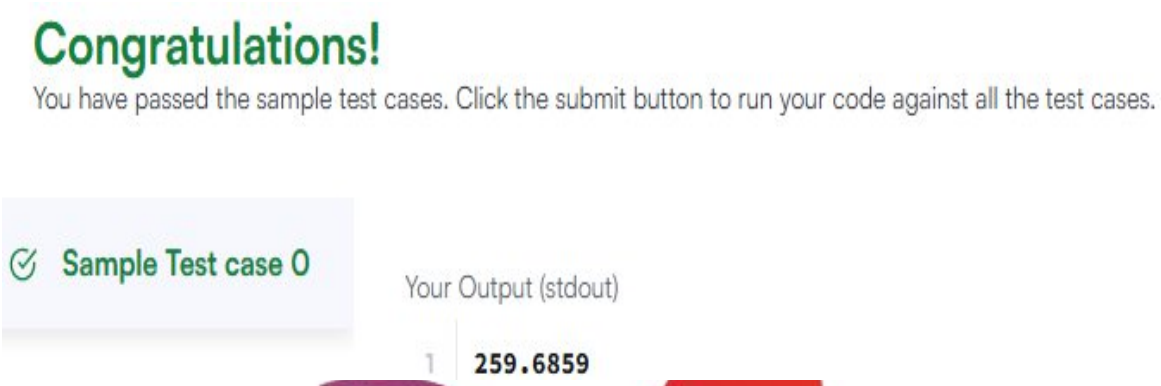
PROGRAM:

```
ELECT ROUND(
    (MAX(LAT_N) - MIN(LAT_N)) +
    (MAX(LONG_W) - MIN(LONG_W)),
    4
```

) AS MANHATTAN_DISTANCE

FROM STATION;

OUTPUT:



40.consider p1(a,c) and p2(b,d) o be two points on a 2D plane where (a,b) are the respective minimum and maximum values of *Northern Latitude* (*LAT_N*) and (c,d)

re the respective minimum and maximum values of *Western Longitude* (*LONG_W*) in **STATION**.

Query the Euclidean Distance between points

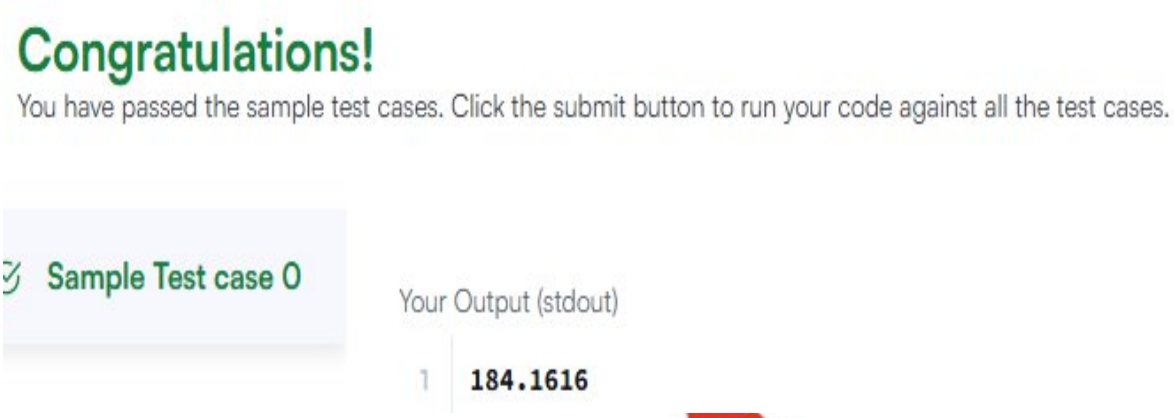
Input Format

The **STATION** table is described as follows:

PROGRAM:

```
SELECT ROUND(
   SQRT(
      POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
      POWER(MAX(LONG_W) - MIN(LONG_W), 2)
   ),
   4
) AS EUCLIDEAN_DISTANCE
FROM STATION
```

OUTPUT:



41. Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

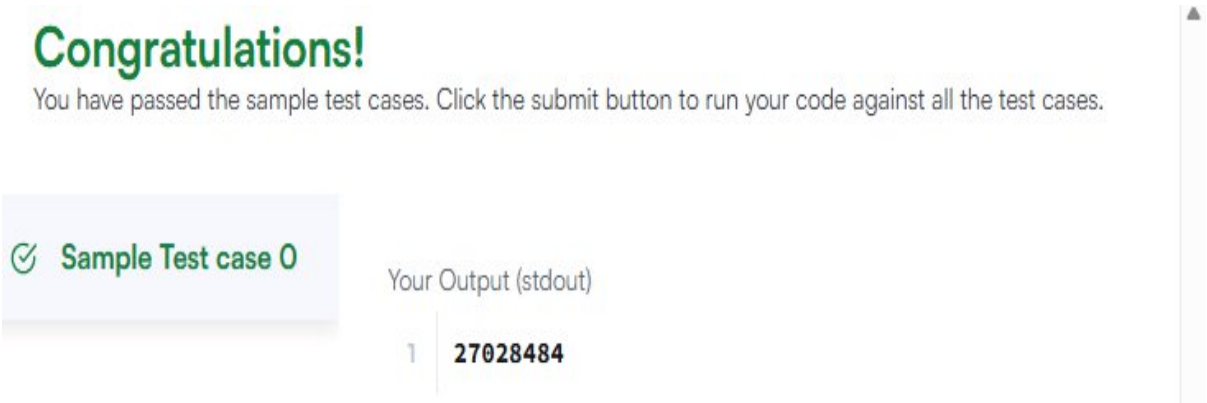
PROGRAM:

```
SELECT SUM(CITY.Population)
FROM CITY
JOIN COUNTRY
```

ON CITY.CountryCode = COUNTRY.Code

WHERE COUNTRY.Continent = 'Asia';

OUTPUT:



42. Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

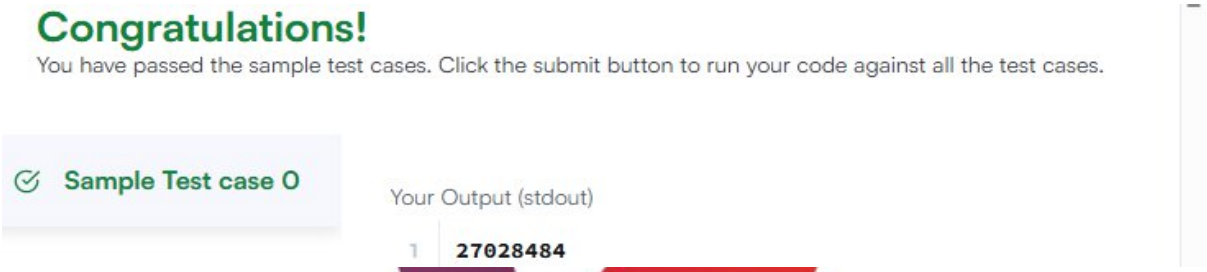
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

SELECT SUM(CITY.Population)

```
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
WHERE COUNTRY.Continent = 'Asia';
```

OUTPUT:



43. Given the **CITY** and **COUNTRY** tables, query the names of all cities where the *CONTINENT* is *'Africa'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

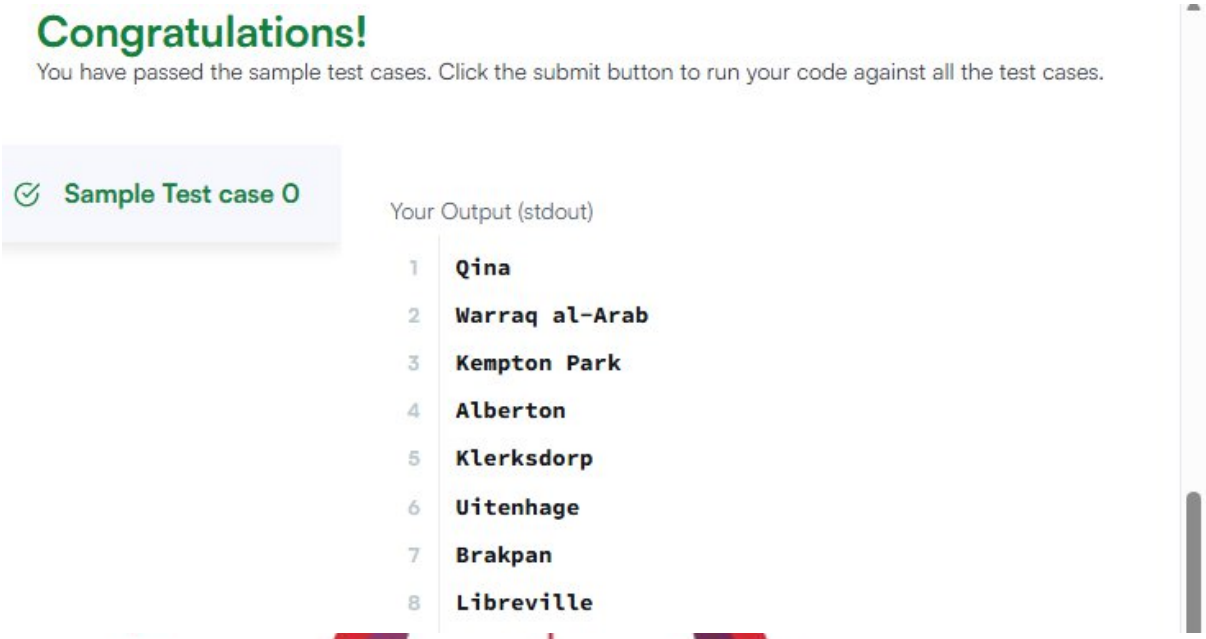
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
ELECT CITY.Name
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
WHERE COUNTRY.Continent = 'Africa';
```

OUTPUT:



44. Given the **CITY** and **COUNTRY** tables, query the names of all the continents (*COUNTRY.Continent*) and their respective average city populations (*CITY.Population*) rounded *down* to the nearest integer.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

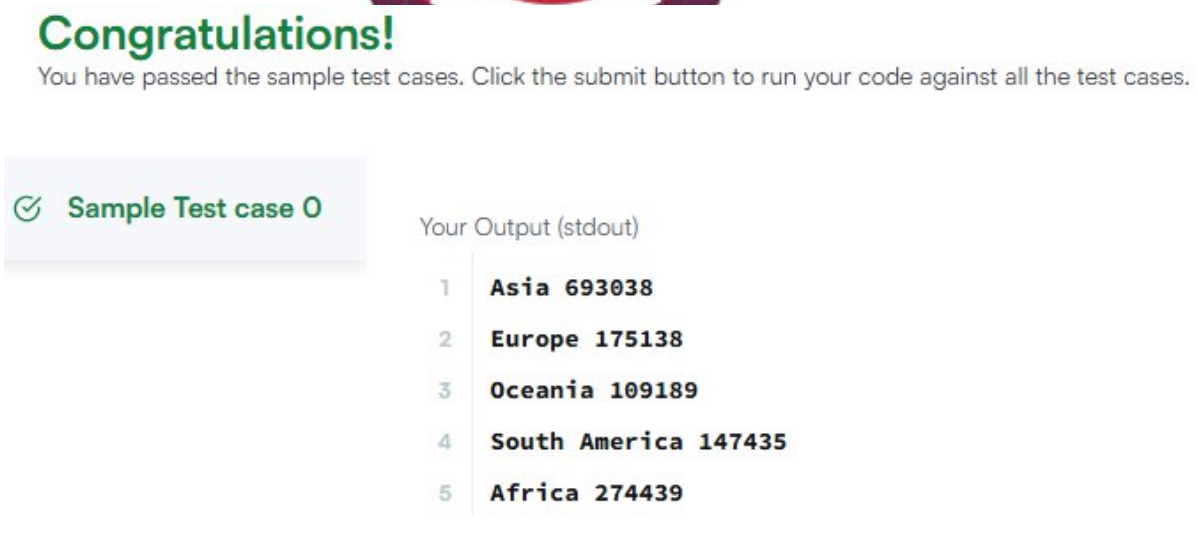
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

PROGRAM:

```
SELECT COUNTRY.Continent,
       FLOOR(AVG(CITY.Population))
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent
```

OUTPUT:



45. You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

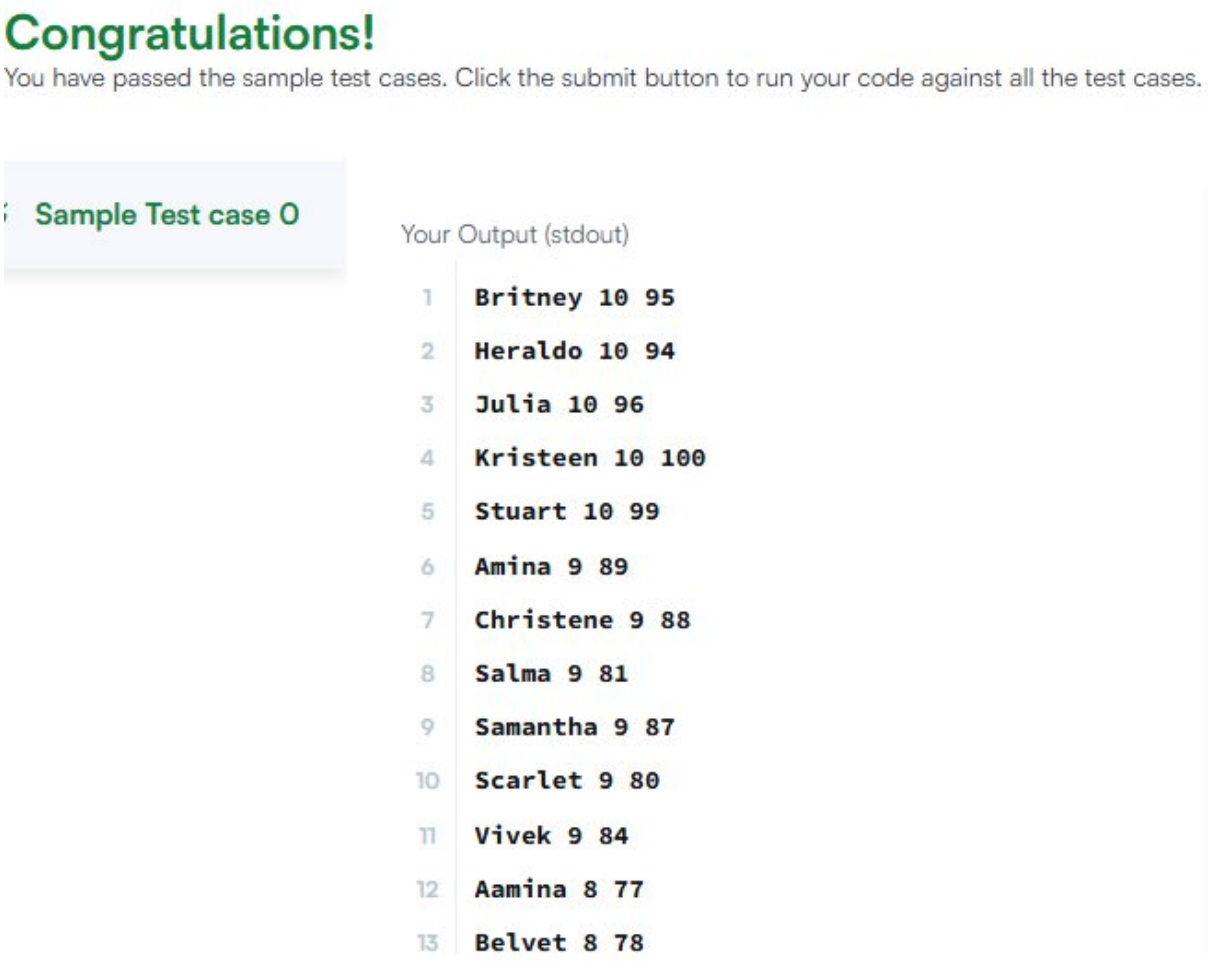
Column	Type
ID	Integer
Name	String
Marks	Integer

PROGRAM:

```
SELECT
    CASE
        WHEN G.Grade < 8 THEN NULL
        ELSE S.Name
    END,
    G.Grade,
    S.Marks
FROM Students S
JOIN Grades G
ON S.Marks BETWEEN G.Min_Mark AND G.Max_Mark
ORDER BY
    G.Grade DESC,
    CASE
        WHEN G.Grade >= 8 THEN S.Name
    END ASC,
    CASE
        WHEN G.Grade < 8 THEN S.Marks
```

END ASC;

OUTPUT:



46. Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

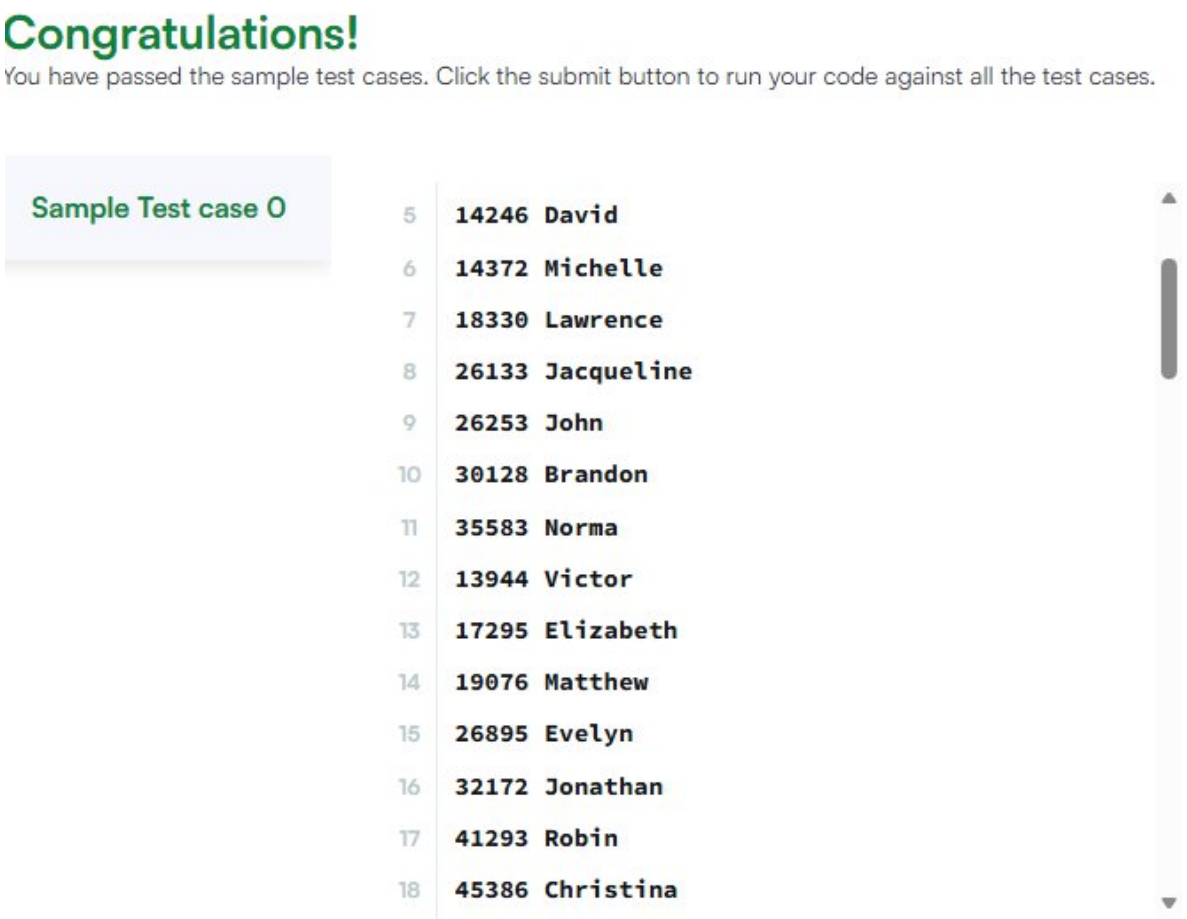
Column	Type
hacker_id	Integer
name	String

culty: The difficult_level is the level of difficulty of the challenge, ar

PROGRAM:

```
SELECT h.hacker_id,
       h.name
FROM Hackers h
JOIN Submissions s
ON h.hacker_id = s.hacker_id
JOIN Challenges c
ON s.challenge_id = c.challenge_id
JOIN Difficulty d
ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(DISTINCT s.challenge_id) > 1
ORDER BY COUNT(DISTINCT s.challenge_id) DESC,
         h.hacker_id ASC;
```

OUTPUT:



47. Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of the wands that Ron's interested in, sorted in order of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand (the higher the power, the better the wand is).

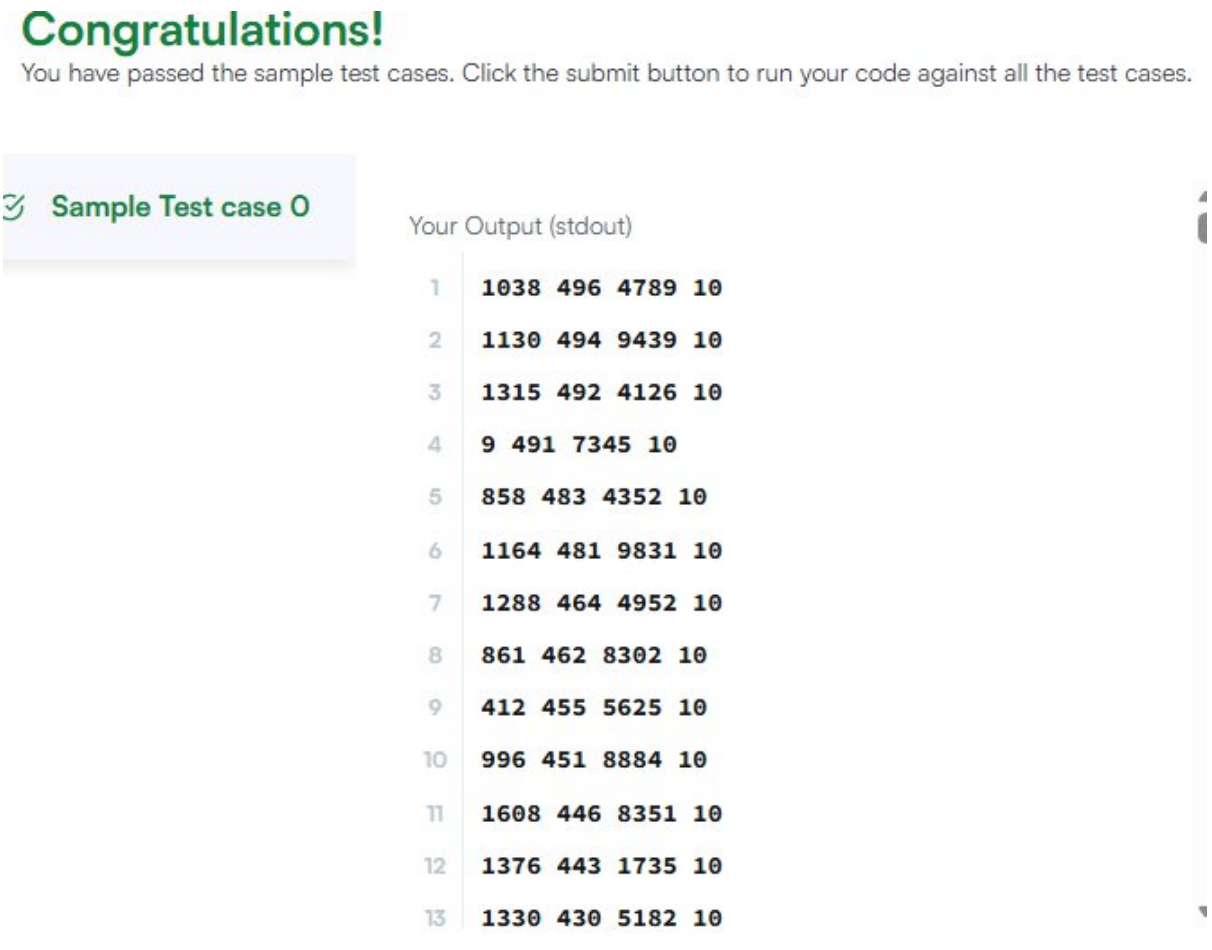
Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

PROGRAM:

```
SELECT w.id,
       wp.age,
       w.coins_needed,
       w.power
FROM Wands w
JOIN Wands_Property wp
ON w.code = wp.code
WHERE wp.is_evil = 0
  AND w.coins_needed = (
      SELECT MIN(w2.coins_needed)
      FROM Wands w2
      JOIN Wands_Property wp2
      ON w2.code = wp2.code
      WHERE wp2.is_evil = 0
        AND w2.power = w.power
```

```
        AND wp2.age = wp.age
  )
ORDER BY w.power DESC,
         wp.age DESC;
```

OUTPUT:



AUTONOMOUS

48. Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

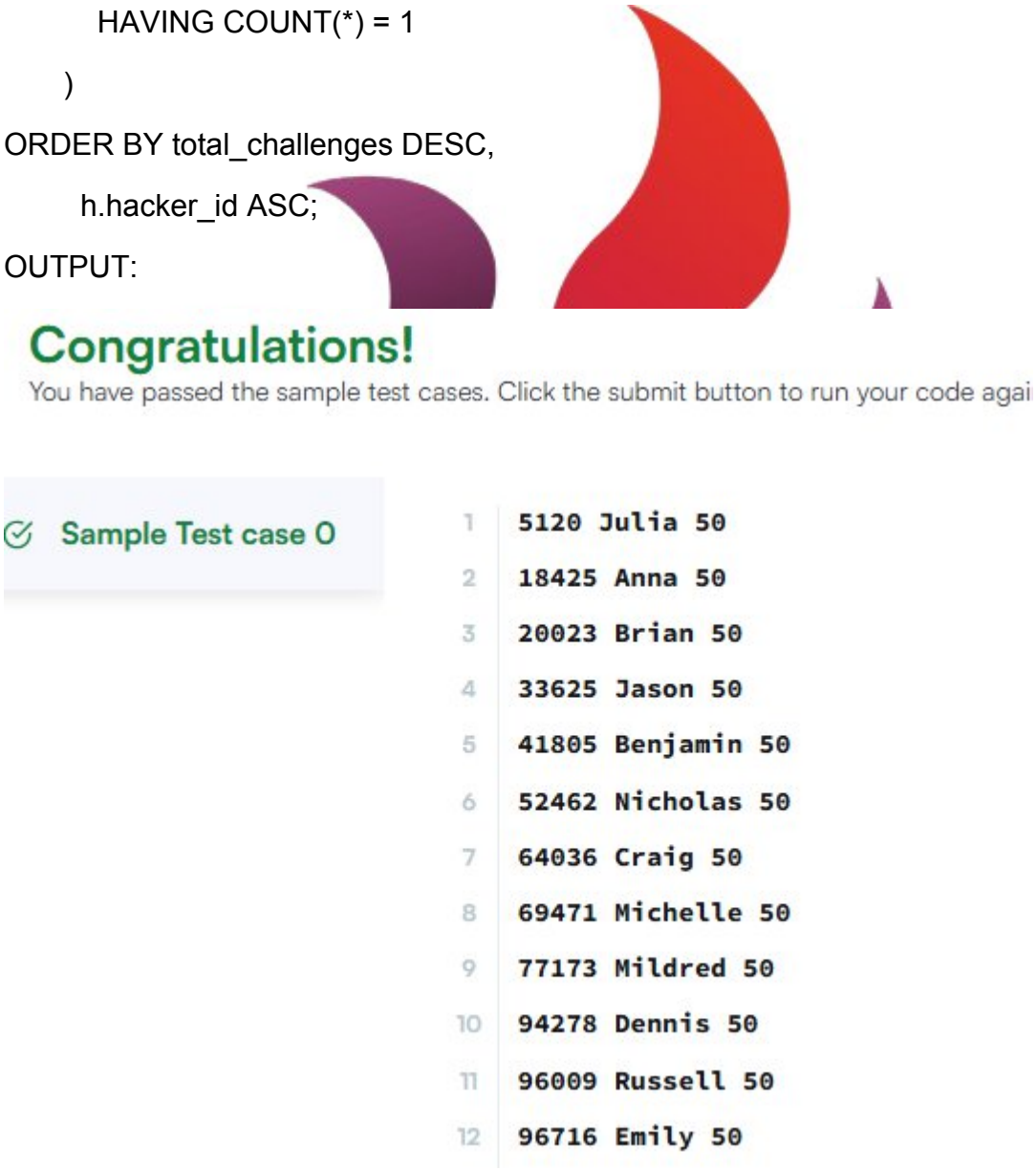
Column	Type
hacker_id	Integer
name	String

PROGRAM:

```
SELECT h.hacker_id,
       h.name,
       COUNT(c.challenge_id) AS total_challenges
FROM Hackers h
JOIN Challenges c
ON h.hacker_id = c.hacker_id
GROUP BY h.hacker_id, h.name
HAVING COUNT(c.challenge_id) = (
          SELECT MAX(cnt)
          FROM (
             SELECT COUNT(*) AS cnt
             FROM Challenges
             GROUP BY hacker_id
          ) t
       )
    OR COUNT(c.challenge_id) IN (
          SELECT cnt
```

```
        FROM (
            SELECT COUNT(*) AS cnt
            FROM Challenges
            GROUP BY hacker_id
        ) t
        GROUP BY cnt
        HAVING COUNT(*) = 1
    )
ORDER BY total_challenges DESC,
        h.hacker_id ASC;
```

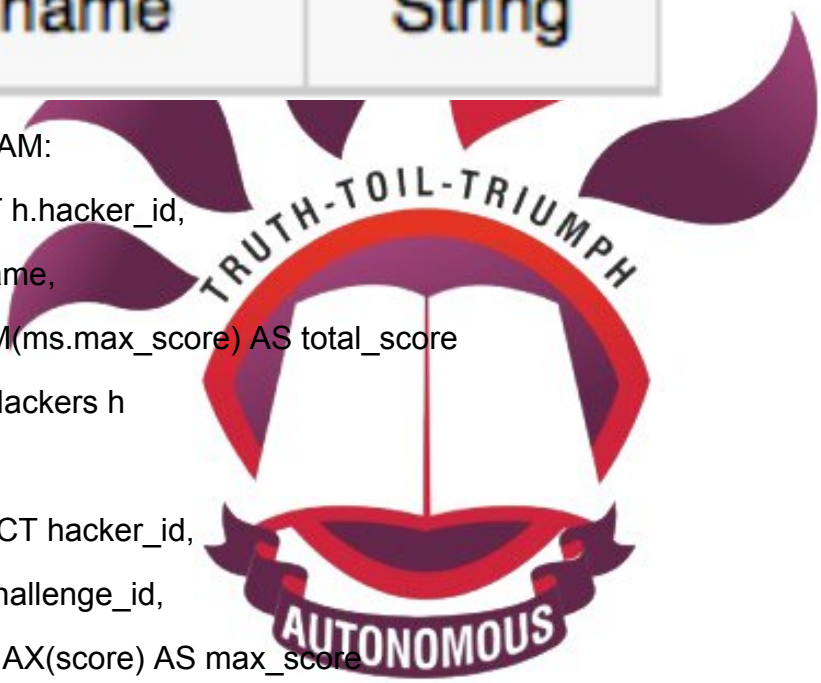
OUTPUT:



49. You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of

Column	Type
hacker_id	Integer
name	String

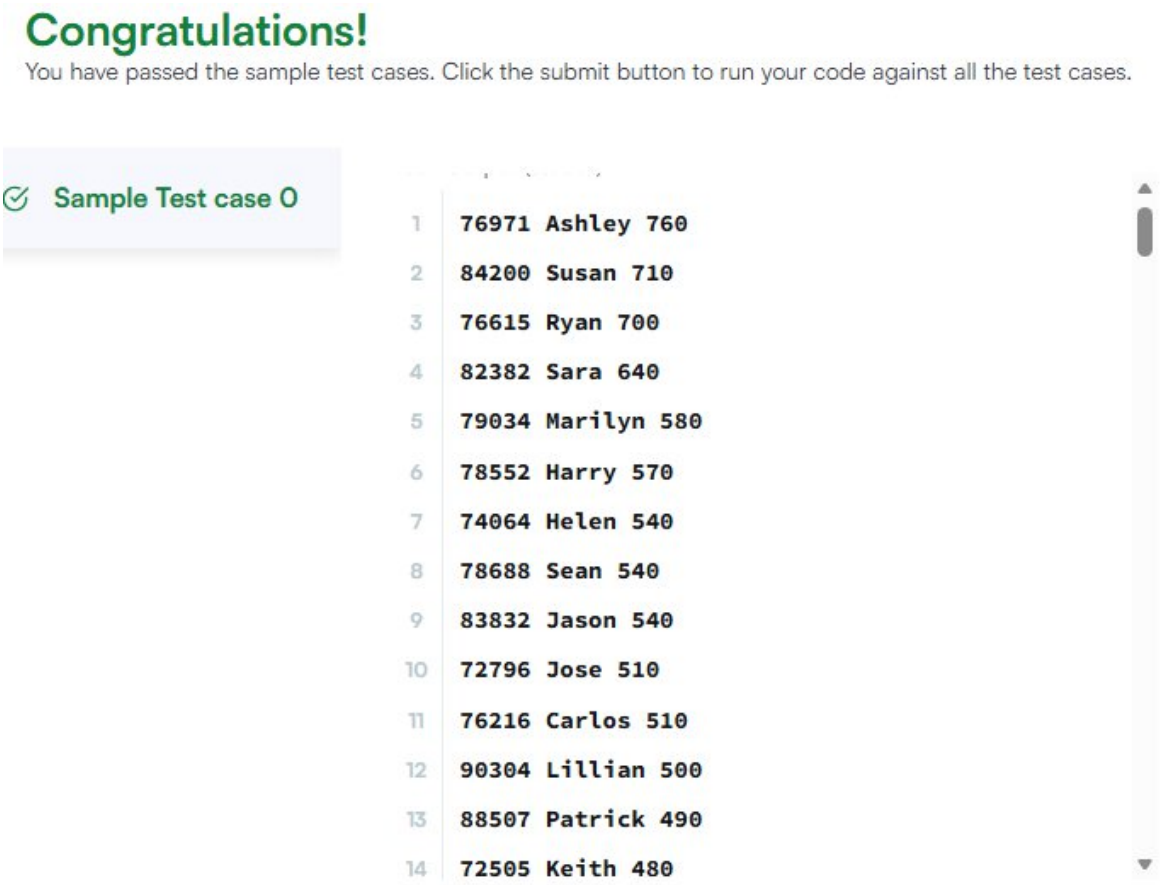


PROGRAM:

```
SELECT h.hacker_id,
       h.name,
       SUM(ms.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT hacker_id,
           challenge_id,
           MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) ms
ON h.hacker_id = ms.hacker_id
GROUP BY h.hacker_id, h.name
HAVING SUM(ms.max_score) > 0
ORDER BY total_score DESC,
```

h.hacker_id ASC;

OUTPUT:



50. ou are given a table, *Projects*, containing three columns: *Task_ID*, *Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	Type
Task_ID	Integer
Start_Date	Date
End_Date	Date

f the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

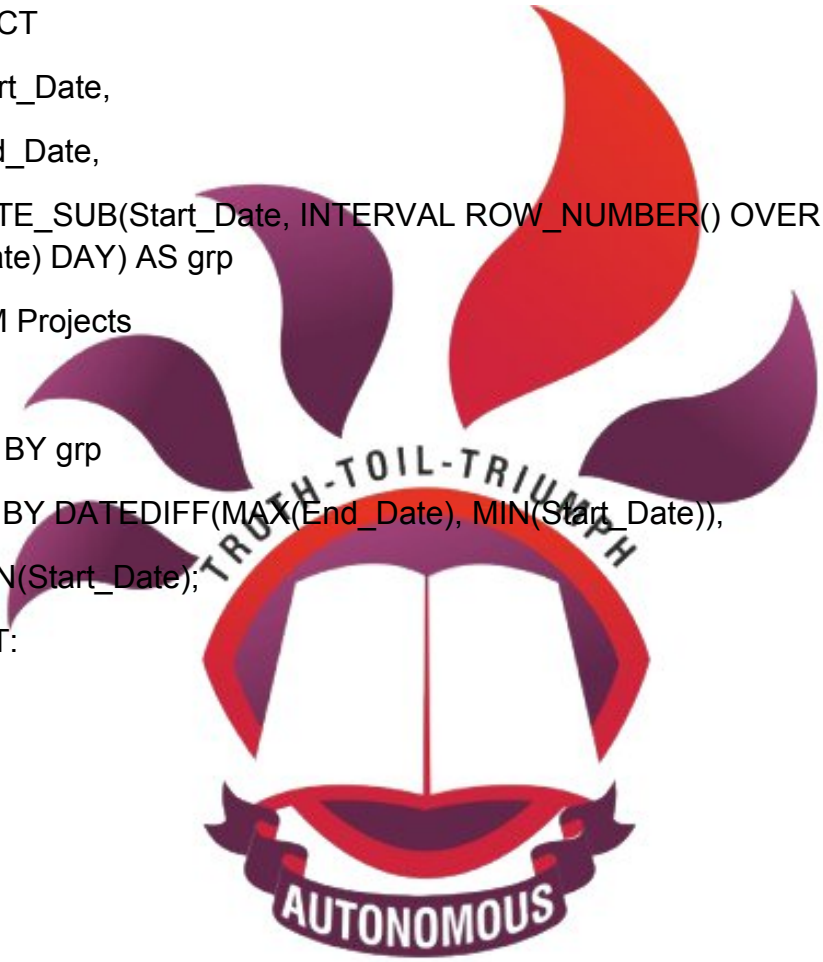
HACKERANK

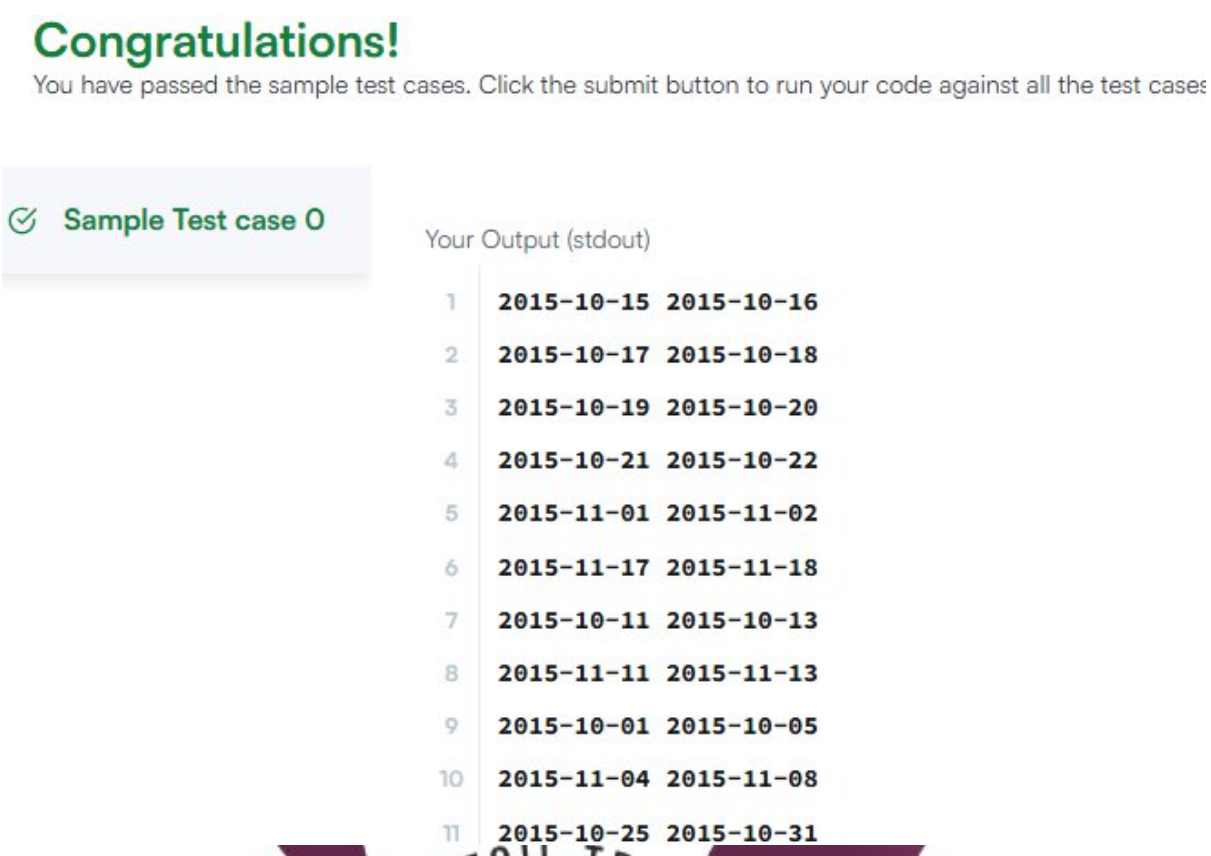
Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project

PROGRAM:

```
FROM
(
    SELECT
        Start_Date,
        End_Date,
        DATE_SUB(Start_Date, INTERVAL ROW_NUMBER() OVER (ORDER BY Start_Date) DAY) AS grp
    FROM Projects
) t
GROUP BY grp
ORDER BY DATEDIFF(MAX(End_Date), MIN(Start_Date)),
         MIN(Start_Date);
```

OUTPUT:





51. You are given three tables: *Students*, *Friends* and *Packages*. *Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month).

Column	*Type*
ID	*Integer*
Name	*String*

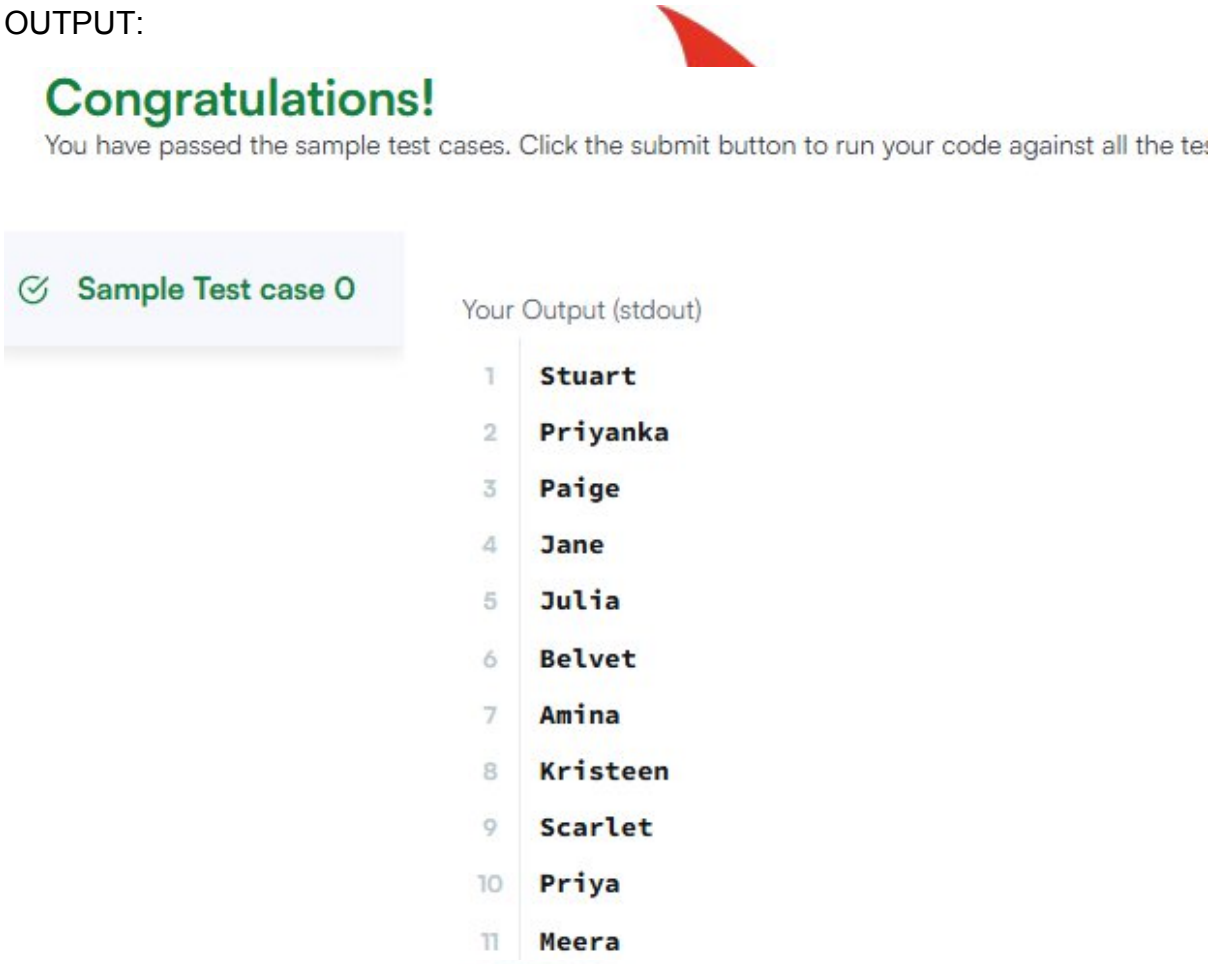
Students

PROGRAM:

```
SELECT s.Name
FROM Students s
JOIN Friends f
    ON s.ID = f.ID
```

```
JOIN Packages p1
    ON s.ID = p1.ID
JOIN Packages p2
    ON f.Friend_ID = p2.ID
WHERE p2.Salary > p1.Salary
ORDER BY p2.Salary;
```

OUTPUT:



52. ou are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	*Type*
X	*Integer*
Y	*Integer*

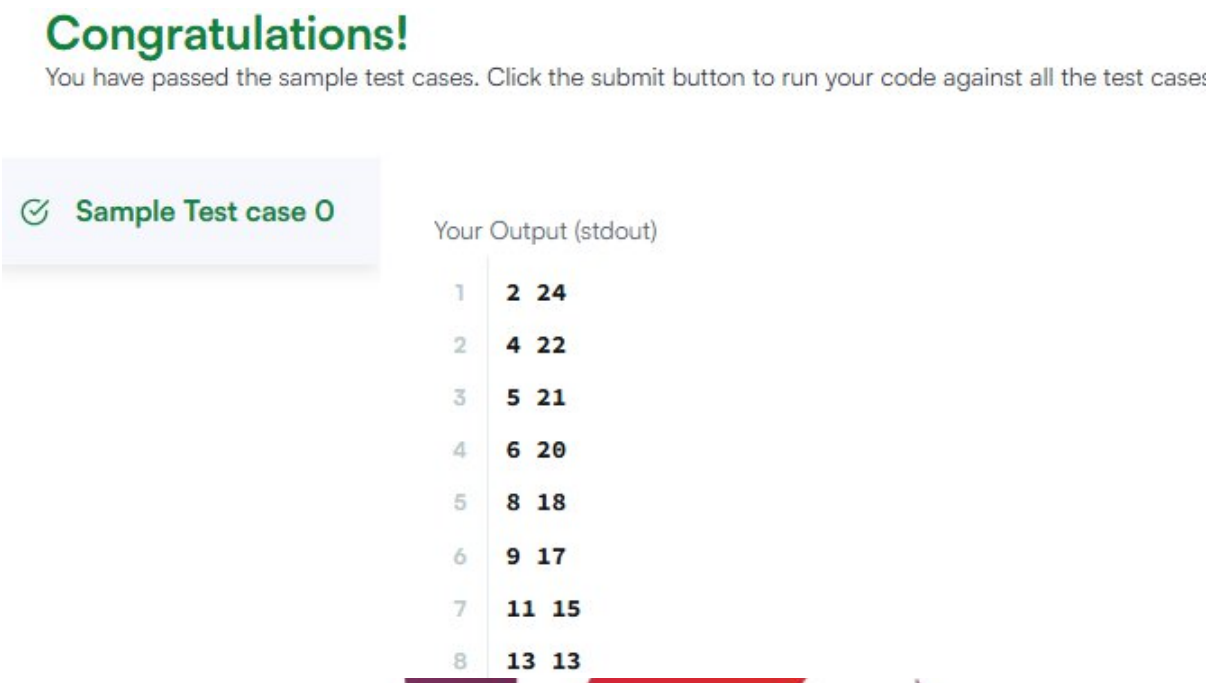
Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of X. List the rows such that $X_1 \leq Y_1$.

PROGRAM:

```
SELECT DISTINCT
    f1.X,
    f1.Y
FROM Functions f1
JOIN Functions f2
    ON f1.X = f2.Y
   AND f1.Y = f2.X
WHERE
    f1.X < f1.Y
    OR (
        f1.X = f1.Y
        AND (
            SELECT COUNT(*)
            FROM Functions f3
            WHERE f3.X = f1.X
              AND f3.Y = f1.Y
        ) > 1
    )
ORDER BY f1.X;
```

OUTPUT:



53. ulia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least

Column	Type
hacker_id	Integer
name	String

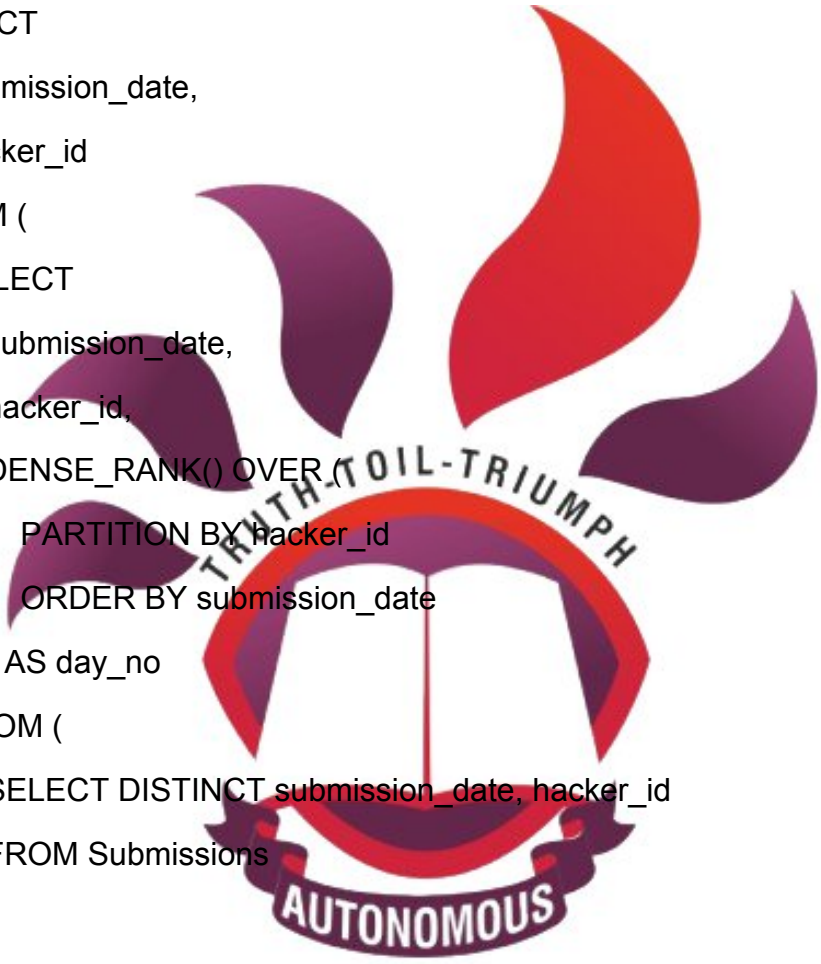
PROGRAM:

```
WITH daily_submissions AS (
    SELECT
        submission_date,
        hacker_id,
```

```
        COUNT(*) AS total_submissions
    FROM Submissions
    GROUP BY submission_date, hacker_id
),

continuous_hackers AS (
    SELECT
        submission_date,
        hacker_id
    FROM (
        SELECT
            submission_date,
            hacker_id,
            DENSE_RANK() OVER (
                PARTITION BY hacker_id
                ORDER BY submission_date
            ) AS day_no
        FROM (
            SELECT DISTINCT submission_date, hacker_id
            FROM Submissions
        ) s
    ) x
    WHERE DATEDIFF(submission_date, '2016-03-01') + 1 = day_no
),

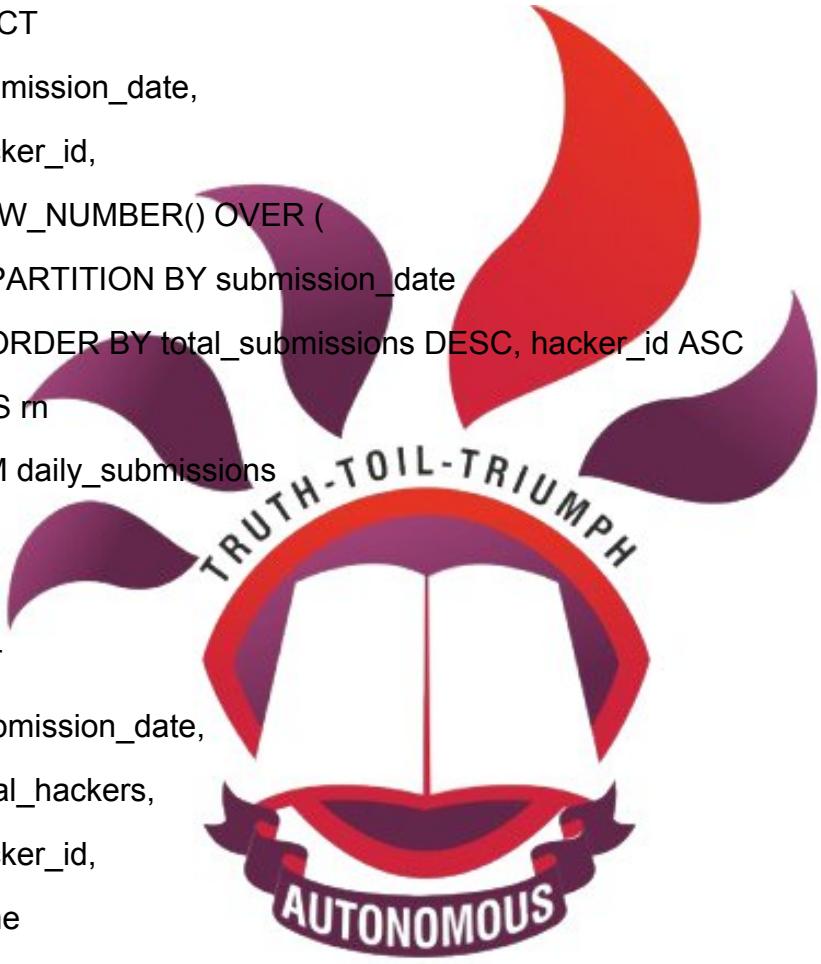
daily_count AS (
    SELECT
        submission_date,
```



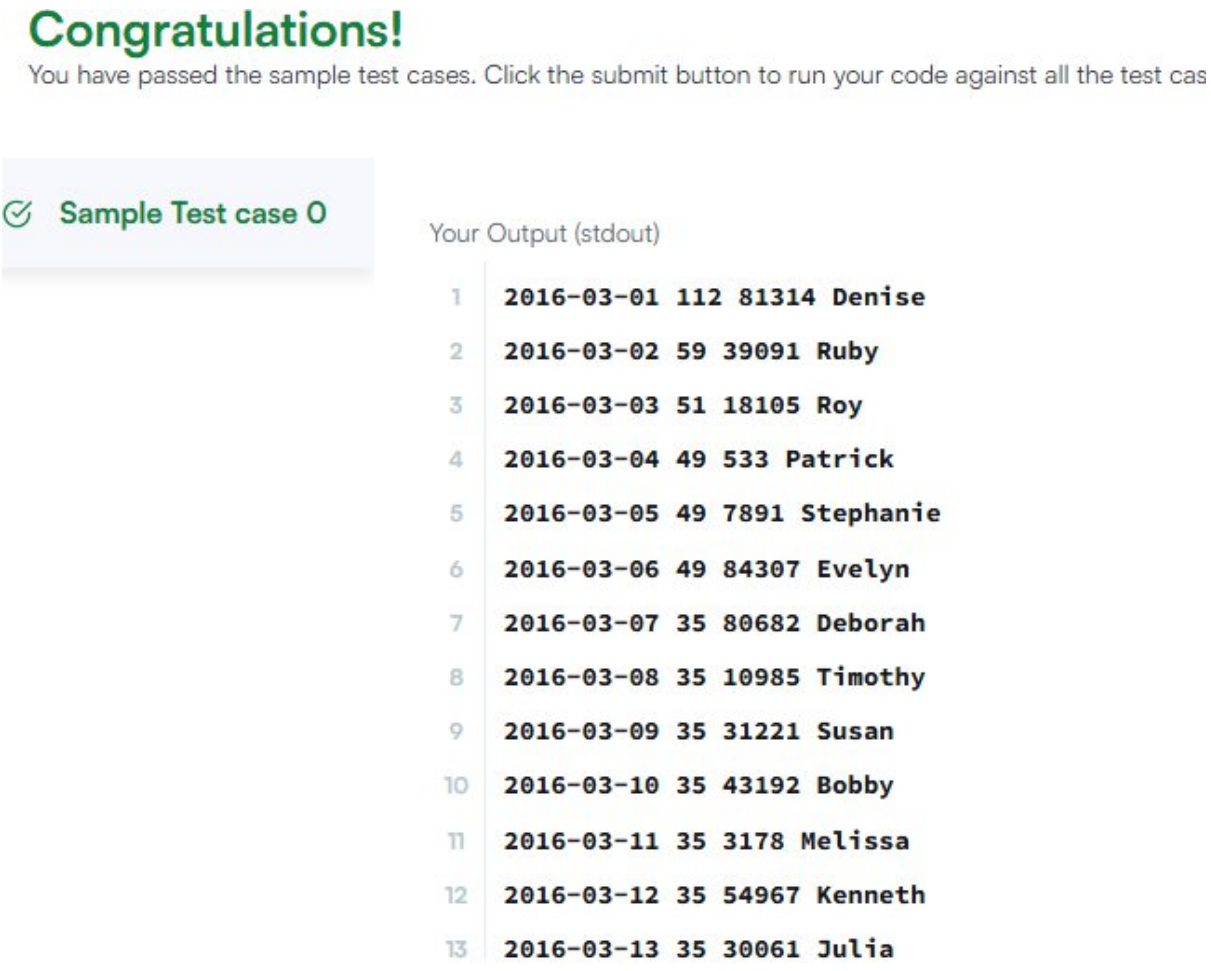
```
        COUNT(DISTINCT hacker_id) AS total_hackers
    FROM continuous_hackers
    GROUP BY submission_date
),

top_hacker AS (
    SELECT
        submission_date,
        hacker_id,
        ROW_NUMBER() OVER (
            PARTITION BY submission_date
            ORDER BY total_submissions DESC, hacker_id ASC
        ) AS rn
    FROM daily_submissions
)

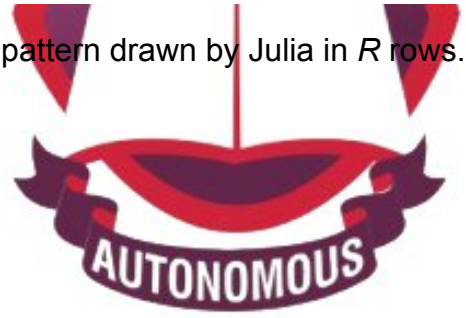
SELECT
    dc.submission_date,
    dc.total_hackers,
    th.hacker_id,
    h.name
FROM daily_count dc
JOIN top_hacker th
    ON dc.submission_date = th.submission_date
   AND th.rn = 1
JOIN Hackers h
    ON th.hacker_id = h.hacker_id
ORDER BY dc.submission_date;
```



OUTPUT:



54. *P(R)* represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:




```
* * * * *
* * * *
* * *
* *
*
```

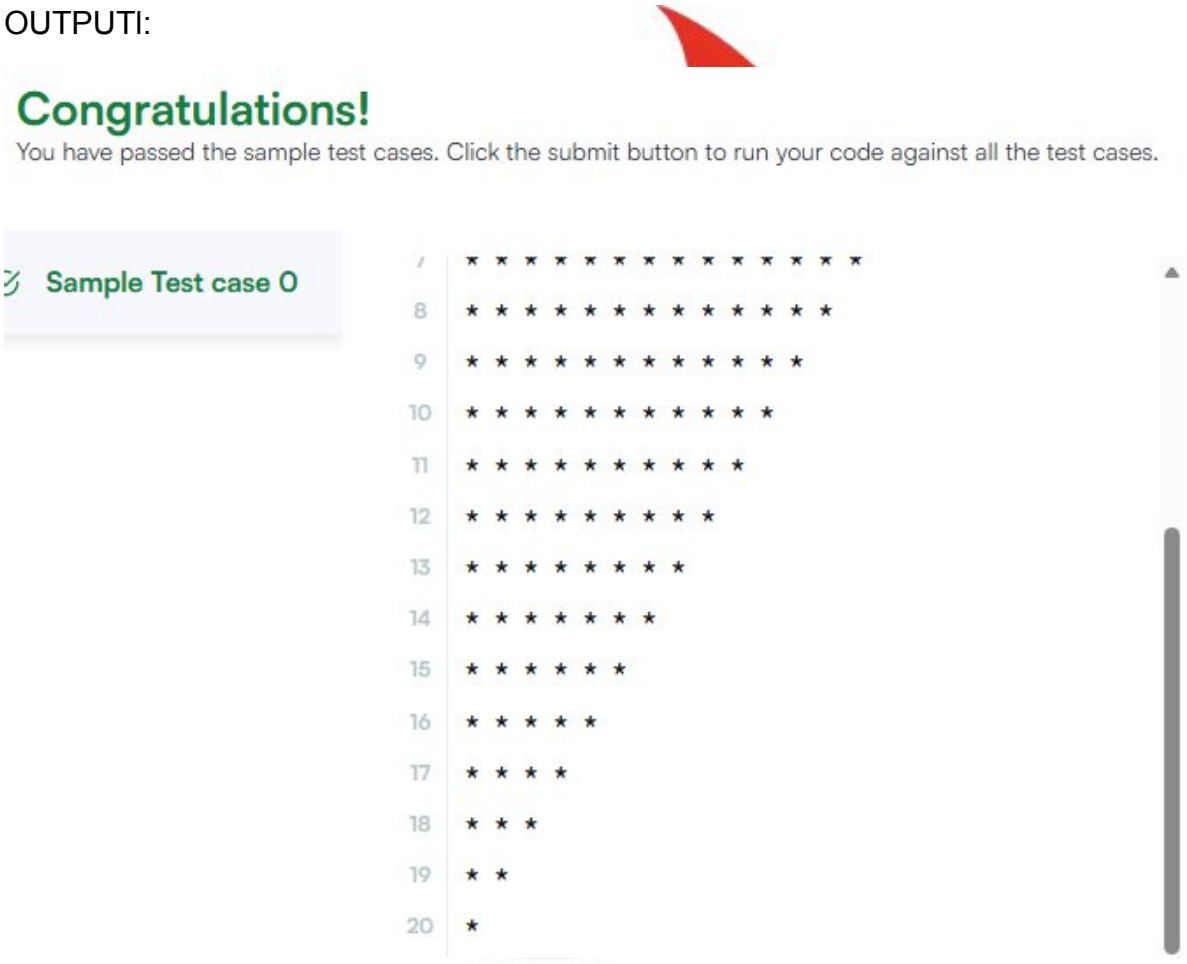
Write a query to print the pattern *P(20)*.

PROGRAM:

```
ITH RECURSIVE pattern AS (
    SELECT 20 AS n
    UNION ALL
```

```
    SELECT n - 1
    FROM pattern
    WHERE n > 1
)
SELECT REPEAT('* ', n) AS pattern
FROM pattern;
```

OUTPUTI:



55. *P(R)* represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

```
*
* *
* * *
* * * *
* * * * *
```

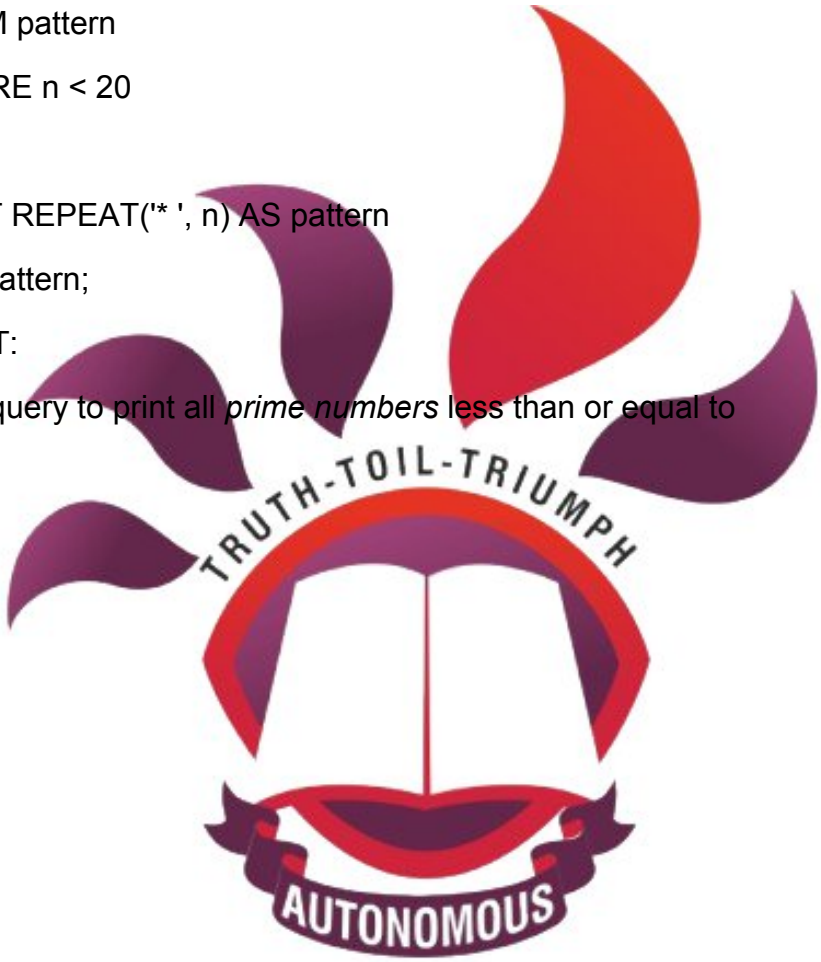
Write a query to print the pattern *P(20)*.

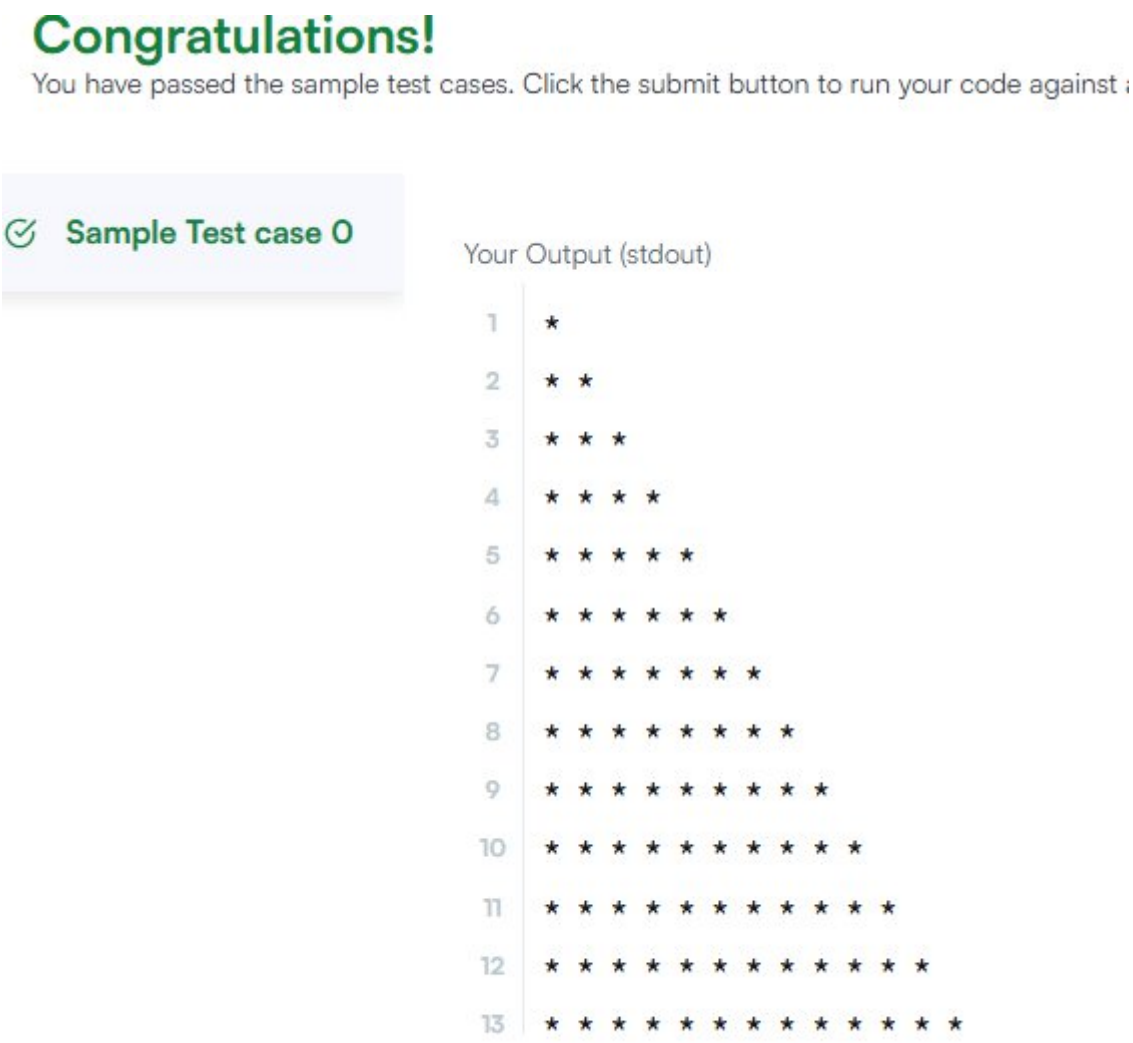
PROGRAM:

```
ITH RECURSIVE pattern AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1
    FROM pattern
    WHERE n < 20
)
SELECT REPEAT('* ', n) AS pattern
FROM pattern;
```

OUTPUT:

Write a query to print all *prime numbers* less than or equal to





56. Write a query to print all *prime numbers* less than or equal to

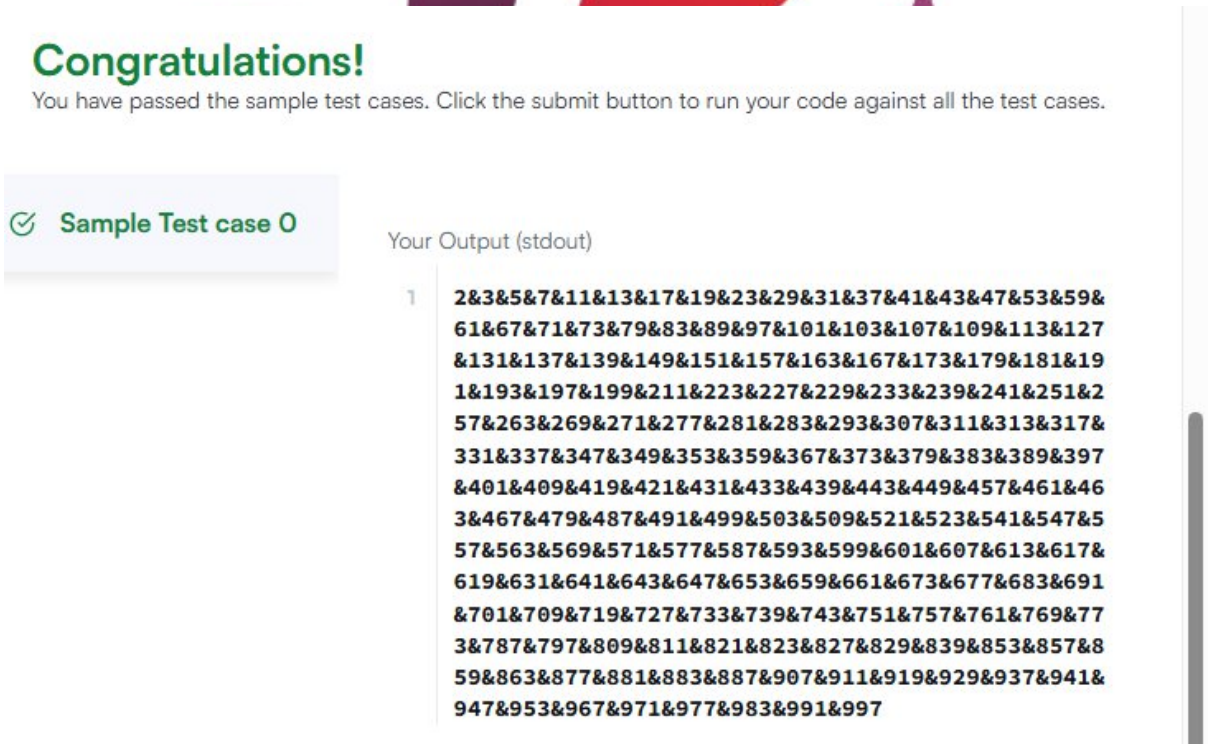
Print your result on a single line, and use the ampersand For example, the output for all prime numbers 2&3&5&7

PROGRAM:

```
WITH RECURSIVE nums AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n + 1
    FROM nums
    WHERE n < 1000
```

```
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
FROM nums
WHERE NOT EXISTS (
    SELECT 1
    FROM nums d
    WHERE d.n < nums.n
      AND d.n > 1
      AND d.n <= SQRT(nums.n)
      AND MOD(nums.n, d.n) = 0
```

OUTPUT:



HACKERANK



KHAN ACADEMY

1. ;/;Here's a table containing a TODO list with the number of minutes it will take to cop;mplete each item. Insert another item to your todo list with the estimated min’utes it will take.

PROGRAM:

```
    CREATE TABLE store (
  id INTEGER,
  item_name TEXT,
  category TEXT,
  price INTEGER,
  quantity INTEGER
);

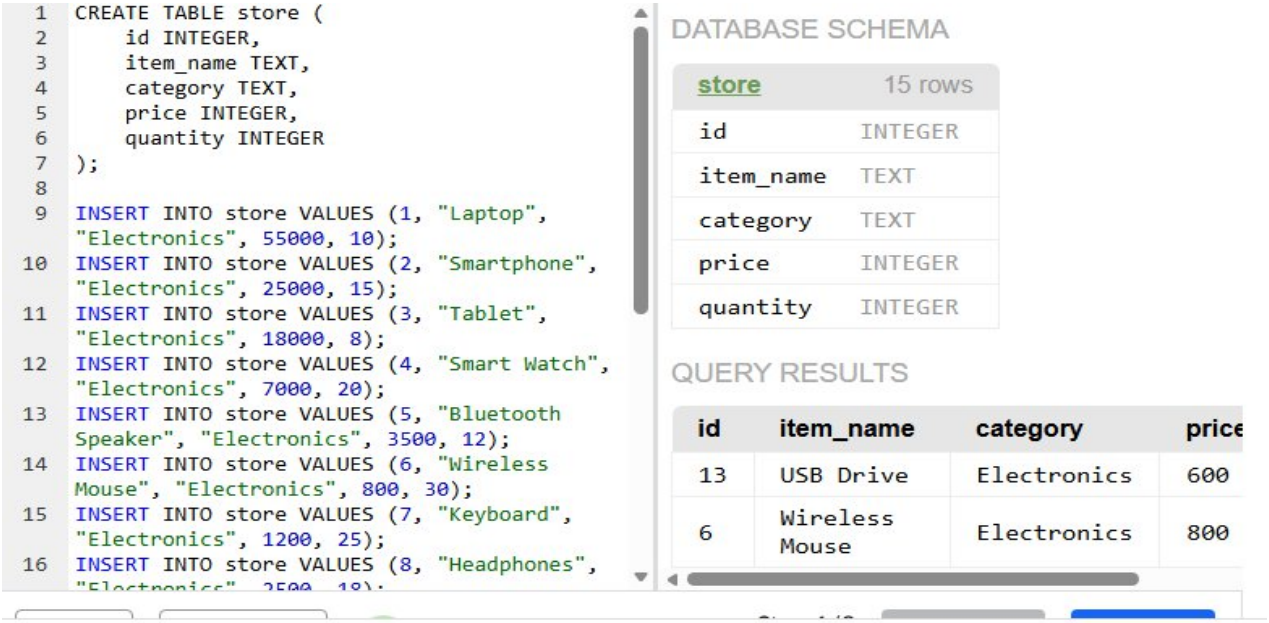
INSERT INTO store VALUES (1, "Laptop", "Electronics", 55000, 10);
INSERT INTO store VALUES (2, "Smartphone", "Electronics", 25000, 15);
INSERT INTO store VALUES (3, "Tablet", "Electronics", 18000, 8);
INSERT INTO store VALUES (4, "Smart Watch", "Electronics", 7000, 20);
INSERT INTO store VALUES (5, "Bluetooth Speaker", "Electronics", 3500, 12);
INSERT INTO store VALUES (6, "Wireless Mouse", "Electronics", 800, 30);
INSERT INTO store VALUES (7, "Keyboard", "Electronics", 1200, 25);
INSERT INTO store VALUES (8, "Headphones", "Electronics", 2500, 18);
INSERT INTO store VALUES (9, "Monitor", "Electronics", 12000, 7);
INSERT INTO store VALUES (10, "Printer", "Electronics", 9000, 5);
INSERT INTO store VALUES (11, "Webcam", "Electronics", 2200, 10);
INSERT INTO store VALUES (12, "Power Bank", "Electronics", 1500, 16);
INSERT INTO store VALUES (13, "USB Drive", "Electronics", 600, 40);
INSERT INTO store VALUES (14, "External Hard Disk", "Electronics", 5000, 9);
INSERT INTO store VALUES (15, "Router", "Electronics", 3000, 11);

SELECT * FROM store
ORDER BY price;

SELECT AVG(price) AS average_price
FROM store;
```

OUTPUT:

KHAN ACADEMY



2. Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

PROGRAM::

```
CREATE TABLE store (
    id INTEGER,
    item_name TEXT,
    category TEXT,
    price INTEGER,
    quantity INTEGER
);

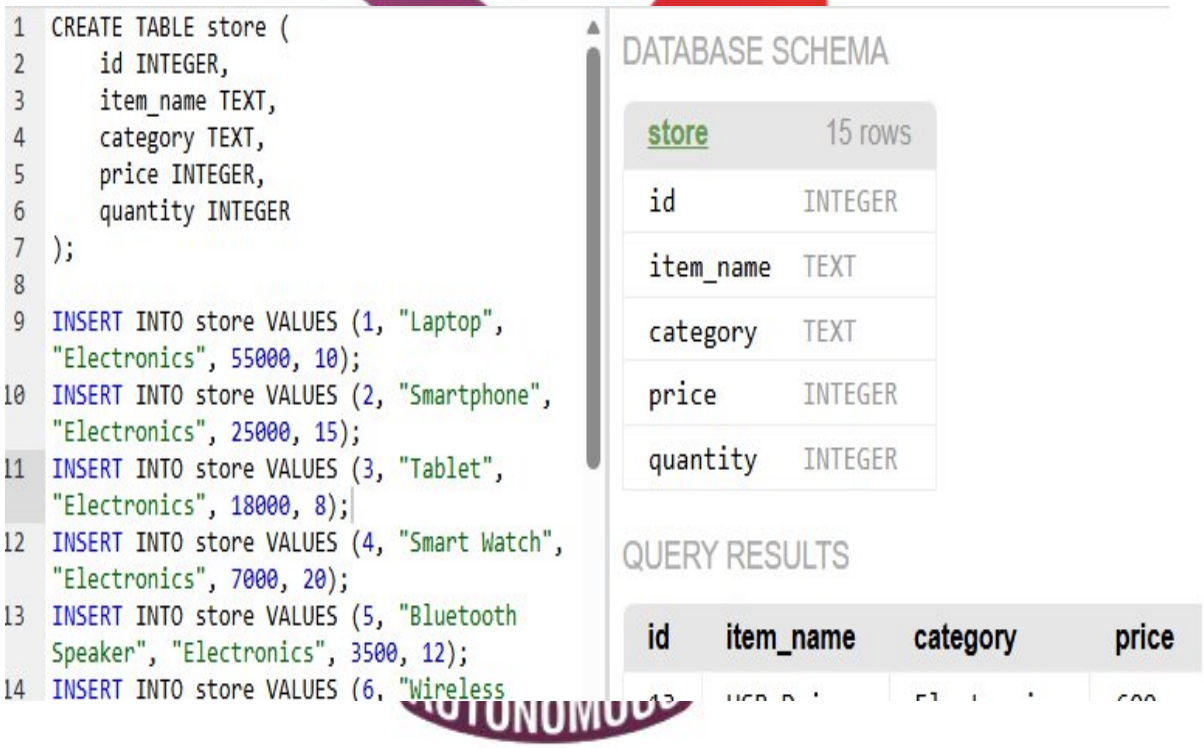
INSERT INTO store VALUES (1, "Laptop", "Electronics", 55000, 10);
INSERT INTO store VALUES (2, "Smartphone", "Electronics", 25000, 15);
INSERT INTO store VALUES (3, "Tablet", "Electronics", 18000, 8);
INSERT INTO store VALUES (4, "Smart Watch", "Electronics", 7000, 20);
INSERT INTO store VALUES (5, "Bluetooth Speaker", "Electronics", 3500, 12);
INSERT INTO store VALUES (6, "Wireless Mouse", "Electronics", 800, 30);
INSERT INTO store VALUES (7, "Keyboard", "Electronics", 1200, 25);
INSERT INTO store VALUES (8, "Headphones", "Electronics", 2500, 18);
INSERT INTO store VALUES (9, "Monitor", "Electronics", 12000, 7);
INSERT INTO store VALUES (10, "Printer", "Electronics", 9000, 5);
```

```
INSERT INTO store VALUES (11, "Webcam", "Electronics", 2200, 10);
INSERT INTO store VALUES (12, "Power Bank", "Electronics", 1500, 16);
INSERT INTO store VALUES (13, "USB Drive", "Electronics", 600, 40);
INSERT INTO store VALUES (14, "External Hard Disk", "Electronics", 5000, 9);
INSERT INTO store VALUES (15, "Router", "Electronics", 3000, 11);

SELECT * FROM store
ORDER BY price;

SELECT AVG(price) AS average_price
FROM store;
```

OUTPUT:



3. Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles.

PROGRAM:

```
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
```

```
    mood TEXT,
    duration INTEGER,
    released INTEGER);

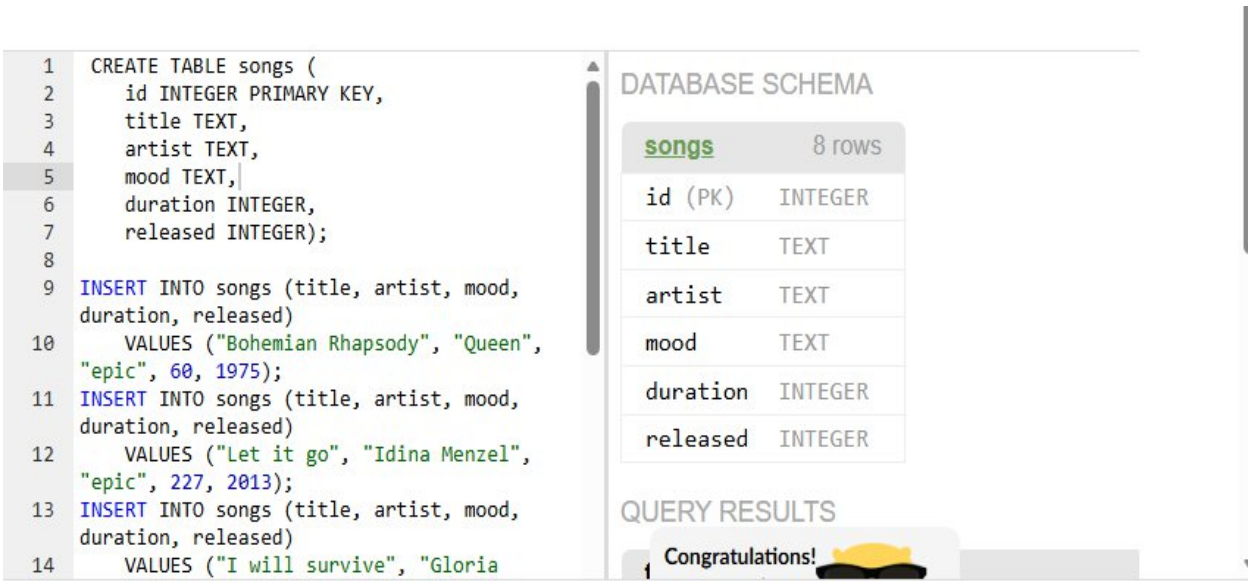

INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);
    SELECT title
FROM songs;
SELECT title
FROM songs
WHERE mood = 'epic'
   OR released > 1990;
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

KHAN ACADEMY

SELECT title

OUTPUT:



4. We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the title of all the songs by the artist named 'Queen'.

PROGRAM:

CREATE TABLE artists (

id INTEGER PRIMARY KEY AUTOINCREMENT,

name TEXT,

country TEXT,

genre TEXT);

INSERT INTO artists (name, country, genre)

VALUES ("Taylor Swift", "US", "Pop");

INSERT INTO artists (name, country, genre)

VALUES ("Led Zeppelin", "US", "Hard rock");

INSERT INTO artists (name, country, genre)

VALUES ("ABBA", "Sweden", "Disco");

INSERT INTO artists (name, country, genre)

```
    VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Guns N' Roses", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Gloria Estefan", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Bob Marley", "Jamaica", "Reggae");


CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT,
    title TEXT);

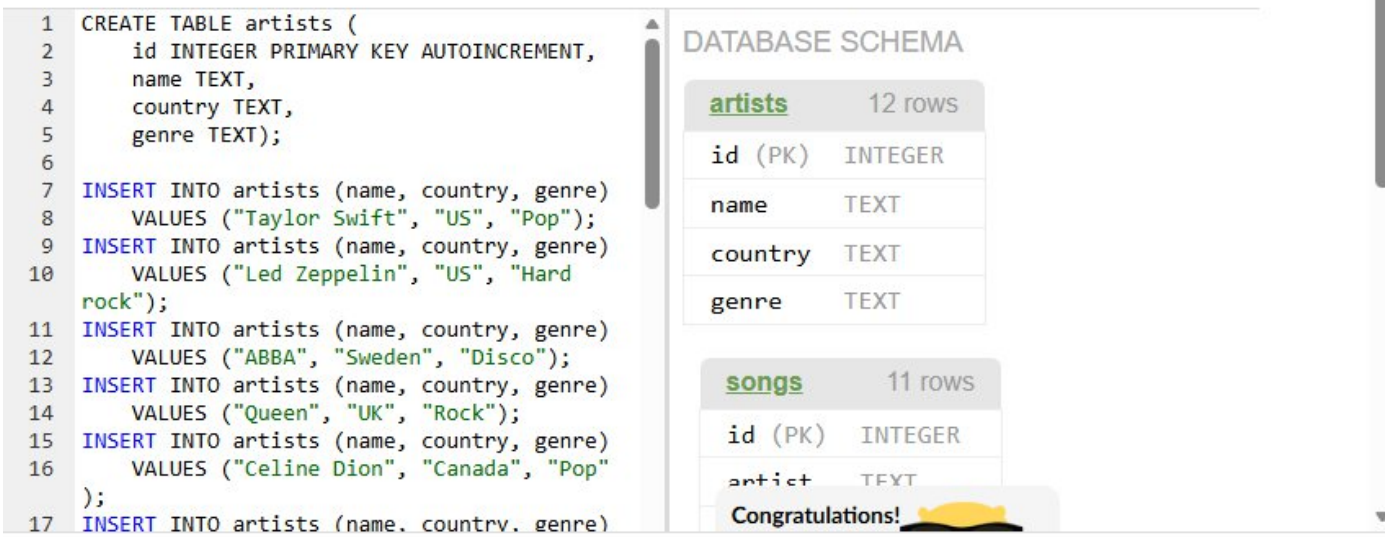

INSERT INTO songs (artist, title)
    VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
```

```
    VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
    VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
    VALUES ("Guns N' Roses", "Don't cry");
    SELECT title
FROM songs
WHERE artist = 'Queen';
SELECT name
FROM artists
WHERE genre = 'Pop';
SELECT title
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

```
FROM songs
WHERE artist IN (
    SELECT name
    FROM artists
    WHERE genre = 'Pop'
);
```

OUTPUT:



5. We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column.

PROGRAM:

```
    CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER);
```

```
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);

INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);

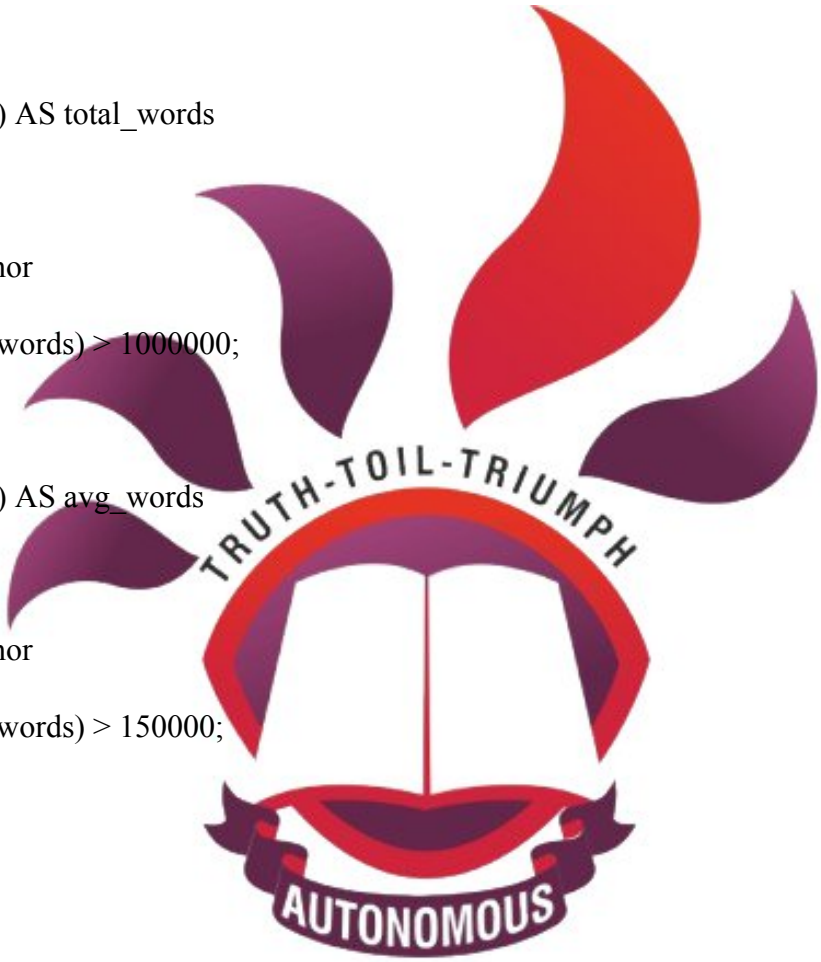
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

KHAN ACADEMY

```
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Return of the King", 134462);
SELECT author,
       SUM(words) AS total_words
FROM books
GROUP BY author
HAVING SUM(words) > 1000000;
SELECT author,
       AVG(words) AS avg_words
FROM books
GROUP BY author
HAVING AVG(words) > 150000;
```

OUTPUT:



KHAN ACADEMY

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER);

INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter
and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter
and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter
and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter
and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter
and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter
```

DATABASE SCHEMA

books	15 rows
id (PK)	INTEGER
author	TEXT
title	TEXT
words	INTEGER

QUERY RESULTS

author	total_words
J.K. Rowling	1086594

author	avg_words
J.K. Rowling	155227.7142857143

Step 1/2

6. We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column.

PROGRCREATE TABLE student_grades (

id INTEGER PRIMARY KEY AUTOINCREMENT,

name TEXT,

number_grade INTEGER,

fraction_completed REAL);

INSERT INTO student_grades (name, number_grade, fraction_completed)

VALUES ("Winston", 90, 0.805);

INSERT INTO student_grades (name, number_grade, fraction_completed)

VALUES ("Winnefer", 95, 0.901);

INSERT INTO student_grades (name, number_grade, fraction_completed)

VALUES ("Winsteen", 85, 0.906);

KHAN ACADEMY

```
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winstonia", 82, 0.9045);
SELECT
   name,
   number_grade,
   ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;

/* Step 2 Query */
SELECT COUNT(*),
   CASE
      WHEN number_grade > 90 THEN "A"
      WHEN number_grade > 80 THEN "B"
      WHEN number_grade > 70 THEN "C"
      ELSE "F"
   END AS "letter_grade"
FROM student_grades
GROUP BY letter_grade;AM:
```

OUTPUT:

KHAN ACADEMY

```
CREATE TABLE student_grades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    number_grade INTEGER,
    fraction_completed REAL);

INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
```

DATABASE SCHEMA

student_grades	6 rows
id (PK)	INTEGER
name	TEXT
number_grade	INTEGER
fraction_completed	REAL

QUERY RESULTS

name	number_grade	percent_cc
Winston	90	81
Winnefer	95	90
Winsteen	85	91
Wincifer	66	71

7. Pick one of those data sets or create a data set like that, and use advanced SELECT queries to discover things about the data. What sort of questions might one have about that data, like if they were using it for an app or a business idea? Here are some ideas:

- What are average, max, and min values in the data?
- What about those numbers per category in the data (using HAVING)?
- What ways are there to group the data values that don't exist yet (using CASE)?
    - What interesting ways are there to filter the data (using AND/OR)?

PROGRAM:

```
CREATE TABLE pokemon (
id INTEGER PRIMARY KEY,
name TEXT,
type TEXT,
hp INTEGER,
attack INTEGER,
defense INTEGER,
speed INTEGER
```

```
);

INSERT INTO pokemon VALUES
(1,'Bulbasaur','Grass',45,49,49,45),
(2,'Ivysaur','Grass',60,62,63,60),
(3,'Venusaur','Grass',80,82,83,80),
(4,'Charmander','Fire',39,52,43,65),
(5,'Charmeleon','Fire',58,64,58,80),
(6,'Charizard','Fire',78,84,78,100),
(7,'Squirtle','Water',44,48,65,43),
(8,'Wartortle','Water',59,63,80,58),
(9,'Blastoise','Water',79,83,100,78),
(10,'Pikachu','Electric',35,55,40,90);

-- 1. Average, Maximum, and Minimum Attack
SELECT AVG(attack) AS Avg_Attack,
       MAX(attack) AS Max_Attack,
       MIN(attack) AS Min_Attack
FROM pokemon;

-- 2. Average HP by Type
SELECT type, AVG(hp) AS Avg_HP
FROM pokemon
GROUP BY type;

-- 3. Types with Average Attack Greater Than 60
```

```
SELECT type, AVG(attack) AS Avg_Attack
FROM pokemon
GROUP BY type
HAVING AVG(attack) > 60;

-- 4. Categorize Pokémon by Speed
SELECT name, speed,
CASE
    WHEN speed >= 90 THEN 'Very Fast'
    WHEN speed >= 70 THEN 'Fast'
    WHEN speed >= 50 THEN 'Medium'
    ELSE 'Slow'
END AS Speed_Category
FROM pokemon;

-- 5. High Attack and High Speed
SELECT *
FROM pokemon
WHERE attack >= 80 AND speed >= 80;

-- 6. Fire or Electric Pokémon
SELECT *
FROM pokemon
WHERE type = 'Fire' OR type = 'Electric';

-- 7. Strongest Attack by Type
```



```
SELECT type, MAX(attack) AS Strongest_Attack

FROM pokemon

GROUP BY type;


-- 8. Count Pokémon by Type

SELECT type, COUNT(*) AS Total_Pokemon

FROM pokemon

GROUP BY type;


-- 9. Pokémon with High Defense

SELECT name, defense

FROM pokemon

WHERE defense > 75;


-- 10. Total Power Score

SELECT name,

(hp + attack + defense + speed) AS Total_Stats

FROM pokemon

ORDER BY Total_Stats DESC;
```

OUTPUT:

KHAN ACADEMY

```
CREATE TABLE pokemon (
    id INTEGER PRIMARY KEY,
    name TEXT,
    type TEXT,
    hp INTEGER,
    attack INTEGER,
    defense INTEGER,
    speed INTEGER
);

INSERT INTO pokemon VALUES
(1,'Bulbasaur','Grass',45,49,49,45),
(2,'Ivysaur','Grass',60,62,63,60),
(3,'Venusaur','Grass',80,82,83,80),
(4,'Charmander','Fire',39,52,43,65),
(5,'Charmeleon','Fire',58,64,58,80),
(6,'Charizard','Fire',78,84,78,100),
(7,'Squirtle','Water',44,48,65,43),
(8,'Wartortle','Water',59,63,80,58),
(9,'Blastoise','Water',79,83,100,78),
(10,'Pikachu','Electric',35,55,40,90);
```

DATABASE SCHEMA

pokemon	10 rows
id (PK)	INTEGER
name	TEXT
type	TEXT
hp	INTEGER
attack	INTEGER
defense	INTEGER
speed	INTEGER

QUERY RESULTS

Avg_Attack	Max_Attack	Min_Attack

7. We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person.

PROGRAM:

CREATE TABLE persons (

id INTEGER PRIMARY KEY AUTOINCREMENT,

name TEXT,

age INTEGER);

INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);

INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);

INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);

INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);

INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);

CREATE table hobbies (

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);


INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age)
VALUES ("Alex", 18);

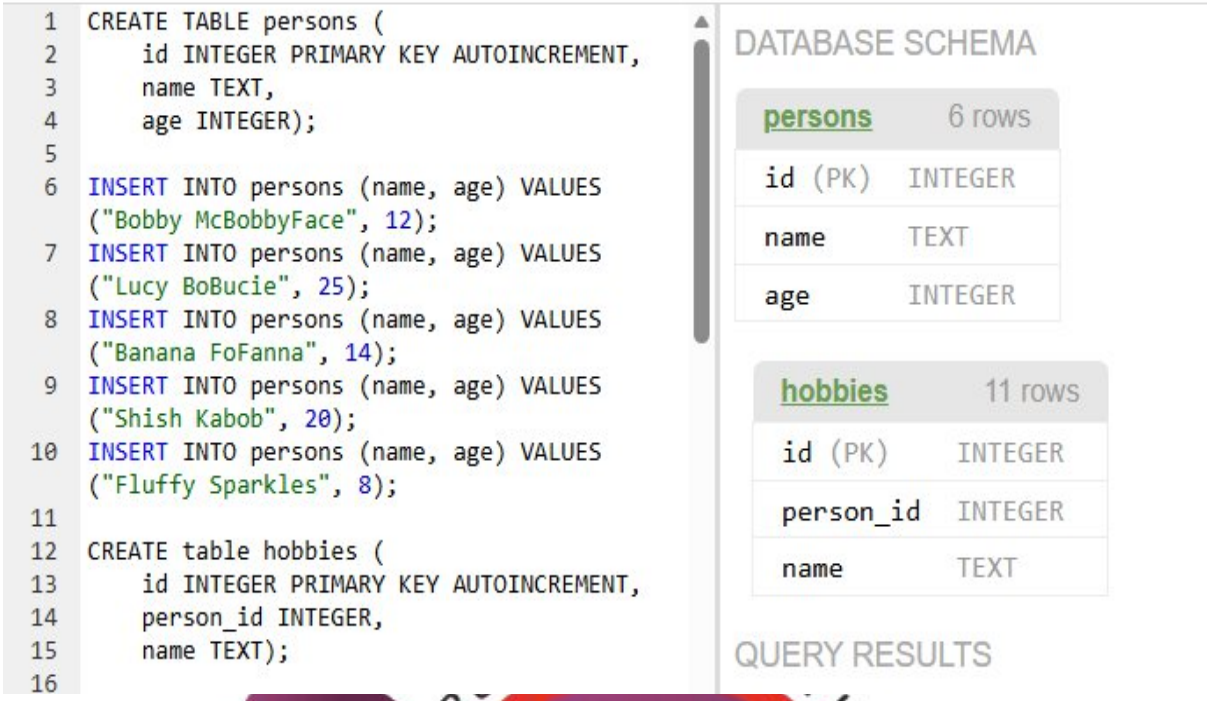

INSERT INTO hobbies (person_id, name)
VALUES (6, "coding");
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

ON persons.id = hobbies.person_id

WHERE persons.name = "Bobby McBobbyFace";

OUTPUT:



```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER);

INSERT INTO persons (name, age) VALUES
("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES
("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES
("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES
("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES
("Fluffy Sparkles", 8);

CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);
```


8. We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY.

PROGCREATE TABLE customers (

id INTEGER PRIMARY KEY AUTOINCREMENT,

name TEXT,

email TEXT);

INSERT INTO customers (name, email) VALUES ("Doctor Who", "doctorwho@timelords.com");

INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");

INSERT INTO customers (name, email) VALUES ("Captain Awesome", "captain@awesome.com");

KHAN ACADEMY

```
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER,
    item TEXT,
    price REAL);


INSERT INTO orders (customer_id, item, price)
    VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "TARDIS", 1000000.00);
    SELECT customers.name,
       customers.email,
       orders.item,
       orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
SELECT
    customers.name,
    customers.email,
    SUM(orders.price) AS total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
```



GROUP BY customers.id

ORDER BY total_spent DESC;RAM:

OUTPUT:

```
1  CREATE TABLE customers (
2      id INTEGER PRIMARY KEY AUTOINCREMENT,
3      name TEXT,
4      email TEXT);
5
6  INSERT INTO customers (name, email) VALUES
   ("Doctor Who", "doctorwho@timelords.com");
7  INSERT INTO customers (name, email) VALUES
   ("Harry Potter", "harry@potter.com");
8  INSERT INTO customers (name, email) VALUES
   ("Captain Awesome", "captain@awesome.com");
9
10 CREATE TABLE orders (
11     id INTEGER PRIMARY KEY AUTOINCREMENT,
12     customer_id INTEGER,
13     item TEXT,
14     price REAL);
15
16 INSERT INTO orders (customer_id, item,
   price)
17     VALUES (1, "Sonic Screwdriver", 1000.00
   );
```

DATABASE SCHEMA

customers	3 rows
id (PK)	INTEGER
name	TEXT
email	TEXT

orders	3 rows
id (PK)	INTEGER
customer_id	INTEGER
item	TEXT
price	REAL

QUERY RESULTS

9. We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

PROGRAM:

CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,

title TEXT,

released INTEGER,

sequel_id INTEGER);

```
INSERT INTO movies
    VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
    VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
    VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
    VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
    VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
INSERT INTO movies
    VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
    VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
    VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);
    SELECT
    m1.title,
    m2.title
FROM movies m1
LEFT OUTER JOIN movies m2
ON m1.sequel_id = m2.id;
```

OUTPUT:

KHAN ACADEMY

```
CREATE TABLE movies (id INTEGER PRIMARY KEY
AUTOINCREMENT,
    title TEXT,
    released INTEGER,
    sequel_id INTEGER);

INSERT INTO movies
    VALUES (1, "Harry Potter and the
Philosopher's Stone", 2001, 2);
INSERT INTO movies
    VALUES (2, "Harry Potter and the
Chamber of Secrets", 2002, 3);
INSERT INTO movies
    VALUES (3, "Harry Potter and the
Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
    VALUES (4, "Harry Potter and the Goblet
of Fire", 2005, 5);
INSERT INTO movies
    VALUES (5, "Harry Potter and the Order
of the Phoenix ", 2007, 6);
INSERT INTO movies
    VALUES (6, "Harry Potter and the Half
```

DATABASE SCHEMA

movies	8 rows
id (PK)	INTEGER
title	TEXT
released	INTEGER
sequel_id	INTEGER

QUERY RESULTS

title	title
Harry Potter and the Philosopher's Stone	Harry Potter and the Chamber of Secrets
Harry Potter and the Chamber of	Harry Potter and the Prisoner of

10. We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies.

PROGRAM:

CREATE TABLE persons (

id INTEGER PRIMARY KEY AUTOINCREMENT,

fullname TEXT,

age INTEGER);

INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");

INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");

INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");

INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");

INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");

CREATE table hobbies (

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);

INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");

CREATE table friends (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person1_id INTEGER,
    person2_id INTEGER);

INSERT INTO friends (person1_id, person2_id)
    VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id)
    VALUES (2, 3);
    SELECT persons.fullname, hobbies.name
FROM persons
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

```
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT
    p1.fullname AS person,
    p2.fullname AS friend
FROM friends
JOIN persons p1
    ON friends.person1_id = p1.id
JOIN persons p2
    ON friends.person2_id = p2.id;
```

OUTPUT:



```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    fullname TEXT,
    age INTEGER);

INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");

CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);

INSERT INTO hobbies (person_id, name)
```

DATABASE SCHEMA

persons	5 rows
id (PK)	INTEGER
fullname	TEXT
age	INTEGER

hobbies	10 rows
id (PK)	INTEGER
person_id	INTEGER
name	TEXT

friends	2 rows

11. We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected.

KHAN ACADEMY

PROGRAM:

```
CREATE table documents (
id INTEGER PRIMARY KEY AUTOINCREMENT,
title TEXT,
content TEXT,
author TEXT);


INSERT INTO documents (author, title, content)
VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
INSERT INTO documents (author, title, content)
VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post.");
INSERT INTO documents (author, title, content)
VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates, heights.");

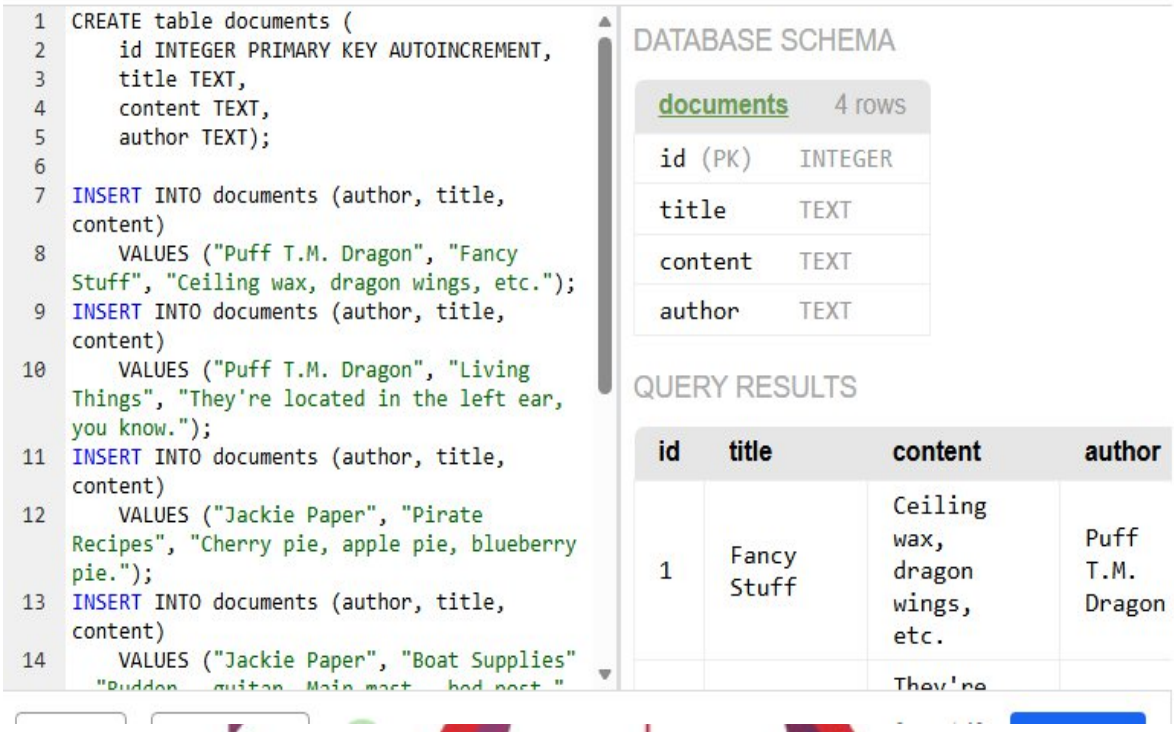

SELECT * FROM documents;
UPDATE documents
SET author = "Jackie Draper"
WHERE author = "Jackie Paper";
SELECT *
FROM documents;
DELETE FROM documents
```

```
WHERE title = "Things I'm Afraid Of";

SELECT *

FROM documents;
```

OUTPUT:



```
CREATE table documents (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    content TEXT,
    author TEXT);

INSERT INTO documents (author, title,
content)
    VALUES ("Puff T.M. Dragon", "Fancy
Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title,
content)
    VALUES ("Puff T.M. Dragon", "Living
Things", "They're located in the left ear,
you know.");
INSERT INTO documents (author, title,
content)
    VALUES ("Jackie Paper", "Pirate
Recipes", "Cherry pie, apple pie, blueberry
pie.");
INSERT INTO documents (author, title,
content)
    VALUES ("Jackie Paper", "Boat Supplies"
```

DATABASE SCHEMA

documents 4 rows

id (PK)	INTEGER
title	TEXT
content	TEXT
author	TEXT

QUERY RESULTS

id	title	content	author
1	Fancy Stuff	Ceiling wax, dragon wings, etc.	Puff T.M. Dragon

12. Think about your favorite apps, and pick one that stores your data- like a game that stores scores, an app that lets you post updates, etc. Now in this project, you're going to imagine that the app stores your data in a SQL database (which is pretty likely!), and write SQL statements that might look like their own SQL.

- CREATE a table to store the data.
- INSERT a few example rows in the table.
- Use an UPDATE to emulate what happens when you edit data in the app.
- Use a DELETE to emulate what happens when you delete data in the app

PROGRAM:

```
CREATE TABLE notes (
    id INTEGER PRIMARY KEY,
    title TEXT,
```

```
    content TEXT,
    created_date TEXT
);

-- Insert example notes
INSERT INTO notes VALUES
(1, 'Shopping List', 'Buy milk, eggs, and bread', '2026-07-02');

INSERT INTO notes VALUES
(2, 'Homework', 'Complete SQL project', '2026-07-02');

INSERT INTO notes VALUES
(3, 'Workout', 'Run 5 km in the evening', '2026-07-02');

-- View all notes
SELECT * FROM notes;

-- Update a note (editing in the app)
UPDATE notes
SET content = 'Buy milk, eggs, bread, and fruits'
WHERE id = 1;

-- View updated note
SELECT * FROM notes WHERE id = 1;

-- Delete a note (removing it from the app)
```

KHAN ACADEMY

DELETE FROM notes

WHERE id = 3;

-- View remaining notes

SELECT * FROM notes;/* What does the app's SQL look like? */

OUTPUT:

```
CREATE TABLE notes (
    id INTEGER PRIMARY KEY,
    title TEXT,
    content TEXT,
    created_date TEXT
);

-- Insert example notes
INSERT INTO notes VALUES
(1, 'Shopping List', 'Buy milk, eggs, and bread', '2026-07-02');

INSERT INTO notes VALUES
(2, 'Homework', 'Complete SQL project', '2026-07-02');

INSERT INTO notes VALUES
(3, 'Workout', 'Run 5 km in the evening', '2026-07-02');

-- View all notes
SELECT * FROM notes;
```

DATABASE SCHEMA

notes	2 rows
id (PK)	INTEGER
title	TEXT
content	TEXT
created_date	TEXT

QUERY RESULTS

id	title	content	created_da
1	Shopping List	Buy milk, eggs, and bread	2026-07-0

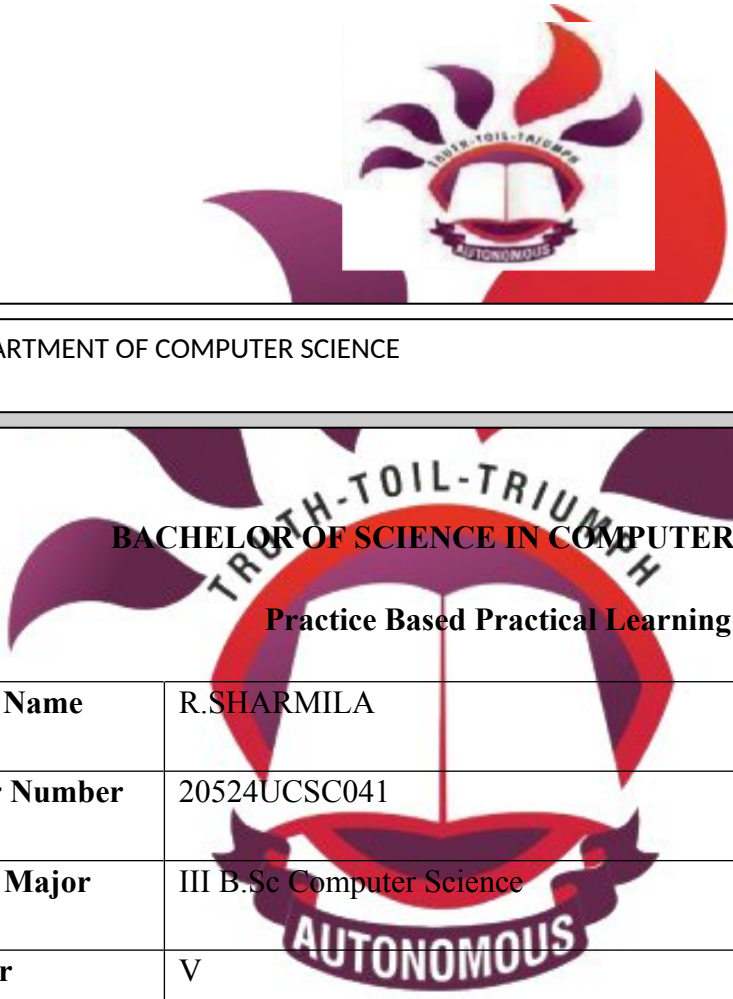


THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram - 605 403



DEPARTMENT OF COMPUTER SCIENCE

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

Practice Based Practical Learning

Student Name	R.SHARMILA
Register Number	20524UCSC041
Class & Major	III B.Sc Computer Science
Semester	V
Course Code	UCSR513
Course Title	Advanced Database Management System Practical
Academic Year	2026-2027

Course Incharge	Dr. S Manimekalai

Course In-Charge Head of the Department

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join		

	• Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX.NO:01: Creation of DDL operations using Books Table**

DATE:19.6.2026

AIM

To create a Books table, insert book records, display all records, and retrieve books published from the year 2020 onwards

Software Used

Oracle/MySQL/sqlite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects in a database.

DDL COMMANDS:

● Create

● Alter

● Drop

● Truncate

CREATE:

Create command is used to create a new table and also use constraints.

Syntax

sql

```
CREATE TABLE table_name(
    column1 datatype constraints,
    column2 datatype constraints,
    column3 datatype constraints
);
```

Table Structure

sql

```
SQL> CREATE TABLE books(
    book_id INTEGER PRIMARY KEY,
   title TEXT,
```

```
    author TEXT,
    published_year INTEGER
);
```

Table Created.

ALTER:

Alter command is used to alter a table or make some changes in a table. It has methods such as ADD, MODIFY, DROP and RENAME.

ADD:

Add command is used to insert a new attribute/column with data type.

Syntax:

sql

```
ALTER TABLE table_name ADD column_name datatype;
```

Query:

sql

```
SQL> ALTER TABLE books ADD price INTEGER;
```

Table Altered.

MODIFY:

Modify command is used to modify or change the data type and to increase the data size in the table.

Syntax:

sql

ALTER TABLE table_name MODIFY column_name datatype;

Query:

sql

SQL> ALTER TABLE books MODIFY title VARCHAR(150);

Table Altered.

DROP:

Drop command is used to delete the attribute/column in a table.

Syntax:

sql

ALTER TABLE table_name DROP COLUMN column_name;

Query:

sql

SQL> ALTER TABLE books DROP COLUMN price;

Table Altered.

RENAME:

Rename command is used to re-change the table name.

sql

SQL> RENAME TABLE books TO library_books;

```

Table renamed.
```

sql

```
ALTER TABLE books RENAME TO library_books;
```

Column Rename

sql

```
ALTER TABLE library_books RENAME COLUMN title TO book_title;
```

TRUNCATE:

Truncate command is used to delete all values in the table.

Syntax:

sql

```
TRUNCATE TABLE table_name;
```

Query:

sql

```
TRUNCATE TABLE library_books;
```

DROP:

Drop command is used to delete the table and all its data permanently.

Syntax:

sql

```
DROP TABLE table_name;
```

Query:

sql

DROP TABLE library_books;

Explanation

● Create a table named `books`.

● Insert book details into the table.

● Display all book records.

● Retrieve books published in or after the year 2020 and sort them by published year.

Create Table

sql

```
CREATE TABLE books(
    book_id INTEGER PRIMARY KEY,
    title TEXT,
    author TEXT,
    published_year INTEGER
);
```

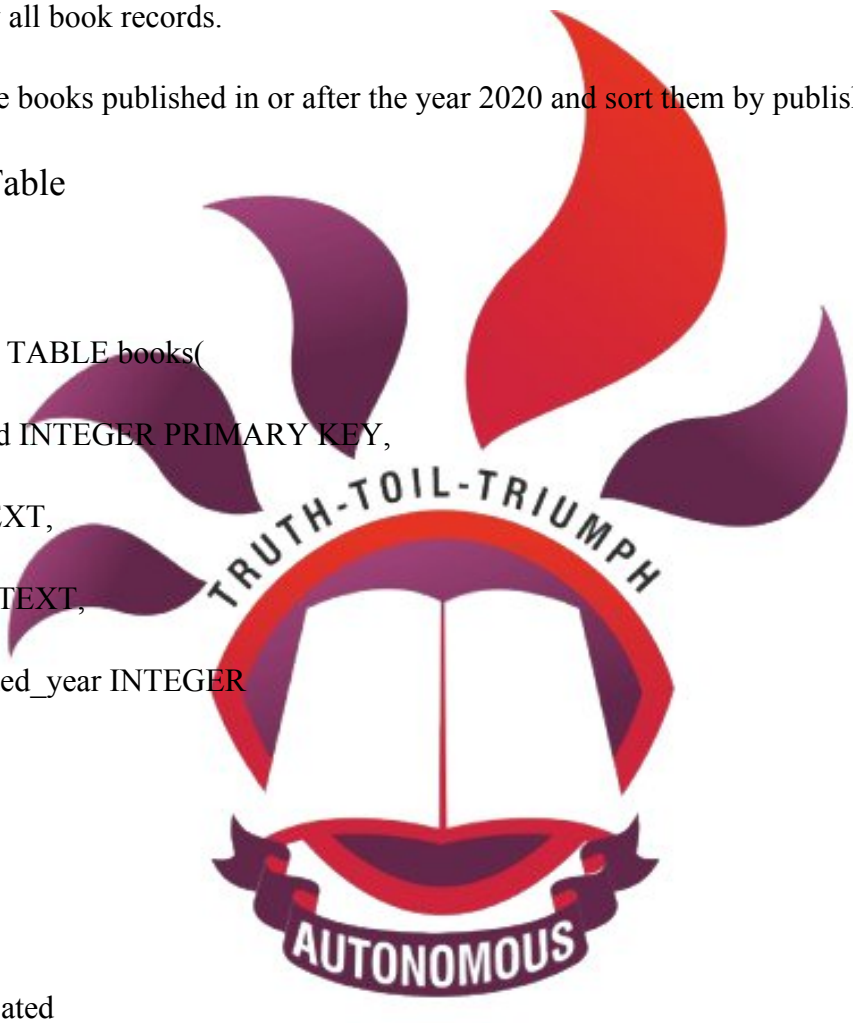


Table Created

Sample Data

sql

INSERT INTO books VALUES(1, 'Atomic Habits', 'James Clear', 2018);

INSERT INTO books VALUES(2, 'The Midnight Library', 'Matt Haig', 2020);

INSERT INTO books VALUES(3, 'Project Hail Mary', 'Andy Weir', 2021);

INSERT INTO books VALUES(4, 'The Psychology of Money', 'Morgan Housel', 2020);

INSERT INTO books VALUES(5, 'Klara and the Sun', 'Kazuo Ishiguro', 2021);

INSERT INTO books VALUES(6, 'Tomorrow and Tomorrow and Tomorrow', 'Gabrielle Zevin', 2022);

Records Inserted

SQL Query

Query1:

Display all book records

sql

SQL> SELECT * FROM books;

Resultant Query

book_id	title	author	published_year
1	Atomic Habits	James Clear	2018
2	The Midnight Library	Matt Haig	2020

3	Project Hail Mary	Andy Weir	2021
4	The Psychology of Money	Morgan Housel	2020
5	Klara and the Sun	Kazuo Ishiguro	2021
6	Tomorrow	Tomorrow Gabrielle Zevin	2022

Query2:

Display books published from the year 2020 onwards

sql

```
SQL> SELECT *
FROM books
WHERE published_year >= 2020
ORDER BY published_year;
```

Resultant Query

book_id	title	author	published_year
2	The Midnight Library	Matt Haig	2020
4	The Psychology of Money	Morgan Housel	2020
3	Project Hail Mary	Andy Weir	2021
5	Klara and the Sun	Kazuo Ishiguro	2021
6	Tomorrow and Tomorrow	Tomorrow Gabrielle Zevin	2022

RESULT:

Thus, the Books table was created successfully, records were inserted, all book details were displayed, and the books published from the year 2020 onwards were retrieved successfully.

EX.NO:02: Creation of DML Operations using Products Table

DATE:25.6.2026

Aim

To create a Products table, insert product records, display all records, update product details, delete a product record, and retrieve products based on their prices.

Software Used

Oracle/MySQL/sqlite3

Description

DML (Data Manipulation Language) is used to insert, modify, delete, and retrieve data stored in database tables.

DML COMMANDS:

- INSERT
- UPDATE
- DELETE
- SELECT

INSERT

INSERT command is used to add new records into a table.

Syntax

sql id="jv7v6q"

INSERT INTO table_name(column1,column2,column3)

VALUES(value1,value2,value3);

Query

sql id="qz2m6a"

SQL> INSERT INTO products VALUES(1, 'Laptop', 55000);

1 row inserted.

UPDATE

UPDATE command is used to modify existing records in a table.

Syntax

sql id="k8p0xv"

UPDATE table_name

SET column_name=value

WHERE condition;

Query

sql id="nq5m2e"

SQL> UPDATE products

SET price=58000

WHERE id=1;

1 row updated.

DELETE

DELETE command is used to remove existing records from a table.

Syntax

sql id="c7q1hs"

DELETE FROM table_name

WHERE condition;

Query

sql id="6v7j3n"

SQL> DELETE FROM products

WHERE id=5;

1 row deleted.

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

sql id="0yq7fr"

SELECT * FROM table_name;

Query

sql id="9q5z8k"

SQL> SELECT * FROM products;

Explanation

● Create a table named `products`.

● Insert product details into the table.

● Display all product records.

● Update the price of a product.

● Delete a product record.

● Retrieve products whose price is 30000 or above and sort them by price.

Create Table

sql id="a4n8ws"

```
CREATE TABLE products(
    id INTEGER PRIMARY KEY,
    name TEXT,
    price INTEGER
```

);

Table Created.

Sample Data

sql id="m3x7pk"

INSERT INTO products VALUES(1, 'Laptop', 55000);

INSERT INTO products VALUES(2, 'Keyboard', 1500);

INSERT INTO products VALUES(3, 'Smartphone', 35000);

INSERT INTO products VALUES(4, 'Headphones', 2500);

INSERT INTO products VALUES(5, 'Tablet', 28000);

INSERT INTO products VALUES(6, 'Monitor', 32000);

Records Inserted.

SQL Query

Query1:

Display all product records

sql id="7d5k2m"

SQL> SELECT * FROM products;

Resultant Query

id	name	price
1	Laptop	55000

2	Keyboard	1500
3	Smartphone	35000
4	Headphones	2500
5	Tablet	28000
6	Monitor	32000

Query2:

Update the price of Laptop

sql id="c4v8tz"

SQL> UPDATE products

SET price=58000

WHERE id=1;

1 row updated.

Updated Record

id	name	price
1	Laptop	58000

Query3:

Delete Tablet's record

sql id="w7k1fc"

SQL> DELETE FROM products

WHERE id=5;

1 row deleted.

Remaining Records

id	name	price
1	Laptop	58000
2	Keyboard	1500
3	Smartphone	35000
4	Headphones	2500
6	Monitor	32000

Query4:

Display products whose price is 30000 or above

sql id="h5q8vn"

```
SQL> SELECT *

FROM products

WHERE price >= 30000

ORDER BY price;
```

Resultant Query

id	name	price
6	Monitor	32000

3	Smartphone	35000
1	Laptop	58000

RESULT

Thus, the Products table was created successfully, records were inserted, product details were displayed, the product price was updated, a product record was deleted, and the products whose price was 30000 or above were retrieved successfully.

EX.NO:03: Creation of DQL Operations using Sales Table

DATE:

Aim

To create a Sales table, insert sales records, display all records, calculate total sales and average sales for each category, and retrieve categories based on total sales using DQL operations.

Software Used

Oracle/MySQL/sqlite3

Description

DQL (Data Query Language) is used to retrieve and display data stored in database tables. The main command used in DQL is SELECT.

DQL COMMANDS:

- SELECT
- GROUP BY
- SUM

● AVG

● HAVING

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

sql id="m4t6nx"

SELECT * FROM table_name;

Query

sql id="r8v2kp"

SQL> SELECT * FROM sales;

GROUP BY

GROUP BY command is used to group rows having the same values in specified columns.

Syntax

sql id="f7q3wc"

SELECT column_name, SUM(column_name)

FROM table_name

GROUP BY column_name;

SUM

SUM function is used to calculate the total value of a numeric column.

Syntax

sql id="k9z5ad"

SELECT SUM(column_name)

FROM table_name;

AVG

AVG function is used to calculate the average value of a numeric column.

Syntax

sql id="p2n6yv"

SELECT AVG(column_name)

FROM table_name;

HAVING

HAVING command is used to filter grouped records based on a condition.

Syntax

`sql id="x3c8lm"

SELECT column_name, SUM(column_name)

FROM table_name

GROUP BY column_name

HAVING condition;

Explanation

● Create a table named `sales`.

● Insert sales details into the table.

● Display all sales records.

● Calculate the total sales for each category.

● Calculate the average sales for each category.

● Retrieve categories whose total sales are greater than 50000.

Create Table

sql id="a7k2pd"

```
CREATE TABLE sales(
    sale_id INTEGER PRIMARY KEY,
    product_name TEXT,
    category TEXT,
    amount INTEGER
);
```

Table Created.

Sample Data

sql id="v5m9qr"

```
INSERT INTO sales VALUES(1, 'Laptop', 'Electronics', 55000);
INSERT INTO sales VALUES(2, 'Mobile', 'Electronics', 30000);
INSERT INTO sales VALUES(3, 'Chair', 'Furniture', 12000);
INSERT INTO sales VALUES(4, 'Table', 'Furniture', 18000);
INSERT INTO sales VALUES(5, 'Headphones', 'Electronics', 8000);
INSERT INTO sales VALUES(6, 'Sofa', 'Furniture', 25000);
```

Records Inserted.

SQL Query

Query1:

Display all sales records

`sql id="n6w4sz"

SQL> SELECT * FROM sales;

Resultant Query

Saleid	product name	category	amount
1	Laptop	Electronics	55000
2	Mobile	Electronics	30000
3	Chair	Furniture	12000
4	Table	Furniture	18000
5	Headphones	Electronics	8000
6	Sofa	Furniture	25000

Query2:

Display total sales for each category

sql id="c8p3vy"

SQL> SELECT category, SUM(amount) AS total_sales

FROM sales

GROUP BY category;

Resultant Query

category	total_sales

Electronics	93000
Furniture	55000

Query3:

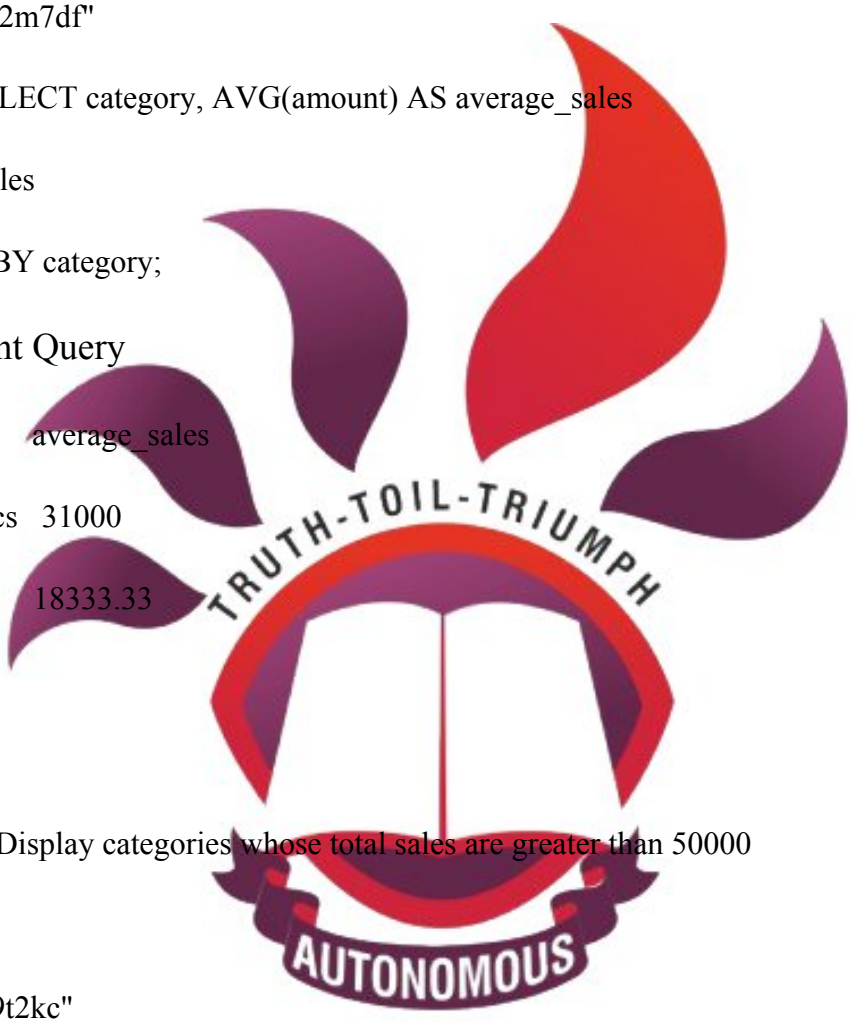
Display average sales for each category

sql id="w2m7df"

```
SQL> SELECT category, AVG(amount) AS average_sales
FROM sales
GROUP BY category;
```

Resultant Query

```
category      average_sales
Electronics   31000
Furniture     18333.33
```



Query4: Display categories whose total sales are greater than 50000

sql id="e9t2kc"

```
SQL> SELECT category, SUM(amount) AS total_sales
FROM sales
GROUP BY category
HAVING SUM(amount) > 50000;
```

Resultant Query

category total_sales

Electronics 93000

Furniture 55000

RESULT:

Thus, the Sales table was created successfully, records were inserted, all sales details were displayed, total and average sales were calculated for each category, and the categories having total sales greater than 50000 were retrieved successfully.

EX.NO: 04: Creation of Transaction Operations using Bank Transactions Table

DATE: _____

Aim

To create a `bank_transactions` table, insert transaction records, perform transaction operations using **COMMIT, ROLLBACK, SAVEPOINT**, and verify the changes made to the database.

Software Used

Oracle / MySQL / SQLite3

Description

Transaction Control Language (TCL) is used to manage transactions in a database. It helps to save or undo the changes made to the database.

TRANSACTION COMMANDS:

- COMMIT
- ROLLBACK

● SAVEPOINT

● ROLLBACK TO SAVEPOINT

COMMIT

COMMIT command is used to permanently save all the changes made during a transaction.

Syntax

sql

```
COMMIT;
```

Query

sql

```
SQL> COMMIT;
```

Transaction committed.

ROLLBACK

command is used to undo the changes made to the database before the changes are committed.

Syntax

sql

```
ROLLBACK;
```

Querysql

```
SQL> ROLLBACK;
```

Transaction rolled back.

SAVEPOINT

SAVEPOINT command is used to create a temporary point within a transaction. The transaction can be rolled back to that specific point.

Syntax

sql

SAVEPOINT savepoint_name;

Query

sql

SQL> SAVEPOINT sp1;

Savepoint created.

ROLLBACK TO SAVEPOINT

ROLLBACK TO SAVEPOINT is used to undo the changes made after a particular savepoint without undoing the entire transaction.

Syntaxsql

ROLLBACK TO SAVEPOINT savepoint_name;

Query

sql

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

Explanation

● Create a table named `bank_transactions`.

● Insert transaction details into the table.

● Use COMMIT to permanently save the inserted records.

● Update a customer's balance and create a SAVEPOINT.

● Make another change after the savepoint.

● Use ROLLBACK TO SAVEPOINT to undo the changes made after the savepoint.

● Use ROLLBACK to undo uncommitted changes.

Create Table

sql

```
CREATE TABLE bank_transactions(
    transaction_id INTEGER PRIMARY KEY,
    customer_name TEXT,
    transaction_type TEXT,
    amount INTEGER,
    balance INTEGER
);
```

Table Created

Sample Data

sql

```
INSERT INTO bank_transactions VALUES(201, 'Rahul', 'Deposit', 10000, 50000);

INSERT INTO bank_transactions VALUES(202, 'Anitha', 'Deposit', 15000, 60000);

INSERT INTO bank_transactions VALUES(203, 'Vijay', 'Withdraw', 5000, 25000);

INSERT INTO bank_transactions VALUES(204, 'Sneha', 'Deposit', 20000, 70000);

INSERT INTO bank_transactions VALUES(205, 'Karthik', 'Withdraw', 8000, 32000);
```

Records Inserted

SQL Query

Query 1:

Display all transaction records

`sql

```
SQL> SELECT * FROM bank_transactions;
```

Resultant Query

transaction_id	customer_name	transaction_type	amount	balance
201	Rahul	Deposit	10000	50000
202	Anitha	Deposit	15000	60000
203	Vijay	Withdraw	5000	25000
204	Sneha	Deposit	2500	70000
205	Karthik	Withdraw	8000	32000

Query 2:

Commit the inserted records

sql

SQL> COMMIT;

Transaction committed.

The inserted transaction records are now permanently saved.

Query 3:

Update Rahul's account balance

sql

SQL> UPDATE bank_transactions

SET balance = 55000

HERE transaction_id = 201;

1 row updated.

Updated Record

transaction_id	customer_name	balance
201	Rahul	55000

Query 4:

Create a SAVEPOINT

sql

SQL> SAVEPOINT sp1;

Savepoint created.

Query 5:

Update Anitha's account balance

sql

SQL> UPDATE bank_transactions

SET balance = 65000

WHERE transaction_id = 202;

1 row updated.

Updated Record

Transaction id	Customer name	balance
| 202	Anitha	65000

Query 6:

Rollback to SAVEPOINT

sql

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed

The changes made after SAVEPOINT `sp1` are undone.

Resultant Query

transaction id	Customer name	balance
201	Rahul	55000
202	Anitha	60000

Query 7:

Rollback an uncommitted change

sql

SQL> UPDATE bank_transactions

SET balance = 20000

WHERE transaction_id = 203;

1 row updated.

sql

SQL> ROLLBACK;

Transaction rolled back.

The balance of Vijay is restored to its previous value

Resultant Query

transaction_id	customer_name	balance
| 203	Vijay	25000

RESULT

Thus, the `bank_transactions` table was created successfully, transaction records were inserted, and the transaction operations **COMMIT, ROLLBACK, SAVEPOINT, and ROLLBACK TO SAVEPOINT** were performed successfully.

EX.NO:05 Creation of Join Operations Using Games and Players Table

DATE: 22.6.2026

AIM

To create Games and Players tables, insert records, and retrieve related data using different types of joins such as Inner Join, Left Join, Right Join, Full Join, and Self Join.

Software Used

Oracle / MySQL / SQLite3

DESCRIPTION

Joins are used to combine records from two or more tables based on a related column between them.

JOIN COMMANDS:

• Inner Join

• Left Join

• Right Join

• Full Join

• Self Join

CREATE TABLE

sql

```
CREATE TABLE Games (
    Game_ID INT PRIMARY KEY,
    Game_Name VARCHAR(30),
    Game_Type VARCHAR(20)
);
```

sql

```
CREATE TABLE Players (
    Player_ID INT PRIMARY KEY,
    Player_Name VARCHAR(30),
    Game_ID INT
);
```

INSERT RECORDS

sql

```
INSERT INTO Games VALUES (101, 'Free Fire', 'Action');
INSERT INTO Games VALUES (102, 'Minecraft', 'Adventure');
INSERT INTO Games VALUES (103, 'Chess', 'Strategy');
INSERT INTO Games VALUES (104, 'Subway Surfers', 'Arcade');
```

INSERT INTO Games VALUES (105, 'Valorant', 'Shooting');

sql

INSERT INTO Players VALUES (1, 'Arun', 101);

INSERT INTO Players VALUES (2, 'Bala', 102);

INSERT INTO Players VALUES (3, 'Cathy', 103);

INSERT INTO Players VALUES (4, 'Divya', 101);

INSERT INTO Players VALUES (5, 'Esha', 106);

INNER JOIN

Inner Join returns only the records that have matching values in both tables.

Syntax:

sql

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

Query:

sql

```
SELECT Games.Game_Name, Players.Player_Name
FROM Games
INNER JOIN Players
ON Games.Game_ID = Players.Game_ID;
```

Result:

GAME	PLAYER NAME
Free Fire	Arun
Minecraft	Bala
Chess	Cathy
Free Fire	Divya

LEFT JOIN

Left Join returns all records from the left table and the matching records from the right table.

Syntax:

sql

SELECT columns

FROM table1

LEFT JOIN table2

ON table1.column = table2.column;

Query:

sql

SELECT Games.Game_Name, Players.Player_Name

FROM Games

LEFT JOIN Players

ON Games.Game_ID = Players.Game_ID;

Result:

GAME_NAME	PLAYER_NAME
Free Fire	Arun
Free Fire	Divya
Minecraft	Bala
Chess	Cathy
Subway Surfers	NULL
Valorant	NULL

RIGHT JOIN

Right Join returns all records from the right table and the matching records from the left table.

Syntax:

sql

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

Query:

sql

```
SELECT Games.Game_Name, Players.Player_Name
FROM Games
RIGHT JOIN Players
ON Games.Game_ID = Players.Game_ID;
```

Result:

GAME_NAME	|PLAYER_NAME
Free Fire |	Arun
Minecraft	Bala
Chess	Cathy
Free Fire	Divya
NULL	Esha |

FULL JOIN

Full Join returns all records from both tables. It includes matching and non-matching records.

Syntax:

sql

```
SELECT columns
FROM table1
FULL JOIN table2
ON table1.column = table2.column;
```

Query:

sql

```
SELECT Games.Game_Name, Players.Player_Name
FROM Games
FULL JOIN Players
ON Games.Game_ID = Players.Game_ID;
```

Result query :

GAME_NAME	|PLAYER_NAME
Free Fire	Arun
Free Fire	Divya
Minecraft	Bala
Chess	Cathy
Subway Surfers	NULL
Valorant	| NULL
NULL	| Esha

SELF JOIN

Self Join is a join in which a table is joined with itself.

Query:

`sql

```
SELECT P1.Player_Name AS Player1,
    P2.Player_Name AS Player2
FROM Players P1
JOIN Players P2
ON P1.Game_ID = P2.Game_ID
AND P1.Player_ID < P2.Player_ID;
```

Result query:

PLAYER1	PLAYER2
Arun	Divya

RESULT

Thus, the Games and Players tables were created successfully, records were inserted, and different types of join operations such as Inner Join, Left Join, Right Join, Full Join, and Self Join were executed successfully.

EX. NO: 05: Creation of SQL Functions Using Travel_Bookings Table

DATE:

AIM

To create a Travel_Bookings table, insert booking records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / sqlite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL FUNCTION TYPES:

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS

Aggregate functions are used to perform calculations on a group of rows and return a single result.

Common aggregate functions are:

● COUNT()

● SUM()

● AVG()

● MAX()

● MIN()

Syntax

sql

```
SELECT aggregate_function(column_name)
FROM table_name;
```

STRING FUNCTIONS

String functions are used to perform operations on text or character values.

Common string functions are:

● UPPER()

● LOWER()

● LENGTH()

Syntax

sql

```
SELECT string_function(column_name)
FROM table_name;
```

DATE FUNCTIONS

Date functions are used to perform operations on date values.

Common date functions are:

● CURRENT_DATE

● YEAR()

● MONTH()

● DAY()

Syntax

sql

```
SELECT date_function(date_column)
FROM table_name;
```

Explanation

● Create a table named travel_bookings.

● Insert travel booking details into the table.

● Display all travel booking records.

● Calculate the total number of bookings using COUNT().

● Calculate the total and average booking amount using SUM() and AVG().

● Find the highest and lowest booking amount using MAX() and MIN().

● Display destination names in uppercase and lowercase using UPPER() and LOWER().

● Find the length of each destination name using LENGTH().

● Retrieve booking details based on the travel date.

Create Table

sql

CREATE TABLE travel_bookings (

booking_id INTEGER PRIMARY KEY,

traveller_name TEXT,

destination TEXT,

booking_amount INTEGER,

travel_date DATE

);

Table Created

Sample Data

sql

INSERT INTO travel_bookings VALUES (1, 'Arun', 'Chennai', 4500, '2026-01-10');

INSERT INTO travel_bookings VALUES (2, 'Meena', 'Ooty', 6500, '2026-02-15');

INSERT INTO travel_bookings VALUES (3, 'Kavin', 'Goa', 8500, '2026-03-12');

INSERT INTO travel_bookings VALUES (4, 'Divya', 'Mysore', 5500, '2026-04-18');

INSERT INTO travel_bookings VALUES (5, 'Priya', 'Kerala', 7500, '2026-05-20');

INSERT INTO travel_bookings VALUES (6, 'Rahul', 'Bangalore', 4000, '2026-06-25');

Records Inserted

SQL Query

Query 1

: Display all travel booking records

sql

SELECT * FROM travel_bookings;

Resultant Query

booking_id	Btraveller_name	destination	booking_amount	travel_date
1	Arun	Chennai	4500	2026-01-10
2	Meena	Ooty	6500	2026-02-15
3	Kavin	Goa	8500	2026-03-12
4	Divya	Mysore	5500	2026-04-18
5	Priya	Kerala	7500	2026-05-20
6	Rahul	Bangalore	4000	2026-06-25

AGGREGATE FUNCTIONS

Query 2:

Count the total number of bookings

sql

SELECT COUNT(*) AS Total_Bookings

FROM travel_bookings;

Query 3:

Calculate the total booking amount

sql

SELECT SUM(booking_amount) AS Total_Amount

FROM travel_bookings;

Resultant Query

Total_Amount

42000

Query 4:

Calculate the average booking amount

sql

```
SELECT AVG(booking_amount) AS Average_Amount
FROM travel_bookings;
```

Resultant Query

Average_Amount

7000

Query

Find the highest and lowest booking amount

sql

```
SELECT MAX(booking_amount) AS Highest_Amount,
MIN(booking_amount) AS Lowest Amount
FROM travel_bookings;
```

Resultant Query

Highest_Amount	Lowest_Amount
8500	4000

STRING FUNCTIONS

Query 6:

Display destination names in uppercase

sql

```
SELECT destination, UPPER(destination) AS Upper_Destination
```

FROM travel_bookings;

Resultant Query

destination	Upper Destination
Chennai	CHENNAI
Ooty	OOTY
Goa	GOA
Mysore	MYSORE
Kerala	KERALA
Bangalore	BANGALORE

|Query 7:

Display destination names in lowercase

sql

SELECT destination, LOWER(destination) AS Lower_Destination

FROM travel_bookings;

Resultant Query

| destination | Lower_Destination |

| Chennai | chennai |

| Ooty | ooty |

| Goa | goa |

| Mysore | mysore |

| Kerala | kerala |

| Bangalore | bangalore |

Query 8: Display the length of each destination name

```sql
SELECT destination, LENGTH(destination) AS Destination_Length
FROM travel_bookings;
```

Resultant Query

destination	Destination_Length
Chennai	7
Ooty	4
Goa	3
Mysore	6
Kerala	6
Bangalore	9

DATE FUNCTIONS

Query 9: Display the current date

```sql
SELECT CURRENT_DATE AS Current_Date;
```

Resultant Query

Current_Date

Current system date

Query 10:

Display bookings made for travel after March 2026

sql

```
SELECT traveller_name, destination, travel_date
FROM travel_bookings
WHERE travel_date > '2026-03-31';
```

Resultant Query

traveller_name | destination | travel_date |

| Divya | Mysore | 2026-04-18 |

| Priya | Kerala | 2026-05-20 |

| Rahul | Bangalore | 2026-06-25 |

RESULT:

Thus, the Travel_Bookings table was created successfully, travel booking records were inserted, and the required data was retrieved successfully using Aggregate Functions, String Functions, and Date Functions.

EX. NO: 06: Creation of SQL Functions Using Travel_Bookings Table

DATE:10.7.2026

AIM

To create a Travel_Bookings table, insert booking records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / sqlite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL FUNCTION TYPES:

● Aggregate Functions

● String Functions

● Date Functions

AGGREGATE FUNCTIONS

Aggregate functions are used to perform calculations on a group of rows and return a single result.

Common aggregate functions are:

● COUNT()

● SUM()

● AVG()

● MAX()

● MIN()

Syntax

```sql

SELECT aggregate_function(column_name)
```

FROM table_name;

```

### STRING FUNCTIONS

String functions are used to perform operations on text or character values.

Common string functions are:

- UPPER()
- LOWER()
- LENGTH()

### Syntax

```sql
SELECT string_function(column_name)
FROM table_name;
```

DATE FUNCTIONS
```

Date functions are used to perform operations on date values.

Common date functions are:

- CURRENT_DATE
- YEAR()
- MONTH()
- DAY()

Syntax

```sql
SELECT date_function(date_column)
FROM table_name;
```

Explanation

- Create a table named travel_bookings.
- Insert travel booking details into the table.

● Display all travel booking records.

● Calculate the total number of bookings using COUNT().

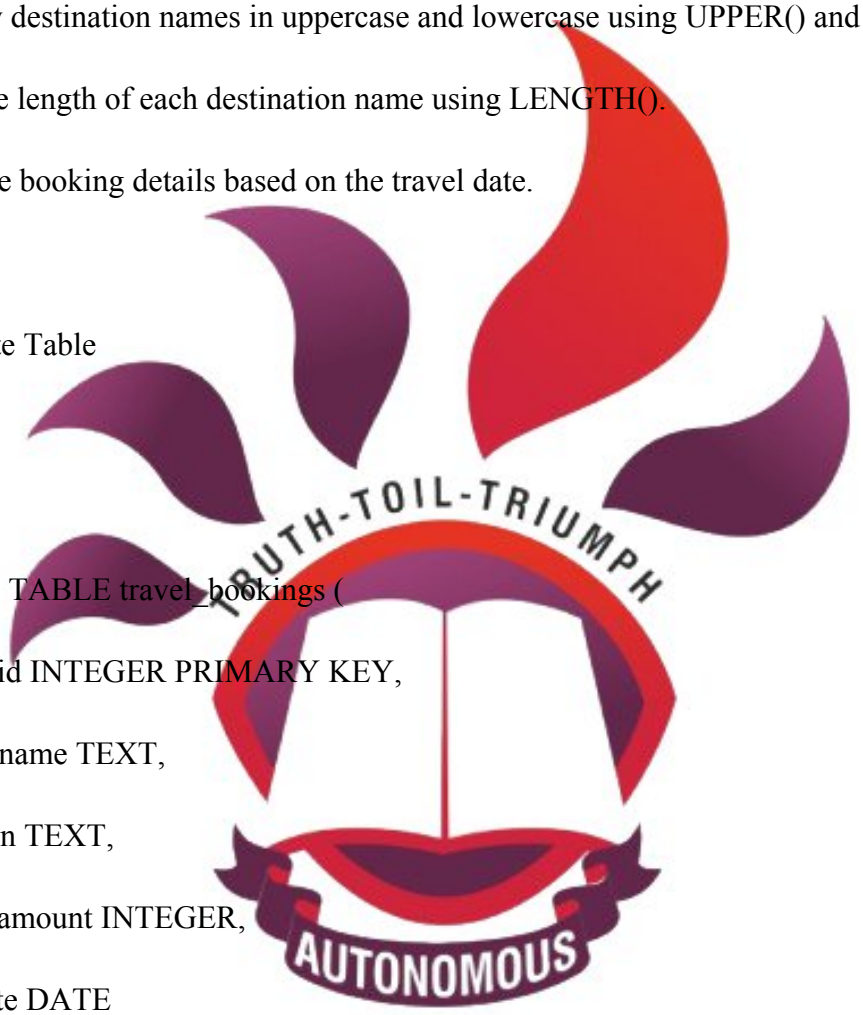
● Calculate the total and average booking amount using SUM() and AVG().

● Find the highest and lowest booking amount using MAX() and MIN().

● Display destination names in uppercase and lowercase using UPPER() and LOWER().

● Find the length of each destination name using LENGTH().

● Retrieve booking details based on the travel date.



Create Table

```sql
CREATE TABLE travel_bookings (
booking_id INTEGER PRIMARY KEY,
traveller_name TEXT,
destination TEXT,
booking_amount INTEGER,
travel_date DATE
);
```

Table Created

Sample Data

```sql
INSERT INTO travel_bookings VALUES (1, 'Arun', 'Chennai', 4500, '2026-01-10');

INSERT INTO travel_bookings VALUES (2, 'Meena', 'Ooty', 6500, '2026-02-15');

INSERT INTO travel_bookings VALUES (3, 'Kavin', 'Goa', 8500, '2026-03-12');

INSERT INTO travel_bookings VALUES (4, 'Divya', 'Mysore', 5500, '2026-04-18');

INSERT INTO travel_bookings VALUES (5, 'Priya', 'Kerala', 7500, '2026-05-20');

INSERT INTO travel_bookings VALUES (6, 'Rahul', 'Bangalore', 4000, '2026-06-25');
```

Records Inserted

SQL Query

Query 1: Display all travel booking records

```sql
SELECT * FROM travel_bookings;
```

Resultant Query

booking_id	traveller_name	destination booking_amount	travel_date
1	Arun	Chennai	4500	2026-01-10
2	Meena	Ooty	6500	2026-02-15
3	Kavin	Goa	8500	2026-03-12
4	Divya	Mysore	5500	2026-04-18
5	Priya	Kerala	7500	2026-05-20
6	Rahul	Bangalore	4000	2026-06-25

AGGREGATE FUNCTIONS

Query 2: Count the total number of bookings

```sql

SELECT COUNT(*) AS Total_Bookings

FROM travel_bookings;

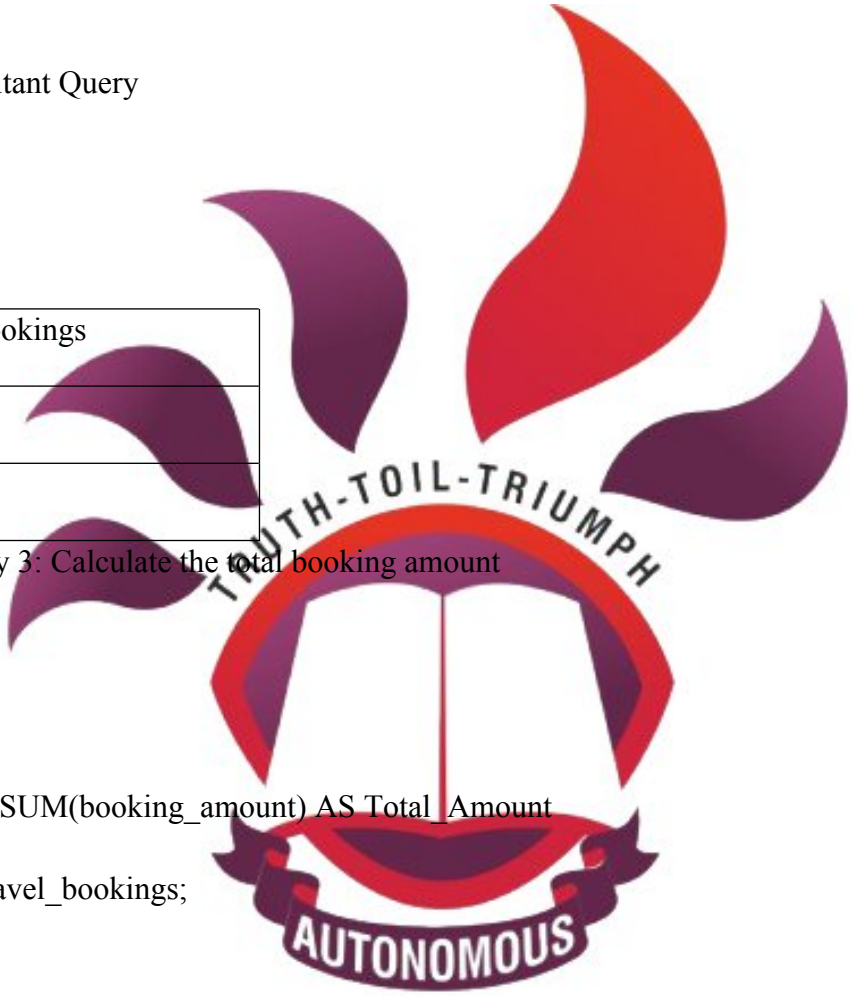
```

Resultant Query

|

Total Bookings
6

Query 3: Calculate the total booking amount




```sql

SELECT SUM(booking_amount) AS Total_Amount

FROM travel_bookings;

```

Resultant Query

|

Total Amount
42000

|

Query 4: Calculate the average booking amount

```sql
SELECT AVG(booking_amount) AS Average_Amount
FROM travel_bookings;
```

Resultant Query

Average_Amount
7000

Query 5: Find the highest and lowest booking amount

```sql
```

SELECT MAX(booking_amount) AS Highest_Amount,

MIN(booking_amount) AS Lowest_Amount

FROM travel_bookings;

```

### Resultant Query

|

| Highest Amount | LowestAmount |
|---|---|
| 8500 | 4000 |

|

# STRING FUNCTIONS

### Query 6: Display destination names in uppercase

```sql

SELECT destination, UPPER(destination) AS Upper_Destination

FROM travel_bookings;

```
```

Resultant Query

destination	Upper Destination
Chennai	CHENNAI
Ooty	OOTY
Goa	GOA
Mysore	MYSORE
Kerala	KERALA
Bangalore	BANGALORE

Query 7: Display destination names in lowercase

sql

```
SELECT destination, LOWER(destination) AS Lower_Destination
FROM travel_bookings;
```

Resultant Query

|

destination	Lower Destination
Chennai	chennai

Ooty	ooty
Goa	goa
Mysore	mysore
Kerala	kerala
Bangalore	bangalore

|

Query 8: Display the length of each destination name

sql

```
SELECT destination, LENGTH(destination) AS Destination_Length
FROM travel_bookings;
```

Resultant Query

|

destination	Destination_Length
| Chennai	7
| Ooty	4
| Goa	3
| Mysore	6
| Kerala	6
| Bangalore	9

DATE FUNCTIONS

Query 9: Display the current date

sql

```
SELECT CURRENT_DATE AS Current_Date;
```

Resultant Query

Current_Date
Current system date

Query 10: Display bookings made for travel after March 2026

sql

```
SELECT traveller_name, destination, travel_date
FROM travel_bookings
WHERE travel_date > '2026-03-31';
```

Resultant Query

|

traveller_name	destination	travel_date |

| Divya	Mysore	2026-04-18
| Priya	Kerala	2026-05-20
| Rahul	Bangalore	2026-06-25 |

RESULT:

Thus, the Travel_Bookings table was created successfully, travel booking records were inserted, and the required data was retrieved successfully using Aggregate Functions, String Functions, and Date Functions.

EX. NO: 07: Creation of Subqueries and Nested Queries using Products

DATE:14.7.2026

Aim

To create a Products table, insert product records, and retrieve data using Subqueries and Nested Queries.

Software Used

Oracle / MySQL / sqlite3

Description

A Subquery is a query written inside another SQL query. It is used to retrieve intermediate results and use them in the main query.

A Nested Query is a query placed inside another query to perform operations based on the result of the inner query.

SUBQUERY TYPES:

• Single Row Subquery

• Multiple Row Subquery

• Nested Subquery

SINGLE ROW SUBQUERY

A Single Row Subquery returns only one value or one row and is used with comparison operators such as =, >, <, >= and <=.

Syntax:

```sql
SELECT column_name
FROM table_name
WHERE column_name >
(SELECT column_name
FROM table_name
WHERE condition);
```

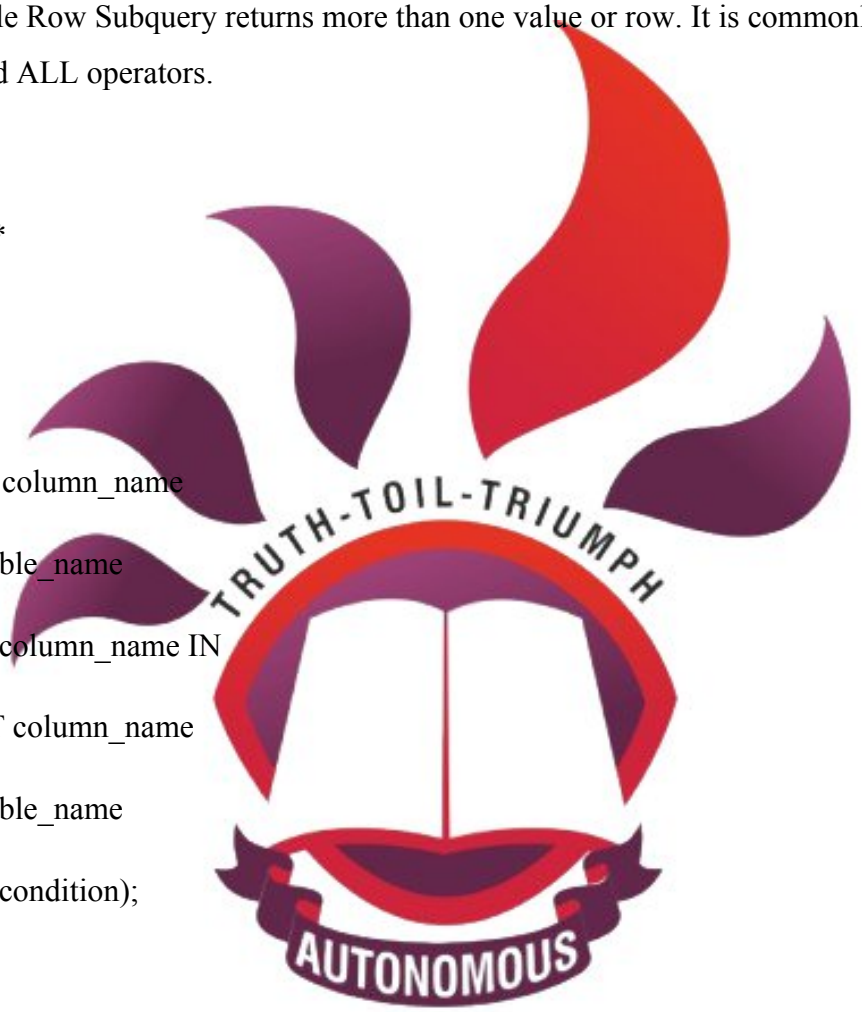
```

### MULTIPLE ROW SUBQUERY

A Multiple Row Subquery returns more than one value or row. It is commonly used with IN, ANY, and ALL operators.

Syntax:**

```sql
SELECT column_name
FROM table_name
WHERE column_name IN
(SELECT column_name
FROM table_name
WHERE condition);
```



NESTED SUBQUERY

A Nested Subquery contains one query inside another query. The result of the inner query is used by the outer query.
```

Syntax:

sql

SELECT column_name

FROM table_name

WHERE column_name =

(SELECT column_name

FROM table_name

WHERE condition);

```

### Explanation




• Create a table named products.

• Insert product details into the table.

• Display all product records.

• Find products whose price is greater than the average price.

• Find products that belong to the Electronics or Clothing categories.

• Find the product that has the highest price.

• Retrieve products based on the price of another product.
```

Create Table

```sql
CREATE TABLE products (
product_id INTEGER PRIMARY KEY,
product_name TEXT,
category TEXT,
price INTEGER
);
```

Table Created

Sample Data

```sql
INSERT INTO products VALUES (1, 'Laptop', 'Electronics', 55000);

INSERT INTO products VALUES (2, 'T-Shirt', 'Clothing', 1200);
```

INSERT INTO products VALUES (3, 'Headphones', 'Electronics', 3500);

INSERT INTO products VALUES (4, 'Smartphone', 'Electronics', 30000);

INSERT INTO products VALUES (5, 'Jeans', 'Clothing', 2500);

INSERT INTO products VALUES (6, 'Watch', 'Accessories', 5000);

```

**Records Inserted**

SQL Query

### Query 1: Display all product records

sql

SELECT * FROM products;

Resultant Query

| productid | productname | category | price |
|---|---|---|---|
```

1	Laptop	Electronics	55000
2	T-Shirt	Clothing	1200
3	Headphones	Electronics	3500
4	Smartphone	Electronics	30000
5	Jeans	Clothing	2500
6	Watch	Accessories	5000

Query 2: Display products whose price is greater than the average price

```sql
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price)
OM products);
```

Resultant Query

productname	price
Laptop	55000
Smartphone	30000

Query 3: Display products that belong to Electronics or Clothing category

```sql

SELECT product_name, category

FROM products

WHERE category IN (SELECT category

FROM products

WHERE category IN ('Electronics', 'Clothing'));

```

Resultant Query

|

product_name	category
Laptop	Electronics
T-Shirt	Clothing
Headphones	Electronics
Smartphone	Electronics
Jeans	Clothing

|

Query 4: Display the product with the highest price

```sql
```

```
SELECT product_name, price

FROM products

WHERE price = (SELECT MAX(price)

FROM products);
```

Resultant Query

|

productname	price
Laptop	55000

Query 5: Display products whose price is greater than T-Shirt's price

```sql
SELECT product_name, price

FROM products

WHERE price > (SELECT price

FROM products

WHERE product_name = 'T-Shirt');
```

Resultant Query

|

product_name	price
| Laptop	55000
| Headphones	3500
| Smartphone	30000
| Jeans	250|
| Watch	5000

RESULT:

Thus, the Products table was created successfully, product records were inserted, and the required product details were retrieved successfully using Subqueries and Nested Queries.

EX. NO:08: Creation of Views, Indexes and Sequences Using Patient Details**

DATE:

Aim

To create and perform operations on Views, Indexes, and Sequences using SQL commands.

Software Used

Oracle SQL / SQL Command Line

Description

View

A View is a virtual table created from one or more tables using a SELECT query. It does not store the actual data separately.

Index:**

An Index is used to improve the speed of data retrieval from a table.

Sequence:**

A Sequence is a database object used to generate unique numeric values automatically.

1. VIEW

Definition

A View is a virtual table based on the result of a SQL query.

Syntax

```sql
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Create Table

sql

```
CREATE TABLE patient_details (
patient_id NUMBER PRIMARY KEY,
patient_name VARCHAR2(50),
department VARCHAR2(30),
consultation_fee NUMBER
);
```

Table Created

Insert Records

sql

INSERT INTO patient_details VALUES (301, 'Kavin', 'Cardiology', 800);

INSERT INTO patient_details VALUES (302, 'Meena', 'General', 400);

INSERT INTO patient_details VALUES (303, 'Arun', 'Neurology', 1000);

INSERT INTO patient_details VALUES (304, 'Divya', 'Cardiology', 900);

INSERT INTO patient_details VALUES (305, 'Priya', 'General', 500);

COMMIT;

Records Inserted

Create View

sql

CREATE VIEW cardiology_patients AS

SELECT patient_id, patient_name, consultation_fee

FROM patient_details

WHERE department = 'Cardiology';

```

View Created**

Display View

```sql

SELECT * FROM cardiology_patients;

```

Resultant Query

|

Patient id	patient name	consultation fee |
301	Kavin	800 |
304	Divya	900 |

Explanation
```

The `cardiology_patients` view displays only the patients who belong to the Cardiology department.

2. INDEX

Definition

An Index is a database object that improves the speed of searching and retrieving records from a table.

Syntax

```sql
CREATE INDEX index_name
ON table_name (column_name);
```

Create Index

sql

```
CREATE INDEX idx_patient_name
ON patient_details(patient_name);
```

```

Index Created**

Use Index in Search

sql

SELECT *

FROM patient_details

WHERE patient_name = 'Kavin';

Resultant Query

|

| patientid | patientname | department | consultation_fee |
|---|---|---|---|
| 301 | Kavin | Cardiology | 800 |

### Explanation

The index `idx_patient_name` is created on the `patient_name` column. It helps improve the performance of searches based on patient names.
```

3. SEQUENCE

Definition

A Sequence is a database object that automatically generates a series of unique numeric values.

Syntax

sql

```
CREATE SEQUENCE sequence_name
START WITH initial_value
INCREMENT BY increment_value;
```

Create Sequence

sql

```
CREATE SEQUENCE patient_seq
START WITH 306
INCREMENT BY 1;
```

Sequence Created

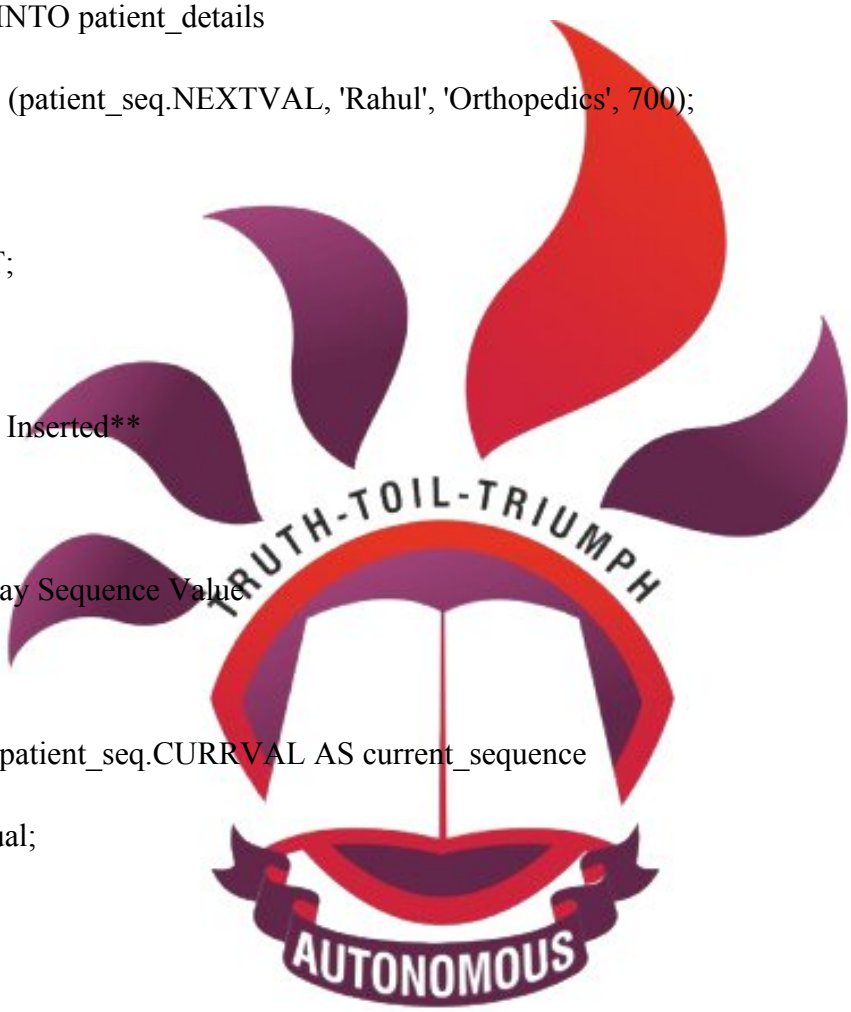
Insert Record Using Sequence

sql

```
INSERT INTO patient_details
VALUES (patient_seq.NEXTVAL, 'Rahul', 'Orthopedics', 700);

COMMIT;
```

Record Inserted



Display Sequence Value

sql

```
SELECT patient_seq.CURRVAL AS current_sequence
FROM dual;
```

Resultant Query

|

Current sequence

306

Insert Another Record Using Sequence

sql

```
INSERT INTO patient_details
VALUES (patient_seq.NEXTVAL, 'Anitha', 'Dermatology', 600);

COMMIT;
```

Record Inserted**

Display All Patient Records

sql

```
SELECT * FROM patient_details
ORDER BY patient_id;
```

Resultant Query

|

Patientid	patientname	department	consultationfee
301	Kavin	Cardiology	800
302	Meena	General	400
303	Arun	Neurology	1000
304	Divya	Cardiology	900
305	Priya	General	500
306	Rahul	Orthopedics	700
307	Anitha	Dermatology	600

Important Commands

Drop View

sql

DROP VIEW cardiology_patients;

```

**View Dropped**

Drop Index

sql

DROP INDEX idx_patient_name;`
```

Index Dropped**

Drop Sequence

```sql
DROP SEQUENCE patient_seq;
```

Sequence Dropped**

Result

Thus, the View, Index, and Sequence were created and successfully operated using SQL commands. The View was used to display selected records, the Index was used to improve data retrieval, and the Sequence was used to generate unique numeric values automatically.

EX. NO:09: Creation of Stored Procedures and Functions

DATE:25.7.2026

Aim

To create and execute Stored Procedures and Functions using PL/SQL.

Oracle SQL / SQL Command Line

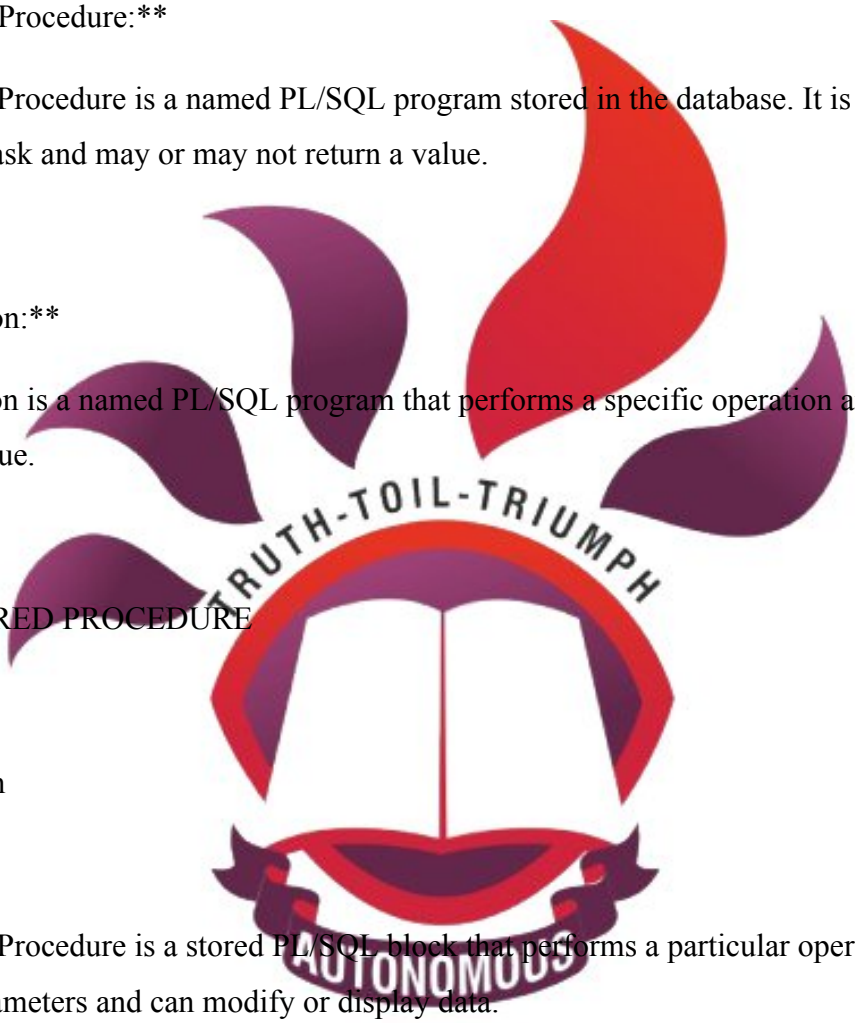
Description

Stored Procedure:

A Stored Procedure is a named PL/SQL program stored in the database. It is used to perform a specific task and may or may not return a value.

Function:

A Function is a named PL/SQL program that performs a specific operation and always returns a single value.



1. STORED PROCEDURE

Definition

A Stored Procedure is a stored PL/SQL block that performs a particular operation. It can accept input parameters and can modify or display data.

Syntax

`sql

CREATE OR REPLACE PROCEDURE procedure_name

(parameter_name IN datatype)

IS

BEGIN

    statements;

END;

/

Create Table

```sql
CREATE TABLE food_orders (
    order_id NUMBER PRIMARY KEY,
    customer_name VARCHAR2(50),
    food_item VARCHAR2(50),
    order_amount NUMBER
);
```

Table Created**

Insert Records

```sql

INSERT INTO food_orders VALUES (101, 'Arun', 'Burger', 250);

INSERT INTO food_orders VALUES (102, 'Meena', 'Pizza', 550);

INSERT INTO food_orders VALUES (103, 'Kavin', 'Biryani', 450);

INSERT INTO food_orders VALUES (104, 'Divya', 'Pasta', 350);

INSERT INTO food_orders VALUES (105, 'Priya', 'Fried Rice', 300);

COMMIT;

```

Records Inserted**

Display Food Order Records

sql

SELECT * FROM food_orders;

Resultant Query

|

order_id customername food_item order_amount	

101	Arun	Burger	250 |
102	Meena	Pizza	550 |
103	Kavin	Biryani	450 |
104	Divya	Pasta	350 |
105	Priya	Fried Rice	300 |

Create Stored Procedure

The following procedure displays the details of a food order based on the order ID.

```sql
CREATE OR REPLACE PROCEDURE display_order (
    p_order_id IN NUMBER
)
IS
    v_customer food_orders.customer_name%TYPE;
    v_food food_orders.food_item%TYPE;
    v_amount food_orders.order_amount%TYPE;
BEGIN
    SELECT customer_name, food_item, order_amount
    INTO v_customer, v_food, v_amount
    FROM food_orders
    WHERE order_id = p_order_id;
```

```
    DBMS_OUTPUT.PUT_LINE('Customer Name: ' || v_customer);
    DBMS_OUTPUT.PUT_LINE('Food Item: ' || v_food);
    DBMS_OUTPUT.PUT_LINE('Order Amount: ' || v_amount);
END;
/
```



Procedure Created**

Execute Stored Procedure

```
sql
SET SERVEROUTPUT ON;

EXEC display_order(103);
```

Resultant Query

Customer Name	Food Item	Order Amount

Kavin	Biryani	450

Output

```text
Customer Name: Kavin
Food Item: Biryani
Order Amount: 450
```

Explanation

The procedure `display_order` accepts the order ID as an input parameter and retrieves the corresponding customer name, food item, and order amount from the `food_orders` table.

2. FUNCTION

Definition

A Function is a stored PL/SQL program that performs a specific operation and returns a single value.

Syntax

```sql
CREATE OR REPLACE FUNCTION function_name
(parameter_name IN datatype)
RETURN datatype
IS
BEGIN
    statements;
    RETURN value;
END;
/
```

Create Function

The following function returns the order category based on the order amount.

```sql
CREATE OR REPLACE FUNCTION get_order_category (
    p_amount IN NUMBER
)
```

```
RETURN VARCHAR2
IS
    v_category VARCHAR2(15);
BEGIN
    IF p_amount >= 500 THEN
        v_category := 'Premium';
    ELSIF p_amount >= 400 THEN
        v_category := 'Large';
    ELSIF p_amount >= 300 THEN
        v_category := 'Medium';
    ELSE
        v_category := 'Small';
    END IF;

    RETURN v_category;
END;
/
```

Function Created

Execute Function

```sql
SELECT order_id, customer_name, order_amount,
    get_order_category(order_amount) AS category
FROM food_orders;
```

Resultant Query

orderid| customername	orderamount	category
101	Arun	250	Small
102	Meena	550	Premium
103	Kavi	450	Large
104	D	350	Medium
105	Priy	300	Medium

Explanation

The function `get_order_category` accepts the order amount as an input and returns the corresponding order category. The function is called inside the SELECT statement for each order.

Important Commands

Display Stored Procedure

sql

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'DISPLAY_ORDER';

```

### Resultant Query

| object_name | object_type |
| --- | --- |
| DISPLAY_ORDER | PROCEDURE |

|

Display Stored Function

```sql

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'GET_ORDER_CATEGORY';

```
```

Resultant Query

|

object_name	object_type |
GET_ORDER_CATEGORY	FUNCTION

Drop Procedure

```sql
DROP PROCEDURE display_order;
```

Procedure Dropped

Drop Function

```sql
DROP FUNCTION get_order_category;
```

Function Dropped**

Result

Thus, the Stored Procedure and Function were created and executed successfully using PL/SQL. The Stored Procedure was used to retrieve food order details, and the Function was used to determine the order category based on the order amount.

EX. NO: 10Creation of Triggers using Bank Accounts Table**

DATE:12.8.2026

Aim

To create and execute Triggers using PL/SQL to automatically perform an action when an INSERT, UPDATE, or DELETE operation occurs on a table.

Software Used

Oracle SQL / SQL Command Line

Description

A Trigger is a stored PL/SQL program that is automatically executed when a specified event occurs on a table or database.

Triggers can be used to automatically perform operations such as validating data, preventing invalid values, and maintaining audit records when a particular event takes place.

1. CREATE TABLE

```sql
CREATE TABLE bank_accounts (
    account_id NUMBER PRIMARY KEY,
    customer_name VARCHAR2(50),
    account_type VARCHAR2(20),
    balance NUMBER
);
```

Table Created

Insert Records

sql

```
INSERT INTO bank_accounts VALUES (301, 'Arun', 'Savings', 25000);
INSERT INTO bank_accounts VALUES (302, 'Meena', 'Current', 40000);
INSERT INTO bank_accounts VALUES (303, 'Kavin', 'Savings', 18000);
INSERT INTO bank_accounts VALUES (304, 'Divya', 'Salary', 30000);
INSERT INTO bank_accounts VALUES (305, 'Priya', 'Savings', 22000);
```

sql

COMMIT;

Records Inserted**

Display Bank Account Records

sql

SELECT * FROM bank_accounts;

Resultant Query

account_id	customer_name	account_type	balance
301	Arun	Savings	25000
302	Meena	Current	40000
303	Kavin	Savings	18000
304	Divya	Salary	30000
305	Priya	Savings	22000

---

2. BEFORE INSERT TRIGGER

Definition

A BEFORE INSERT Trigger is executed automatically before a new record is inserted into a table.

Syntax

```sql
CREATE OR REPLACE TRIGGER trigger_name
BEFORE INSERT ON table_name
FOR EACH ROW
BEGIN
   statements;
END;
/
```



Create Trigger

The following trigger checks whether the balance is negative before inserting a bank account record.

```sql
CREATE OR REPLACE TRIGGER check_account_balance
BEFORE INSERT ON bank_accounts
FOR EACH ROW
BEGIN
    IF :NEW.balance < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Balance cannot be negative');
    END IF;
END;
/
```

Trigger Created

Execute Trigger

```sql
INSERT INTO bank_accounts
VALUES (306, 'Rahul', 'Savings', -5000);
```

Output

```text
ORA-20001: Balance cannot be negative
```

The trigger prevents the insertion of an invalid negative balance.

---

3. BEFORE UPDATE TRIGGER

Definition

A BEFORE UPDATE Trigger is executed automatically before an existing record is updated.

Create Trigger

The following trigger prevents the account balance from being updated to a negative value.

```sql
```

```
CREATE OR REPLACE TRIGGER check_updated_balance

BEFORE UPDATE OF balance ON bank_accounts

FOR EACH ROW

BEGIN

   IF :NEW.balance < 0 THEN

      RAISE_APPLICATION_ERROR(-20002, 'Updated balance cannot be negative');

   END IF;

END;

/
```



Trigger Created

Execute Trigger

```sql
UPDATE bank_accounts

SET balance = -1000

WHERE account_id = 303;
```

Output

```text
ORA-20002: Updated balance cannot be negative
```

The trigger prevents the invalid balance from being updated.

---

4. AFTER INSERT TRIGGER

Definition

An AFTER INSERT Trigger is executed automatically after a new record is successfully inserted into a table.

Create Audit Table

```sql
CREATE TABLE account_audit (
```

```
    audit_id NUMBER,
    account_id NUMBER,
    action_type VARCHAR2(20)
);
```

Table Created

Create Sequence

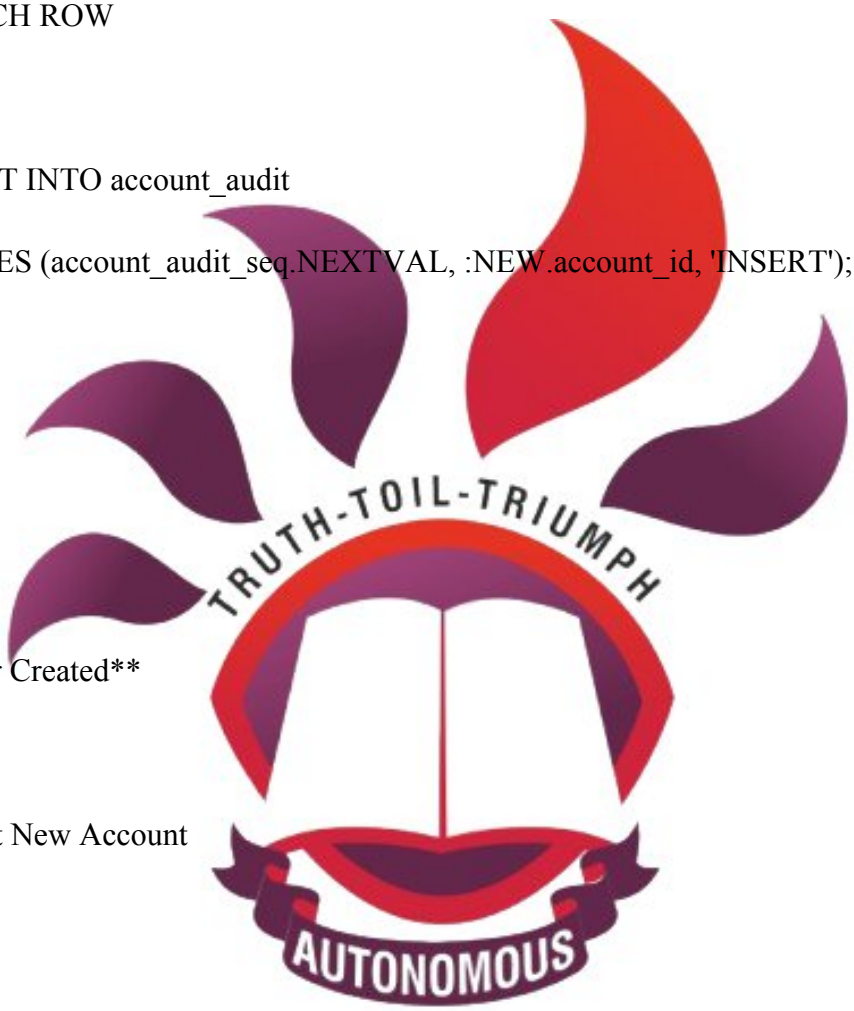
Before executing the above trigger, create the required sequence:

```sql
CREATE SEQUENCE account_audit_seq
START WITH 1
INCREMENT BY 1;
```

Sequence Created

Create Trigger

```sql
CREATE OR REPLACE TRIGGER account_insert_audit
AFTER INSERT ON bank_accounts
FOR EACH ROW
BEGIN
    INSERT INTO account_audit
    VALUES (account_audit_seq.NEXTVAL, :NEW.account_id, 'INSERT');
END;
/
```



Trigger Created

Insert New Account

```sql
INSERT INTO bank_accounts
VALUES (306, 'Rahul', 'Savings', 15000);

COMMIT;
```

```

**Record Inserted**

### Display Audit Records

```sql
SELECT * FROM account_audit;
```

Resultant Query

audit_id	account_id	action_type
1	306	INSERT

---

5. DISPLAY FINAL BANK ACCOUNT RECORDS

```sql
SELECT * FROM bank_accounts
```
```

ORDER BY account_id;

```

### Resultant Query

|

| account_id customer_name account_type \| balance \| |
|---|
| 301 Arun \| Savings \| 25000 \| |
| 302 \| Meena \| Current \| 40000 \| |
| 303 \| Kavin \| Savings \| 18000 \| |
| 304 \| Divya \| Salary \| 30000 \| |
| 305 \| Priya \| Savings \| 22000 \| |
| 306 \| Rahul \| Savings \| 15000 \| |

Important Commands

Display Triggers

```sql

SELECT trigger_name, status

FROM user_triggers;

```

Resultant Query
```

| trigger_name | status |

| ---------- | ------- |

| CHECK_ACCOUNT_BALANCE | ENABLED |

| CHECK_UPDATED_BALANCE | ENABLED |

| ACCOUNT_INSERT_AUDIT | ENABLED |

Drop Trigger

```sql
DROP TRIGGER check_account_balance;
```

Trigger Dropped

```sql
DROP TRIGGER check_updated_balance;
```

Trigger Dropped

```sql

DROP TRIGGER account_insert_audit;

```

Trigger Dropped

Result

Thus, the Triggers were created and executed successfully using PL/SQL. The triggers automatically validated account balance before insertion and updation and recorded new account insertion details in the audit table.

EX. NO: 11 Creation of Cursor Implementation using Bus Reservation Table

DATE:19.8.2026

Aim

To implement and execute a Cursor using PL/SQL to retrieve and process records from a database table.

Software Used

Oracle SQL / SQL Command Line

Description

A Cursor is a pointer used to process the rows returned by a SQL query one row at a time.

Cursors are useful when each record needs to be processed individually.

There are two types of cursors:

1. Implicit Cursor
2. Explicit Cursor

In this practical, an Explicit Cursor is implemented.

1. CREATE TABLE

```sql
CREATE TABLE bus_reservation (
    reservation_id NUMBER PRIMARY KEY,
    passenger_name VARCHAR2(50),
```

```
    bus_route VARCHAR2(50),
    ticket_fare NUMBER
);
```

Table Created

Insert Records

```sql
INSERT INTO bus_reservation VALUES (401, 'Arun', 'Chennai to Madurai', 650);
INSERT INTO bus_reservation VALUES (402, 'Meena', 'Madurai to Coimbatore', 550);
INSERT INTO bus_reservation VALUES (403, 'Kavin', 'Chennai to Pondicherry', 300);
INSERT INTO bus_reservation VALUES (404, 'Divya', 'Trichy to Chennai', 700);
INSERT INTO bus_reservation VALUES (405, 'Priya', 'Salem to Madurai', 450);
```

```sql
COMMIT;
```

Records Inserted

Display Reservation Records

sql

SELECT * FROM bus_reservation;

```

Resultant Query

|

| reservation_id | passenger_name | bus_route | ticket_fare |
|---|---|---|---|
| 401 | Arun | Chennai to Madurai | 650 |
| 402 | Meena | Madurai to Coimbatore | 550 |
| 403 | Kavin | Chennai to Pondicherry | 300 |
| 404 | Divya | Trichy to Chennai | 700 |
| 405 | Priya | Salem to Madurai | 450 |

|

---

## 2. EXPLICIT CURSOR
```

Definition

An Explicit Cursor is a cursor that is declared and controlled by the programmer. It is used to retrieve multiple rows from a table and process them one by one.

Syntax

```sql
DECLARE
   CURSOR cursor_name IS
      SELECT statement;
BEGIN
   OPEN cursor_name;

   LOOP
      FETCH cursor_name INTO variables;

      EXIT WHEN cursor_name%NOTFOUND;

      statements;
   END LOOP;
```

```
    CLOSE cursor_name;
END;
/
```

Create Explicit Cursor

The following cursor retrieves each passenger name and ticket fare from the `bus_reservation` table.

```sql
SET SERVEROUTPUT ON;

DECLARE
    CURSOR bus_cursor IS
        SELECT passenger_name, ticket_fare
        FROM bus_reservation;

    v_passenger_name bus_reservation.passenger_name%TYPE;
    v_ticket_fare bus_reservation.ticket_fare%TYPE;
```

```
BEGIN
    OPEN bus_cursor;

    LOOP
        FETCH bus_cursor INTO v_passenger_name, v_ticket_fare;

        EXIT WHEN bus_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Passenger: ' || v_passenger_name ||
            ' | Fare: ₹' || v_ticket_fare
        );
    END LOOP;

    CLOSE bus_cursor;
END;
/
```

Output

```text
Passenger: Arun | Fare: ₹650
Passenger: Meena | Fare: ₹550
Passenger: Kavin | Fare: ₹300
Passenger: Divya | Fare: ₹700
Passenger: Priya | Fare: ₹450
```

---

3. CURSOR WITH CONDITION

The following cursor displays only the passengers whose ticket fare is greater than ₹500.

```sql
SET SERVEROUTPUT ON;

DECLARE
    CURSOR bus_cursor IS
        SELECT reservation_id, passenger_name, ticket_fare
        FROM bus_reservation
```

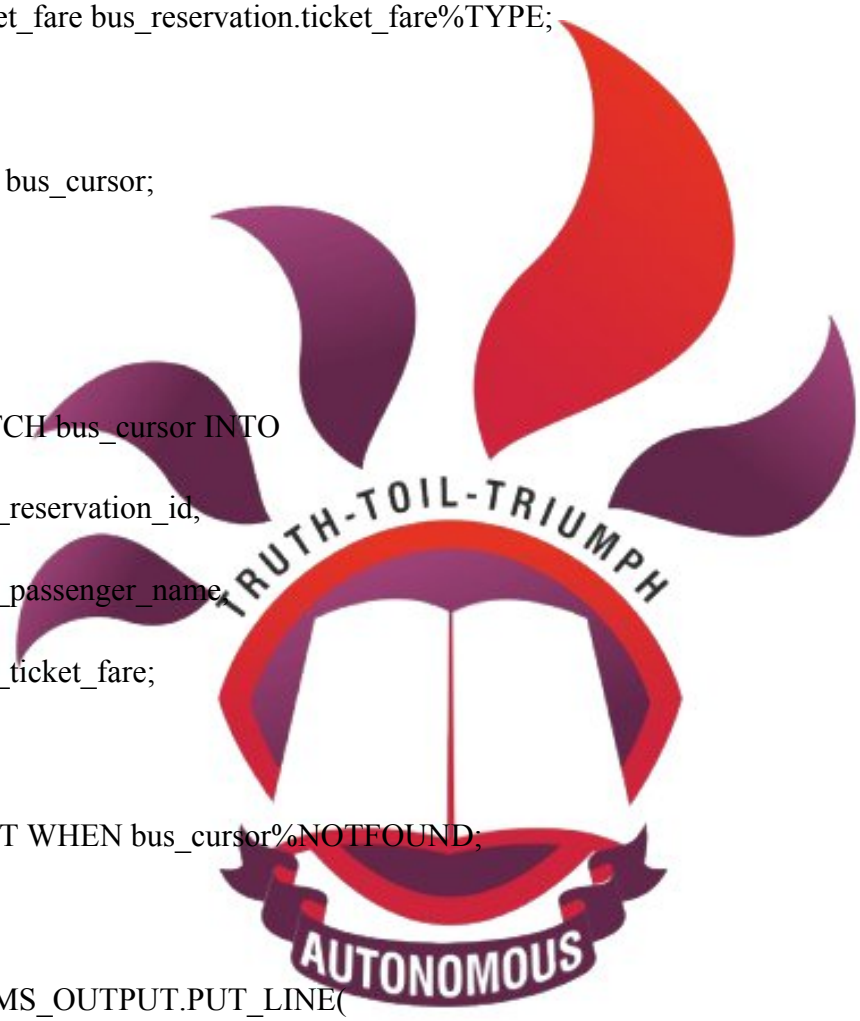
```
    WHERE ticket_fare > 500;

    v_reservation_id bus_reservation.reservation_id%TYPE;
    v_passenger_name bus_reservation.passenger_name%TYPE;
    v_ticket_fare bus_reservation.ticket_fare%TYPE;
BEGIN
    OPEN bus_cursor;

    LOOP
        FETCH bus_cursor INTO
            v_reservation_id,
            v_passenger_name,
            v_ticket_fare;

        EXIT WHEN bus_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Reservation ID: ' || v_reservation_id ||
            ' | Passenger: ' || v_passenger_name ||
            ' | Fare: ₹' || v_ticket_fare
        );
```



```
    END LOOP;

    CLOSE bus_cursor;
END;
/
```

Resultant Query

reservation_id	passenger_name	ticket_fare
401	Arun	650
402	Meena	550
404	Divya	700

---

4. CURSOR ATTRIBUTES

Cursor attributes provide information about the current status of a cursor.

Important Cursor Attributes

| Cursor Attribute | Description

| `%FOUND` | Returns TRUE if the last fetch returned a row |

| `%NOTFOUND` | Returns TRUE if the last fetch did not return a row |

| `%ROWCOUNT` | Returns the number of rows processed |

| `%ISOPEN` | Returns TRUE if the cursor is currently open |

---

5. CURSOR USING %ROWCOUNT

The following program displays the total number of reservations processed by the cursor.

```sql
SET SERVEROUTPUT ON;

DECLARE
   CURSOR bus_cursor IS
      SELECT passenger_name, ticket_fare
      FROM bus_reservation;

   v_passenger_name bus_reservation.passenger_name%TYPE;
   v_ticket_fare bus_reservation.ticket_fare%TYPE;
```

```
BEGIN
    OPEN bus_cursor;

    LOOP
        FETCH bus_cursor INTO v_passenger_name, v_ticket_fare;

        EXIT WHEN bus_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Passenger: ' || v_passenger_name ||
            ' | Fare: ₹' || v_ticket_fare
        );
    END LOOP;

    DBMS_OUTPUT.PUT_LINE(
        'Total Reservations Processed: ' || bus_cursor%ROWCOUNT
    );

    CLOSE bus_cursor;
END;
/
```



```

Output

```text
Passenger: Arun | Fare: ₹650
Passenger: Meena | Fare: ₹550
Passenger: Kavin | Fare: ₹300
Passenger: Divya | Fare: ₹700
Passenger: Priya | Fare: ₹450
Total Reservations Processed: 5
```

Result

Thus, the Cursor was successfully implemented using PL/SQL. The explicit cursor was used to retrieve and process bus reservation records one by one, apply conditions, and count the number of processed records.

EX. NO: 12 Creation of PL/SQL Exception Handling using Restaurant Orders Table

DATE:28.8.2026
```

Aim

To implement Exception Handling in PL/SQL and handle runtime errors using predefined and user-defined exceptions.

Software Used

Oracle SQL / SQL Command Line

Description

An Exception is an error that occurs during the execution of a PL/SQL program.

Exception Handling is used to handle runtime errors without terminating the program abnormally.

The basic structure of exception handling is:

```sql
DECLARE
    declarations;
```

```
BEGIN
    statements;
EXCEPTION
    WHEN exception_name THEN
        statements;
END;
/
```

Types of Exceptions

Type	Description
Predefined Exception	Built-in exceptions provided by Oracle
User-Defined Exception	Exception created by the programmer

---

1. CREATE TABLE

```sql
CREATE TABLE restaurant_orders (
```

```
    order_id NUMBER PRIMARY KEY,
    customer_name VARCHAR2(50),
    food_item VARCHAR2(50),
    amount NUMBER
);
```

Table Created**

Insert Records

```sql
INSERT INTO restaurant_orders VALUES (501, 'Arun', 'Biryani', 450);
INSERT INTO restaurant_orders VALUES (502, 'Meena', 'Pizza', 850);
INSERT INTO restaurant_orders VALUES (503, 'Kavin', 'Burger', 350);
INSERT INTO restaurant_orders VALUES (504, 'Divya', 'Fried Rice', 550);
INSERT INTO restaurant_orders VALUES (505, 'Priya', 'Pasta', 650);
```

```sql
COMMIT;
```

Records Inserted

Display Order Records

sql

SELECT * FROM restaurant_orders;

```

Resultant Query

|

| order_id | customer_name | food_item | amount |
|---|---|---|---|
| 501 | Arun | Biryani | 450 |
| 502 | Meena | Pizza | 850 |
| 503 | Kavin | Burger | 350 |
| 504 | Divya | Fried Rice | 550 |
| 505 | Priya | Pasta | 650 |

2. NO_DATA_FOUND EXCEPTION

Definition
```

`NO_DATA_FOUND` occurs when a `SELECT INTO` statement does not return any record.

Program

```sql
SET SERVEROUTPUT ON;

DECLARE
    v_customer restaurant_orders.customer_name%TYPE;
    v_amount restaurant_orders.amount%TYPE;
BEGIN
    SELECT customer_name, amount
    INTO v_customer, v_amount
    FROM restaurant_orders
    WHERE order_id = 599;

    DBMS_OUTPUT.PUT_LINE('Customer: ' || v_customer);
    DBMS_OUTPUT.PUT_LINE('Amount: ₹' || v_amount);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE(
            'Exception: Order not found.'
        );
END;
/
```

Output

```
text
Exception: Order not found.
```

Exception Handled Successfully

---

3. TOO_MANY_ROWS EXCEPTION

Definition

`TOO_MANY_ROWS` occurs when a `SELECT INTO` statement returns more than one record.

Program

```sql
SET SERVEROUTPUT ON;

DECLARE
    v_food restaurant_orders.food_item%TYPE;
BEGIN
    SELECT food_item
    INTO v_food
    FROM restaurant_orders
    WHERE amount > 400;

    DBMS_OUTPUT.PUT_LINE('Food Item: ' || v_food);

EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: More than one order found.'
        );
```

END;

/

```

Output

```text

Exception: More than one order found.

```

Exception Handled Successfully**



4. ZERO_DIVIDE EXCEPTION

Definition

`ZERO_DIVIDE` occurs when a number is divided by zero.

Program
```

```sql
SET SERVEROUTPUT ON;

DECLARE
    v_amount NUMBER := 850;
    v_persons NUMBER := 0;
    v_share NUMBER;
BEGIN
    v_share := v_amount / v_persons;

    DBMS_OUTPUT.PUT_LINE(
        'Amount Per Person: ₹' || v_share
    );

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: Number of persons cannot be zero.'
        );
END;
/
```

Output

text

Exception: Number of persons cannot be zero.

Exception Handled Successfully

5. USER-DEFINED EXCEPTION

Definition

A User-Defined Exception is an exception created by the programmer to handle a specific business condition.

In this example, an order amount greater than ₹800 is treated as a high-value order.

Program

sql

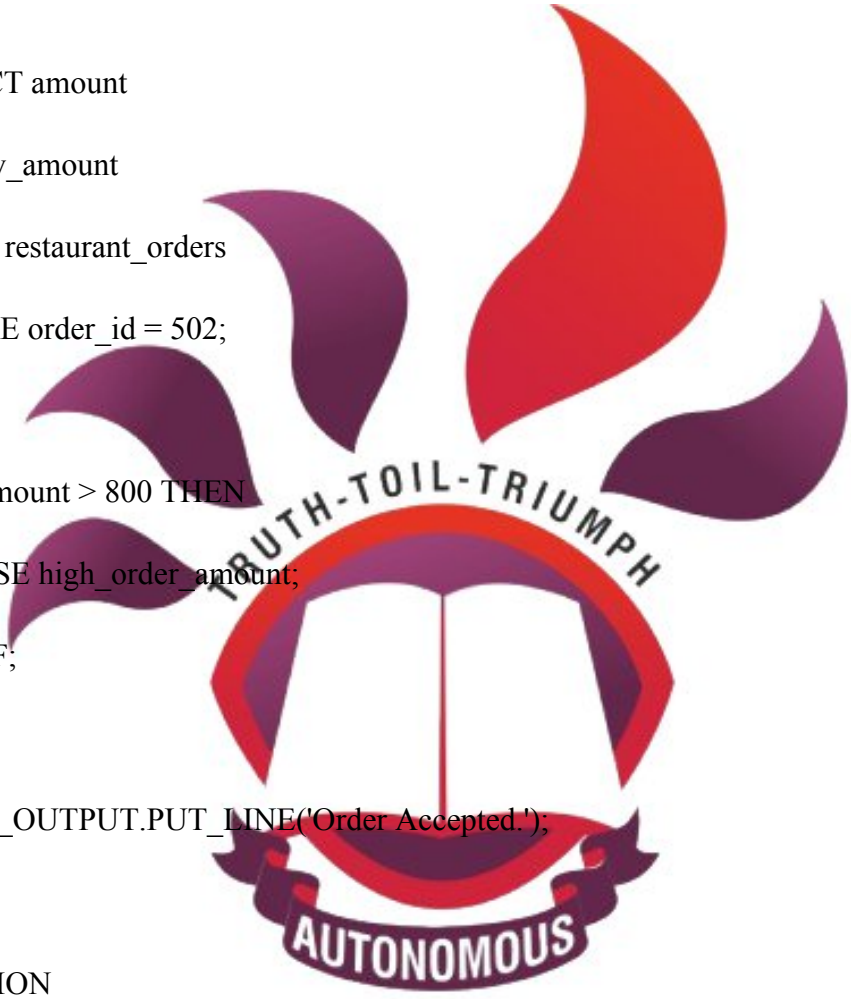
SET SERVEROUTPUT ON;

```
DECLARE
    v_amount restaurant_orders.amount%TYPE;
    high_order_amount EXCEPTION;
BEGIN
    SELECT amount
    INTO v_amount
    FROM restaurant_orders
    WHERE order_id = 502;

    IF v_amount > 800 THEN
        RAISE high_order_amount;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Order Accepted.');

EXCEPTION
    WHEN high_order_amount THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: High-value order detected.'
        );
```



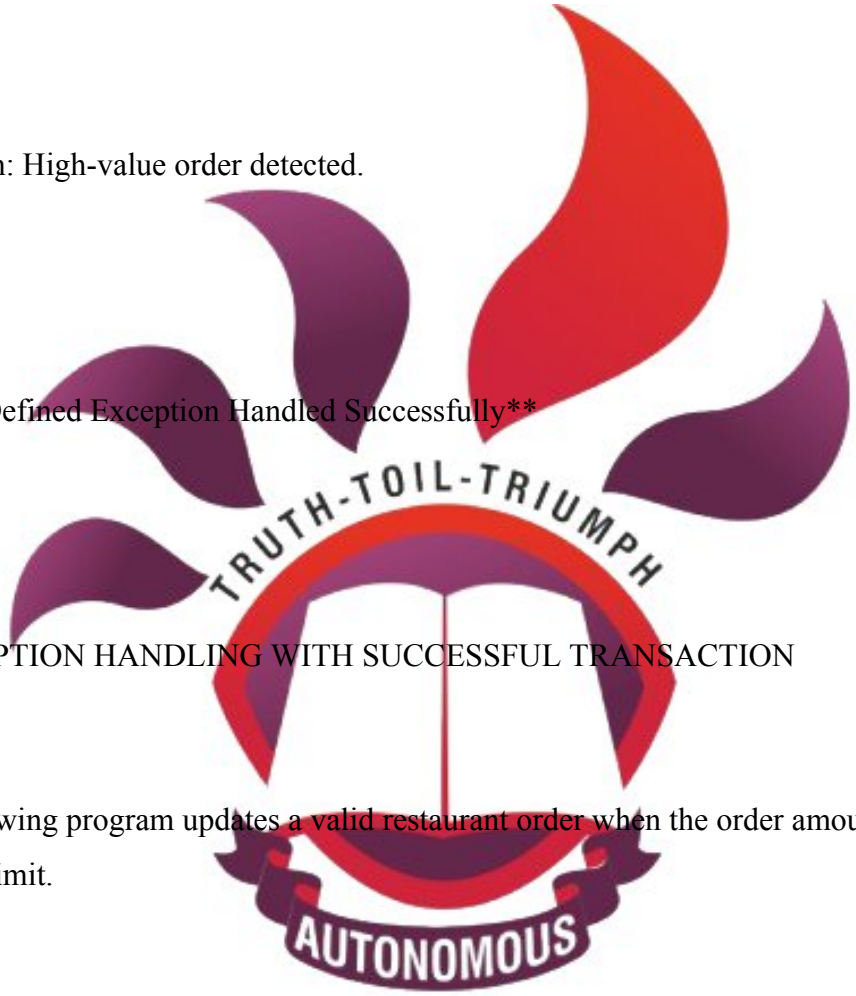
END;

/

Output

```text
Exception: High-value order detected.
```

User-Defined Exception Handled Successfully



6. EXCEPTION HANDLING WITH SUCCESSFUL TRANSACTION

The following program updates a valid restaurant order when the order amount is within the allowed limit.

sql

SET SERVEROUTPUT ON;

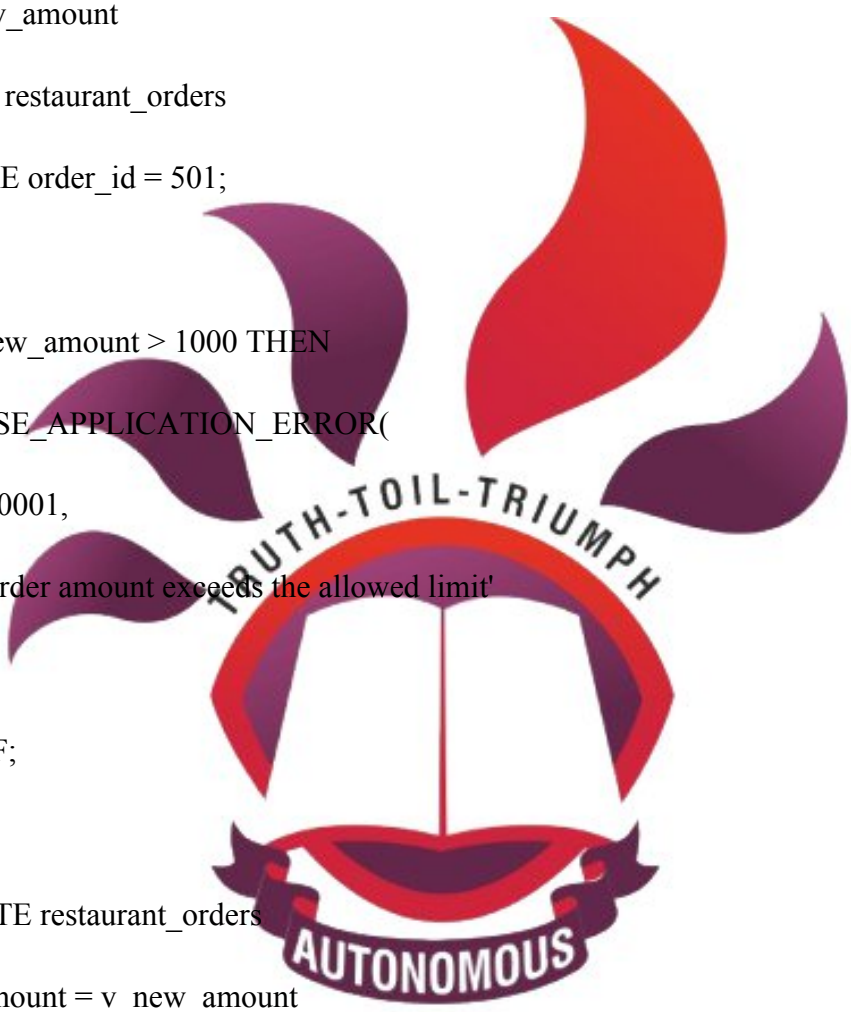
DECLARE

```
    v_amount restaurant_orders.amount%TYPE;
    v_new_amount NUMBER := 750;
BEGIN
    SELECT amount
    INTO v_amount
    FROM restaurant_orders
    WHERE order_id = 501;

    IF v_new_amount > 1000 THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Order amount exceeds the allowed limit'
        );
    END IF;

    UPDATE restaurant_orders
    SET amount = v_new_amount
    WHERE order_id = 501;

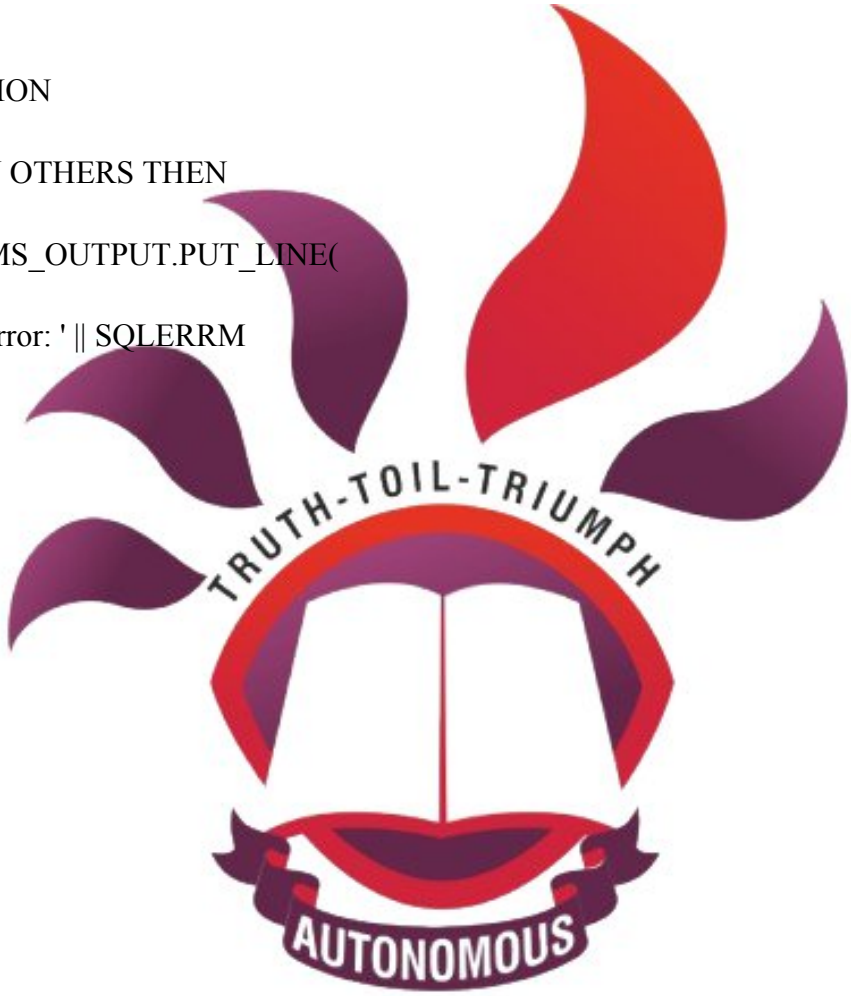
    COMMIT;
```



```
    DBMS_OUTPUT.PUT_LINE('Order Amount Updated Successfully.');
    DBMS_OUTPUT.PUT_LINE(
        'Updated Amount: ₹' || v_new_amount
    );

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/
```

Output




```
text
Order Amount Updated Successfully.
Updated Amount: ₹750
```

Display Updated Order

```sql
SELECT * FROM restaurant_orders
WHERE order_id = 501;
```

Resultant Query

order_id	customer_name	food_item	amount
501	Arun	Biryani	750

Important Predefined Exceptions

Exception	Description
NO_DATA_FOUND	No record is returned
TOO_MANY_ROWS	More than one record is returned
ZERO_DIVIDE	Division by zero occurs
DUP_VAL_ON_INDEX	Duplicate value is inserted into a unique/primary key column
VALUE_ERROR	Invalid value or conversion error occurs

Result

Thus, PL/SQL Exception Handling was successfully implemented. The `NO_DATA_FOUND`, `TOO_MANY_ROWS`, `ZERO_DIVIDE`, and user-defined exceptions were handled successfully using appropriate exception-handling mechanisms.

EX. NO: 13 Creation of PL/SQL Stored Procedures using Employee Attendance Table

DATE:2.9.2026

Aim

To create and execute PL/SQL Stored Procedures to retrieve, update, and process employee attendance records using input and output parameters.

Software Used

Oracle SQL / SQL Command Line

Description

A Stored Procedure is a named PL/SQL program stored in the database. It can be executed whenever required to perform a specific task.

Stored Procedures are useful for performing operations such as displaying records, updating values, and processing data using parameters.

There are three commonly used parameter modes:

| Parameter | Description

| IN | Used to pass a value to the procedure |

| OUT | Used to return a value from the procedure |

| IN OUT | Used to pass and return a value |

---

1. CREATE TABLE

```sql
CREATE TABLE employee_attendance (
    employee_id NUMBER PRIMARY KEY,
    employee_name VARCHAR2(50),
    department VARCHAR2(30),
```

attendance_days NUMBER

);

```

Table Created**

Insert Records

sql

INSERT INTO employee_attendance VALUES (601, 'Arun', 'IT', 24);

INSERT INTO employee_attendance VALUES (602, 'Meena', 'HR', 22);

INSERT INTO employee_attendance VALUES (603, 'Kavin', 'Finance', 26);

INSERT INTO employee_attendance VALUES (604, 'Divya', 'IT', 20);

INSERT INTO employee_attendance VALUES (605, 'Priya', 'Sales', 23);

sql

COMMIT;

**Records Inserted**
```

Display Attendance Records

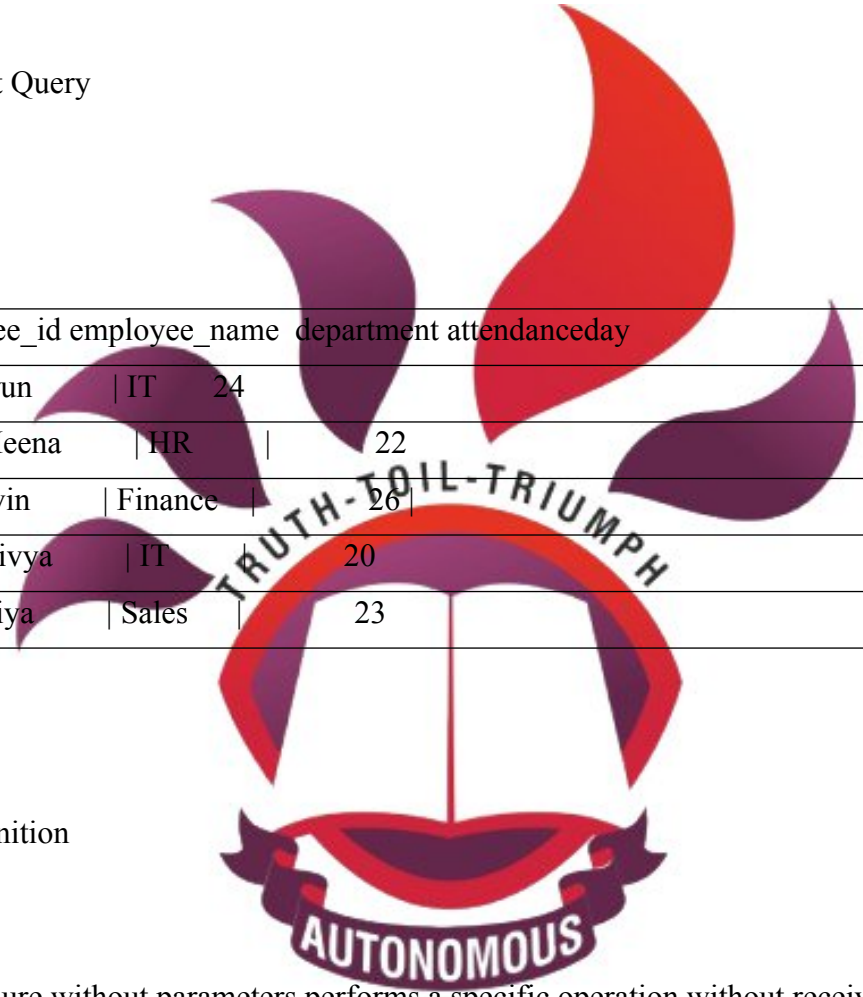
sql

SELECT * FROM employee_attendance;

Resultant Query

|

employee_id employee_name	department attendanceday
601 | Arun	| IT	24
602 | Meena	| HR	| 22
603 Kavin	| Finance	| 26 |
604 | Divya	| IT	| 20
605 | Priya	| Sales	| 23



Definition

A Procedure without parameters performs a specific operation without receiving any input value.

Syntax

```sql
```

```
CREATE OR REPLACE PROCEDURE procedure_name
IS
BEGIN
   statements;
END;
/
```

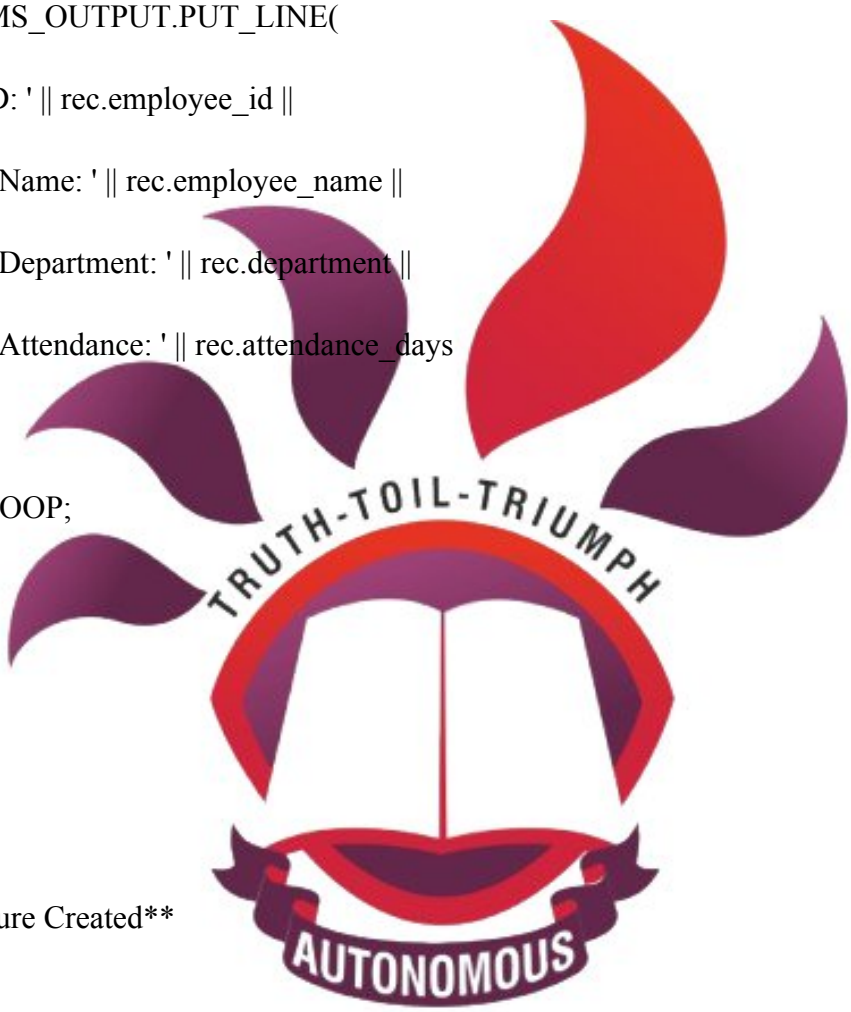
Create Procedure

The following procedure displays all employee attendance records.

```sql
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE display_attendance
IS
BEGIN
   FOR rec IN (
      SELECT employee_id, employee_name,
          department, attendance_days
```

```
        FROM employee_attendance
        ORDER BY employee_id
    )
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            'ID: ' || rec.employee_id ||
            ' | Name: ' || rec.employee_name ||
            ' | Department: ' || rec.department ||
            ' | Attendance: ' || rec.attendance_days
        );
    END LOOP;
END;
/
```



Procedure Created

Execute Procedure

```sql
EXEC display_attendance;
```

```

### Output

text

| | | | |
|---|---|---|---|
| 601 | Arun | Department: IT | Attendance: 24 |
| 602 | Meena | Department: HR | Attendance: 22 |
| 603 | Kavin | Department: Finance | Attendance: 26 |
| 604 | Divya | Department: IT | Attendance: 20 |
| 605 | Priya | Department: Sales | Attendance: 23 |

3. PROCEDURE WITH IN PARAMETER

Definition

An IN parameter is used to pass a value to a procedure.

## Create Procedure

The following procedure updates the attendance days of a particular employee.
```

```
sql

CREATE OR REPLACE PROCEDURE update_attendance (

    p_employee_id IN NUMBER,

    p_days IN NUMBER

)

IS

BEGIN

    UPDATE employee_attendance

    SET attendance_days = p_days

    WHERE employee_id = p_employee_id;


    COMMIT;


    DBMS_OUTPUT.PUT_LINE(

        'Attendance Updated Successfully.'

    );

END;

/
```



Procedure Created

Execute Procedure

sql

```
EXEC update_attendance(604, 25);
```

Output

text

Attendance Updated Successfully.

Display Updated Employee

sql

```
SELECT * FROM employee_attendance
WHERE employee_id = 604;
```

Resultant Query

Employee id	employee name	department	attendance days
604	Divya	IT	25

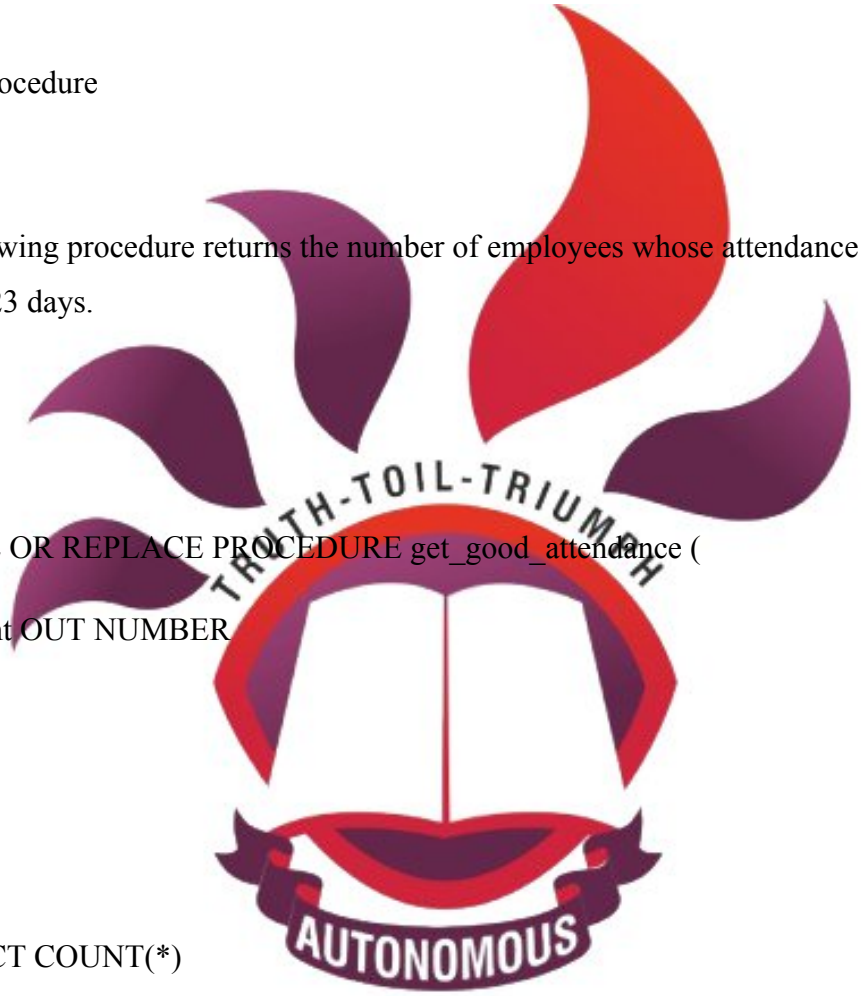
4. PROCEDURE WITH OUT PARAMETER

Definition

An OUT parameter is used to return a value from a procedure to the calling program.

Create Procedure

The following procedure returns the number of employees whose attendance is greater than or equal to 23 days.



sql

```
CREATE OR REPLACE PROCEDURE get_good_attendance (
    p_count OUT NUMBER
)
IS
BEGIN
    SELECT COUNT(*)
    INTO p_count
    FROM employee_attendance
    WHERE attendance_days >= 23;
END;
```

/

```

Procedure Created

Execute Procedure

sql
SET SERVEROUTPUT ON;

DECLARE
    v_count NUMBER;
BEGIN
    get_good_attendance(v_count);

    DBMS_OUTPUT.PUT_LINE(
        'Employees with Good Attendance: ' || v_count
    );
END;
/
```

Output

text

Employees with Good Attendance: 4

5. DISPLAY FINAL ATTENDANCE RECORDS

sql

```
SELECT * FROM employee_attendance
ORDER BY employee_id;
```

Resultant Query

Employee id	employee name	department	attendance_days
601	Arun	IT	24
602	Meena	HR	22
603	Kavin	Finance	26
604	Divya	IT	25
605	Priya	Sales	23

Important Commands

Display Stored Procedures

sql

SELECT object_name, object_type

FROM user_objects

WHERE object_type = 'PROCEDURE';

```

Resultant Query

| OBJECT | NAME | OBJECT_TYPE |
|---|---|---|
| DISPLAY | ATTENDANCE | PROCEDURE |
| UPDATE | ATTENDANCE | PROCEDURE |
| GETGOOD | ATTENDANCE | PROCEDURE |

|

Drop Procedures

sql

DROP PROCEDURE display_attendance;

Procedure Dropped

sql
```

DROP PROCEDURE update_attendance;

Procedure Dropped

sql

DROP PROCEDURE get_good_attendance;

Procedure Dropped

Result

Thus, the PL/SQL Stored Procedures were created and executed successfully. The procedures were used to